



Splunk Enterprise

Search Manual

9.0.10

2026-06-28

Contents

Search Overview	21
Get started with Search	21
Start Here	21
Getting started in your own environment	22
Use the Search app effectively	22
See also	23
Navigating Splunk Web	23
About Splunk Home	23
About the Splunk bar	24
See also	26
About the search language	27
Types of searches	27
Raw event searches	27
Transforming searches	28
Information density	28
Types of commands	28
Why the types of commands matter	29
Streaming and non-streaming commands	29
Processing attributes	30
Distributable streaming	30
Centralized streaming	31
Transforming	31
Generating	31
Orchestrating	32
Dataset processing	32
Search with Splunk Web, CLI, or REST API	32
Search with Splunk Web	32
Search with the CLI or REST API	32

Using the Search App	33
About the Search app	33
Open the Search app	33
The Search summary view	33
The New Search view	35
Anatomy of a search	39
The anatomy of a search	39
About the search pipeline	39
Quotes and escaping characters	40
Fields	41
Help building searches	42
Use the search assistant to build searches	42
Search assistant modes	42
Change the search assistant mode	46
Hide and display the search assistant	47
Change the default search assistant mode for all users	48
See also	49
Help reading searches	49
Syntax highlighting	49
Color themes	51
Auto-format search syntax	52
Number search lines	54
Search bar shortcuts	55
Change the default Search preferences for all users	59
See also	60
Add comments to searches	60
Comments and special characters	61
Comment limitations	61
Use comments to troubleshoot a search	61

Search actions	62
Control search job progress	62
Change the search mode	63
Save the results	63
Other search actions	64
See also	64
Search modes	64
Using the Fast mode	65
Using the Smart mode	66
Using the Verbose mode	66
Comparing search modes	66
See also	67
Search history	67
Search history in the Search Summary view	67
Search history with the Search Assistant	68
Search history with keyboard navigation	68
See Also	69
Search Primer	69
Search command primer	69
Keywords and phrases	70
File paths	70
See Also	70
Wildcards	71
Be efficient and specific	71
Best practices for using wildcards	71
Using wildcards with index= and NOT	72
When to avoid wildcard characters	72
Searching for the asterisk character	74
Other supported wildcards	74

Backslashes	74
More about backslashes	74
Event segmentation and searching	75
Event segmentation at index-time and at search-time	75
Searching and punctuation symbols	75
Major breakers	77
Minor breakers	78
See also	78
Use CASE() and TERM() to match phrases	78
When to use CASE	79
When to use TERM	79
See also	80
Expressions and Predicates	81
Boolean expressions with logical operators	81
Order of evaluation	81
Examples	81
Difference between != and NOT	83
Searching with !=	84
Searching with NOT	84
Using != with the regex command	85
Searching with != or NOT is not efficient	85
Field expressions	85
Use comparison operators to match field values	85
When quotes are required in field expressions	86
Field values that match SPL operators or keywords	86
SPL and regular expressions	86
Pipe characters	87
Backslash characters in regular expressions	87
More about regular expressions	88

Optimizing Searches	88
About search optimization	88
Indexes and searches	88
A tale of two searches	89
See also	92
Quick tips for optimization	92
Limit the data from disk	92
Filter as soon as possible	95
Other techniques for search optimization	96
See also	97
Write better searches	97
Know your type of search	97
Command types and parallel processing	98
Tips for tuning your searches	100
Use fields effectively	100
Summarize your data	101
Use the Search Job Inspector	101
See also	101
Built-in optimization	102
Predicate optimization	102
Predicate merge	103
Predicate pushdown	104
Predicate splitting	105
Projection elimination	106
Stats/chart/timechart to tstats optimization	107
Event types and tags optimization	109
Analyze search optimizations	110
Optimization settings	111
See also	112

Search normalization	112
Benefits of search normalization	112
Predicate flip normalization	113
Predicate sort normalization	113
Disable search normalization	115
Control search execution using directives	115
REQUIRED_TAGS()	116
REQUIRED_EVENTTYPES()	116
READ_SUMMARY()	116
Examples	116
Retrieve Events	118
About retrieving events	118
Events, event data, and fields	118
Use fields to retrieve events	118
Index-time and search-time fields	119
Search examples	120
More about fields	121
Event sampling	121
The event sampling ratio	122
How event sampling works	123
Commands and functions to avoid with event sampling	123
Specify a sampling ratio	123
Event sampling indicators	124
Event sampling with reports and dashboard panels	124
Retrieve events from indexes	124
Specify one or multiple indexes to search	125
Not finding the events you're looking for?	126
Search across one or more distributed search peers	126
Examples	127

Classify and group similar events	127
Save a search as a new event type	128
Use findtypes to discover new event types	129
Use tags to group and find similar events	129
Use the timeline to investigate events	130
Change the timeline format	130
Zoom in and zoom out to investigate events	131
Drill down on event details	133
Drilldown search actions	133
Examples of drilldown search actions	134
Drilldown searches for event timestamps	136
Identify event patterns with the Patterns tab	137
An event patterns example	137
How the Patterns tab works	138
Use the Patterns tab	139
Numbers in the Patterns tab	140
Restricting Patterns tab usage	140
Preview events	141
Limitations using the preview mode	141
Enable events preview mode	141
Specify Time Ranges	142
About searching with time	142
How timestamps are used	142
Timestamps are stored in UNIX time	142
The <code>_time</code> field	142
Specify narrow time ranges	143
See also	143
Select time ranges to apply to your search	143
Select from a list of Preset time ranges	143

Define custom Relative time ranges	147
Define custom Real-time time ranges	148
Define custom Date ranges	149
Define custom Date & Time ranges	150
Use Advanced time range options	151
Customize the list of Preset time ranges	152
Change the default time range	153
See also	153
Specify time modifiers in your search	153
Time modifiers and the Time Range Picker	154
Specify absolute time ranges	154
Specify relative time ranges	155
Relative time modifiers that snap to a time	155
Specify earliest relative time offset and latest time in ad hoc searches	157
Examples of relative time modifiers	157
Examples of chained relative time offsets	157
Examples of searches with relative time modifiers	158
See also	160
Specify time ranges for real-time searches	160
Real-time modifier syntax	160
Real-time searches over "all time"	160
Real-time backfill	161
See also	161
Use time to find nearby events	161
Use time accelerators	162
Use the timeline	162
Search using time bins and spans	162
Using commands to create time bins	162
Time zones and time bins	163

How specifying a time range influences time bins	163
See also	163
How time zones are processed by the Splunk platform	164
Timestamps are stored in UNIX time	164
What time zone is used for timestamps	165
Indexing events using a different time zone	167
How time is interpreted when you search	169
How time zones impact search results	169
See also	170
Subsearches	170
About subsearches	170
Using subsearches	170
Subsearch examples	173
Subsearch performance considerations	177
Output settings for subsearch commands	178
Use subsearch to correlate events	178
Example	179
Change the format of subsearch results	180
Formatting exceptions	180
The search and query fields	181
Examples	182
Create Statistical Tables and Chart Visualizations	182
About transforming commands and searches	182
Transforming commands	182
Table, chart, and report examples	183
Real-time reporting	183
See also	183
Create time-based charts	184

The timechart command	184
Examples	184
See also	184
Create charts that are not (necessarily) time-based	184
The chart command	184
Examples	184
Visualize field value highs and lows	185
The top and rare commands	185
Examples	185
A more complex example of the top command	186
Create reports that display summary statistics	186
The stats and eventstats commands	186
Examples	187
Look for associations, statistical correlations, and differences in search results	187
The associate command	188
The correlate command	188
The diff command	188
Build a chart of multiple data series	188
Scenario	189
Walkthrough	189
See also	190
Compare hourly sums across multiple days	190
Scenario	191
Solution	191
Drill down on tables and charts	191
Open a non-transforming search in Pivot to create tables and charts	192
Open a search in Pivot	192
Save the finished pivot table or chart	193

About permissions for datasets created though this method	193
Search and Report in Real Time	194
About real-time searches and reports	194
Real-time search mechanics	194
Indexed real-time search	195
See also	197
Real-time searches and reports in Splunk Web	197
Real-time searches in Splunk Web	197
Real-time reports in Splunk Web	198
See also	198
Real-time searches and reports in the CLI	198
See also	199
Expected performance and known limitations of real-time searches and reports	199
Indexing throughput	199
Real-time search windows	200
See also	201
How to restrict usage of real-time search	201
Disable real-time search in indexes.conf	201
Disable real-time search for a user or role	201
Set search limits on real-time searches	202
Set indexer limits for real-time search	202
See also	203
Evaluate and Manipulate Fields	203
About evaluating and manipulating fields	203
Use the eval command and functions	203
Types of eval expressions	204
Defining calculated fields	207
Use lookup to add fields from lookup tables	207

Example	207
See also	208
Extract fields with search commands	208
Extract fields using regular expressions	208
Extract fields from .conf files	209
Extract fields from events formatted as tables	209
Extract fields from events formatted in XML	209
Extract fields from XML and JSON documents	209
Extract fields from events based on form templates	209
Evaluate and manipulate fields with multiple values	209
About multivalue fields	209
Evaluate multivalue fields	210
Manipulate multivalue fields	211
See also	212
Calculate Statistics	212
About calculating statistics	212
Use the stats command and functions	213
About the stats commands and functions	213
Stats, eventstats, and streamstats	213
Examples	214
Use stats with eval expressions and functions	216
Use eval expressions to count the different types of requests against each Web server	216
Use eval expressions to categorize and count fields	217
See also	217
Add sparklines to search results	218
Using sparklines with the stats and chart commands	219
Memory and stats search performance	221
Advanced Statistics	222

About advanced statistics	222
See also	222
Commands for advanced statistics	223
Commands that find anomalies	223
Commands for predicting and trends	223
Commands for reports, charts, and maps	224
See also	224
About anomaly detection	225
Overview of anomaly detection	225
Effective anomaly detection	225
See also	225
Finding and removing outliers	226
What is an outlier?	226
Statistical outliers	226
Identifying outliers	227
Removing outliers in charts	228
Manage alerts for outliers	229
See also	230
Detecting anomalies	230
Finding spikes in your data	230
See also	231
Detecting patterns	231
Detecting patterns in events	232
Use the cluster command parameters wisely	232
Other commands to use with the cluster command	232
Return the 3 most recent events in each cluster	232
See also	233
About time series forecasting	233
Commands for time series forecasting	233

See also	235
Machine Learning	235
Splunk Machine Learning Toolkit	235
Group and Correlate Events	235
About event grouping and correlation	235
Use time to identify relationships between events	237
About transactions	237
Transaction search	237
Transaction search example	237
Using stats instead of transaction	238
Transactions and macro search	239
Identify and group events into transactions	239
Transaction search options	240
Example transaction search	241
Manage Jobs	242
About jobs and job management	242
Inspecting jobs and managing jobs	242
Job menu	243
Edit search job settings	243
Managing long-running jobs	244
Administering jobs	245
See also	246
Extending job lifetimes	246
Default job lifetimes	247
Automatic lifetime extensions	247
Changing the lifetime for the current job	247
Changing the lifetime for active jobs	247
Expired jobs	247

Changing the default lifetime values	248
Share jobs and export results	249
Share a job with others	249
Export job results to a file	252
Manage search jobs	252
Opening the Jobs page	252
Refreshing the jobs list	253
View and compare jobs	253
Inspect jobs	254
Extending search job lifetimes	255
Share jobs	256
Export jobs	256
Delete jobs	257
View search job properties	257
Open the Search Job Inspector	258
What the Search Job Inspector shows you	258
Open the Job Details dashboard to get a concise overview of your search job	259
Review the specific execution costs and properties of your search job	260
Execution costs	260
Search job properties	262
Examples of Search Job Inspector output	267
See also	267
Dispatch directory and search artifacts	267
Dispatch directory location	268
Dispatch directory contents	268
Dispatch directory maintenance	273
Limit search process memory usage	275
What does this threshold do?	275
Enable a search process memory threshold	276

Where is threshold activity logged?	276
Manage Splunk Enterprise jobs from the OS	277
Manage jobs in *nix	277
Manage jobs in Windows	278
Use the filesystem to manage jobs	278
Save and Schedule Searches	279
Saving searches	279
See Also	280
Scheduling searches	280
See Also	280
Run Federated Searches Across Multiple Splunk Deployments	280
About federated search	280
Components of a typical federated search setup	281
How federated search works	282
Kinds of federated searches you can set up	283
About the standard and transparent modes	283
About federated search and Splunk security and IT products	286
Set up federated search between Splunk platform deployments	286
Migrate from hybrid search to federated search	287
Comparing hybrid search and federated search	287
Transparent or standard mode?	288
Move to federated search	289
Service accounts and federated search security	292
Federated search security models	292
Step one: Create a service account role on the remote deployment	292
Step two: Create a new service account user on the remote deployment and assign the role to it	294
Additional security for standard mode federated providers: Federated indexes	295
About HTTPS with TLS 1.2 encryption for federated search	295

Set the app context for standard mode federated providers	296
Benefits of setting an app context	296
Find installed apps and their application short names	297
Create standard mode federated providers that have the same host name but different app contexts	297
Duplicate knowledge objects on local and remote search heads	297
Custom knowledge object coordination for standard mode federated providers	298
Example of a custom lookup in a standard mode federated search	298
The importance of duplication over naming	299
Ensure a custom knowledge object exists on federated and remote search heads	299
Help with knowledge objects	300
Define a federated provider	301
Prerequisites	301
Steps	302
Troubleshoot a federated provider connection	304
About creating multiple federated provider definitions for the same host name and port	304
Next step for a standard mode federated provider	305
Next step for a transparent mode federated provider	305
Create a federated index	305
Specifying remote datasets	305
Ensure federated index replication to search head cluster members in your local Splunk Enterprise deployment	306
Prerequisites for federated index creation	307
Steps	307
Give your users access to federated indexes	308
Reference federated indexes in federated searches	309
Run federated searches	309
General requirements for federated search	309
Search over a standard mode federated provider	310

Standard mode federated search examples	312
Search over a transparent mode federated provider	314
Troubleshooting federated searches	314
Export Search Results	316
Export search results	316
What are the available export methods?	316
Export options	317
Supported export formats	317
Export data using Splunk Web	318
When exporting triggers your search to run again	319
Extend the session timeout when exporting large amounts of data	319
Forward data to third-party systems	319
Use reports to send results to stakeholders	320
Export data using the CLI	320
Export data using the Splunk REST API	320
Export data	321
See also	322
Export data using the Splunk SDKs	322
Use Python SDK to export data	322
Use Java SDK to export data	324
Use JavaScript Export to export data	324
Export data using the dump command	325
Forward data to third-party systems	325
Write Custom Search Commands	326
About writing custom search commands	326
Search Examples and Walkthroughs	326
Calculate sizes of dynamic fields	326
Scenario	326

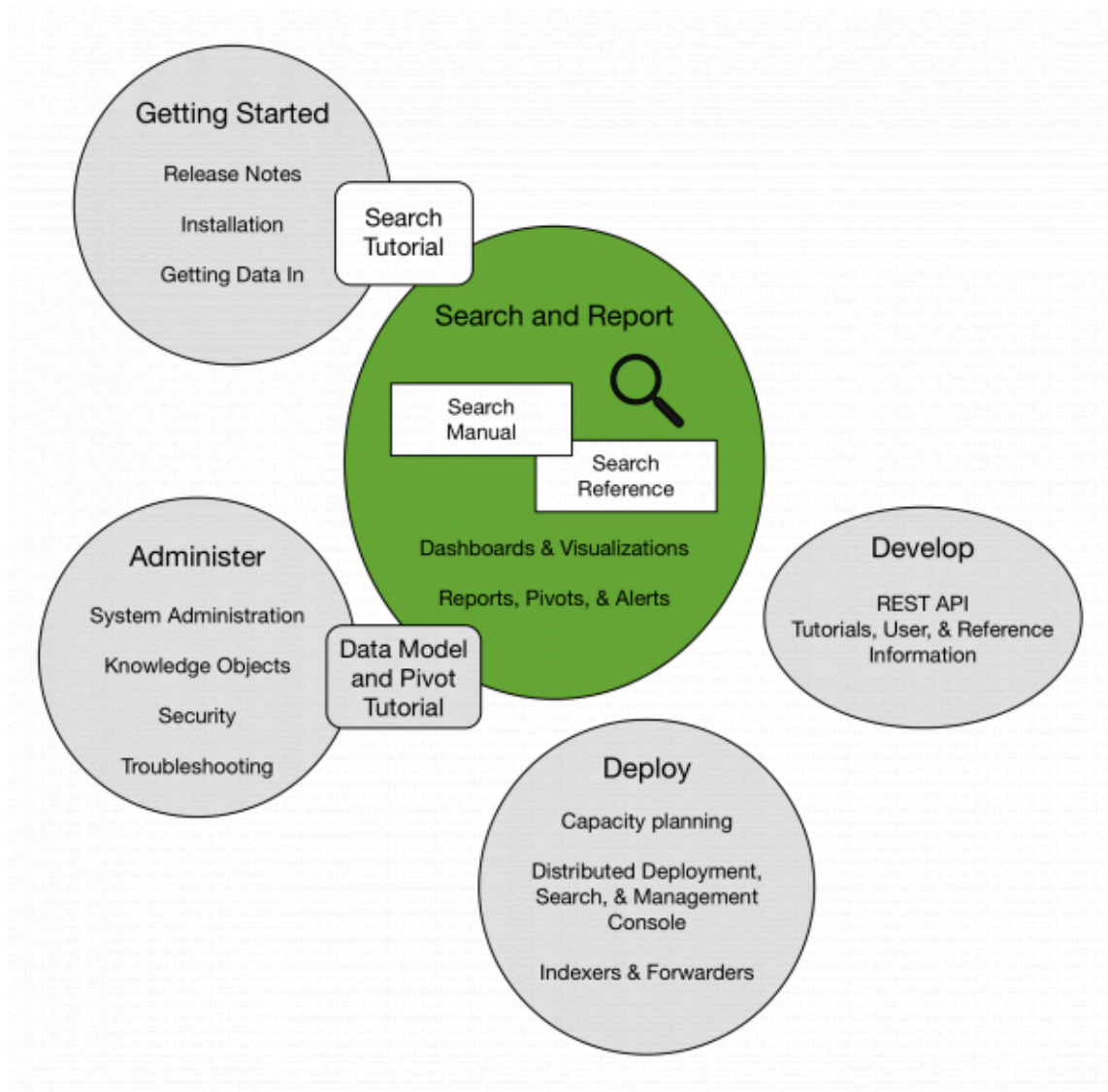
Walkthrough 327

Search Overview

Get started with Search

This manual discusses the **Search & Reporting app** and how to use the Splunk search processing language (**SPL**).

The Search app, the short name for the Search & Reporting app, is the primary way you navigate the data in your Splunk deployment. The Search app consists of a web-based interface (Splunk Web), a command line interface (CLI), and the Splunk SPL.



Start Here

If you are new to Splunk Search, the best way to get acquainted is to start with the [Search Tutorial](#). The Search Tutorial introduces you to the Search and Reporting app and guides you through adding data, searching your data, and building simple reports and dashboards.

The Search Tutorial provides a great foundation for understanding Splunk Search.

Getting started in your own environment

After you complete the Search Tutorial, you should learn about the types of data you can explore, how Splunk software indexes data, and about Splunk knowledge objects.

Here are the resources to look at:

- Upload data to your Splunk deployment. See the [Getting Data In](#) manual.
- Understand how indexing works. See the [Managing Indexers and Clusters of Indexers](#) manual.
- Understand fields and knowledge objects, such as host, source type, and event type. See the [Knowledge Manager Manual](#).

Use the Search app effectively

And of course you need to learn how to use the Search app effectively, which is the focus of this manual. This manual contains detailed information about how to search your data.

Basic Search app skills

- [Navigating Splunk Web](#)
- [Using the Search app](#)
- [Types of searches](#)
- [Types of commands](#)

Detailed Search information

- [Retrieving events](#)
- [Specifying time ranges](#)
- [Optimizing searches](#)
- [Creating tables and charts](#)
- [Evaluating and manipulating fields](#)
- [Calculating statistics and advanced statistics](#)
- [Grouping and correlating events](#)
- [Managing search jobs](#)

Search command reference

For a catalog of search commands and arguments that make up the Splunk SPL, see the [Search Reference](#).

Distributed Search

If you are using Splunk Enterprise, **distributed search** provides a way to scale your deployment by separating the search management and presentation layer from the indexing and search retrieval layer. For an introduction to distributed search, see the *Distributed Search Manual*.

See also

[Navigating Splunk Web](#)

[Using Splunk Search](#)

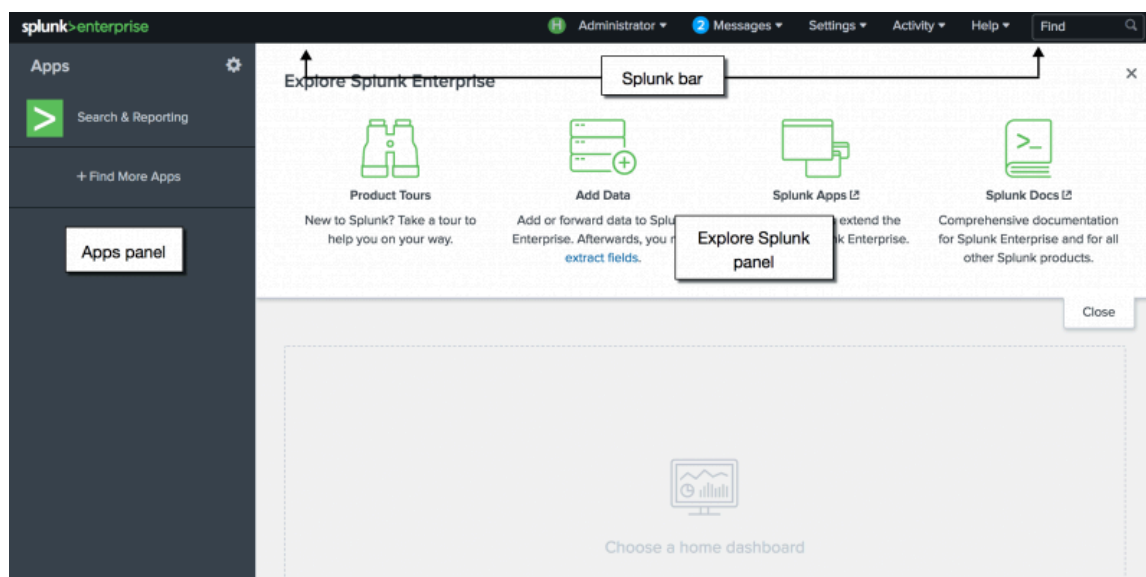
Navigating Splunk Web

This topic discusses navigating the different views in **Splunk Web**, the Splunk web browser interface.

About Splunk Home

Splunk Home is your interactive portal to the data and apps in your Splunk deployment. The first time you log into your Splunk deployment, you land in Splunk Home. All of your apps appear on this page.

The main parts of Splunk Home include the Splunk bar, the Apps menu, the Explore panel, and a custom default dashboard (not shown here).



Your Splunk account might be configured to start in another view instead of Splunk Home, such as Search or Pivot in the **Search & Reporting** app.

Apps panel

The Apps panel lists the apps that are installed on your Splunk instance that you have permission to view. Select the app from the list to open it. The **Search & Reporting** app is often referred to as Splunk Search. When you have more than one app, you can drag and drop the apps within the workspace to rearrange them.

You can perform the following actions.

- Click the gear icon to view and manage the apps that are installed in your Splunk deployment.
- Click the plus icon to browse for more apps to install.

Explore panel

The options in the Explore panel help you to get started. Click on the icons to open the **Add Data** view, browse for new apps, open the user documentation, or open Splunk Answers.

Home dashboard

Below the Explore panel is the home dashboard. When you first open Splunk Home, there is no default dashboard.

Click in the area labeled **Choose a home dashboard** to select a default dashboard.

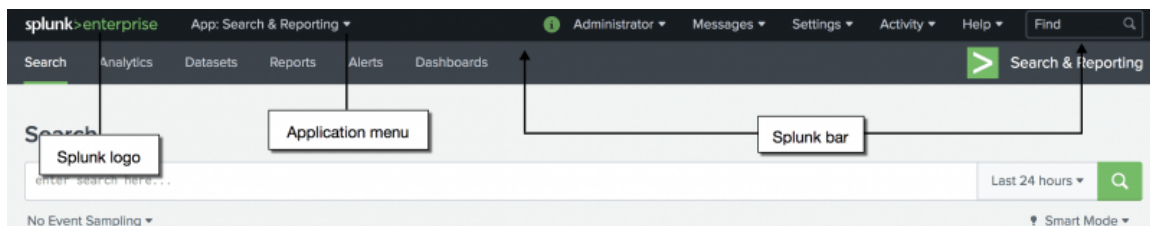
If you are new to Splunk software, hold off selecting a default dashboard until you have created and saved a few searches. You might want to create a dashboard of your own and use that as your default dashboard.

For more information about dashboards, see the [Dashboards and Visualizations](#) manual.

About the Splunk bar

Use the Splunk bar to navigate Splunk Web. You can use it to switch between apps, add data, manage settings and edit your Splunk configuration, view system-level messages, monitor the activity of your search jobs and alerts, and get help using Splunk software.

The Splunk bar in another view, such as the Search view in the **Search & Reporting** app, also includes an **App** menu next to the Splunk logo. Use the **App** menu to quickly switch between the Splunk applications that you have installed on your computer.

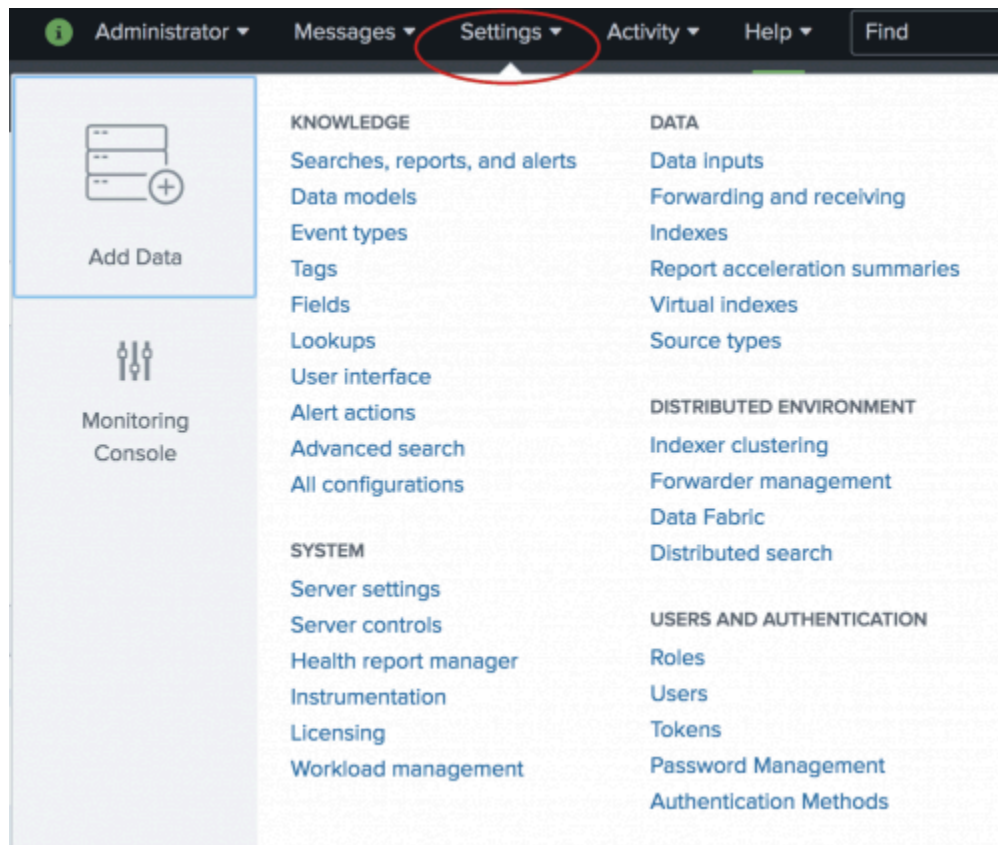


Return to Splunk Home

Click the Splunk logo on the navigation bar to return to Splunk Home from any other view in Splunk Web.

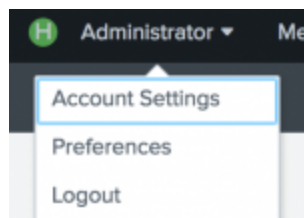
Settings menu

The Settings menu lists the configuration pages for Knowledge objects, Distributed environment settings, System and licensing, Data, and Authentication settings. If you do not see some of these options, you do not have the permissions to view or edit them.



Account menu

Use the **Account** menu to edit your account settings or log out of this Splunk installation. The **Account menu** is called "Administrator" because that is the default user name for a new installation. You can change this display name by selecting **Edit account** and changing the **Full name**. Other settings you can edit include: the time zone settings, the default app for this account, and the account's password.



Messages menu

All system-level error messages are listed on the **Messages** menu. When you have a new message to review, a numerical notification appears next to the **Messages** menu. The notification indicates the number of messages that you have.

2 Messages ▾

Activity menu

The Activity menu lists shortcuts to the Jobs and Triggered alerts views.

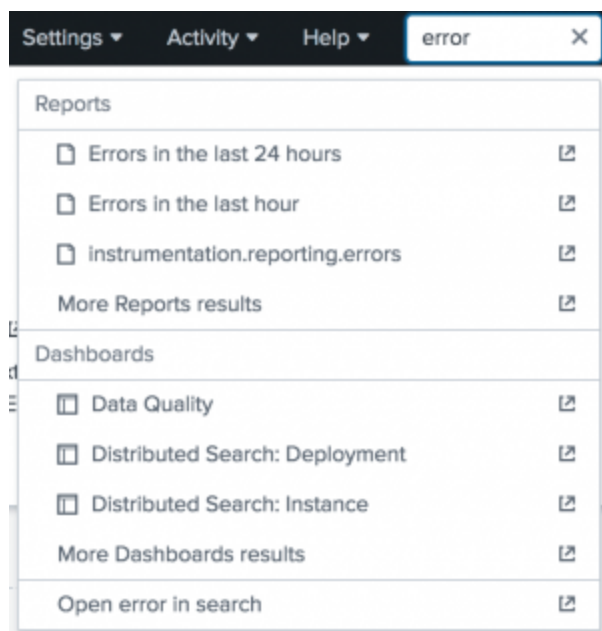
- Click **Jobs** to open the search jobs manager window, where you can view and manage currently running searches.
- Click **Triggered Alerts** to view scheduled alerts that are triggered.

Help

Click **Help** to see links to Video Tutorials, Splunk Answers, the Splunk Support Portal, and online Documentation.

Find

Use **Find** to search for objects within your Splunk deployment. **Find** performs matches that are not case sensitive on the ID, labels, and descriptions in saved objects. For example, if you type **error**, it returns the saved objects that contain that term.



These saved objects include **Reports** and **Dashboards**. The results appear in the list separated by the categories where they exist.

You can also run a search for **error** in the **Search & Reporting** app by clicking **Open error in search**.

See also

[About the Search app](#)

About the search language

The Splunk **Search Processing Language** (SPL) encompasses all the search commands and their functions, arguments and clauses. Search commands tell Splunk software what to do to the events you retrieved from the indexes. For example, you need to use a command to filter unwanted information, extract more information, evaluate new fields, calculate statistics, reorder your results, or create a chart.

Some search commands have functions and arguments associated with them. Use these functions and their arguments to specify how the commands act on your results and which fields they act on. For example, you can use functions to format the data in a chart, describe what kind of statistics to calculate, and specify what fields to evaluate. Some commands also use clauses to specify how to group your search results.

To get familiar with SPL, read these topics in this manual:

- [Types of searches](#)
- [Types of commands](#)
- [Using Splunk Search](#)

For more details on SPL syntax, see [Understanding SPL syntax](#), in the *Search Reference*.

For information about functions, see

- [Evaluation functions](#) in the *Search Reference*
- [Statistical and charting functions](#) in the *Search Reference*

Types of searches

As you search, you will begin to recognize patterns and identify more information that can be useful as searchable fields. You can configure Splunk software to recognize these new fields as you index new data, or you can create new fields as you search. Whatever you learn, you can use, add, and edit this knowledge about fields, events, and transactions to your event data. This capturing of knowledge helps you to construct more efficient searches and build more detailed reports.

Before delving into the language and syntax of search, you should ask what you are trying to accomplish. Generally, after getting data into your Splunk deployment, you want to:

- Investigate to learn more about the data you just indexed or to find the root cause of an issue.
- Summarize your search results into a report, whether tabular or other visualization format.

Because of this, you might hear us refer to two types of searches: Raw event searches and transforming searches.

Raw event searches

Raw event searches are searches that just retrieve events from an index or indexes, and are typically used when you want to analyze a problem. Some examples of these searches include: checking error codes, correlating events, investigating security issues, and analyzing failures. These searches do not usually include search commands (except **search** itself), and the results are typically a list of raw events.

- Read more about raw event searches starting with the topic [About retrieving events](#).

Transforming searches

Transforming searches are searches that perform some type of statistical calculation against a set of results. These are searches where you first retrieve events from an index and then pass the events into one or more search commands. These searches will always require fields and at least one of a set of statistical commands. Some examples include: getting a daily count of error events, counting the number of times a specific user has logged in, or calculating the 95th percentile of field values.

- Read more about the structure of a search in [About the search processing language syntax](#).
- Read more about using subsearches to filter results in [About subsearches](#).
- Read more about transforming searches and commands starting with the topic [About transforming commands and searches](#).

Information density

Whether you are retrieving raw events or building a report, you should also consider whether you are running a search for *sparse* or *dense* information:

- **Sparse searches** are searches that look for a single event or an event that occurs infrequently within a large set of data. You have probably heard these referred to as 'needle in a haystack' or "rare term" searches. Some examples of these searches include: searching for a specific and unique IP address or error code.
- **Dense searches** are searches that scan through and report on many events. Some examples of these searches include: counting the number of errors that occurred or finding all events from a specific host.

See [How search types affect Splunk Enterprise performance](#) in the *Capacity Planning Manual*.

Types of commands

As you learn about Splunk SPL, you might hear the terms streaming, generating, transforming, orchestrating, and data processing used to describe the types of search commands. This topic explains what these terms mean and lists the commands that fall into each category.

There are six broad categorizations for almost all of the search commands:

- distributable streaming
- centralized streaming
- transforming
- generating
- orchestrating
- dataset processing

These categorizations are not mutually exclusive. Some commands fit into only one categorization. The **stats** command is an example of a command that fits only into the transforming categorization. Other commands can fit into multiple categorizations. For example a command can be streaming and also generating.

For a complete list of commands that are in each type, see [Command types](#) in the *Search Reference*.

Why the types of commands matter

Although it can be easy to get confused by the different categories of commands, having a solid understanding of the differences between types of commands will help you understand the implications for how and where data is processed, and optimize the performance of your searches.

For example, suppose you have a search that uses the following commands in this order:

```
search... | lookup... | where... | eval... | sort... | where... |...
```

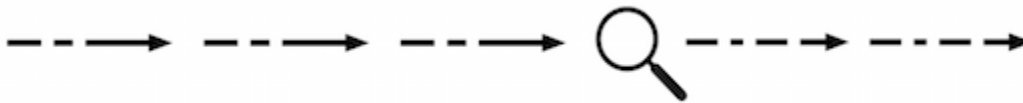
The first 4 commands, from the **search** to **eval** commands, are distributable streaming commands that can all be processed on the indexers. As a result, when the search is run, the **search head** pushes the search to the indexers.

Since the **sort** command is not a distributable streaming command and needs all of the events in one place, the events that are returned from the first 4 commands are then sent back to the search head for sorting. As a result, the rest of the search after the **sort** command must also be processed on the search head. This is true even if the commands that follow **sort** are distributable streaming commands, like the second **where** command in the search.

Once search processing moves to the search head, it can't be moved back to the indexer. With this in mind, you should put non-streaming commands as late as possible in your searches to make them run efficiently. To find out more about how the types of commands used in searches can affect performance, see [Write better searches](#).

Streaming and non-streaming commands

A **streaming command** operates on each event as it is returned by a search. Essentially one event in and one (or no) event out.

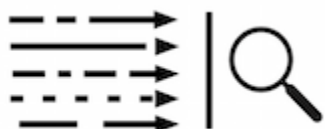


For example, the **eval** command can create a new field, **full_name**, to contain the concatenation of the value in the **first_name** field, a space, and the value in the **last_name** field.

```
... | eval full_name = first_name." ".last_name
```

The **eval** command evaluates each event without considering the other events.

A **non-streaming command** requires the events from all of the indexers before the command can operate on the entire set of events. Many transforming commands are non-streaming commands. There are also several commands that are not transforming commands but are also non-streaming. These non-transforming, non-streaming commands are most often dataset processing commands.



For example, before the **sort** command can begin to sort the events, the entire set of events must be received by the **sort** command. Other examples of non-streaming commands include **dedup** (in some modes), **stats**, and **top**.

Non-streaming commands force the entire set of events to the search head. This requires a lot of data movement and a loss of parallelism.

For information on how to mitigate the cost of non-streaming commands, see [Write better searches](#) in this manual.

Processing attributes

The following table describes the processing differences between some of the types of commands.

	Distributable streaming	Centralized streaming	Data processing (non-streaming)	Transforming
Can run on indexers	Y	N	N	N
Can output before final input	Y	Y	N	N
Outputs events if inputs are events	Y	Y	Y	N

When a command is run it outputs either events or results, based on the type of command. For example, when you run the **sort** command, the input is events and the output is events in the sort order you specify. However, transforming commands do not output events. Transforming commands output results. For example the **stats** command outputs a table of calculated results. The events used to calculate those results are no longer available. After you run a transforming command, you can't run a command that expects events as an input.

Data processing commands are non-streaming commands that require the entire dataset before the command can run. These commands are not transforming, not distributable, not streaming, and not orchestrating. The **sort** command is an example of a data processing command. See [Data processing commands](#).

Distributable streaming

A streaming command operates on each event returned by a search. For distributable streaming, the order of the events does not matter. A distributable streaming command is a command that can be run on the indexer, which improves processing time. The other commands in a search determine if the distributable streaming command is run on the indexer:

- If all of the commands before the distributable streaming command can be run on the indexer, the distributable streaming command is run on the indexer.
- If any one of the commands before the distributable streaming command must be run on the search head, the remaining commands in the search **must be** run on the search head. When the search processing moves to the search head, it can't be moved back to the indexer.

Distributable streaming commands can be applied to subsets of indexed data in a parallel manner. For example, the **rex** command is streaming. It extracts fields and adds them to events at search time.

Some of the common distributable streaming commands are: [eval](#), [fields](#), [makemv](#), [rename](#), [regex](#), [replace](#), [strcat](#), [typer](#), and [where](#).

For a complete list of distributable streaming commands, see [Streaming commands](#) in the *Search Reference*.

Centralized streaming

For centralized streaming commands, the order of the events matters. A centralized streaming command applies a transformation to each event returned by a search. But unlike distributable streaming commands, a centralized streaming command only works on the search head. You might also hear the term "stateful streaming" to describe these commands.

Centralized streaming commands include: [head](#), [streamstats](#), some modes of [dedup](#), and some modes of [cluster](#).

Transforming

A **transforming command** orders the search results into a data table. These commands "transform" the specified cell values for each event into numerical values that Splunk software can use for statistical purposes. Transforming commands are not streaming. Also, transforming commands are required to transform search result data into the data structures that are required for visualizations such as column, bar, line, area, and pie charts.

Transforming commands include: [chart](#), [timechart](#), [stats](#), [top](#), [rare](#), and [addtotals](#) when it is used to calculate column totals (not row totals).

For more information about transforming commands and their role in create statistical tables and chart visualizations, see [About transforming commands and searches](#) in the this manual.

For a complete list of transforming commands, see [Transforming commands](#) in the *Search Reference*.

Generating

A **generating command** returns information or generates results. Some generating commands can return information from an index, a data model, a lookup, or a CSV file without any transformations to the information. Other generating commands generate results, usually for testing purposes.

Generating commands are either event-generating (distributable or centralized) or report-generating. Most report-generating commands are also centralized. Depending on which type the command is, the results are returned in a list or a table.

Generating commands do not expect or require an input. Generating commands are usually invoked at the beginning of the search and with a leading pipe. That is, there cannot be a search piped into a generating command. The exception to this is the **search** command, because it is implicit at the start of a search and does not need to be invoked.

Examples of generating commands include: [dbinspect](#), [datamodel](#), [inputcsv](#), [inputlookup](#), [makeresults](#), [metadata](#), [pivot](#), [search](#), and [tstats](#)

For a complete list of generating commands, see [Generating commands](#) in the *Search Reference*.

Note: A comment inserted before a generating command causes the search to fail. For example, the following search fails because the commented text precedes **tstats**, which is a generating command:

```
| ``This search returns an error`` | tstats count WHERE host=x BY source.
```

Orchestrating

An **orchestrating command** is a command that controls some aspect of how the search is processed. It does not directly affect the final result set of the search. For example, you might apply an orchestrating command to a search to enable or disable a search optimization that helps the overall search complete faster.

Examples of orchestrating commands include [redistribute](#), [noop](#), and [localop](#). The [lookup](#) command also becomes an orchestrating command when you use it with the **local=t** argument.

Dataset processing

There are a handful of commands that require the entire dataset before the command can run. These commands are referred to as dataset processing commands. These commands are not transforming, not distributable, not streaming, and not orchestrating. Some of these commands fit into other types in specific situations or when specific arguments are used.

Examples of data processing commands include: [sort](#), [eventstats](#), and some modes of [cluster](#), [dedup](#), and [fillnull](#).

For a complete list of dataset processing commands, see [Dataset processing commands](#) in the *Search Reference*.

Search with Splunk Web, CLI, or REST API

You can perform searches using Splunk Web and the Splunk REST API. If you use Splunk Enterprise, you can also run a search from the command line interface (CLI). Which tool is best can sometimes depend on what you want from your search.

If you need to be able to search a Splunk Enterprise and Splunk Cloud Platform deployment together in a single search, you must configure hybrid searching. See [Configure hybrid search](#) in the *Splunk Cloud Platform Admin Manual*.

Search with Splunk Web

When you search with Splunk Web, you are using the Search app, and you can control the search experience by selecting a search mode (Fast, Verbose, Smart). Depending on the mode you select, Splunk software automatically discovers and extracts fields other than the default fields, returns results as an events list or a table, and runs the calculations required to generate the event timeline. Calculating the event timeline is very expensive because it creates buckets and keeps the statistics for events and fields in a dispatch directory such that it is available when the user clicks a bar on the timeline.

- Read more about how to ["Set search mode to adjust your search experience"](#) in this manual.

Search with the CLI or REST API

When you run a search through the command line interface (CLI) or use the search jobs endpoint in the REST API to create a search, the search goes directly to the Splunk search engine without going through **Splunk Web**. These searches can complete much faster than the searches in Splunk Web because Splunk software does not calculate or generate the event timeline. Instead, search results are displayed as a raw events list or a table, depending on the type of search.

- Read more ["About CLI searches"](#) in the *Search Reference*.
- Read about ["Creating searches using the REST API"](#) in the *REST API Reference Manual*.

Using the Search App

About the Search app

The Search & Reporting app, referred to as the *Search app*, is the application that you use to search and create reports on your data.

This topic describes the views and elements that comprise the Search app.

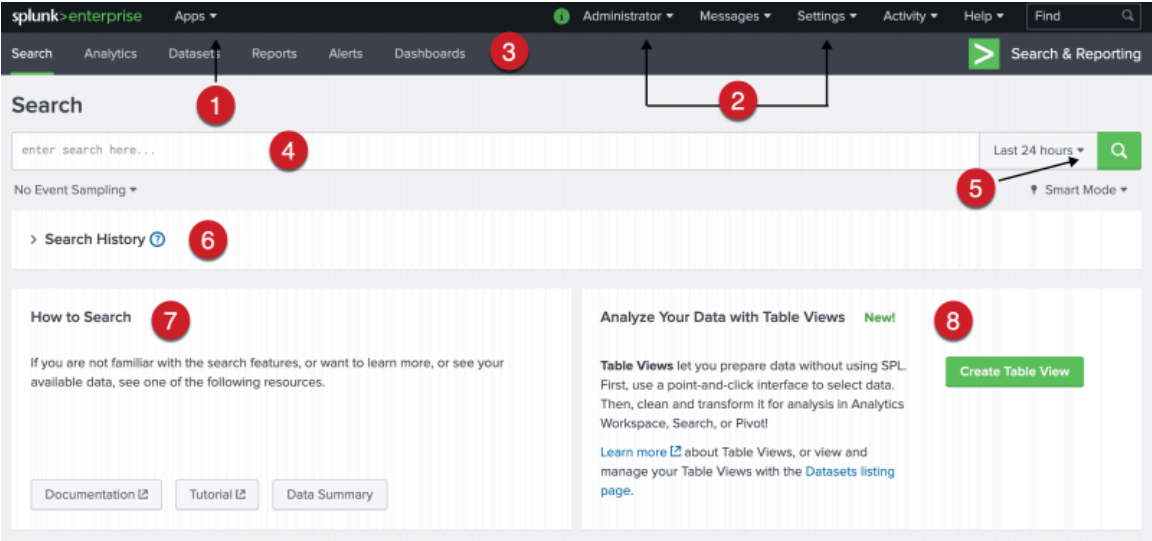
Open the Search app

To open the Search app, from Splunk Home click **Search & Reporting** in the **Apps** panel. This opens the Search Summary view in the Search & Reporting app.

The Search summary view

Before you run a search, the Search summary view displays the following elements: App bar, Search bar, Time range picker, **How to Search** panel, **Search History** panel, and the **Analyze Your Data with Table Views** panel.

Some of these are common elements that you see on other views. Elements that are unique to the Search Summary view are the panels below the Search bar.



Number	Element	Description
1	Applications menu	Switch between Splunk applications that you have installed. The current application, Search & Reporting app, is listed. This menu is on the Splunk bar.
2	Splunk bar	Edit your Splunk configuration, view system-level messages, and get help on using the product.
3	Apps bar	Navigate between the different views in the application you are in. For the Search & Reporting app

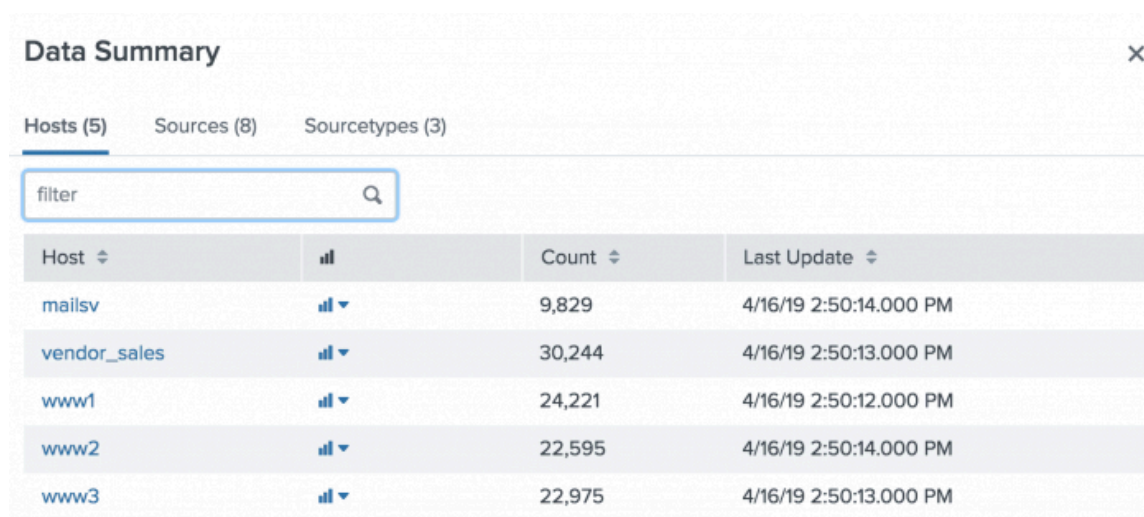
Number	Element	Description
		the views are: Search, Datasets, Reports, Alerts, and Dashboards.
4	Search bar	Specify your search criteria.
5	Time range picker	Specify the time period for the search, such as the last 30 minutes or yesterday. The default is Last 24 hours .
6	Search history	View a list of the searches that you have run. The search history appears after you run your first search.
7	How to Search	Use the links to learn more about how to start searching your data, as well a summary of the data that you have access to.
8	Analyze Your Data with Table Views	Create curated collections of event data into datasets that you design for a specific business purpose.

Data summary

The Data Summary dialog box shows three tabs: Hosts, Sources, Sourcetypes. These tabs represent searchable fields in your data. Selecting a host, source, or source type from the Data Summary dialog box is a great way to see how your data is turned into events.

Host

The **host** of an event is the host name, IP address, or fully qualified domain name of the network machine from which the event originated. In a distributed environment, you can use the host field to search data from specific machines.



Host		Count	Last Update
mailsv		9,829	4/16/19 2:50:14.000 PM
vendor_sales		30,244	4/16/19 2:50:13.000 PM
www1		24,221	4/16/19 2:50:12.000 PM
www2		22,595	4/16/19 2:50:14.000 PM
www3		22,975	4/16/19 2:50:13.000 PM

Source

The **source** of an event is the file or directory path, network port, or script from which the event originated.

Data Summary ×
 Hosts (5) **Sources (8)** Sourcetypes (3)

Source 		Count 	Last Update 
tutorialdata.zip:/mailsv/secure.log		9,829	4/16/19 2:50:14.000 PM
tutorialdata.zip:/vendor_sales/vendor_sales.log		30,244	4/16/19 2:50:13.000 PM
tutorialdata.zip:/www1/access.log		13,628	4/16/19 2:50:12.000 PM
tutorialdata.zip:/www1/secure.log		10,593	4/16/19 2:50:12.000 PM
tutorialdata.zip:/www2/access.log		12,912	4/16/19 2:50:14.000 PM
tutorialdata.zip:/www2/secure.log		9,683	4/16/19 2:50:13.000 PM
tutorialdata.zip:/www3/access.log		12,992	4/16/19 2:50:13.000 PM
tutorialdata.zip:/www3/secure.log		9,983	4/16/19 2:50:12.000 PM

Source type

The **source type** of an event tells you what kind of data it is, usually based on how the data is formatted. This classification lets you search for the same type of data across multiple sources and hosts.

Data Summary ×
 Hosts (5) Sources (8) **Sourcetypes (3)**
 

Sourcetype 		Count 	Last Update 
access_combined_wcookie		39,532	4/16/19 2:50:14.000 PM
secure		40,088	4/16/19 2:50:14.000 PM
vendor_sales		30,244	4/16/19 2:50:13.000 PM

In this example, source types are:

- **access_combined_wcookie:** Apache web server logs
- **secure:** Secure server logs
- **vendor_sales:** Global sales vendors

For information about which source type is assigned to your data, see [Why source types matter](#) in the *Getting Data In* manual.

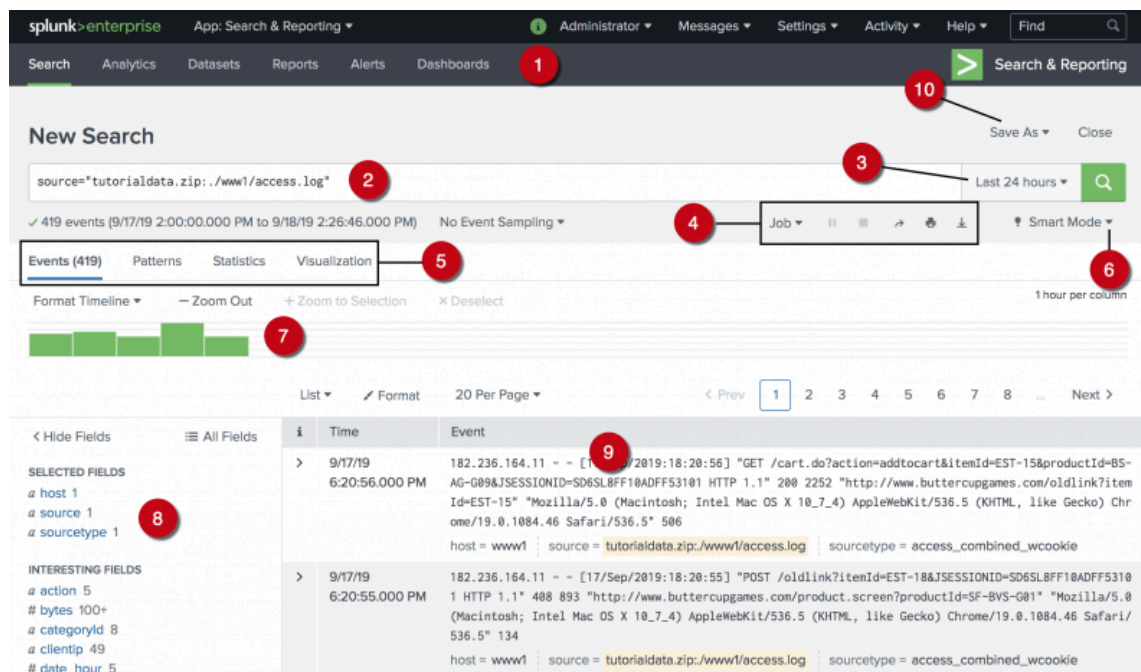
The New Search view

The **New Search** view opens after you run a search or when you click the **Search** tab to start a new search. The App bar, Search bar, and Time range picker are still available in this view. Additionally, this view contains many more elements: search action buttons and search mode selector; counts of events; job status bar; and tabs for Events, Patterns, Statistics, and Visualizations.

You can type `index=_internal` in the Search bar and press **Enter** to look at the events from the internal log files on your Splunk instance.

If you followed the steps [to get data into your Splunk deployment](#) in the *Search Tutorial*, you can type `buttercupgames` in the Search bar and press **Enter** to search for the "buttercupgames" keyword in your events.

In this view, the App bar, Search bar and Time range picker are also available. The **New Search** view contains many more elements such as search action buttons, a search mode selector, counts of events, a job status bar, and results tabs for Events, Patterns, Statistics, and Visualizations.



Number	Element	Description
1	Apps bar	Navigate between the different views in the Search & Reporting app: Search, Metrics, Datasets, Reports, Alerts, and Dashboards.
2	Search bar	Specify your search criteria.
3	Time range picker	Specify the time period for the search.
4	Search action buttons	Actions that you can perform, including working with your search Job, sharing, printing, and exporting your search results.
5	Search results tabs	The tab that your search results appear on depends on your search. Some searches produce a set of events, which appear on the Events tab. Other searches transform the data in events to produce search results, which appear on the Statistics tab.
6	Search mode	Use the search mode selector to provide a search experience that fits your needs. The modes are Smart (default), Fast, and Verbose.

Number	Element	Description
	menu	
7	Timeline	A visual representation of the number of events that occur at each point in time. Peaks or valleys in the timeline can indicate spikes in activity or server downtime. The timeline options are located above the timeline. You can format the timescale, zoom out, or zoom to a selected set of events.
8	Fields sidebar	Displays a list of the fields discovered in the events. The fields are grouped into Selected Fields and Interesting Fields .
9	Events viewer	Displays the events that match your search. By default, the most recent event is listed first. In each event, the matching search terms are highlighted. To change the event view, use the List , Format , and Per Page options.
10	Save As menu	Use the Save As menu to save your search results as a Report, Dashboard Panel, Alert, or Event Type.

App bar

Use the App bar to navigate between the different views in the Search & Reporting app: Search, Pivot, Reports, Alerts, and Dashboards. There are entire manuals devoted to these other capabilities.

- [Alerting manual](#)
- [Dashboards and Visualizations](#)
- [Pivot manual](#)
- [Reporting manual](#)

Search bar

Use the search bar to specify your search criteria in Splunk Web. Type your search string and press **Enter**, or click the Search icon which is on the right side of the search bar.



Time range picker

Time is the single most important search parameter that you specify.

Use the time range picker to retrieve events over a specific time period. For **real-time searches** you can specify a window over which to retrieve events. For **historical searches**, you can restrict your search by specifying a relative time range such as 15 minutes ago, Yesterday, and so on. You can also restrict your searches using a specific date and time range. The time range picker has many preset time ranges that you can select from, but you can also type a custom time range.

For more information, see [About searching with time](#).

Timeline

The timeline is a visual representation of the number of events that occur at each point in time in your results. Peaks or valleys in the timeline can indicate spikes in activity or server downtime. The timeline options are located above the timeline. You can zoom in, zoom out, and change the scale of the chart.

When you click a point on the timeline or use one of the timeline options, the display of the timeline changes based on the events returned from your search. A new search is not run.

Use the **Format Timeline** drop-down list to hide or change the scale of the timeline.

Search actions

There are a wide range of search actions you can perform, including working with your search Jobs, saving, sharing, exporting, and printing your search results.

For more information, see:

- [Perform actions on running searches](#)
- [About jobs and job management](#)
- [Export search results](#)

Search mode

You can use the search mode selector to provide a search experience that fits your needs. The modes are Smart (default), Fast, and Verbose.

For more information, see [Search modes](#).

Fields sidebar

To the left of the events list is the Fields sidebar. As events are retrieved that match your search, the Fields sidebar shows the **Selected Fields** and **Interesting Fields** in the events. These are the fields that the Splunk software extracts from your data.

When you first run a search the **Selected Fields** list contains the default fields host, source, and sourcetype. The default fields appear in every event.

Interesting Fields are fields that appear in at least 20% of the events.

Next to the field name is a count of how many distinct values there are in that field. Click on any field name to show more information about that field, such as a count and percentage of events that include each value.

Note: If the number of distinct values in a field exceeds 100, the field summary statistics begins discarding some of the statistical information to keep memory usage low. For these fields an approximate distinct count is computed instead of an exact distinct count. This is an intended feature to conserve memory.

Anatomy of a search

A search consists of a series of commands that are delimited by pipe (|) characters. The first whitespace-delimited string after each pipe character controls the command used. The remainder of the text for each command is handled in a manner specific to the given command.

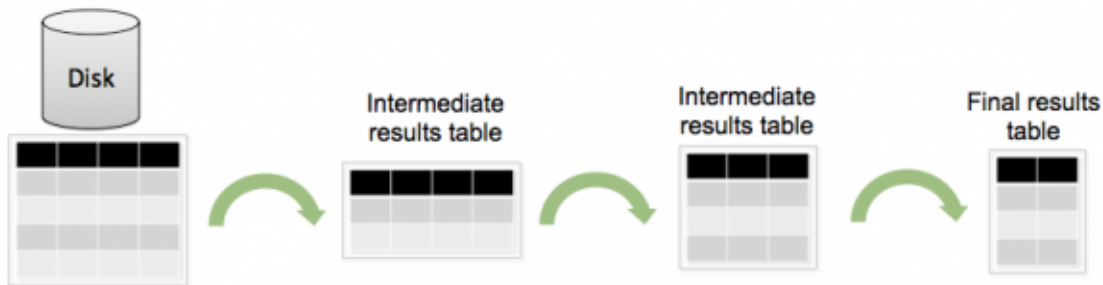
This topic discusses an anatomy of a Splunk search and some of the syntax rules shared by each of the commands and syntax rules for fields and field values.

The anatomy of a search

To better understand how search commands act on your data, it helps to visualize all your indexed data as a table. Each search command redefines the shape of your table.

For example, let's take a look at the following search.

```
sourcetype=syslog ERROR | top user | fields - percent
```



The Disk represents all of your indexed data. The Disk is a table of a certain size with columns that represent fields and rows that represent events. The first intermediate results table shows fewer rows--representing the subset of events retrieved from the index that matched the search terms "sourcetype=syslog ERROR". The second intermediate results table shows fewer columns, representing the results of the top command, "top user", which summarizes the events into a list of the top 10 users and displays the user, count, and percentage. Then, "fields - percent" removes the column that shows the percentage, so you are left with a smaller final results table.

About the search pipeline

The "search pipeline" refers to the structure of a Splunk search, in which consecutive commands are chained together using a pipe character, "|". The pipe character tells Splunk software to use the output or result of one command (to the left of the pipe) as the input for the next command (to the right of the pipe). This enables you to refine or enhance the data at each step along the pipeline until you get the results that you want.

A Splunk search starts with search terms at the beginning of the pipeline. These search terms are keywords, phrases, boolean expressions, key/value pairs, etc. that specify which events you want to retrieve from the index(es). See ["About retrieving events"](#).

The retrieved events can then be passed as inputs into a search command using a pipe character. Search commands tell Splunk software what to do to the events after you retrieved them from the index(es). For example, you might use commands

to filter unwanted information, extract more information, evaluate new fields, calculate statistics, reorder your results, or create a chart. Some commands have functions and arguments associated with them. These functions and their arguments enable you to specify how the commands act on your results and which fields to act on; for example, how to create a chart, what kind of statistics to calculate, and what fields to evaluate. Some commands also enable you to use clauses to specify how you want to group your search results.

- For more information about what you can do with search commands, see ["About the search processing language"](#).
- In the *Search Reference*, for a list of search commands, see the ["Command quick reference"](#) and the individual search command reference topics for syntax and usage information.

Quotes and escaping characters

Generally, you need quotes around phrases and field values that include white spaces, commas, pipes, quotes, or brackets. Quotes must be balanced, an opening quote must be followed by an unescaped closing quote. For example:

- A search such as `error | stats count` will find the number of events containing the string `error`.
- A search such as `... | search "error | stats count"` would return the raw events containing the literal string `error`, a pipe character (`|`), `stats`, and `count`, in that order.

Additionally, you want to use quotes around keywords and phrases if you don't want to search for their default meaning, such as Boolean operators and field-value pairs. For example:

- A search for the keyword AND without meaning the Boolean operator: `error "AND"`
- A search for this field-value phrase: `error "startswith=Error"`

The backslash character (\) is used to escape quotes, pipes, and itself. Backslash escape sequences are still expanded inside quotes. For example:

- The sequence `\|` as part of a search will send a pipe character to the command, instead of having the pipe split between commands.
- The sequence `\"` will send a literal quote to the command, for example for searching for a literal quotation mark or inserting a literal quotation mark into a field using rex.
- The `\\` sequence will be available as a literal backslash in the command.

If Splunk software does not recognize a backslash sequence, it will not alter it.

- For example, `\s` in a search string will be available as `\s` to the command, because `\s` is not a known escape sequence.
- However, in the search string, `\\s` will be available as `\s` to the command, because `\\` is a known escape sequence that is converted to `\`.

Asterisks (*) cannot be searched for using a backslash to escape the character. Splunk software treats the asterisk character as a major breaker. Because of this, it will never be in the index. If you want to search for the asterisk character, you have to run a post-filtering regex search on your data:

```
index=_internal | regex ".*\*.*"
```

For more information about major breakers, read ["Overview of event processing"](#) in the Getting Data in Manual.

Examples

Example 1: The `myfield` field is created with the value of `6`.

```
... | eval myfield="6"
```

Example 2: The `myfield` field is created with the value of `"`.

```
... | eval myfield="\'"
```

Example 3: The `myfield` field is created with the value of `\`.

```
... | eval myfield="\\"
```

Example 4: This search would produce an error because of unbalanced quotation marks.

```
... | eval myfield="\'
```

Fields

Events and results flowing through the Splunk search pipeline exist as a collection of fields. Fields can fundamentally come from the Splunk index, for example, `_time` as the time of the event, `source` as the filename, and so on. Or can be derived from a wide variety of sources at search time, such as eventtypes, tags, regex extractions using the **rex** command, totals coming from the **stats** command, and so on.

For a given event, a given field name might be present or absent. If present, it might contain a single value or multiple values. Each value is a text string. Values might be of positive length (a string, or text) or zero length (empty strings, or `""`).

Numbers, for example, are strings that contain the number. For example, a field containing a value of the number **10** contains the characters **1** and **0**: `"10"`. Commands that take numbers from values automatically convert them internally to numbers for calculations.

Null field

A null field is not present on a particular result or event. Other events or results in the same search might have values for this field. For example, the **fillnull** command adds a field and default value to events or results that lack fields present on other events or results in the search.

Empty field

An empty field is shorthand for a field that contains a single value that is the empty string.

Empty value

A value that is the empty string, or `""`. You can also describe this as a zero-length string.

Multivalue field

A field that contains more than one value. For example, events such as email logs often have multivalue fields in the To: and Cc: information. See [Manipulate and evaluate fields with multiple values](#) in the *Search Manual*.

Help building searches

The Splunk Search Processing Language (SPL) includes commands and functions that you can use to build searches. All of the commands and functions are documented in the [Search Reference](#).

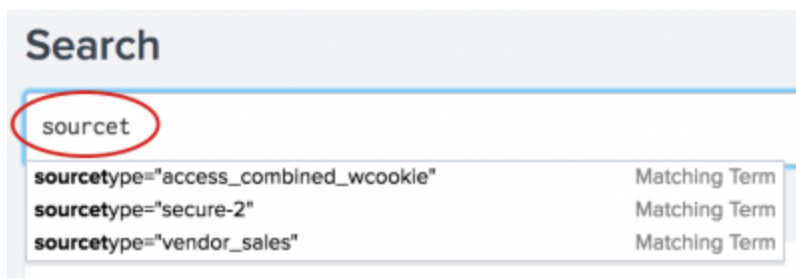
Splunk Web has several built-in features to help you build and parse searches.

- Search assistant modes
- Syntax highlighting
- Auto-format search syntax
- Numbering search lines
- Shortcuts

This topic discusses using the search assistant. See [Help reading searches](#) for information about syntax highlighting, auto-formatting, line numbers, and shortcuts.

Use the search assistant to build searches

When you type a few letters or a term into the Search bar, the search assistant shows you terms and searches that match what you typed.



The **Matching Terms** are based on the terms that are indexed from your data. The **Matching Searches** are based on your recent searches.

The list continues to update as you type.

To add an item in the list to your search criteria you can click on an item, or use the arrow keys to highlight the item and press **Enter**.

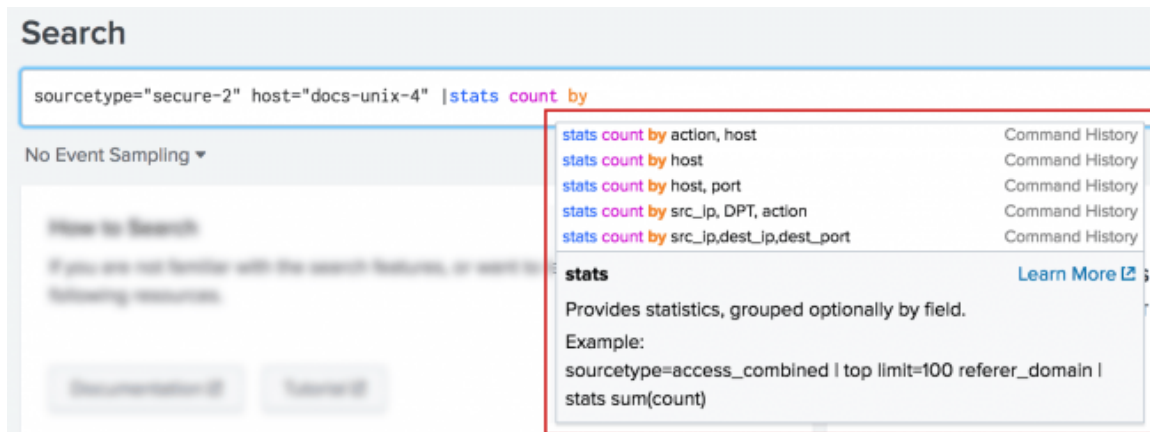
Search assistant modes

The search assistant has three modes: Full, Compact, and None. The default mode is Compact.

Compact mode

The Compact mode displays a list of matching terms and searches when you type. When you type a pipe (|) character to indicate that you want to use a command, a list of the SPL commands appears. You can type a letter to jump to the section of the list that begins with that letter. For example, if you type the letter **s**, the list displays all of the commands that begin with the letter **s**.

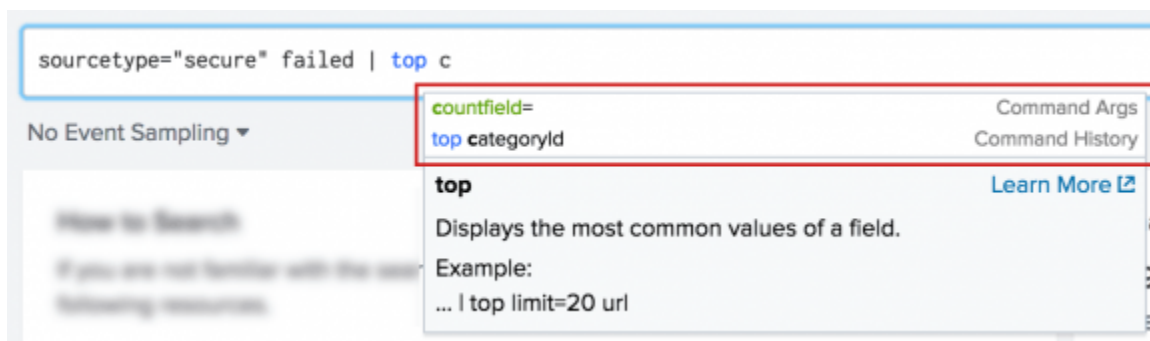
When you type a command, a list appears showing Command History and Matching Searches. Initially, the Command History shows some command examples. As you use a command in your searches, the Command History displays your uses of the command instead of the examples.



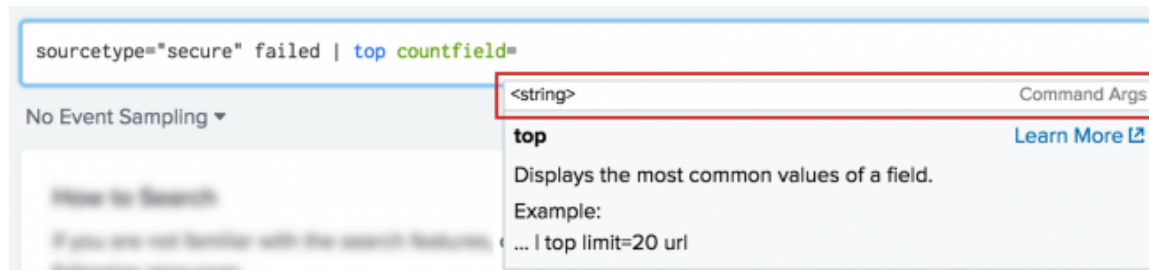
Below the list are a brief description of the command and an example. The **Learn More** link opens the [Search Reference](#) in a new window and displays documentation about the command.

Note: To access the **Learn More** link from your keyboard, use your arrow keys to highlight the command or attribute name. Press **Tab** to highlight the **Learn More** link and then press **Enter** to activate the link.

If you type something after the command, the search assistant shows any command arguments or history that match what you type.



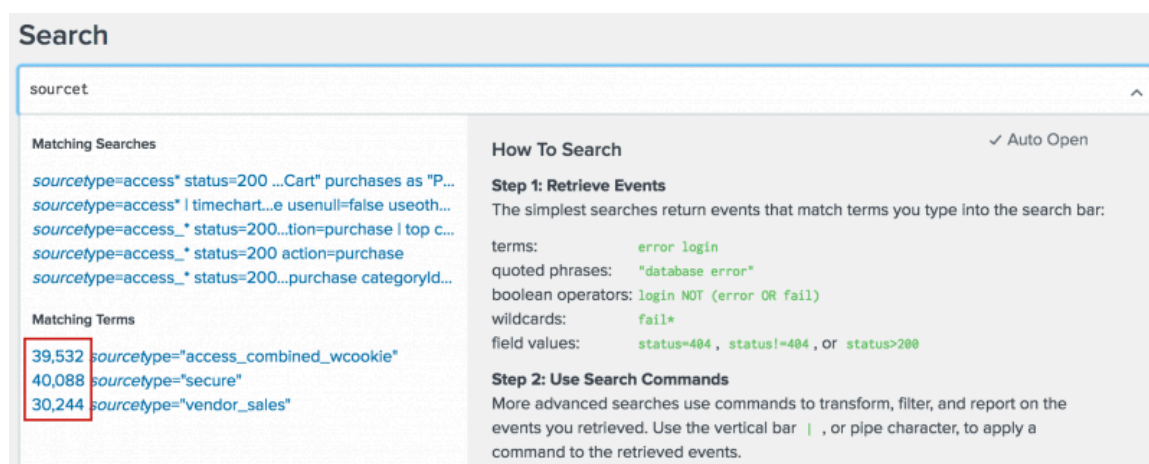
The search assistant can also show you the data type that an argument requires. Type the argument in the Search bar. Include the equal (=) symbol, if that is part of the argument syntax. In the following example, the search assistant shows that a `<string>` value is required for the `countfield` argument.



Full mode

The Full mode displays a list of matching terms and searches when you type, along with a count of how many times a term appears in your indexed data. This count tells you how many search results will be returned if you search on that term. If a term or phrase is not in the list, the term is not in your indexed data.

The Full mode also provides suggestions in the How To Search section on ways that you can retrieve events and use the search commands.



When you type a command in the Search bar, the list of matching terms and searches is replaced with the **Command History** list.

To add an item in the Command History list to your search criteria click on an item, or use the arrow keys to highlight the item and press **Enter**.

The search assistant displays a brief description of the command and several examples. There are two links next to the command description: Help and More.

- The **Help** link opens the [Search Reference](#) in a new window and displays documentation about the command.
- The **More** link expands the information about the command that is displayed on the screen.

Search

sourcetype="secure" failed | [top](#)

Command History

- ... | top categoryId
- ... | top ESXHost
- ... | top user
- ... | top dest_ip
- ... | top dest_port

top ✓ Auto Open

[Help](#) [More >](#)

Displays the most common values of a field.

Examples

Return the 20 most common values of the "url" field.

... | top limit=20 url

Return top URL values.

... | top url

Return top "user" values for each "host".

... | top user by host

When you select the **More** link, several new sections appear. The **Details** section provides a detailed description of the command. The **Syntax** section shows the basic syntax for the command. The **Related** section lists commands that are related to the command that you typed. If the command has complex syntax, click the **More** link next to the syntax to expand the syntax.

Search

sourcetype="secure" failed | [top](#)

Matching Searches

- sourcetype="secure" failed | top port*
- sourcetype="secure" failed | top limit=5 port, host*
- sourcetype="secure" failed | top limit=5 userid*
- sourcetype="secure" failed | top host*

Command History

- ... | top categoryId
- ... | top ESXHost
- ... | top port
- ... | top user
- ... | top dest_ip

top ✓ Auto Open

[Help](#) [Less](#)

Displays the most common values of a field.

Details

Finds the most frequent tuple of values of all fields in the field list, along with a count and percentage. If a the optional by-clause is provided, finds the most frequent values for each distinct tuple of values of the group-by fields.

Syntax

[More >](#)

top (in? (((showcount=boo)((showperc=boo)((limit=int)((countfield=string)| (percentfield=string)((useother=boo)((otherstr=string))) * field-list ((by field-list)?

Related

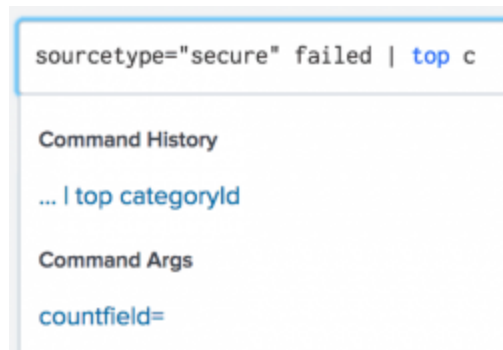
rare, sitop, stats

Examples

Return the 20 most common values of the "url" field.

... | top limit=20 url

If you type something after the command, the search assistant shows any command arguments or history that match what you type.



None mode

You can turn off the search assistant by changing the mode to **None**.

Change the search assistant mode

The default search assistant mode is Compact. You can change the search assistant mode or temporarily hide the search assistant while you build your search.

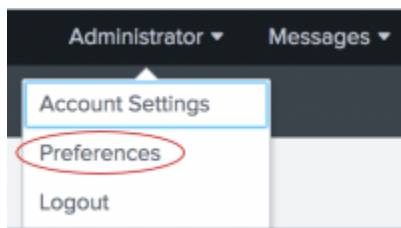
When you change the search assistant mode, the change affects only your user account.

Prerequisites

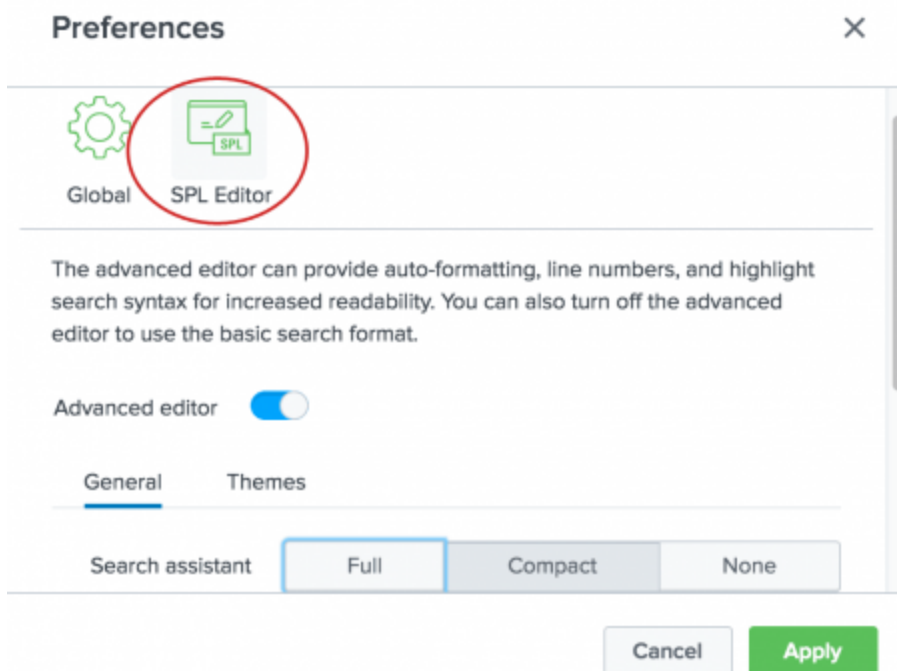
- If the Search bar contains a search that you have not run, run the search before you change the search assistant mode. Otherwise, the search is lost when you change modes. Running the search adds the search to the search history, where you can access it after you change the mode.
- If you have a Splunk Free license, you cannot change the search assistant mode. The User account menu, where the Preferences options resides, is not available in Splunk Free. To learn about what is and is not included in Splunk Free, see [About Splunk Free](#) in the *Admin manual*.

Steps

1. On the Splunk bar, select **[User_account_name] > Preferences**.



2. Click **SPL Editor**.
3. Verify that the **Advanced editor** is turned on.
4. For Search assistant, click on the mode that you want to use, **Full**, **Compact**, or **None**.



5. Click **Apply**.

Hide and display the search assistant

By default, the search assistant opens when you type something into the Search bar. You can turn off or hide the search assistant.

Turn off the search assistant

To turn off the search assistant, change the search assistant mode to **None**.

Hide the search assistant

The options for hiding the search assistant depend on the mode that you are using.

Compact mode

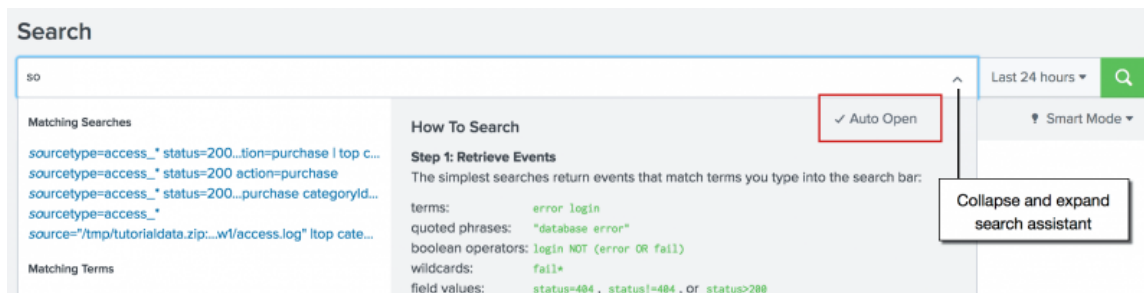
You cannot hide the search assistant. You can only turn off the search assistant.

Full mode

To hide the search assistant in Full mode, you turn off the **Auto Open** feature and collapse the search assistant drop-down.

1. In the search assistant window, click **Auto Open**. This removes the check mark next to **Auto Open**.
2. Click the collapse and expand button on the right side of the Search bar to hide the search assistant.

The search assistant remains hidden until you use the expand button to show the search assistant again. See [Unhide the search assistant window](#) in this topic.



Note: When you uncheck **Auto Open** and click the collapse button the search assistant is hidden, even when you start a new search or close and reopen Splunk Web. The search assistant remains hidden until you unhide it.

Unhide the search assistant

If the search assistant is hidden, click the expand button on the right side of the Search bar and click **Auto Open**.

If these steps do not unhide the search assistant window, then either the search assistant is turned off or there is no assistance for what you have typed into the Search bar.

To turn the Search Assistant back on, you need to change the search assistant mode to **Compact** or **Full**.

Change the default search assistant mode for all users

Individual users can change the default search assistant setting for themselves. The default search assistant mode can also be changed globally, for all users.

Prerequisites

- Only users with file system access, such as system administrators, can change the default search assistant mode for all users.
- Know how to edit configuration files. Review the steps in [How to edit a configuration file](#) in the *Splunk Enterprise Admin Manual*.

CAUTION: Never change or copy the configuration files in the default directory. The files in the default directory must remain intact and in their original location. Make changes to the files in the local directory.

Steps

1. Open or create a local **user-prefs.conf** file for the Search app at \$SPLUNK_HOME/etc/apps/<app_name>/local if you are using *nix, or %SPLUNK_HOME%\etc\apps\<app_name>\local if you are using Windows.
2. In the **[general]** stanza, change the search assistant mode by selecting one of the other mode values. Choose from **full**, **compact**, or **none**. For example: **search_assistant=full**.
3. Restart the Splunk Enterprise instance.

See also

Related information

[Help reading searches](#)

[Add comments to searches](#)

[Anatomy of a search](#)

Help reading searches

Search strings can be long and difficult to read. The Search bar contains features to help you read, parse, or interpret the Splunk Search Processing Language (SPL) syntax. The syntax highlighting feature displays parts of SPL in different colors. Syntax highlighting is available in two different color themes.

In addition to color themes, you can use auto-formatting and line numbers to help read searches. There are keyboard shortcuts available to help you find information in your search syntax.

Syntax highlighting

With syntax highlighting the SPL commands, arguments, functions, and keywords are color-coded to make it easier to read a search.

Consider the following search.

```
sourcetype=access_* | timechart count(eval(action=purchase)) BY productName usenull=false useother=false
```

With syntax highlighting turned on, searches can be easier to read. Syntax highlighting shows commands, arguments, functions, and keywords in different colors. The following image shows a search string with syntax highlighting.

New Search

```
sourcetype=access* | timechart count(eval(action="purchase")) BY productName usenull=false useother=false
```

Syntax validation

If a command, argument, function, or boolean operator is not spelled or capitalized correctly, the term is not highlighted in color. The lack of color alerts you to incorrect syntax.

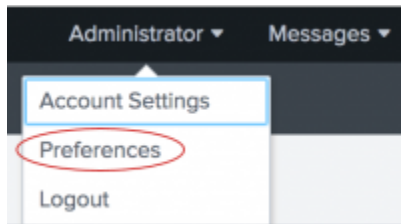
If you specify an incorrect data type for an argument, the data type appears in red. For example, the `limit` argument for the `top` command expects an integer. If you type `...|top limit=false` the term `false` is highlighted in red because it is not an integer.

Turn off syntax highlighting

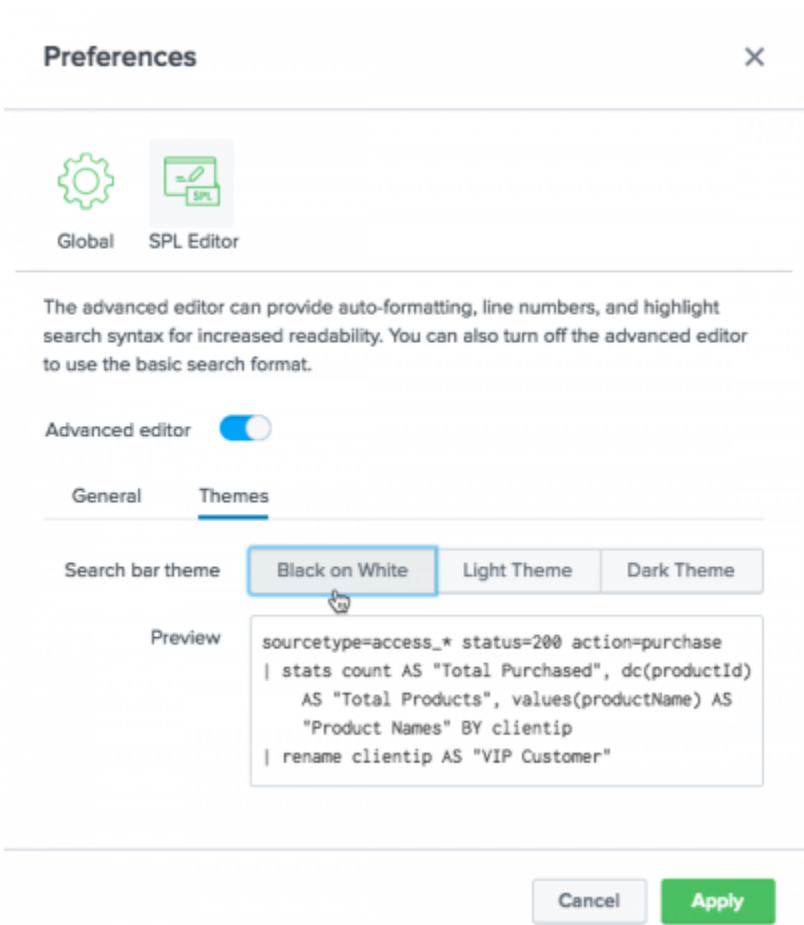
You can turn syntax highlighting colors off by changing the color theme to **Black on White**. This is useful for people who have difficulty distinguishing between different colors.

Note: You cannot turn off or change syntax highlighting if you have a Splunk Free license. See [About Splunk Free](#) in the *Admin manual*.

1. On the Splunk bar, select **[User_account_name] > Preferences**.



2. Click **SPL Editor**.
3. On the **Themes** tab, click **Black on White**.



4. Click **Apply**.

Color themes

You can change the appearance of your search criteria by specifying a color theme. There are several themes to choose from.

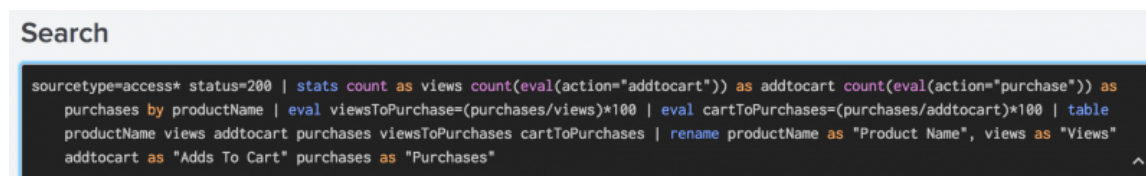
Theme name	Description	Notes
Black on White	White background. Black text. No other colors.	Useful for people who have difficulty distinguishing between different colors.
Light Theme	White background. Black text. Colors for commands, arguments, functions, keyword modifiers, and Boolean operators.	Default theme
Dark Theme	Black background. Light grey text. Colors for commands, arguments, functions, keyword modifiers, and Boolean operators.	

Color codes

The color coding that is used for the search syntax depends on the color theme that is implemented. The **Light** theme is the default theme. The color codes for the **Light** and **Dark** themes are described in the following table.

Syntax component	Color	Example
Commands		
Command arguments		=false
Functions		
Keyword modifiers and Boolean operators		productName
Inline comments		

The following image shows syntax highlighting with the **Dark** theme.



Change the color theme

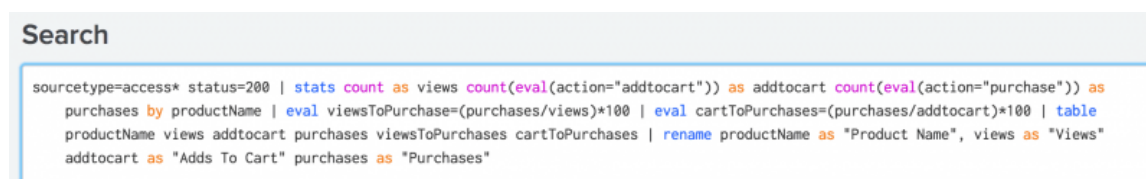
You change the color theme in the Search bar by using the account menu. You cannot change the color theme if you have a Splunk Free license. See [About Splunk Free](#) in the *Admin manual*.

1. On the Splunk bar, select **[User_account_name] > Preferences**.
2. Click **SPL Editor**.
3. On the **Themes** tab, select the color theme that you want to use.
4. Click **Apply**.

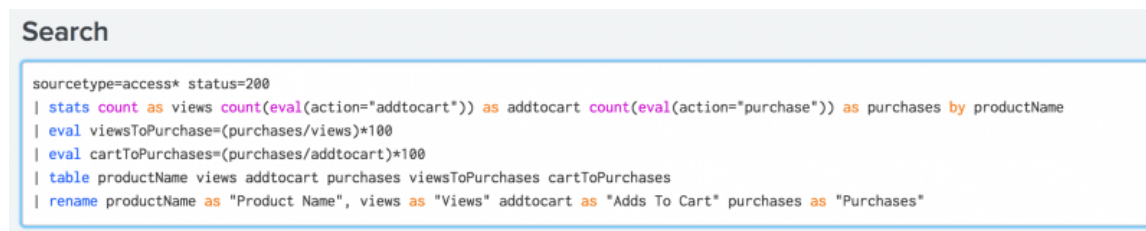
Auto-format search syntax

As you build a search, you can set up the Splunk software to format the search syntax as you type. Auto-format makes your searches more readable. Each pipe section is parsed onto a separate line. Any subsearches are indented.

The following image shows how a search appears in the Search bar when auto-format is turned off.



When auto-format is turned on, this same search is parsed as shown in the following image.



Characters that trigger auto-format

Character	Automatic formatting
Pipe ()	The pipe is placed on a new line to separate each new piped section of your search criteria.
Left square bracket ([)	The left square bracket, which signifies the start of a subsearch, is placed on a new line and indented several spaces.

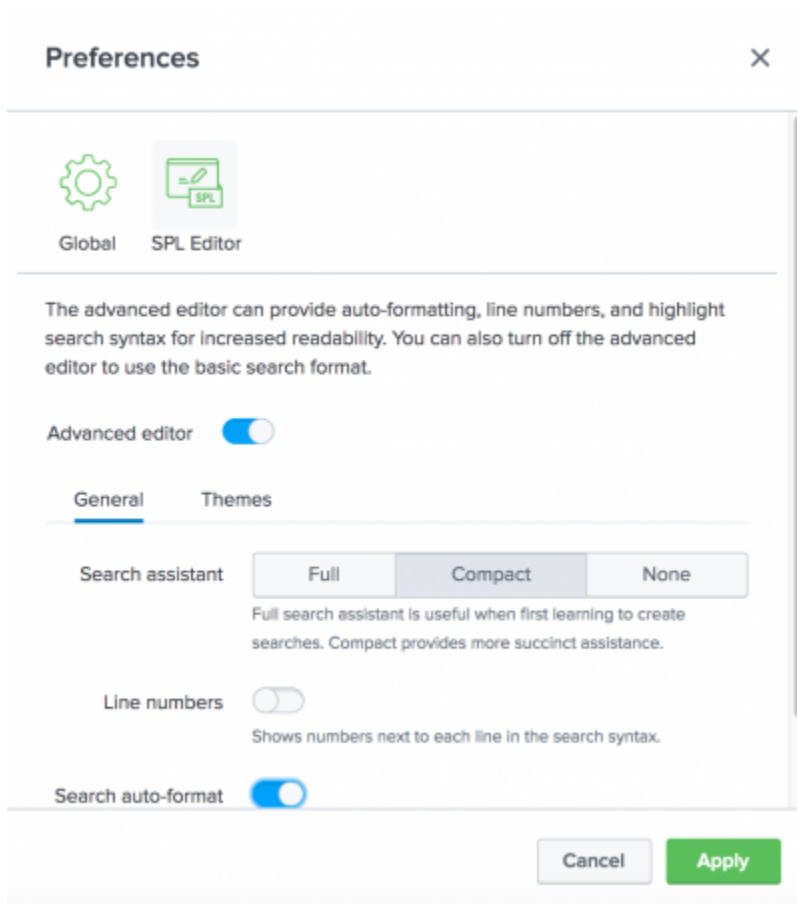
Note: If the pipe or left bracket is inside a quoted string, the auto-format is not triggered.

Turn on Search auto-format

By default, automatic formatting of search syntax is turned off. You can turn on the automatic formatting of the search syntax in the Settings dialog box.

Changing the options in the Settings dialog box changes the setting only for you. It does not impact the setting for other users.

1. On the Splunk bar, select **[User_account_name] > Preferences**.
2. Click **SPL Editor**.
3. On the **General** tab click **Search auto-format**.



4. Click **Apply**.

Auto-formatting is applied to new searches that you type into the Search bar. If you already have a search in the Search bar, use the [Search bar shortcuts](#) to apply auto-formatting to that search.

Why are my searches not auto-formatting?

The auto-format feature works on searches that you type into the Search bar. If you paste a search into the Search bar or select a search from **Search History**, the search is not automatically formatted even when the auto-format feature is turned on.

To apply auto-formatting to a search that you paste into the Search bar or select from Search History, use the following keyboard shortcut to apply auto-formatting to that search.

- On Linux or Windows use **Ctrl + **
- On Mac OSX use **Command + **

Number search lines

To make reading your searches easier, you can display line numbers in the Search bar. The following image shows both line numbers and auto-formatting turned on.

Search

```

1 sourcetype=access* status=200
2 | stats count as views count(eval(action="addtocart")) as addtocart count(eval(action="purchase")) as purchases by productName
3 | eval viewsToPurchase=(purchases/views)*100
4 | eval cartToPurchases=(purchases/addtocart)*100
5 | table productName views addtocart purchases viewsToPurchase cartToPurchases
6 | rename productName as "Product Name", views as "Views" addtocart as "Adds To Cart" purchases as "Purchases"

```

Turn on line numbering

By default, line numbering is turned off. You turn on line numbering in the Preferences dialog box.

1. On the Splunk bar, select **[User_account_name] > Preferences**.
2. Click **SPL Editor**.
3. On the **General** tab click **Line numbers**.
4. Click **Apply**.

Changing the options in the **Preferences** dialog box changes the setting only for you. It does not impact the preferences set for other users. See [Change the default Search preferences for all users](#).

A row in the Search bar is not a line

The line numbering feature applies numbers only to lines. A row in the Search bar is not necessarily a line. You might have a long line that spans multiple rows in the Search bar but is still only one line.

For example, if you paste a long search into the Search bar that has not been formatted with multiple lines, the search has one line number and spans multiple rows.

You can create lines in the Search bar by using the following methods.

- The Search auto-formatting feature is turned on and you type a pipe character or left square bracket.
- You use the [Search bar shortcuts](#) to auto-format the current search.
- You press **Shift + Enter** to split the active row at the cursor. Pressing **Enter** does not create a new line in the Search bar.

Search bar shortcuts

In the Search bar, you can use keyboard shortcuts to help you develop, read, and parse your search criteria.

Make searches easier to read

Long searches can be difficult to read. For example, the following search uses multiple commands and includes many occurrences of renaming columns in the search results.

```

sourcetype=access_* status=200 | stats count AS views count(eval(action="addtocart")) AS addtocart
count(eval(action="purchase")) AS purchases by productName | eval viewsToPurchase=(purchases/views)*100 | eval

```

```
cartToPurchases=(purchases/addtocart)*100 | table productName views addtocart purchases viewsToPurchases
cartToPurchases | rename productName AS "Product Name", views AS "Views", addtocart as "Adds To Cart",
purchases AS "Purchases"
```

The following image shows how this search appears in the Search bar.

Search

```
sourcetype=access* status=200 | stats count as views count(eval(action="addtocart")) as addtocart count(eval(action="purchase")) as
purchases by productName | eval viewsToPurchase=(purchases/views)*100 | eval cartToPurchases=(purchases/addtocart)*100 | table
productName views addtocart purchases viewsToPurchases cartToPurchases | rename productName as "Product Name", views as "Views"
addtocart as "Adds To Cart" purchases as "Purchases"
```

You can use a keyboard shortcut to parse each pipe section on a separate line. Any subsearches are indented. The [auto-format feature](#) does not need to be turned on to use these keyboard shortcuts.

- On Linux or Windows use **Ctrl + **
- On Mac OSX use **Command + **
- You can also use **Ctrl + Shift + F** or **Command + Shift + F**, which works well with many non-English keyboards.

The results of the shortcut are shown in the following image.

Search

```
sourcetype=access* status=200
| stats count as views count(eval(action="addtocart")) as addtocart count(eval(action="purchase")) as purchases by productName
| eval viewsToPurchase=(purchases/views)*100
| eval cartToPurchases=(purchases/addtocart)*100
| table productName views addtocart purchases viewsToPurchases cartToPurchases
| rename productName as "Product Name", views as "Views" addtocart as "Adds To Cart" purchases as "Purchases"
```

You can also use **Shift + Enter** to force a new line. See [Line and word shortcuts](#).

Expand your search

For long searches, or searches that contain [search macros](#) or [saved searches](#), it can be difficult to see the entire search in the Search bar.

You can see the contents of your entire search by using a keyboard shortcut, **Command+Shift+E (Mac OSX)** or **Control+Shift+E (Linux or Windows)** from the Search bar in the Search page. This opens a preview that displays the expanded search string, including all search macros and saved searches. If syntax highlighting or line numbering are turned on, those features also appear in the preview.

You can copy parts of the search from the preview window. You can also click **Open in Search** in the preview window to run your search in a new window. See [Preview your search](#).

Highlight search terms

- To highlight all of the occurrences of a word in the search, double-click on that word.

Search

```
sourcetype=access* status=200 | stats count as views count(eval(action=addtocart)) as addtocart count(eval(action=purchase)) as purchases by productName | eval viewsToPurchase=(purchases/views)*100 | eval cartToPurchases=(purchases/addtocart)*100 | table productName views addtocart purchases viewsToPurchase cartToPurchases | rename productName as "Product Name", views as "Views" addtocart as "Adds To Cart" purchases as "Purchases"
```

Locate matching parenthesis

- Position your cursor immediately after an open or close parenthesis. The matching parenthesis is highlighted.

Search

```
sourcetype=access* status=200 | stats count as views count(eval(action=addtocart)) as addtocart count(eval(action=purchase)) as purchases by productName | eval viewsToPurchase=(purchases/views)*100 | eval cartToPurchases=(purchases/addtocart)*100 | table productName views addtocart purchases viewsToPurchase cartToPurchases | rename productName as "Product Name", views as "Views" addtocart as "Adds To Cart" purchases as "Purchases"
```

Undo and Redo shortcuts

Use these keyboard shortcuts to undo and redo actions in the Search bar.

Action	Linux or Windows	Mac OSX
Undo the previous action.	Ctrl + Z	Command + Z
Redo the previous action.	Ctrl + Y or Ctrl + Shift + Z	Command + Y or Command + Shift + Z

Search history shortcuts

Use these keyboard shortcuts to scroll through your search history.

Action	Linux or Windows	Mac OSX
Scroll to the previous search.	Alt + P	Ctrl + P
Scroll to the next search.	Alt + N	Ctrl + N

Search assistant window shortcuts

With the Compact mode of the search assistant, you can use keyboard shortcuts to select items in the list and close and reopen the search assistant window.

Action	Linux or Windows	Mac OSX
Move your cursor into the search assistant window.	Down arrow key	Down arrow key
Close the search assistant window.	ESC	ESC
Reopen the search assistant window.	Ctrl + Space	Control + Space
Select an item in the search assistant window and insert it into the Search bar.	Use the Up arrow and Down arrow keys to highlight the item and press Enter .	Use the Up arrow and Down arrow keys to highlight the item and press Enter .
Toggle between the list and the Learn More link in the search assistant window.	Tab	Tab

Find and replace shortcuts

Use the following keyboard shortcuts to find and replace terms in the Search bar.

Action	Linux or Windows	Mac OSX
Find a term.	Ctrl + F	Command + F
Find and replace a term.	Ctrl + H	Command + Option + F

Line and word shortcuts

The distinction between rows and lines is important to understand when you use keyboard shortcuts to manipulate rows or lines in your search criteria in the Search bar.

- Long searches appear on multiple rows in the Search bar.
- If the search is not parsed, the search is one line.
- If the search is parsed, separating each piped section and subsearch into its own line, a row is the same as a line.

Action	Linux or Windows	Mac OSX
Split the active row at the cursor.	Shift + Enter	Shift + Enter
Remove the active line. If the search is one line with multiple rows and not parsed into separate lines, the entire search is removed.	Ctrl + D	Command + D
Copy the active row and place the copy below the active row.	Alt + Shift + Down arrow	Command + Option + Down arrow
Copy the active row and place the copy above the active row.	Alt + Shift + Up	Command + Option +

Action	Linux or Windows	Mac OSX
	arrow	Up arrow
Move the active row down one row.	Alt + Down arrow	Option + Down arrow
Move the active row up one row.	Alt + Up arrow	Option + Up arrow
Remove the search criteria from the cursor to the end of the row.	Alt + Delete	Control + K
Remove the search criteria from the cursor to the start of the row.	Alt + Backspace	Command + Delete
Remove the word or space to the right of the cursor.	Ctrl + Delete	Control + Delete
Remove the word or space to the left of the cursor.	Ctrl + Backspace	Option + Delete

Change the default Search preferences for all users

The previous sections describe how individual users can change the default Search preferences for syntax highlighting, auto-formatting, and line numbering features for themselves.

The default Search preferences can also be changed globally for all users.

Prerequisites

- Only users with file system access, such as system administrators, can change the default Search preferences for all users. If you are using Splunk Cloud Platform and want to change the default Search settings for your Splunk system, open a Support ticket.
- Review the steps in [How to edit a configuration file](#) in the *Admin Manual*.

Note: Never change or copy the configuration files in the **default** directory. The files in the **default** directory must remain intact and in their original location. Make the changes in the **local** directory.

Steps

- Open the local **user-prefs.conf** file for the Search app. For example, `$SPLUNK_HOME/etc/apps/<app_name>/local`.
- Under the [general] stanza, you can change the settings listed in the following table.

Feature	Attribute syntax	Default setting
Syntax highlighting	search_syntax_highlighting = light, dark, or black-white	light
Auto-formatting	search_auto_format = <boolean>	false
Line numbering	search_line_numbers = <boolean>	false

- Restart the Splunk instance.

See also

Related information

[Add comments to searches](#)

[Help building searches](#)

Add comments to searches

You can add inline comments to the search string of a saved search by enclosing the comments in backtick characters (``).

Use inline comments to:

- Explain each "step" of a complicated search that is shared with other users.
- Discuss ways of improving a search with other users.
- Leave notes for yourself in unshared searches that are works in progress.
- Troubleshoot searches by running them with chunks of SPL "commented out".

Here is an example of a search with comments:

```
sourcetype=access_* status=200 ``Get all successful website access events.``
| stats count AS views count(eval(action="addtocart")) AS addtocart count(eval(action="purchase")) AS purchases
by productName ``Create counts of site views, add-to-cart actions, and purchase actions. Break them out by
product name. ``
| eval cartToPurchases=(purchases/views)*100 ``Find the ratio of site views to purchases.``
|eval cartToPurchases=(purchases/addtocart)*100 ``Find the ratio of add-to-cart actions to purchases.``
| table productName views addtocart purchases viewsToPurchases cartToPurchases ``Put all this data into a
table.``
| rename productName AS "Product Name", views AS "Views", addtocart AS "Add To Cart", purchases AS "Purchases"
``Rename some table columns.``
```

In the Search Bar, the comments for this search are color-coded to make the comments easier to find:

Search

```
sourcetype=access_* status=200 '''Get all successful website access events.'''
| stats count AS views count(eval(action="addtocart")) AS addtocart count(eval(action="purchase")) AS purchases by productName
'''Create counts of site views, add-to-cart actions, and purchase actions. Break them out by product name.'''
| eval viewsToPurchases=(purchases/views)*100 '''Find the ratio of site views to purchases.'''
| eval cartToPurchases=(purchases/addtocart)*100 '''Find the ratio of add-to-cart actions to purchases.'''
| table productName views addtocart purchases viewsToPurchases cartToPurchases '''Put all this data into a table.'''
| rename productName AS "Product Name", views AS "Views", addtocart as "Adds To Cart", purchases AS "Purchases" '''Rename some table columns.'''
```

The search processor replaces comments with a space when the search is run. For example, this comment
`...| stats '''Let's use 'stats' to count this'''count` becomes `...| stats count`, when run not
`... | statscount`.

Comments and special characters

- SPL comments support Unicode characters.
- Comments ignore the backslash escape character. For example:
`'''This is still a comment. It is not escaped.'''`
- Single and double backticks within a comment are considered to be part of the comment. For example:
`'''This comment has an extra backtick at the end. It's a valid comment. `'''`

Comment limitations

The following table describes limitations associated with using comments:

Comment limitation	Example
A comment inserted before a generating command causes the search to fail. Common generating commands are <code>tstats</code> , <code>makesresults</code> , and <code>gentimes</code> .	<code>'''This search returns an error''' tstats count WHERE host=x BY source</code>
You can't insert comments inside a quoted string.	<code>"""The search processor treats this as an SPL string, not a comment."""</code>

Use comments to troubleshoot a search

You can use the inline comments to troubleshoot a search.

The following search example is attempting to return the bytes for the individual indexes. However, the search has the wrong field in the `stats` command <split-by clause>.

```
index=_internal source=*license* type=usage | stats sum(b) BY index
```

You can comment out portions of your search to help identify problems. In this search, the `stats` portion of the search is commented out.

```
index=_internal source=*license* type=usage ``| stats sum(b) BY index``
```

The results show the correct name for the field. You need to specify **idx** as the field name instead of **index**.

Here is the revised search with the comments removed and the correct field in the **stats** command:

```
index=_internal source=*license* type=usage | stats sum(b) BY idx
```

(Thanks to Splunk user [Runals](#) for this example.)

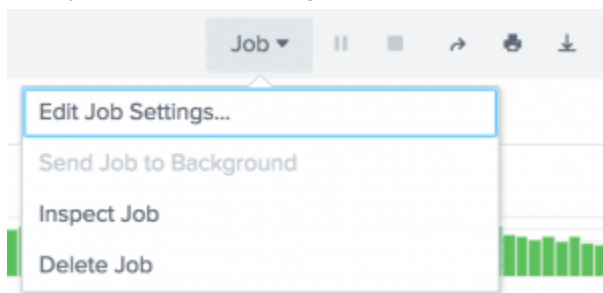
Search actions

Splunk software provides a set of controls that you can use to manage "in process" searches and to create reports and dashboards.

Control search job progress

After you launch a search, you can access and manage information about the search **job** without leaving the Search view.

1. After your search is running, paused, or finalized, click **Job** from the Search actions group.



2. Select an option from the list.
 - **Edit job settings.** Opens the Job Settings dialog, where you can change the read permissions for the job, extend the job lifespan, and get a URL for the job. You can use the URL to share the job with others or to add a bookmark to the job in your Web browser.
 - **Send job to the background.** Runs the job on the background. Use this option if the search job is slow to complete. This enables you to work on other activities, including running a new search job.
 - **Inspect job.** Opens the Search Job Inspector window and displays information and metrics about the search job. You can select this action while the search is running or after the search completes. For more information, see [View search job properties](#).
 - **Delete job.** Deletes the current job, even if that job is running, paused, or has finalized. After you delete the job you can still save the search as a report.

For more information, see [About jobs and job management](#).

Change the search mode

The search mode controls the search experience. The default search mode is Smart Mode.

Fast Mode

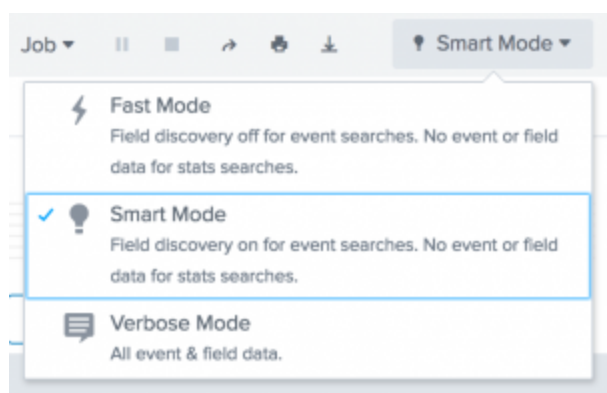
Speeds up searches by cutting down on the amount of event information that the search returns.

Verbose Mode

Returns as much event information as possible.

Smart Mode

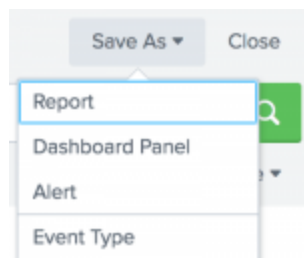
Automatically toggles the search behavior between Fast Mode and Verbose Mode, based on the type of search that you are running.



For more information, see [Search modes](#) in this manual.

Save the results

The **Save as** menu lists options for saving the results of a search as a report, dashboard panel, alert, and event type.



Report

Saves a search as a **report** to use the search again later. You can run the report again from the Reports page. You access the Reports page from the App bar. Read more about how to [Create and edit reports](#) in the *Reporting Manual*.

Dashboard Panel

Generates a dashboard **panel** based on your search and add it to a new or existing dashboard. To learn more, see the [Dashboard overview](#) in the *Dashboards and Visualizations* manual.

Alert

Defines an **alert** based on your search. An alert runs a report in the background (either on a **schedule** or in **real time**). When the search returns results that meet a condition you have set in the alert definition, the alert is triggered. For more information, see the [Alerting Manual](#).

Event Type

Classify events that have common characteristics. If the search does not include a **pipe operator** or a **subsearch**, you can use this option to save the search as an event type. For more information, see [About event types](#) and [Define event types in Splunk Web](#) in the *Knowledge Manager* manual.

Other search actions

Between the job progress controls and search mode selector are three buttons which enable you to **Share**, **Export**, and **Print** the results of a search.

- Click **Share** to share the job. When you select this, the job's lifetime is extended to 7 days and read permissions are set to Everyone. For more information about jobs, see [About jobs and job management](#) in this manual.
- Click **Export** to export the results. You can select to output to CSV, raw events, XML, or JSON and specify the number of results to export.
- Click **Print** to send the results to a printer that has been configured.

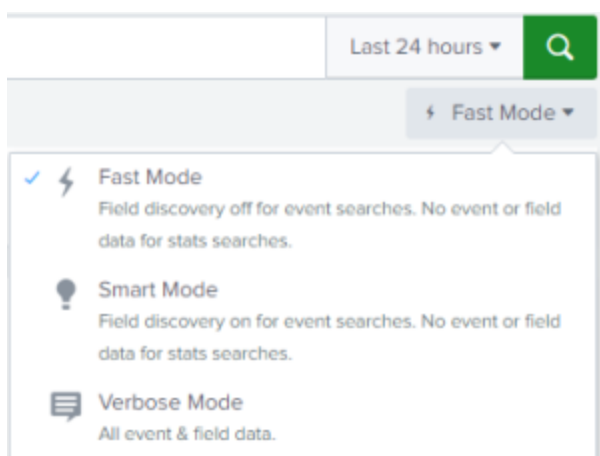
Additionally, use the **Close** button next to **Save as** menu to cancel the search and return to Splunk Home.

See also

[About the Search app](#)

Search modes

You can use the Search Mode selector to provide a search experience that fits your needs. The search mode selector is on the right side of and slightly below the Search bar.



You can choose from the following modes in the search mode selector:

- Fast
- Smart

- Verbose

Depending on the mode you set when you run a search, you can see all the data available for your search, but at the expense of longer search times, or you can speed up and streamline your search in certain ways. The Fast and Verbose modes represent the two ends of that search mode spectrum.

The default Smart mode is the best choice for most searches and is the middle ground between the Fast and Verbose modes. The Smart mode switches between the Fast and Verbose modes depending on the type of search that you are running.

When you run a **report**, Splunk platform uses the same search mode that was in effect when you initially saved the search. When you schedule a report, Splunk platform uses the Smart search mode, by default, and doesn't allow you to specify a different search mode.

Using the Fast mode

The Fast mode prioritizes the performance of the search and only returns essential event data, such as **default fields** and fields that were extracted at index time. Because **field discovery** for events is turned off, the Fast mode doesn't perform any search-time field extraction. However, the Fast mode does return search-time fields that you explicitly add to the search.

Since the Fast mode is optimized for speed, there might be disparities in values of fields compared to the Smart or Verbose modes. But, the number of results returned in searches using the Fast mode are consistent with the number of results returned by the other modes.

The Fast mode does the following:

- Extracts default fields such as **host**, **source**, and **sourcetype**. The fast mode only returns information on indexed and default fields, and any search-time extracted fields that you explicitly include in your search. If you're searching on specific indexed fields, those fields are also extracted.
- For searches with **transforming commands**, depicts search results by displaying a visualization, such as a chart.
- For searches that don't include transforming commands, displays event lists and event timelines.

Most of the time, your ad hoc searches should use the Smart mode, but there are certain specific cases where it makes sense to use the Fast mode instead. For example, use the Fast mode in the following situations:

- When you need to do basic **field discovery** and you don't care about the accuracy of your field extractions. For example, perhaps you just want to see what is in a particular source type, source, or host field.
- When you don't know what's in your index. For example, perhaps you want to know what indexed fields are in your index.
- When you want to know how many events are in your index and how frequently they occur in your data. For example, perhaps you want to find out the volume of events and approximately how many events you're getting during a particular slice of time, such as 500,000 events in a day.
- When you want to run reporting searches that include transforming commands that return results in charts or visualizations.
- When you don't care whether your events have extracted fields that aren't mentioned in your search.

For more information about what the Splunk platform does when field discovery is turned on or off, see [When Splunk Enterprise extracts fields](#) in the *Knowledge Manager Manual*.

Using the Smart mode

All reports run in Smart mode, the default search mode, after they are first created. The Smart mode returns the best results for whatever search or report you run. If you search on events, field discovery for event searches is turned on and you get all the event information you need. If you run a transforming search, the Splunk platform favors speed over thoroughness and takes you quickly to the report result table or visualization.

When you run a Smart mode search that *does not* include transforming commands, the search behaves as if it were in Verbose mode.

- All possible fields are discovered.
- The full event list and event timeline are generated. No event table or visualization will appear because you need transforming commands to make those happen.

When you run a Smart mode search that includes transforming commands, the search behaves as if it were in Fast mode.

- Field discovery is disabled.
- Your search does not waste time generating the event list and event timeline, and jumps you straight to the report result table or visualization.

For more information about transforming commands and transforming searches, see [About reporting commands](#) in the *Search Manual*.

Using the Verbose mode

The Verbose mode returns all of the field and event data it possibly can, even if it means the search takes longer to complete, and even if the search includes reporting commands.

- Your search discovers all of the fields it can. This includes default fields, automatic search-time field extractions, and all user-defined index-time and search-time field extractions. Discovered fields are displayed in the left-hand fields sidebar in the Events results tab.
- Your search returns an event list view of results and generates the search timeline. It also generates report tables and visualizations if your search includes reporting commands.

You may want to use the Verbose mode if you are putting together a transforming search, but are not exactly sure which fields you need to report on, or if you need to verify that you are summarizing the correct events.

Note: Reports cannot benefit from report acceleration when you run them in Verbose mode. If you turn on report acceleration for a report and it has been running faster as a result, be aware that if you switch the mode of the search to Verbose it will run at a slower, non-accelerated pace.

Report acceleration is designed to be used with slow-completing searches that have over 100,000 events and which utilize **transforming commands**. For more information see [Accelerate reports](#), in the *Reporting Manual*.

Comparing search modes

The following table compares the key differences between the different search modes.

Search mode	Optimized for	Field extraction behavior	Displays
Fast	- Search Performance - Transforming searches	- Field Discovery is off. - Only returns search-time fields that you explicitly add to the search.	Only results
Smart	A healthy compromise between data discoverability and search performance.	Field Discovery is: - On for event searches. - Off for transforming searches.	- Events - Results
Verbose	- Data discoverability - Event searches	- Field Discovery is on. - Your search discovers all of the fields it can.	- Events - Results

See also

[About the Search app](#)

Search history

There are several ways to view your search history for searches that you have previously run.

Note: The search history is available only for the app you're currently using. For example, if you run a search in another Splunk app, the search history for that app will not be visible in the search history in the Search app.

Search history in the Search Summary view

Your full search history appears at the bottom of the [Search Summary view](#).

Use the **Search History** panel to view and interact with the searches that you have previously run.

Click the greater than (>) symbol to expand the display to view your search history. The search history displays as a table with the following columns:

Search

Contains the search string, displayed as plain text so that you can copy the contents. By default, the Search History table truncates the search string to fit on a single line. For longer search strings, you can click the expand icon to the left of the search string to display the full search string.

Actions

Contains the action, **Add to search**. Click **Add to Search** to replace the contents of the search bar with the selected historical search contents.

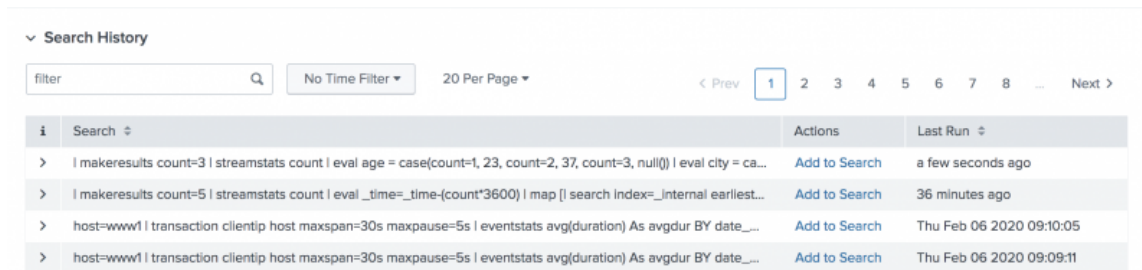
You can use keyboard shortcuts to display the search in a new browser tab.

Windows: Use CTRL+ click on **Add to Search**

Mac: Use Command + click on **Add to Search**

Last Run

Contains the date and time when the search was last run.



i	Search	Actions	Last Run
>	makeresults count=3 streamstats count eval age = case(count=1, 23, count=2, 37, count=3, null()) eval city = ca...	Add to Search	a few seconds ago
>	makeresults count=5 streamstats count eval _time=_time-(count*3600) map [search index=_internal earliest...	Add to Search	36 minutes ago
>	host=www1 transaction clientip host maxspan=30s maxpause=5s eventstats avg(duration) As avgdur BY date_...	Add to Search	Thu Feb 06 2020 09:10:05
>	host=www1 transaction clientip host maxspan=30s maxpause=5s eventstats avg(duration) As avgdur BY date_...	Add to Search	Thu Feb 06 2020 09:09:11

Filter to locate searches

You can filter your search history to quickly find the search that you are looking for. You can filter by keyword or filter by time.

- Type keywords into the **filter** text box to locate historical searches that contain the keyword. For example, type **sourcetype=access_*** to locate the searches that contain this criteria.
- Select from the list of time filters based on when the search was last run. Select either **Today**, **Last 7 Days**, or **Last 30 Days**. To see the entire search history, select **No Time Filter**.

Sort search history

In the Search History table, click the **Search** column header to sort the searches alphabetically by search criteria. Click the **Last Run** column heading to sort the searches by the date that the search was run. You can sort the list in ascending or descending order by clicking the column heading again.

Change the page display

By default, the search history shows your most recent 20 searches. Subsequent pages show older searches. Click **20 Per Page** to change how many searches appear on each page in the search history list. Choose from 10, 20, or 50 searches to display on each page.

Search history with the Search Assistant

Additionally, as you type search criteria into the Search bar, the Search Assistant shows searches from your history as a possible **Matching Search** to the criteria you are typing in.

Search history with keyboard navigation

You can also use these keyboard shortcuts to scroll through the last 100 searches in your search history.

Action	Linux or Windows	Mac OSX
Scroll to the previous search.	Alt + P	Ctrl + P
Scroll to the next search.	Alt + N	Ctrl + N

See Also

[Navigating Splunk Web](#)

[About the Search app](#)

Search Primer

Search command primer

At the beginning of a [search pipeline](#), the **search** command is implied, even though you do not explicitly specify it. If you type in:

```
host=webserver*
```

It is as if you typed in:

```
search host=webserver*
```

Use keywords, phrases, fields, Boolean expressions, and comparison expressions to specify exactly which events you want to retrieve from Splunk indexes.

For specific information see:

- [Wildcards](#)
- [Backslashes](#)
- [Boolean expressions with logical operators](#)
- [Field expressions](#)
- [Difference between NOT and !=](#)
- [Use CASE and TERM to match phrases](#)
- [SPL and regular expressions](#)

Keywords and phrases

By default, when you search with keywords and phrases, Splunk software retrieves events by matching against the raw event field, `_raw`, in your data. When you start adding search modifiers, such as fields like `_time` and `tag`, you are also matching against pieces of information that have been extracted from the `_raw` field.

When searching for strings, which includes keywords and quoted phrases (or anything that's not a search modifier), Splunk software searches the `_raw` field for the matching events or results. Some examples of keywords and phrases are:

```
web
```

```
error
```

```
web error
```

```
"web error"
```

Note that the search for the quoted phrase "web error" is not the same as the search before it. When you search for `web error`, Splunk software returns events that contain both "web" and "error". When you search for "web error", Splunk software only returns events that contain the phrase "web error".

File paths

To search for a file path, such as `D:\Digital\RTFM`, you must escape the backslash characters in the path, for example `D:\\Digital\\RTFM`.

If the file path contains spaces you must enclose the path in quotation marks. For example:

```
"D:\\Digital\\RTFM Backup Folder"
```

A space is considered a major breaker in data. To learn more about major and minor breakers, see [Event segmentation and searching](#).

See Also

- [Understanding SPL syntax](#)
- [About retrieving events](#)
- [About search time ranges](#)

- [Quick tips for optimization](#)

Wildcards

When you need to cast a wide net in your searches, you can use wildcards to match characters in string values. The wildcard that you use depends on the command that you are using the wildcard with:

eval and where commands

Use the LIKE function with the percent (%) symbol as a wildcard for matching multiple characters. Use the underscore (_) character to match a single character. See the [like \(TEXT, PATTERN\)](#) function in the list of Comparison and Conditional eval functions.

All other commands

Use the asterisk (*) character as a wildcard to match an unlimited number of characters in a string. For example,

`"my*"` matches `myhost1` and `myhost.ny.mydomain.com`.

Be efficient and specific

If you specify an asterisk with no other criteria, you are asking to match everything. Yes, everything. All events are retrieved, up to the maximum limit. A search to match everything is both inefficient and time consuming. You'll use a lot of system resources, which can prevent others from running their searches. Additionally, you might wait a long time for your search results.

To avoid these problems, be as specific as you can in your search criteria.

The more specific your search terms are, the more efficient your search is. Sometimes that means not using a wildcard. Searching for a specific word or phrase is more efficient than a search that uses a wildcard. For example, searching for `access denied` is always better than searching for `access*`.

Best practices for using wildcards

The best way to use a wildcard is at the end of a term, such as `fail*`.

Specify a field-value pair whenever possible to avoid searching the `_raw` field, which is the entire event. For example: `status=fail*`.

When using wildcards in searches, carefully consider whether you're getting the results you expect. For example, the following searches containing wildcards do *not* return any events, as expected:

- `NOT *`
- `sourcetype=_json NOT *`
- `sourcetype=* NOT *`

The following searches containing wildcards *do* return events, as expected:

- `sourcetype=*`
- `index=*`

Using wildcards with index= and NOT *

If you want the inverse of a wildcard when using **index=** and **NOT ***, you must use **(NOT *)** to produce the most accurate search results. For example, the following searches do not return events, as expected.

- `index=<index name> (NOT *)`
- `index=* (NOT *)`

The following searches incorrectly return events, because **NOT *** is not enclosed in parentheses.

- `index=<index name> NOT *`
- `index=* NOT *`

When to avoid wildcard characters

There are several situations in which you should avoid using wildcard characters.

Avoid using wildcards in the middle of a string

Certain searches with wildcard characters in the middle of a word or string might cause inconsistent results. This is especially true if the string contains punctuation, such as an underscore `_` or dash `-` character.

If your search displays a warning message indicating that a term contains a wildcard with punctuation in the middle of a word or string, rewrite your search. For example, if your search contains the following string:

```
(host=sh-*splunkcloud* OR host=si-*splunkcloud*)
```

Modify your search to use the following string instead:

```
( ( host=sh-* OR host=si-*) host=*.splunkcloud* )
```

A search that uses a wildcard in the middle of the term returns inconsistent results because of the way in which data that contains punctuation is indexed and searched. For example, suppose you have the following list of product IDs.

```
DB-SG-G01
DC-SG-G02
MB-AG-G07
MB-AG-T01
SC-MG-G01
SF-BV5-G01
SG-SH-G05
WC-SH-A02
WC-SH-G04
```

Say you create a search that looks for all of the product IDs that begin with the letter S and end in G01, like this:

```
productID=S*G01
```

Your search will fail because of the way that events are indexed. When the events with the product IDs are indexed, the product IDs are broken up into segments. For example, the product ID **SC-MG-G01** has these segments: **SC** , **MG** , **G01** . There is no segment that starts with an S and ends with G01, which is what the search **productID=S*G01** specifies. Because there are no segments that match your search, no results are found.

The solution to this problem?

- If the number of product IDs is small, specify the exact product IDs in your search rather than using a wildcard. For example:

```
productID=SC-MG-G01 OR productID=SF-BVS-G01
```

- If the number of product IDs is large, use a **lookup** instead of a wildcard.

To learn more about how punctuation can impact using wildcards, see [Event segmentation and searching](#).

Avoid using wildcards to match punctuation

Punctuation are characters that are not numbers or letters. If you want to match part of a string that includes punctuation, specify each string with the punctuation that you are searching for.

For example, you have the following values in the **uri_path** field in your events.

```
/cart.do  
/cart/error.do  
/cart/success.do  
/category.screen  
/oldlink  
/product.screen  
/productscreen.html  
/show.do  
/stuff/logo.ico
```

You want to match every **uri_path** that starts with **/cart** . The problem is that the paths contain a forward slash (/) character and period (.) character. Instead of specifying a wildcard character for the punctuation such as **/cart*** , specify the punctuation directly in your search criteria. For example,

```
...uri_path=/cart.do OR uri_path=/cart/error.do OR uri_path=/cart/success.do
```

Avoid using wildcards as prefixes

When you use a wildcard character at the beginning of a string, the search must look at every string to determine if the end of the string matches what you specify after the asterisk. Using a prefix wildcard is almost like using a wildcard by itself. Prefix wildcards might cause performance issues.

Avoid using wildcards at the beginning of search terms.

Searching for the asterisk character

You can't search for the asterisk (`*`) character directly because the character is reserved as a wildcard.

However, you can search for a term without the asterisk and then use either the **where** or **regex** command to filter the results.

For example, to search for a term that contains an asterisk such as `*78`, use these steps:

1. First search for `78` without the asterisk, which returns all events that contain the number.
2. Follow that with `| regex _raw=\*78` to return only those events that contain `*78`.

The backslash (`\`) is used in the regular expression to not interpret, or escape, the asterisk character. See the [regex](#) command.

Other supported wildcards

The LIKE function supports using other wildcards for pattern matching. The percent (`%`) symbol is used as a wildcard for matching multiple characters. The underscore (`_`) character is used to match a single character.

These wildcards are only applicable to the LIKE function. See [like \(TEXT, PATTERN\)](#) in the list of Comparison and Conditional functions.

You can use the percent (`%`) wildcard anywhere in the PATTERN.

Backslashes

To pass a literal backslash in an argument to a Splunk Search Processing Language (SPL) command, you must escape the backslash by using the double-slash (`\\`) string in your search. Any commands that execute subsequent to that initial escaping might need additional escaping, especially commands that use regular expressions because backslashes have special meaning in regular expressions. For information about backslashes in regular expressions, see [SPL and regular expressions](#).

SPL uses the asterisk (`*`) as a wildcard character. The backslash symbol can't be used to escape the asterisk in search strings.

More about backslashes

For more information:

- See [SPL and regular expressions](#)
- See [Support for backslash characters \(\ \) in the fields command](#) in the *Search Reference Manual*.
- See [Support for backslash characters \(\ \) in the rename command](#) in the *Search Reference Manual*.

- See [Quotes and escaping characters in the search command](#) in the *Search Reference Manual*.

Event segmentation and searching

When data is added to your Splunk instance, the indexer looks for segments in the data. Data is segmented by separating terms into smaller pieces, first with **major breakers** and then with **minor breakers**. These breakers are characters like spaces, periods, and colons. There are lists of the major and minor breakers later in this topic.

Suppose an event begins with an IP address and a date, such as `91.205.189.15 - - [13/Aug/2022:18:22:16]`. This data is broken into these segments based on the major breakers:

```
91.205.189.15
-
[13/Aug/2022:18:22:16]
```

These major segments are further broken down based on the minor breakers. For example, the IP address is broken into minor segments such as `91`, `205`, `189`, and `15`.

Event segmentation at index-time and at search-time

Event segmentation occurs at index-time and at search-time.

Index-time segmentation

Index-time segmentation affects indexing and search speed, storage size, and the ability to use typeahead functionality in the Search bar.

Search-time segmentation

Search-time segmentation affects search speed and the ability to create searches by selecting items from the search results.

For more information about the distinction between index-time segmentation and search-time segmentation see [Index time versus search time](#) in the Splunk Enterprise *Managing Indexers and Clusters of Indexers* manual.

In this topic we are going to focus on search-time segmentation and how major and minor breakers impact searching.

Searching and punctuation symbols

If your field values contain punctuation symbols such as quotation marks, periods, and colons, follow these best practices in your searches:

- To match a punctuation symbol, don't use a wildcard to match the symbol. Specify the actual symbol.
- Avoid using wildcards in the middle of a term. Wild cards used in the middle of a term might slow down search performance and might return inconsistent results if the term contains punctuation.
- Avoid using wildcards at the beginning of a value. Wild cards used as the first character in a value will slow down search performance.

See [Wildcards](#) for more information about using wildcards in your search criteria.

Punctuation symbols and segment tokens

Many punctuation symbols are interpreted as major or minor breakers in event data. These breakers are used to parse the data into small segments.

To search for values that contain punctuation, enclose the data in quotation marks.

For example, an IP address such as 91.205.189.15 is broken into segment tokens based on the period character.

91
205
189
15
91.205
91.205.189
91.205.189.15
205.189
and so forth

To search for this IP address, you must use quotation marks. The quotation marks tell the search to find the complete string **"91.205.189.15"** .

Other examples are a bit more complicated.

Suppose you have data that sometimes appears with quotations and sometimes does not, such as **app="uat_staging-mgr"** and **app=uat_staging-mgr** .

The quotation mark (") is a major breaker. The equal sign (=) is a minor breaker.

The data gets parsed into segments as shown in this table:

Data	Segments from major breakers	Segments from minor breakers
app="uat_staging-mgr"	app= uat_staging-mgr	app uat staging mgr
app=uat_staging-mgr	app=uat_staging-mgr	app uat staging mgr

The quotations around the data make a difference for the major tokens. For **app="uat_staging-mgr"** , the quote is a major breaker and so you end up with these 2 segments:

app=
uat_staging-mgr

Where as with `app=uat_staging-mgmr` , which does not have any part enclosed in quotations, there is no major breaker and the entire term is 1 segment.

Major breakers

Major breakers are a set of characters that are used to divide words, phrases, or terms in the event data into large tokens.

Examples of major breakers are:

- A space
- A newline
- A tab
- Angle brackets < >
- Square brackets []
- Parenthesis ()
- Curly brackets { }
- An exclamation point !
- A question mark ?
- A semicolon ;
- A comma ,
- Single and double quotation marks ' "
- The ampersand sign &
- There are also multiple major breakers that use percent-encoding, primarily for reserved characters. These major breakers begin with a percent symbol followed by a code. For example, %21 is the code for the exclamation point (!) character and %2526 is a double encoded ampersand (&&).

For a complete list of segmenters, see [segmenters.conf](#) file in the Splunk Enterprise *Admin Manual*.

Here is an example of part of an event:

```
91.205.189.15 - - [13/Aug/2022:18:22:16] "GET /oldlink?itemId=EST-14&JSESSIONID=SD6SL7FF7ADFF53113 HTTP 1.1"
```

This partial example gets segmented on the major breakers into the following tokens:

```
91.205.189.15
-
-
13/Aug/2022:18:22:16
GET
/oldlink
```

```
itemId=EST-14  
JSESSIONID=SD6SL7FF7ADFF53113  
HTTP  
1.1
```

Minor breakers

Minor breakers are a set of characters that are used to further divide large tokens into smaller tokens.

Examples of minor breakers are:

- A period .
- A forward slash /
- A double backslash \\
- A colon :
- The equal sign =
- The AT sign @
- The pound sign #
- The dollar sign \$
- The percent sign %
- The dash sign -
- The underscore sign _

For a complete list of segmenters, see [segmenters.conf](#) file in the Splunk Enterprise *Admin Manual*.

See also

- [About event segmentation](#) in the *Getting Data In* manual
- [segmenters.conf](#) file in the Splunk Enterprise *Admin Manual*

Use CASE() and TERM() to match phrases

If you want to search for a specific term or phrase in your Splunk index, use the CASE() or TERM() directives to do an exact match of the entire term.

CASE

Syntax: CASE(<term>)

Description: Search for case-sensitive matches for terms and field values.

TERM

Syntax: TERM(<term>)

Description: Match whatever is inside the parentheses as a single term in the index, even if it contains characters that are usually recognized as **minor breakers**, such as periods or underscores.

The **CASE()** and **TERM()** directives are similar to the **PREFIX()** directive used with the **tstats** command because they match strings in your raw data. For more information about the **PREFIX()** directive, see [tstats](#) in the *Search Reference*.

When to use CASE

By default, searches are case-insensitive. For example, if you search for **Error**, any case of that term is returned, such as **Error**, **error**, and **ERROR**. You can use the CASE directive to perform case-sensitive matches for terms and field values. For example, if you search for **CASE(error)**, your search returns results containing only the specified case of the term, which is **error**.

You can use the CASE directive to search for terms using wildcards. For example, searching for **CASE(%ASA-1*)** returns events matching values such as **%ASA-1-134568** and **%ASA-1-12345**.

Example

The following search only matches events that contain localhost in uppercase in the **host** field.

```
host=CASE(LOCALHOST)
```

When to use TERM

The TERM directive is useful for more efficiently searching for a term that:

- Contains **minor breakers**, such as periods or underscores.
- Is bound by **major breakers**, such as spaces or commas.
- Does not contain major breakers.

When data is indexed, characters such as periods and underscores are recognized as minor breakers between terms. Use the TERM directive to ignore the minor breakers and match whatever is inside the parentheses as a single term. For example, the IP address **127.0.0.1** contains the period (.) minor breaker. If you search for the IP address 127.0.0.1, Splunk software searches for **127 AND 0 AND 1** and returns events that contain those numbers anywhere in the event. If you specify **TERM(127.0.0.1)**, the search treats the IP address as a single term, instead of individual numbers, and returns all events that contain the IP address 127.0.0.1.

The TERM directive only works for terms that are bounded by major or minor breakers, but the term you are searching for cannot **contain** major breakers. For example, you cannot use TERM to search for **Maria Dubois** because there is a space between the names. This is discussed in the examples later in this topic.

When you use the TERM directive, the Splunk software expects to see the term you specify as a token in the **lexicon** in the .tsidx file. For more information about how Splunk software breaks events up into searchable segments, see [About segmentation](#) in *Getting Data In*.

You can use the TERM directive to search for terms using wildcards. For example, searching for **TERM(%ASA-1*)** returns events matching values such as **%ASA-1-134568** and **%ASA-1-12345**.

See [Use the TERM directive to match terms that contain minor breakers](#).

Using TERM() with major and minor breakers

Searches that contain the TERM() directive might return unexpected results if the following conditions are met:

- The term in the TERM directive contains minor breakers, such as a period (.); and
- The raw data for the term is bound by major breakers that end in a number or letter; and
- The search contains other expressions such as **host=*** , **sourcetype=*** or **myfield=*** , OR contains other expressions that include minor breakers, such as **"myfield1.myfield2"** , **"myfield1:myfield2"** .

For example, say your raw data is **foo%20www.something.com%20abc 127.1.1** , which includes the major breaker **%20** .

The following searches don't return results:

- **index=main TERM(www.something.com) host=***
- **index=main TERM(www.something.com) "127.1.1"**

However, the following search does return results:

- **index=main TERM(www.something.com)**

For a full list of minor breakers and multicharacter major breakers, see the `segmenters.conf` file in the Splunk Enterprise *Admin Manual*.

Examples

Searching for **TERM(127.0.0.1)** works for raw data that looks like this:

127.0.0.1 - admin

Both **127.0.0.1** and **admin** are bounded by major breakers, in this case spaces.

However, searching for **TERM(127.0.0.1)** fails for data that looks like this:

ip=127.0.0.1 - user=admin

This is because the equal symbol (=) is a minor breaker, not a major breaker. Additionally, the IP address portion of the event is indexed as: **ip** , **127** , **0** , **1** , and **ip=127.0.0.1** . You are looking for **127.0.0.1** , which is not an indexed term.

If your data looks like this:

ip 127.0.0.1 - user admin

Searching for **TERM(user admin)** fails to return results. The space is a major breaker and the phrase "user admin" is not indexed as a single term. In this situation, use quotation marks to search for a string that contains a space, for example **"user admin"** .

See also

Related information

[Event segmentation and searching](#)

Expressions and Predicates

Boolean expressions with logical operators

The Splunk search processing language (SPL) supports the following logical operators in Boolean expressions: **AND** , **OR** , **NOT** , and **XOR** .

The operators must be capitalized.

The AND operator is always implied between terms, that is: **web error** is the same as **web AND error** . So unless you want to include it for clarity reasons, you should not need to specify the AND operator.

The NOT operator only applies to the term immediately following NOT. To apply to multiple terms, you must enclose the terms in parenthesis.

Note: Inclusion is generally better than exclusion. Searching for "access denied" will yield faster results than NOT "access granted".

Order of evaluation

The order in which the Splunk software evaluates Boolean expressions depends on whether you are using the expression with the **search** command, the **eval** command, or the **where** command. This includes the implied **search** command at the beginning of the search.

The **search** command evaluates OR before AND operators (XOR is not supported). The **eval** and **where** commands evaluate AND before OR operators.

The following table describes the order in which the Boolean expressions are evaluated by the commands.

Order	Search command	Eval and where commands
1	Expressions within parentheses	Expressions within parentheses
2	NOT clauses	NOT clauses
3	OR clauses	AND clauses
4	AND clauses	OR clauses
5		XOR clauses

Examples

The following examples show how Splunk software processes Boolean expressions using logical operators.

Search command example with AND and OR

Consider the following search:

```
host="www1" AND status=200 OR action="addtocart"
```

With the **search** command, the AND is implied between the expressions. The same results are returned if you omit the AND in the search and specify **host="www1" status=200 OR action="addtocart"**.

This search is processed as:

```
host="www1" AND (status=200 OR action="addtocart")
```

This search returns:

- All of the events where the host is **www1** and the status is either **200** or the action is **addtocart**.

With the **search** command, the OR is processed before the AND.

The **where** command processes this search differently, as shown in the next example.

Where command example with AND and OR

Consider the following search:

```
... | where host="www1" AND status=200 OR action="addtocart"
```

This search is processed as:

```
... | where (host="www1" AND status=200) OR action="addtocart"
```

This search returns:

- All of the events where the host is **www1** and the status is **200**.
- All of the events where the action is **addtocart**.

With the **where** command the AND is processed before the OR.

Search command example with NOT

Consider the following search:

```
host="www1" NOT status=200
```

This search returns all `host="www1"` events that have status codes not equal to 200.

Note: Searches that exclude results using the NOT operator are typically less efficient than searches that are more inclusive. As a result, you should avoid using NOT when possible.

Search command examples with AND NOT and NOT OR

Consider the following search:

```
host="www1" NOT status=200 AND NOT status=505
```

It produces the same results as the following search:

```
host="www1" NOT (status=200 OR status=505)
```

These searches return all `host="www1"` events that have status codes not equal to 200 or 505. These searches should have fewer results than a search that just excludes events with status equal to 200.

Note: Searches that exclude results using the NOT operator are typically less efficient than searches that are more inclusive. As a result, you should avoid using NOT when possible.

Difference between != and NOT

When you want to exclude results from your search you can use the NOT operator or the `!=` field expression. However there is a significant difference in the results that are returned from these two methods.

Suppose you have the following events. As you can see, some events have missing values.

ID	Name	Color	Location
101M3	McIntosh	Chestnut	Marin Meadows
104F5	Lyra	Bay	
104M6	Rutherford	Dun	Placer Pastures
101F2	Rarity		Marin Meadows
102M7	Dash	Black	Calaveras Farms
102M1		Roan	

ID	Name	Color	Location
101F6		Chestnut	Marin Meadows
104F4	Pinkie	Sorrel	Placer Pastures
102M8	Spike	Grey	Calaveras Farms

Searching with !=

If you search with the **!=** expression, every event that has a value in the field, where that value does not match the value you specify, is returned. Events that do not have a value in the field are not included in the results.

For example, if you search for **Location!="Calaveras Farms"**, events that do not have Calaveras Farms as the Location are returned. Events that do not have Location value are not included in the results.

```
source="Ponies.csv" Location!="Calaveras Farms"
```

ID	Name	Color	Location
101M3	McIntosh	Chestnut	Marin Meadows
104M6	Rutherford	Dun	Placer Pastures
101F2	Rarity		Marin Meadows
101F6		Chestnut	Marin Meadows
104F4	Pinkie	Sorrel	Placer Pastures

If you search for a Location that does not exist using the **!=** expression, all of the events that have a Location value are returned.

Searching with NOT

If you search with the NOT operator, every event is returned except the events that contain the value you specify. This includes events that do not have a value in the field.

For example, if you search using **NOT Location="Calaveras Farms"**, every event is returned except the events that contain the value "Calaveras Farms". This includes events that do not have a Location value.

```
source="Ponies.csv" NOT Location="Calaveras Farms"
```


ID	Name	Color	Location
101M3	McIntosh	Chestnut	Marin Meadows
104F5	Lyra	Bay	
104M6	Rutherford	Dun	Placer Pastures
101F2	Rarity		Marin Meadows
102M1		Roan	
101F6		Chestnut	Marin Meadows
104F4	Pinkie	Sorrel	Placer Pastures

If you search for a Location that does not exist using NOT operator, all of the events are returned.

Using != with the regex command

If you use regular expressions in conjunction with != in searches, see [regex](#).

Searching with != or NOT is not efficient

Using the != expression or NOT operator to exclude events from your search results is not an efficient method of filtering events. The execution cost for a search is actually less when you explicitly specify the values that you want to include in the search results. For more tips on search optimization, see [Quick tips for optimization](#).

Field expressions

When you add data, Splunk software extracts pairs of information and saves them as fields. Some fields are common to all events, but others are not. Adding fields to your search term gives you a better chance of matching specific events.

If you are searching web access logs for specific HTTP status errors, instead of searching for "web error 404", you can use fields to search for:

```
status=404
```

See [Use fields to retrieve events](#).

Use comparison operators to match field values

You can use comparison operators to match a specific value or a range of field values.

Operator	Example	Result
=	field=foo	Multivalued field values that exactly match "foo".
!=	field!=foo	Multivalued field values that don't exactly match "foo".
<	field<x	Numerical field values that are less than x.
>	field>x	Numerical field values that are greater than x.
<=	field<=x	Numerical field values that are less than and equal to x.
>=	field>=x	Numerical field values that are greater than and equal to x.

For example, to find events that have a delay field that is greater than 10:

```
delay > 10
```

When quotes are required in field expressions

When the value you are searching for contains a breaking character, you must enclose the value in quotation marks.

Examples of breaking characters are spaces, commas, pipes, square brackets, and equals signs. In addition, to search for reserved keywords such as AND, OR, and NOT you must use quotation marks.

Field values that match SPL operators or keywords

There are several field values that match SPL operators or keywords, such as AS, AND, IN, and OR. Here are a few examples:

- **country=IN** for India
- **country=AS** or **state=AS** for American Samoa
- **iso=AND** for Andorra
- **state=OR** for Oregon

To search for field values that match operators or keywords, you must enclose the value in quotation marks. For example: **country="IN"**.

SPL and regular expressions

Regular expressions in the Splunk Search Processing Language (SPL) are Perl Compatible Regular Expressions (PCRE).

You can use regular expressions with the `rex` and `regex` commands. You can also use regular expressions with evaluation functions such as `match` and `replace`. See [Evaluation functions](#) in the *Search Manual*.

The following sections provide guidance on regular expressions in SPL searches.

Pipe characters

A pipe character (|) is used in regular expressions to specify an OR condition. For example, **A | B** means A or B.

Because pipe characters are used to separate commands in SPL, you must enclose a regular expression that uses the pipe character in quotation marks. The following search shows how to use quotation marks around a pipe character, which is interpreted by SPL as a search for the text "expression" OR "with pipe"..

```
...|regex "expression | with pipe"
```

Backslash characters in regular expressions

The backslash character (\) is used in regular expressions to escape any special characters that have meaning in regular expressions, such as periods (.), double quotation marks ("), and backslashes themselves. For example, the period character is used in a regular expression to match any character, except a line break character. If you want to match a period character, you must escape the period character by specifying `\.` in your regular expression.

In searches that include a regular expression that contains a double backslash, like the file path `c:\\temp`, the search interprets the first backslash as a regular expression escape character. The file path is interpreted as `c:\temp`, because one of the backslashes is removed. You must escape both backslash characters in the file path by specifying **4** consecutive backslashes for the root portion of the file path, such as `c:\\\\temp`. For a longer file path, such as `c:\\temp\\example`, you can specify `c:\\\\temp\\\\example` in your regular expression in the search string.

One reason you might need extra escaping backslashes in your searches is that the Splunk platform parses text twice; once for SPL and then again for regular expressions. Each parse applies its own use of backslashes in layers and treats each backslash as a special character that needs an additional backslash to make it literal. As a result, `\\` in SPL becomes `\` before it is parsed as a regular expression, and `\\\\` in SPL becomes `\\` before it is parsed as a regular expression.

See [Backslashes](#) in the *Search Manual*.

Avoid extra escaping backslash characters

To avoid using extra escaping backslashes in your searches, you can use the octal code `\134` or the hexadecimal code `\x5c` in your regular expression. These codes are equivalent to the backslash character and get around the need to double-escape backslashes. For example, consider the following search, which extracts the characters **ABC** that follow 2 backslashes:

```
| makeresults
| eval example="xyz\\ABC"
| rex field=example max_match=3 ".*\\(?:<extract>.*)"
```

The search results look something like this:

time	example	extract
2023-09-20 17:20:59	xyz\\ABC	ABC

Instead of using 3 backslashes, you can get the same search results using `\x5c` in the regular expression, like this:

```
| makeresults  
| eval example="xyz\\ABC"  
| rex field=example max_match=3 ".*\x5c(?<extract>.*)"
```

More about regular expressions

For more information:

- See [Extract fields using regular expressions](#)
- See [About Splunk regular expressions](#) in the *Knowledge Manager Manual*.

Optimizing Searches

About search optimization

Search optimization is a technique for making your search run as efficiently as possible.

When not optimized, a search often runs longer, retrieves larger amounts of data from the indexes than is needed, and inefficiently uses more memory and network resources. Multiply these issues by hundreds or thousands of searches and the end result is a slow or sluggish system.

There are a set of basic principles that you can follow to optimize your searches.

- Retrieve only the required data
- Move as little data as possible
- Parallelize as much work as possible
- Set appropriate time windows

To implement the search optimization principles, use the following techniques.

- Filter as much as possible in the initial search
- Perform joins and lookups on only the required data
- Perform evaluations on the minimum number of events possible
- Move commands that bring data to the search head as late as possible in your search criteria

Indexes and searches

When you run a search, the Splunk software uses the information in the index files to identify which events to retrieve from disk. The smaller the number of events to retrieve from disk, the faster the search runs.

How you construct your search has a significant impact on the number of events retrieved from disk.

When data is indexed, the data is processed into events based on time. The processed data consists of several files:

- The raw data in compressed form (**rawdata**)
- The indexes that point to the raw data (**index files**, also referred to as **tsidx files**)
- Some metadata files

These files are written to disk and reside in sets of directories, organized by age, called **buckets**.

Use indexes effectively

One method to limit the data that is pulled off from disk is to partition data into separate indexes. If you rarely search across more than one type of data at a time, partition different types of data into separate indexes. Then restrict your searches to the specific index. For example, store web access data in one index and firewall data in another. Using separate indexes is recommended for sparse data, which might otherwise be buried in a large volume of unrelated data.

- See [Ways to set up multiple indexes](#) in the *Managing Indexers and Clusters of Indexers* manual
- See [Retrieve events from indexes](#)

A tale of two searches

Some frequently used searches unnecessarily consume a significant amount of system resources. You will learn how optimizing just one search can save significant system resources.

A frequently used search

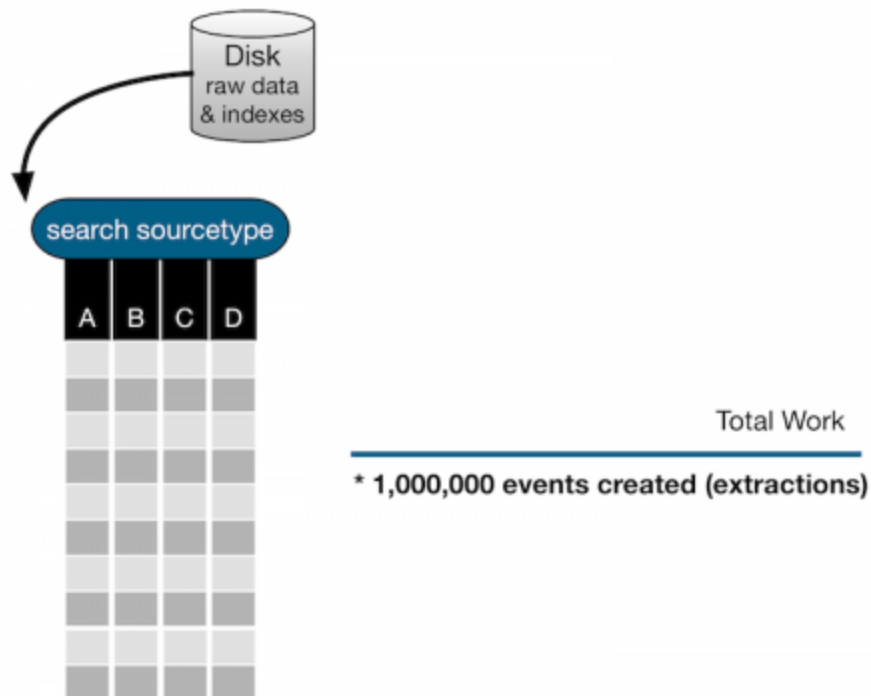
One search that is frequently used is a search that contains a lookup and an evaluation, followed by another search. For example:

```
sourcetype=my_source  
| lookup my_lookup_file D OUTPUTNEW L  
| eval E=L/T  
| search A=25 L>100 E>50
```

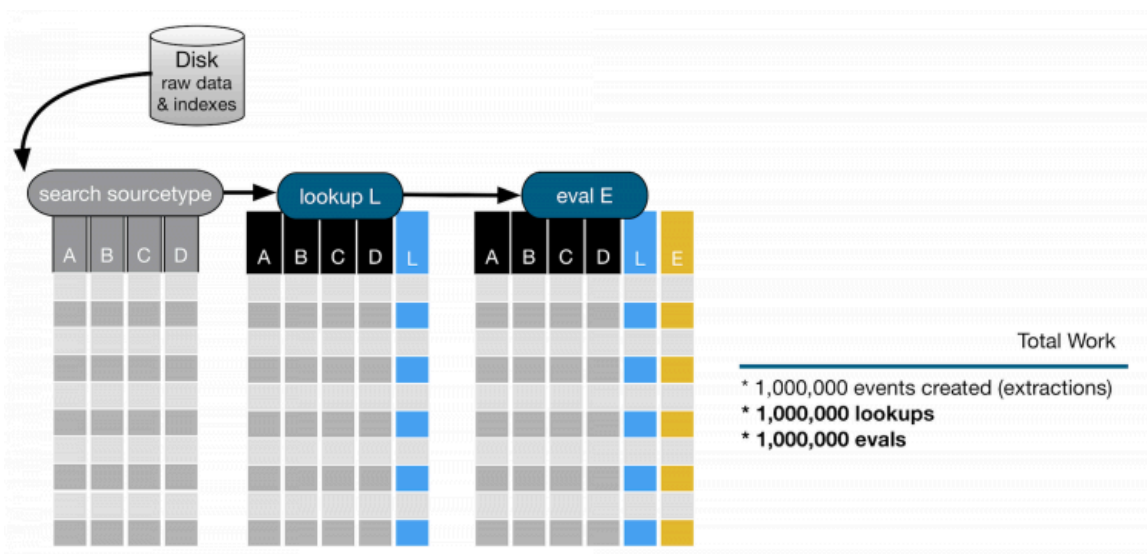
The following diagram shows a simplified, visual representation of this search.



When the search is run, the index is accessed and 1 million events are extracted based on the source type.



In the next part of the search, the **lookup** and **eval** command are run on all 1 million events. Both the **lookup** and **eval** commands add columns to the events, as shown in the following image.



Finally, a second search command runs against the columns A, L, and E.

- For column A, the search looks for values that are equal to 25.
- For column L, which was added as a result of the **lookup** command, the search looks for values greater than 100.
- For column E, which was added as a result of the **eval** command, the search looks for values that are greater than 50.

Events that match the criteria for columns A, L, and E are identified, and 50,000 events that match the search criteria are returned. The following image shows the entire process and the resource costs involved in this inefficient search.



An optimized search

You can optimize the entire search by moving some of the components from the second **search** to locations earlier in the search process.

Moving the criteria **A=25** before the first pipe filters the events earlier and reduces the amount of times that the index is accessed. The number of events extracted is 300,000. This is a reduction of 700,000 compared to the original search. The **lookup** is performed on 300,000 events instead of 1 million events.

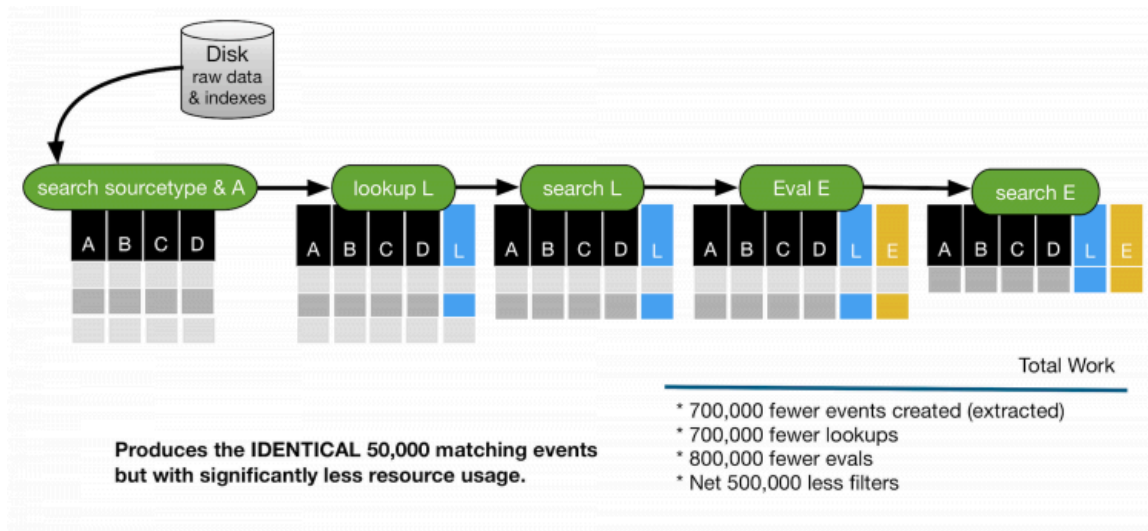
Moving the criteria **L>100** immediately after the **lookup** filters the events further, reducing the number of events returned by 100,000. The **eval** is performed on 200,000 events instead of 1 million events.

The criteria **E>50** is dependent on the results of the **eval** command and cannot be moved. The results are the same as the original search. 50,000 events are returned, but with much less impact on resources.

This is the optimized search.

```
sourcetype=my_source A=25
| lookup my_lookup_file D OUTPUTNEW L
| search L>100
| eval E=L/T
| search E>50
```

The following image shows the impact of rearranging the search criteria.



See also

- [Quick tips for optimization](#)
- [Write better searches](#)
- [Built-in optimizations](#)

Quick tips for optimization

The key to fast searching is to limit the data that needs to be pulled from disk to an absolute minimum. Filter the data as early as possible in the search, so that processing is done on the minimum amount of data necessary.

Limit the data from disk

The techniques to limit the amount of data retrieved from disk range from setting a narrow time window, being as specific as possible, and retrieving the smallest number of events necessary.

Narrow the time window

One of the most effective ways to limit the data that is pulled off from disk is to limit the time range. Use the [time range picker](#) or specify time modifiers in your search to identify the smallest window of time necessary for your search.

If you need to see data from only the last hour, do not use the default time range of **Last 24 hours**.

- See [Select time ranges to apply to your search](#)
- See [Specify time modifiers in your search](#)

If you must use a broad time range, such as **Last week** or **All time**, then use other techniques to limit the amount of data retrieved from disk.

Specify the index, source, or source type

Understanding how your data is organized is important to optimizing your searches. Take the time to learn which indexes contain your data, the sources of your data, and the source types. Knowing this information about your data helps you narrow down your searches.

1. Run the following search.

```
source=*
```

This search not optimized, but it does provide an opportunity for you to learn about the data you have access to.

2. In the **Selected fields** list, click on each type of field and look at the values for host, source, and sourcetype.
3. In the **Interesting fields** list, click on the **index** field. Look at the names of the indexes that you have access to.

Whenever possible, specify the index, source, or source type in your search. When Splunk software indexes data, it automatically tags each event with a number of fields. The index, source, and source type fields are added automatically to each event as default fields. A *default field* is an indexed field that the Splunk software recognizes in your event data at search time. The host, source, and source type fields describe where the event originated.

Be specific

Use the most specific terms in your search that you can. If possible, avoid using wildcard characters.

For example, instead of using a wildcard character for a keyword:

```
*error
```

Use the specific keyword:

```
fatal_error
```

Here is another example.

Instead of using a wildcard character for field values:

```
status=404 OR status=5*
```

Specify each value:

```
status=404 OR status=500 OR status=503
```

Combine a source type or an index with one or more field-value pairs. For example:

```
sourcetype=access_* status=200 action=purchase
```

This search retrieves events from only your web access logs. A wildcard character, `access_*`, is used in the field value to match any Apache web access source type. The source types can be `access_common`, `access_combined`, or `access_combined_wcookie`. Two specific field-value pairs are included in the search, `status=200` and `action=purchase`.

Limit the number of events retrieved

By default, a Splunk search retrieves all events. However in some situations you might want to retrieve a sample set of events, instead of retrieving the entire event set. Limiting the number of events retrieved is useful in several situations:

- You are creating a search and want to determine if you are retrieving the correct events
- You need only a subset or sample set of events for your search

You can specify a limit to the number of events retrieved in a couple of ways:

Use the head command

The `head` command retrieves only the most recent N events for a historical search, or the first N captured events for a realtime search. For example:

```
sourcetype=access_* | head 1000 ...
```

Use event sampling

Event sampling uses a ratio that you specify to select events. For example, if the sample ratio value is 100, each event has a 1 in 100 chance of being included in the result set. To learn more about event sampling and sampling ratios, see [Event sampling](#).

By default, event sampling is not active. You must specify a sampling ratio before you run your search. In Splunk Web, click the **Sampling** drop-down and choose a sampling ratio.

Use the TERM directive to match terms that contain minor breakers

The TERM directive is useful when you are searching for a term that contains minor breakers, such as periods or underscores, but does not contain major breakers. The term must be bound by major breakers, such as spaces or commas.

For example, the IP address `192.0.2.255` contains the period (.) minor breaker. If you specify `TERM(192.0.2.255)`, the Splunk software treats the IP address as a single term, instead of individual numbers.

Using the TERM directive to search for terms that contain minor breakers improves search performance. See [Event segmentation and searching](#).

Avoid using NOT expressions

More resources are used tracking NOT expressions than if you specify what you are looking for. Where ever possible, avoid using NOT expressions. For example, instead of using a string of NOT or != expressions such as:

```
(NOT host=d NOT host=e)
```

or

```
(host!=d AND host!=e)
```

Use the specific terms you are searching for:

```
(host=a OR host=b OR host=c) .
```

To learn more, see [Difference between NOT and !=](#).

Filter as soon as possible

Filter results as soon as possible before performing calculations. You can use field-value pairs and commands to filter results.

Use field-value pairs before the first pipe

Field-value pairs are indexed. Specifying field-value pairs before the first pipe is an efficient way to filter out events.

For example, in the following search the term **status=404** is in a separate search:

```
ERROR | search status=404
```

Move the term **status=404** before the first pipe:

```
ERROR status=404
```

Here is another example.

The second search includes the term **clientip="10.0.0.0/8"** . There is no reason to wait to filter on that term.

```
ERROR | stats sum(bytes) as sum by clientip | search sum >1048576 AND clientip="10.0.0.0/8"
```

Move the term **clientip="10.0.0.0/8"** to filter out all other clientip addresses before the **stats** command.

```
ERROR clientip="10.0.0.0/8" | stats sum(bytes) by clientip | search sum > 1048576
```

Use filtering commands before calculating commands

Use filtering commands, such as **where**, before commands that perform calculations, such as **eval**.

For example, this search has a **where** command after the **eval** command. The search does not require the results of the **eval** command before the **where** command is run.

```
field1=value | eval KB=bytes/1024 | where field2=field3
```

Move the **where** command to filter the results before the **eval** command is processed:

```
field1=value | where field2=field3 | eval KB=bytes/1024
```

Filter unnecessary fields from search results

You can remove unnecessary fields from the search results by using commands such as **fields**. Removing extraneous fields from searches make them more efficient because search doesn't have to process superfluous data. Other search optimizations that work on a definite set of fields, such as predicate pushdown and transforming commands, already reduce the number of fields that are propagated and, therefore, can make the **fields** command less impactful.

Use non-streaming commands as late as possible

Postpone commands like **sort** and **stats** as late as possible in your search. These commands are referred to as non-streaming commands. Before these commands can run, the entire result set must be returned. For example, the results cannot be sorted until all of the results are available.

For an explanation about the differences between streaming and non-streaming commands, see [Types of commands](#).

For a list of of commands by type, see [Command types](#) in the *Search Reference*.

Other techniques for search optimization

There are a few other techniques that you can use to optimize your searches.

- Store your apps on a fast, local disk, not on network file system (NFS). Loading apps on NFS can become a performance bottleneck.
- Use post-process searches in dashboards. See [Searches power dashboards and forms](#) in *Dashboards and Visualizations*.
- Use **summary indexing**, **report acceleration**, and **data model acceleration** features.
- Use **Fast Mode** to increase the speed of searches by reducing the event data that the search returns. See [Search modes](#).

See also

Related information

[About search optimization](#)

[Write better searches](#)

[Built-in optimizations](#)

Write better searches

This topic examines some causes of slow searches and includes guidelines to help you write searches that run more efficiently. Many factors can affect the speed of your searches, including:

- The volume of data that you are searching
- How your searches are constructed
- The number of concurrent searches

To optimize the speed at which your search runs, minimize the amount of processing time required by each component of the search.

Know your type of search

The recommendations for optimizing searches depend on the type of search that you run and the characteristics of the data you are searching. Searches fall into two types, that are based on the goal you want to accomplish. Either a search is designed to retrieve events or a search is designed to generate a report that summarizes or organizes the data.

See [Types of searches](#).

Searches that retrieve events

Raw event searches retrieve events from a Splunk index without any additional processing of the events that are retrieved. When retrieving events from the index, be specific about the events that you want to retrieve. You can do this with keywords and field-value pairs that are unique to the events.

If the events you want to retrieve occur frequently in the dataset, the search is called a *dense search*. If the events you want to retrieve are rare in the dataset, the search is called a *sparse search*. Sparse searches that run against large volumes of data take longer than dense searches against the same data set.

See [Use fields to retrieve events](#).

Searches that generate reports

Report-generating searches, or transforming searches, perform additional processing on events after the events are retrieved from an index. This processing can include filtering, transforming, and other operations using one or more statistical functions against the set of results. Because this processing occurs in memory, the more restrictive and specific you are retrieving the events, the faster the search will run.

See [About transforming commands and searches](#).

Command types and parallel processing

Some commands process events in a stream. There is one event in and one, or no, event out. These are referred to as **streaming commands**. Examples of streaming commands are **where**, **eval**, **lookup**, and **search**.

Other commands require all of the events from all of the indexers before the command can finish. These are referred to as **non-streaming commands**. Examples of non-streaming commands are **stats**, **sort**, **dedup**, **top**, and **append**.

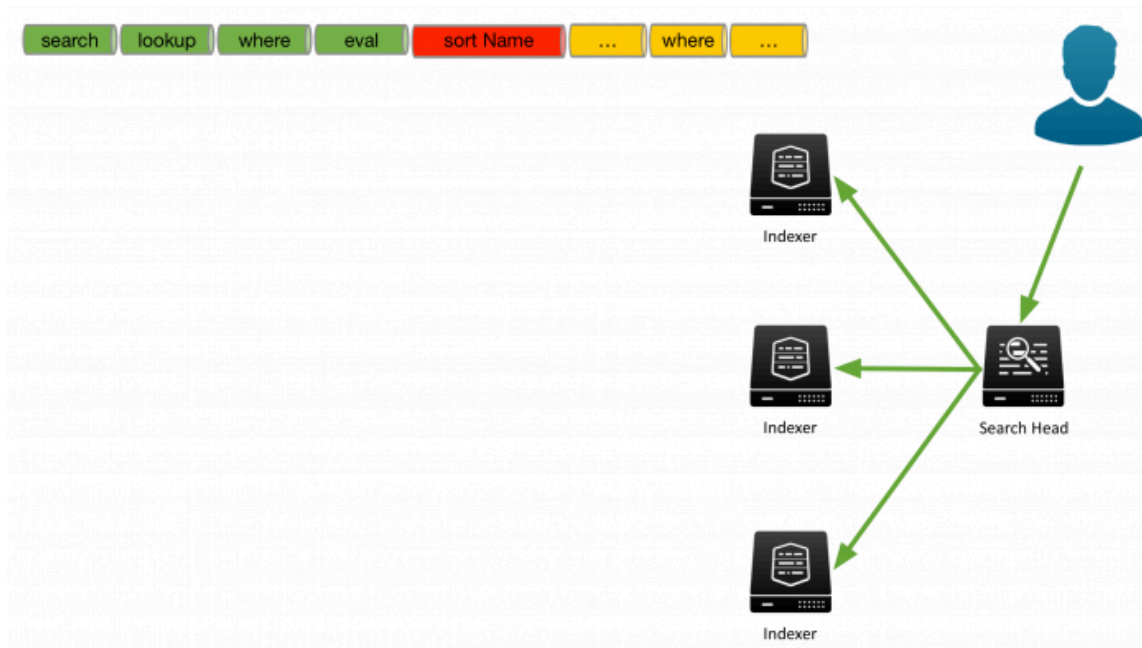
Non-streaming commands can run only when all of the data is available. To process non-streaming commands, all of the search results from the indexers are sent to the search head. When this happens, all further processing must be performed by the search head, rather than in parallel on the indexers.

Parallel processing example

Non-streaming commands that are early in your search reduce parallel processing.

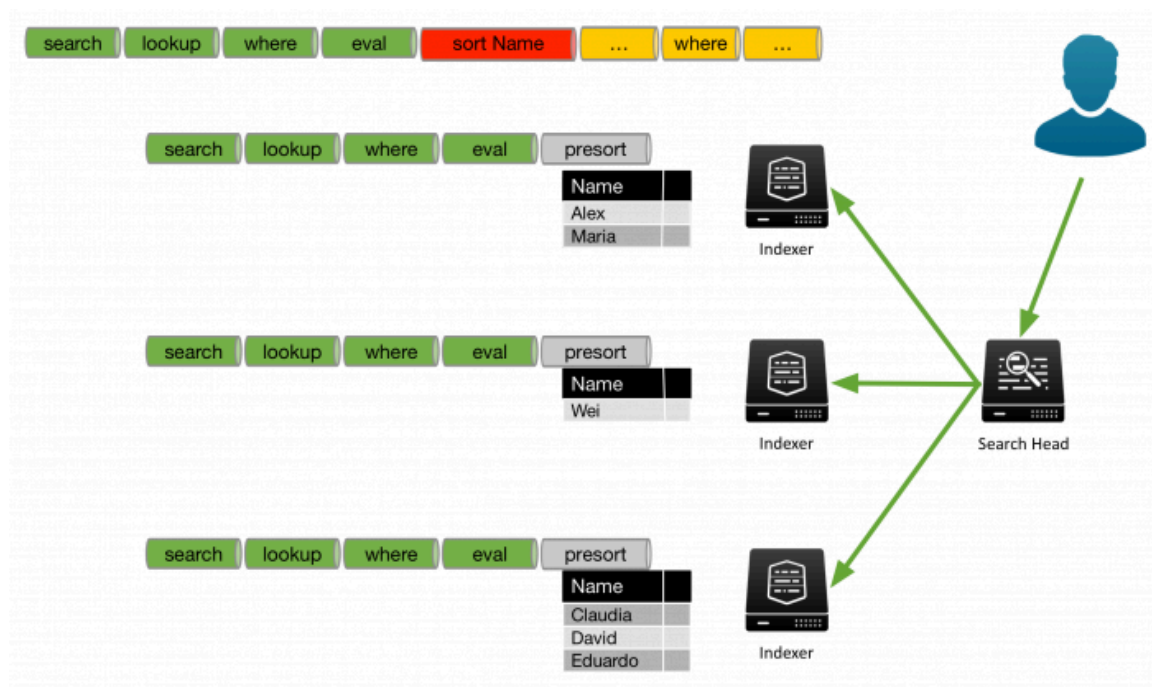
For example, the following image shows a search that a user has run. The search starts with the **search** command, which is implied as the first command in the Search bar. The search continues with the **lookup**, **where**, and **eval** commands. The search then contains a **sort**, based on the **Name** field, followed by another **where** command.

The search is sent to the search head and distributed to the indexers to process as much of the search as possible on the indexers.



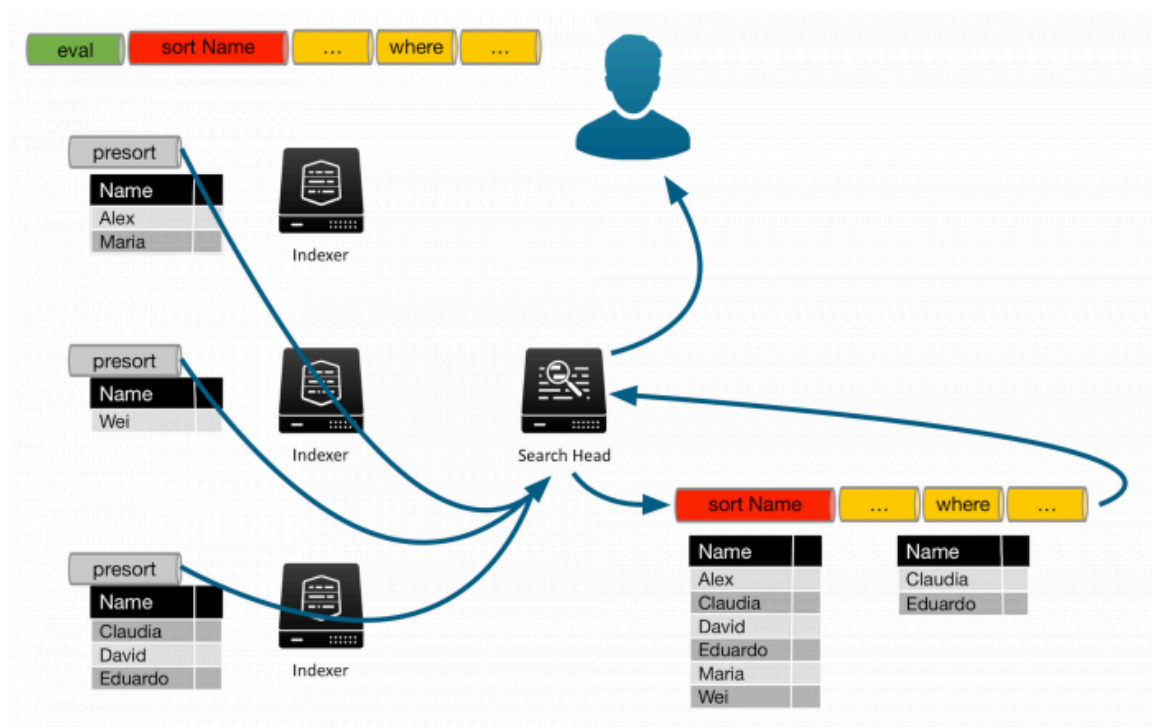
For the events that are on each indexer, the indexer processes the search until the indexer encounters a non-streaming command. In this example, the indexers process the search through the **eval** command. To perform the **sort**, all of the results must be sent to the search head for processing.

However, the results that are on each indexer can be sorted by the indexer. This is referred to as a *presort*. In this example the sort is on the **Name** field. The following image shows that the first indexer returns the names Alex and Maria. The second indexer returns the name Wei. The third indexer returns the names Claudia, David, and Eduardo.



To return the full list of results sorted by name, all of the events that match the search criteria must be sent to the search head. When all of the results are on the search head, the rest of the search must be processed on the search head. In this example the **sort** and any remaining commands are processed on the search head.

The following image shows that each indexer has presorted the results, based on the **Name** field. The results are sent to the search head, and are appended together. The search head then sorts the entire list into the correct order. The search head processes the remaining commands in the search to produce the final results. In this example, that includes the second **where** command. The final results are returned to the user.



When part or all of a search is run on the indexers, the search processes in parallel and search performance is much quicker.

To optimize your searches, place non-streaming commands as late as possible in your search string.

Tips for tuning your searches

In most cases, your search is slow because of the complexity of your query to retrieve events from index. For example, if your search contains extremely large OR lists, complex subsearches (which break down into OR lists), and types of phrase searches, it takes longer to process. This section discusses some tips for tuning your searches so that they are more efficient.

Performing statistics with a BY clause on a set of field values that have a high cardinality, lots of uncommon or unique values, requires a lot of memory. One possible remedy is to decrease the value for the `chunk_size` setting used with the `tstats` command. Additionally, reducing the number of distinct values that the BY clause must process can also be beneficial.

Restrict searches to the specific index

If you rarely search across more than one type of data at a time, partition your different types of data into separate indexes. Then restrict your searches to the specific index. For example, store Web access data in one index and firewall data in another. This is recommended for sparse data, which might otherwise be buried in a large volume of unrelated data.

See [Create custom indexes](#) in *Managing Indexers and Clusters of Indexers* and [Retrieve events from indexes](#) in this manual.

Use fields effectively

Searches with fields are faster when they use fields that have already been extracted (indexed fields) instead of fields extracted at search time. For more information about indexed fields and default fields, see [About fields](#) in the *Knowledge Manager Manual*.

Use indexed and default fields

Use indexed and default fields whenever you can to help search or filter your data efficiently. At index time, Splunk software extracts a set of default fields that are common to each event. These fields include `host`, `source`, and `sourcetype`. Use these fields to filter your data as early as possible in the search so that processing is done on a minimum amount of data.

For example, if you're building a report on web access errors, search for those specific errors before the reporting command:

```
sourcetype=access_* (status=4* OR status=5*) | stats count by status
```

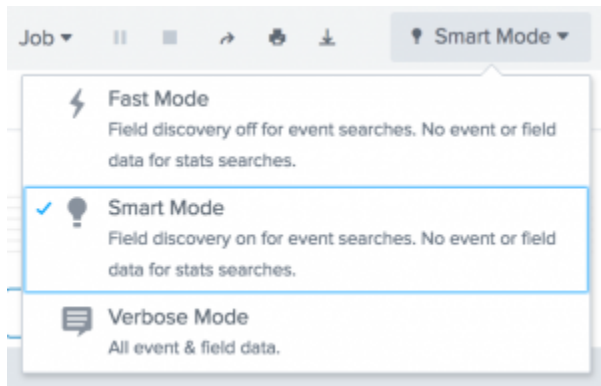
Specify indexed fields with <field>::<value>

You can also run efficient searches for fields that have been indexed from structured data such as CSV files and JSON data sources. When you do this, replace the equal sign with double colons, like this: `<field>::<value>`.

This syntax works best in searches for fields that have been indexed from structured data, though it can be used to search for default and custom indexed fields as well. You cannot use it to search on **search-time** fields.

Disable field discovery to improve search performance

If you don't need additional fields in your search, set **Search Mode** to a setting that disables field discovery to improve search performance in the timeline view or use the **fields** command to specify only the fields that you want to see in your results.



However, disabling field discovery prevents automatic **field extraction**, except for fields that are required to fulfill your search, such as fields that you are specifically searching on and **default fields** such as **_time**, **host**, **source**, and **sourcetype**. The search runs faster because Splunk software is no longer trying to extract every field possible from your events.

Search mode is set to **Smart** by default. Set the search mode to **Verbose** if you are running searches with reporting commands, you don't know what fields exist in your data, and think you might need the fields to help you narrow down your search.

See [Set search modes](#).

Also see the topic about the [fields](#) command in the *Search Reference*.

Summarize your data

It can take a lot of time to search through very large data sets. If you regularly generate reports on large volumes of data, use summary indexing to pre-calculate the values that you use most often in your reports. Schedule saved searches to collect metrics on a regular basis, and report on the summarized data instead of on raw data.

See [Use summary indexing for increased reporting efficiency](#).

Use the Search Job Inspector

The **Search Job Inspector** is a tool you can use both to troubleshoot the performance of a search and to determine which phase of the search takes the greatest amounts of time. It dissects the behavior of your searches to help you understand the execution costs of knowledge objects such as event types, tags, lookups, search commands, and other components within the search.

See [View search job properties](#) in this manual.

See also

- [About search optimization](#)
- [Quick tips for optimization](#)

- [Built-in optimizations](#)

Built-in optimization

The Splunk software includes built-in optimizations that analyze and process your searches for maximum efficiency.

One goal of these optimizations is to filter results as early as possible. Filtering reduces the amount of data to process for the search. Early filtering avoids unnecessary lookups and evaluation calculations for events that are not part of the final search results.

Another goal is to process as much as possible in parallel on the indexers. The built-in optimizations can reorder search processing, so that as many commands as possible are run in parallel on the indexers before sending the search results to the search head for final processing. You can see the reordered search using the Job Inspector. See [Analyze search optimizations](#).

Predicate optimization

Predicates act as filters to remove unnecessary events as your search is processed. The earlier in the search pipeline that a predicate is applied, the more efficient is the use of system resources. If a predicate is applied early, when events are fetched, only the events that match the predicate are fetched. If the same predicate is applied later in the search, then resources are wasted fetching more events than are necessary.

There are several built-in optimizations that focus on predicates:

- Predicate merge
- Predicate pushdown
- Predicate splitting

Types of predicates

There are two types of predicates, simple predicates and complex predicates.

- A simple predicate is a single conditional of the form **field = value**.
- A complex predicate is a combination of several conjunctions (AND and OR) and disjunctions (NOT). For example, **Field1 = Value1 OR Field2 = Value2 AND Field3 = Value3**.

A complex predicate can be merged or split apart to optimize a search.

In Splunk SPL, there are two commands that perform predicate-based filtering, **where** and **search**.

An example of using the **where** command for filtering is:

```
index="_internal" | where bytes > 10
```

An example of using the **search** command for filtering is:

```
index="_internal" | search bytes > 10
```

Search-based predicates are a subset of where-based predicates. In other words, search-based predicates can be replaced by where-based predicates. However, where-based predicates cannot be replaced by search-based predicates.

Predicate merge

The predicate merge optimization takes two predicates and merges the predicates into one predicate. For example, consider the following search:

```
index=main | search a > 10 AND fieldA = "New"
```

There are two search commands in this example. The **search** command before the pipe, which is implied at the beginning of every search, and the explicit **search** command after the first pipe. With the predicate merge optimization, the predicates in the second **search** command are merged with the predicates in the first **search** command. For example:

```
index=main AND a > 10 AND fieldA = "New"
```

In some cases, predicates used with the **where** command can also be merged. For example, consider the following search:

```
index=main | where fieldA = "New"
```

With the predicate merge optimization, the predicates specified with the **where** command are merged with the predicates specified with the **search** command.

```
index=main AND fieldA = "New"
```

Issues with field extractions and predicate merge

An **inline field extraction** requires special handling if the regular expression pattern extracts a subtoken. The field must be set to **indexed=false** in the **fields.conf** file. See [Example inline field extraction configurations](#) in the *Knowledge Manager Manual*.

Consider the following sample event:

```
Mon Apr 17 16:08:16 2017 host=10.10.1.1 Login name=John SUCCESS FRANCE
```

You create an extracted field called **country** that uses the following regular expression:

```
SUCCESS\s+?\S{3}
```

- **SUCCESS** matches the characters SUCCESS literally and is case sensitive.
- **\s** matches any whitespace character (space, tab, new line).
- **+?** is a quantifier that matches between one and unlimited times, the fewest needed to match the pattern.
- **\S** matches any non-whitespace character.
- **{3}** is a quantifier that matches exactly 3 non-whitespace characters

For the sample event, the following regular expression extracts the first 3 characters of the word FRANCE or **FRA**. The extraction **FRA** is a subtoken of the indexed term FRANCE.

When you search using the extracted field, for example:

```
index=main | search country=FRA
```

The search is optimized, using the predicate merge optimizer:

```
index=main country=FRA
```

However, the search returns no results because **FRA** is not part of the index. **FRANCE** is the indexed term.

To overcome this issue, you must add the following setting to the **fields.conf** file:

```
[country]  
indexed=false
```

Alternatively, you can disable the built-in optimizations. See [Optimization settings](#).

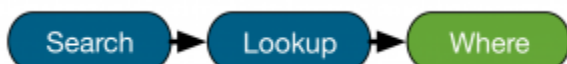
Predicate pushdown

The predicate pushdown optimization analyzes a search and reorders the search processing so that predicates are processed as early as possible.

Example of a simple predicate pushdown

You perform the following search:

```
sourcetype=access* (status=401 OR status=403) | lookup usertogroup user OUTPUT group | where  
src_category="email_server"
```



The `sourcetype=access* (status=401 OR status=403)` portion of the search retrieves 50,000 events. The `lookup` is performed on all 50,000 events. Then the `where` command is applied and filters out events that are not `src_category="email_server"`. The result is that 45,000 events are discarded and 5,000 events are returned in the search results.

If the search criteria used with the `where` command is applied before the `lookup`, more events are filtered out of the results. Only 5,000 events are retrieved from disk before the `lookup` is performed.

With predicate pushdown, the search is reordered for processing. The `where` command is eliminated by moving the search criteria `src_category="email_server"` before the first pipe.

```
sourcetype=access* (status=401 OR status=403) src_category="email_server" | lookup usertogroup user OUTPUT
group
```



Example of a complex predicate pushdown

Consider the following search fragment:

```
... | eval x=if(isnull(x) OR x="", "missing", x) | where x = "hello"
```

In this situation, the `eval` command is assigning a default value when a field is null or empty. This is a common pattern in Common Information Model (CIM) data models.

The built-in optimization reorganizes the search criteria before processing the search. The `where` command is moved before the `eval` command.

```
... | where x = "hello" | eval x=if(isnull(x) OR x="", "missing", x)
```

Predicate splitting

Predicate splitting is the action of taking a predicate and dividing, or splitting, the predicate into smaller predicates. The predicate splitting optimizer can then move the smaller predicates, when possible, to an earlier place in the search.

Consider the following search:

```
index="_internal" component = "SearchProcess" | eval a = (x + y) | where a > 200 AND x < 10
```

Because `field a` is generated as part of the `eval` command, it cannot be processed earlier in the search. However, `field x` exists in the events and can be processed earlier. The predicate splitting optimization separates the components of the `where` portion of the search. The search is reordered to process eligible components earlier.

```
index="_internal" component = "SearchProcess" | where x < 10 | eval a = (x + y) | where a > 200
```

The portion of the **where** command **x < 10** is moved to before the **eval** command. This move reduces the number of results that the **eval** command must process.

Projection elimination

This optimization analyzes your search and determines if any of the generated fields specified in the search are not used to produce the search results. If generated fields are identified that can be eliminated, an optimized version of the search is run. Your search syntax remains unchanged.

For example, consider the following search:

```
index=_internal | eval c = x * y / z | stats count BY a, b
```

The **c** field that is generated by **eval c = x * y / z** is not used in the **stats** calculation. The **c** field can be eliminated from the search.

Your search syntax remains:

```
index=_internal | eval c = x * y / z | stats count BY a, b
```

But the optimized search that is run is:

```
index=_internal | stats count BY a, b
```

Here is another example:

```
| tstats count FROM datamodel=internal_audit_logs
```

For buckets whose data models are not built, this expands to the following fallback search:

```
| search ((index=* OR index=_) index=_audit)
| eval nodename = "Audit"
| eval is_searches=if(searchmatch("(action=search NOT dmauditsearch)"),1,0),
  is_not_searches=1-is_searches, is_modify=if(searchmatch("(action=edit_user
  OR action=edit_roles OR action=update)"),1,0), is_not_modify=1-is_modify
| eval nodename = if(nodename == "Audit"
  AND searchmatch("(action=search NOT dmauditsearch)"), mvappend(nodename,
  "Audit.searches"), nodename)
```

```
| eval is_realtime=case(is_realtime == 0, "false", is_realtime == 1, "true",
  is_realtime == "N/A", "false"), search_id=replace(search_id,"",""),
  search=replace(search,"",""), search_type=case((id LIKE "DM_"
  OR savedsearch_name LIKE "_ACCELERATE_DM%"), "dm_acceleration",
  search_id LIKE "scheduler%", "scheduled", search_id LIKE "rt%", "realtime",
  search_id LIKE "subsearch%", "subsearch", (search_id LIKE "SummaryDirector%"
  OR search_id LIKE "summarize_SummaryDirector%"), "summary_director",
  1=1, "adhoc")
| rename is_realtime AS Audit.searches.is_realtime
  search_id AS Audit.searches.search_id
  search AS Audit.searches.search
  search_type AS Audit.searches.search_type
| eval nodename = if(nodename == "Audit" AND searchmatch("(action=edit_user
  OR action=edit_roles OR action=update)"), mvappend(nodename,
  "Audit.modify"), nodename)
| rename action AS Audit.action info AS Audit.info object AS Audit.object
  operation AS Audit.operation path AS Audit.path user AS Audit.user
  exec_time AS Audit.exec_time result_count AS Audit.result_count
  savedsearch_name AS Audit.savedsearch_name
  scan_count AS Audit.scan_count total_run_time AS Audit.total_run_time
  is_searches AS Audit.is_searches is_not_searches AS Audit.is_not_searches
  is_modify AS Audit.is_modify is_not_modify AS Audit.is_not_modify
| addinfo type=count label=prereport_events
| stats count
```

This search can be optimized to this syntax:

```
| search ((index=* OR index=_) index=_audit) | stats count
```

Eliminating unnecessary generated fields, or projections, leads to better search performance.

Stats/chart/timechart to tstats optimization

This optimization converts qualifying [stats](#), [chart](#), or [timechart](#) searches into [tstats](#) searches. When the search processor applies this optimization to qualifying **stats**, **chart**, or **timechart** searches that aggregate large amounts of data, such searches can complete far faster than they otherwise would. This increase in efficiency happens because **tstats** performs statistical searches on indexed fields in tsidx index summary files rather than the raw data.

For example, consider this search:

```
index=_internal sourcetype=splunkd | stats count
```

The **stats / chart / timechart** to **tstats** optimizer optimizes that search to something like this:

```
| tstats prestats=true summariesonly=false allow_old_summaries=false include_reduced_buckets=true count WHERE
(index=_internal sourcetype=splunkd) | stats count
```

How searches qualify for stats/chart/timechart to tstats optimization

The search optimizer applies **stats** / **chart** / **timechart** to **tstats** optimization only to **stats** , **chart** , and **timechart** searches that meet an array of conditions.

Qualification for stats/chart/timechart to tstats search optimization	More information
The search must have an explicit or implicit search command followed by a stats , chart , or timechart command.	Here is an example of a search that fits this pattern: <pre>index=_internal sourcetype=splunkd stats count</pre>
The search portion of the search string and the stats , chart , or timechart portion of the search string reference only indexed fields .	The tstats command can perform searches only on indexed fields. For more information about indexed fields and search-time fields, see When Splunk software extracts fields in the <i>Knowledge Manager Manual</i>.
The search does not have search-time field extractions that overwrite the indexed fields in the search and the stats , chart , or timechart portions of the search.	For example, a search with a search-time field extraction for a calculated field that overwrites the host field would not be optimized. At run time, the optimizer reviews all possible configurations to which the user has access. If any of these configurations might lead to overwriting of indexed fields with search-time fields, the optimizer disqualifies the search for tstats optimization. See When Splunk software extracts fields in the <i>Knowledge Manager Manual</i>.
The search uses only statistical or charting functions that are supported by the tstats command.	For example, the earliest , latest , earliest_time , and latest_time functions are not supported by tstats . If the search uses those functions in conjunction with the stats , chart , or timechart commands, it is disqualified for tstats optimization. See the tstats topic.
The search filters out source types that invoke multikv to extract values from table-formatted events.	These are source types that are configured with KV_MODE=multi in the Advanced tab of the Edit Source Types dialog box. If a search does not explicitly filter out source types, the optimizer then checks all source types available to the search. If the optimizer finds any source types that are configured with KV_MODE=multi , it does not apply stats / chart / timechart to tstats optimization to the search. See Manage source types in the <i>Getting Data In</i> manual.
The search does not invoke verbose mode .	See Search modes .
The search is not a real-time search .	See About real-time searches and reports .

Qualification for stats/chart/timechart to tstats search optimization	More information
The search does not require subsecond timestamp resolution.	Searches that require result timestamps to have millisecond or smaller time resolution are disqualified. See About searching with time .
The search does not use event sampling.	See Event sampling .
The search does not include subsearches .	See About subsearches .
The search is not running in an environment that has been configured to rename source types at search time.	See Rename source types in the <i>Getting Data In</i> manual.
The search is not running in an environment where <code>status_buckets > 0</code> .	A positive value for the <code>status_buckets</code> setting causes timelines to be generated by <code>stats</code> searches, which disqualifies them for <code>tstats</code> optimization. The <code>status_buckets</code> setting can be set through a POST operation to the search/jobs REST API endpoint, or through a direct change to the <code>status_buckets</code> setting in <code>limits.conf</code> . See the Search endpoint descriptions in the <i>REST API Reference Manual</i> .
The search does not run over virtual indexes.	Does not apply to Splunk Cloud Platform. Virtual indexes are used only by Splunk Analytics for Hadoop. See Search a virtual index in the <i>Splunk Analytics for Hadoop</i> manual.

Disabling the stats/chart/timechart to tstats optimizer on a per-search basis

The `stats / chart / timechart` to `tstats` optimizer can cause certain kinds of searches to run slower.

For example, this optimizer does not cooperate with searches that populate [summary indexes](#), because it does not respect the fields created by summary indexing commands. This optimizer may cause searches that perform `stats`, `chart`, or `timechart` operations on summary indexes to perform extra work on a fallback search, thus slowing their performance.

You can disable this optimization for individual searches.

If you want to run a `stats` search without the optimization, append

| `noop search_optimization.replace_stats_cmds_with_tstats=f` to the search string.

If you want to run a `chart` or `timechart` search without the optimization, append

| `noop search_optimization.replace_chart_cmds_with_tstats=f` to the search string.

Event types and tags optimization

Event typing and tagging can take a significant amount of search processing time. This is especially true with Splunk Enterprise Security where there are many event types and tags defined. The more event types and tags that are defined in the system, the greater the annotation costs.

The built-in event types and tags optimization applies to [transforming searches](#) and data model acceleration searches.

Transforming searches

When a transforming search is run without the built-in optimization, all of the event types and tags are uploaded from the configuration system to the search processor. This upload happens whether the search requires the event types and tags or not. The search processing attempts to apply the event types and tags to each event.

For example, consider the following search:

```
index=_internal | search tag=foo | stats count by host
```

This search needs only the tag **foo**. But without the optimization, all of the event types and tags are uploaded. The built-in optimization solves this problem by analyzing the search and only uploading the event types and tags that are required. If no event types or tags are needed, none are uploaded which improves search performance.

For transforming searches, this optimization is controlled by several settings in the **limits.conf** file, under the **[search_optimization::required_field_values]** stanza. These settings are turned on by default. There is no need to change these settings, unless you need to troubleshoot an issue.

Data model acceleration searches

Another problem that can be solved by the event type and tags optimization is specifically targeted at data model acceleration. With a configuration setting in the **datamodels.conf** file, you can specify a list of tags that you want to use with the data model acceleration searches. You specify the list under the **[dm.name]** stanza with the **tags_whitelist** setting.

For detailed information about the **tags_whitelist** setting, including usage examples, see [Set a tag whitelist for better data model performance](#) in the *Knowledge Manager Manual*.

Analyze search optimizations

You can analyze the impact of the built-in optimizations by using the Job Inspector.

Determine search processing time

You can use the Job Inspector to determine whether the built-in search optimizations are helping your search to complete faster.

1. Run your search.
2. From the search action buttons, select **Job > Inspect job**.
3. In the Job Inspector window, review the message at the top of the window. The message is similar to "The search has completed and returned X results by scanning X events in X seconds." Make note of the number of seconds to complete the search.
4. Close the Job inspector window.
5. In the Search bar, add **|noop search_optimization=false** to the end of your search. This turns off the built-in optimizations for this search.

6. Run the search.
7. Inspect the job and compare the message at the top of the Job Inspector window with the previous message. This message specifies how many seconds the search processing took without the built-in optimizations.

View the optimized search

You can compare your original search with the optimized search that is created when the built-in optimizations are turned on.

1. Run your search.
2. From the search action buttons, select **Job, Inspect job**.
3. In the Job Inspector window, expand the **Search job properties** section and scroll to the **normalizedSearch** property. This property shows the internal search that is created based on your original search.
4. Scroll to the **optimizedSearch** property. This property shows the search that is created, based on the **normalizedSearch**, when the built-in optimizations are applied.

Optimization settings

By default, the built-in optimizations are turned on.

Turn off optimization for a specific search

In some very limited situations, the optimization that is built into the search processor might not optimize a search correctly. In these situations, you can troubleshoot the problem by turning off the search optimization for that specific search.

Turning off the search optimization enables you to determine if the cause of unexpected or limited search results is the search optimization.

You turn off the built-in optimizations for a specific search by using the **noop** command.

You add **noop search_optimization=false** at the end of your search. For example:

```
| datamodel Authentication Successful_Authentication search | where Authentication.user = "fred" | noop  
search_optimization=false
```

Turn off all optimizations

You can turn off all of the built-in optimizations for all users.

Splunk Cloud Platform

To turn off the built-in optimizations for all users, request help from Splunk Support. If you have a support contract, file a new case using the Splunk Support Portal at [Support and Services](#). Otherwise, contact [Splunk Customer Support](#).

Splunk Enterprise

To turn off the built-in optimizations for all users, follow these steps.

Prerequisites

- Only users with file system access, such as system administrators, can turn off the built-in optimizations using a configuration file.
- Review the steps in [How to edit a configuration file](#) in the Splunk Enterprise *Admin Manual*.
- You can have configuration files with the same name in your default, local, and app directories. Read [Where you can place \(or find\) your modified configuration files](#) in the Splunk Enterprise *Admin Manual*.

CAUTION: Never change or copy the configuration files in the default directory. The files in the default directory must remain intact and in their original location.

Steps

1. Open or create a local limits.conf file for the Search app at `$SPLUNK_HOME/etc/apps/search/local`.
2. Under the `[search_optimization]` stanza, set **enabled=false**.

See also

- [About optimization](#)
- [Quick tips for optimization](#)
- [Write better searches](#)

In the *Search Reference*:

- The [noop](#) command

Search normalization

When you use the **search** or **where** command in a search string, the **SPL** processor might reorder the expression statement that follows the command for normalization purposes. The SPL processor applies two kinds of normalization logic to search strings: predicate flip and predicate sort.

For more information about predicates and predicate-based search optimization, see [Built-in optimization](#).

Use the [Job Inspector](#) to see the results of search normalization and optimization. See [Analyze search optimizations](#).

Benefits of search normalization

Some search optimizations perform better after search normalization. For example, the predicate merge optimization cannot merge **where** statements that place a field value before a field name. However, if the SPL processor applies predicate flip normalization to those statements so that the field name precedes the field value, the predicate merge optimization can merge it.

See [Predicate merge](#).

Predicate flip normalization

Under predicate flip normalization, the SPL processor takes **where** statements that have field-value pairs where field values are placed ahead of field names and switches them so that the field names come first.

For example, in this search, the field value has been placed ahead of the field:

```
index=main | where "error"=status
```

After normalization, the field name and field value are flipped:

```
index=main | where (status == "error")
```

Predicate flip normalization only works when the SPL processor can distinguish the field name from the field value. The SPL processor puts numeric field values and string field values that are surrounded by quotes on the right side of the operator. When it is possible, the SPL processor flips value-field combinations where the value includes functions, such as **value()=field**.

The SPL processor will not apply predicate flip normalization to boolean, time, and IPv4 fields. For example, with a boolean value-field pair like **true=purchased**, the SPL processor cannot distinguish whether **true** or **purchased** is the field name.

Predicate sort normalization

Under predicate sort normalization, the SPL processor uses lexicographical sorting logic to ensure that **search** expressions and **where** statements are consistently ordered in the same way.

Predicate sort for the search command

When you use the **search** command in a string, the SPL processor applies predicate sort normalization to any boolean expressions that follow it.

For example, the following three searches use the **search** command with a boolean expression. These searches look different, but they produce the same result:

```
| search ( z OR y AND d AND c AND b AND a )
```

```
| search ( d AND z OR y AND c AND b AND a )
```

```
| search ( d AND ( z OR y ) AND ( c AND b AND a ) )
```

After normalization, those strings are reordered so that they share the following form:

```
| search ((y OR z) a b c d)
```

Predicate sort for the where command

When you use the **where** command in a string, the SPL processor applies predicate sort normalization to any boolean or arithmetic statements that follow it.

For example, these **where** statements have mathematical expressions that all resolve to the same result, but are ordered differently:

```
| where x = (d+(c-a)+c*b)*b
```

```
| where b*(d+(c-a)+c*b) = x
```

```
| where ((b*c)+d+(c-a))*b = x
```

After normalization, these **where** statements share the following form:

```
| where (x == (((b * c) + (c - a)) + d) * b))
```

Example combining predicate flip and predicate sort

The following example combines predicate flip and predicate sort. Before normalization, you can have the following **where** statements:

```
| where status="error" OR code=500
```

```
| where "error"=status OR code=500
```

```
| where 500=code OR "error"=status
```

After normalization, these **where** statements share the following form:

```
| where ((code == 500) OR (status == "error"))
```

Disable search normalization

If you put your **search** expressions and **where** statements in a specific order for search performance reasons, you might want to disable search normalization. Predicate flip normalization and predicate sort normalization are controlled by separate settings in the `limits.conf` file. For reference, see the `.spec` file at [limits.conf](#). You can disable one kind of normalization and leave the other enabled.

Splunk Cloud Platform

To disable search normalization, request help from Splunk Support. If you have a support contract, file a new case using the Splunk Support Portal at [Support and Services](#). Otherwise, contact [Splunk Customer Support](#).

Splunk Enterprise

To disable search normalization, follow these steps.

Prerequisites

- Only users with file system access, such as system administrators, can disable search normalization using configuration files.
- Review the steps in [How to edit a configuration file](#) in the *Splunk Enterprise Admin Manual*.
- You can have configuration files with the same name in your default, local, and app directories. Read [Where you can place \(or find\) your modified configuration files](#) in the *Splunk Enterprise Admin Manual*.

CAUTION: Never change or copy the configuration files in the default directory. The files in the default directory must remain intact and in their original location. Make changes to the files in the local directory.

Disable predicate flip normalization

1. Open or create a local `limits.conf` file at `$SPLUNK_HOME/etc/system/local`.
2. Under the `[search_optimization::search_flip_normalization]` stanza, set **enabled=false**.

Disable predicate sort normalization

1. Open or create a local `limits.conf` file at `$SPLUNK_HOME/etc/system/local`.
2. Under the `[search_optimization::search_sort_normalization]` stanza, set **enabled=false**.

Control search execution using directives

You can use the following **search execution directives** to control aspects of a search before a search executes and optimize search performance.

- `REQUIRED_TAGS()`
- `REQUIRED_EVENTTYPES()`

- `READ_SUMMARY()`

Note: These directives should be used only by advanced Splunk users who need to exercise fine control over their searches. In most cases, do not use `REQUIRED_TAGS()`, `REQUIRED_EVENTTYPES()` or `READ_SUMMARY()` unless directed to do so by Splunk Support.

You may have already heard of `TERM()` and `CASE()` directives, which are qualifiers that are applied to search terms in searches. Because `TERM()` and `CASE()` don't control how searches are executed or relate to search execution directives, they are not discussed in this section. See [Use CASE\(\) and TERM\(\) to match phrases](#).

REQUIRED_TAGS()

The `REQUIRED_TAGS()` directive turns off the automatic tagging that Splunk performs as part of the background operations for the search command. Use this directive to tell Splunk software not to run all tags when performing its automatic tagging operations because you're only interested in certain tags that you have defined. The `REQUIRED_TAGS()` directive is typically used to improve search performance.

The `intersect="t"` argument that Splunk software adds to the `REQUIRED_TAGS` directive in some data-model-based searches is for internal use only.

REQUIRED_EVENTTYPES()

The `REQUIRED_EVENTTYPES()` directive turns off the automatic eventtypes that Splunk generates as part of the background operations for the search command. Use this directive to restrict the set of event types that are used in your search. The `REQUIRED_EVENTTYPES()` directive is useful for debugging and, in some cases, it can help improve search performance.

READ_SUMMARY()

The `READ_SUMMARY()` directive tells Splunk software to look only at the specified summary, which allows the search processor to leverage existing data model acceleration summary data when it performs event searches. Use the `READ_SUMMARY()` directive to tell Splunk software to display summary data for this search from the specified summary only and ignore the rest of the summaries. The `READ_SUMMARY()` directive is typically used to improve search performance.

Note: This directive is intended primarily for internal use by Splunk software. Do not use `READ_SUMMARY()` unless directed to do so by Splunk Support.

Examples

These examples use the sample data from the Search Tutorial but should work with any format of Apache web access log. To try these examples on your own Splunk instance, you must download the sample data and follow the instructions to [get the tutorial data into Splunk](#). Use the time range **All time** when you run the search.

1. Create custom tags using REQUIRED_TAGS()

In this example, say you create two tags like these:

Tag name	field value pair
tag1	host=www1
tag2	host=www2

Then, run the following search using the time range **All time**:

```
source="tutorialdata.zip:*" | tags
```

The results include the tag field, which associates **tag1** and **tag1** with events that contain the www1 and www2 hosts.

To suppress **tag1** and **tag2** in the results, run the following search using the time range **All time**:

```
source="tutorialdata.zip:*" DIRECTIVES(REQUIRED_TAGS(tags=""))
```

The results do not include the tag field.

To limit your search to just **tag1**, run the following search:

```
source="tutorialdata.zip:*" DIRECTIVES(REQUIRED_TAGS(tags="tag1"))
```

The results include the tag field, which lists the **tag1** tag that is used in the events that contain the www1 host.

2. Restrict event types in searches using REQUIRED_EVENTTYPES()

In this example, say you define the following eventtypes in the eventtypes.conf file:

```
[eventtype1] search = host=www1
```

```
[eventtype2] search = host=www2
```

Then, run this search:

```
source="tutorialdata.zip:*" DIRECTIVES(REQUIRED_EVENTTYPES(eventtypes="eventtype1"))
```

Because the search includes **REQUIRED_EVENTTYPES(eventtypes="eventtype1")**, it is restricted to **eventtype1**. As a result, only **eventtype1** is returned in the **eventtype** field.

Retrieve Events

About retrieving events

When you search, you are seeking to match search terms against segments of your event data. These search terms are keywords, phrases, boolean expressions, field name and value pairs, and so forth that specify which events you want to retrieve from the indexes. Read the [Search command primer](#) to learn how to use the search command effectively.

Your event data might be partitioned into different indexes and across distributed search peers. Read more about how to search across multiple indexes and servers in [Retrieve events from indexes](#).

Events are retrieved from the indexes in reverse time order. The results of a search are ordered from most recent to least recent by default. You can retrieve events faster if you filter by time, whether you are using the timeline to zoom in on clusters of events or applying time ranges to the search itself. For more information, read how to [Use the timeline to investigate events](#) and [About time ranges in search](#).

Events, event data, and fields

The phrase *event data* refers to your data after it has been added to the Splunk index. Events are a single record of activity or instance of this event data. For example, an event might be a single log entry in a log file. Because the Splunk software separates individual events by their time information, an event is distinguished from other events by a timestamp.

Here is a sample event:

```
172.26.34.223 - - [01/Jul/2005:12:05:27 -0700] "GET /trade/app?action=logout HTTP/1.1" 200 2953
```

Events contain pairs of information, or fields. When you add data and it gets indexed, the Splunk software automatically extracts some useful fields for you, such as the host the event came from and the [type of data source](#) it is.

Use fields to retrieve events

Fields are searchable name/value pairings in **event data**. All fields have names and can be searched with those names. Searches with field expressions are more precise (and therefore more efficient) than searches using only keywords and quoted phrases.

Look at the following search:

```
host=webserver
```

In this search, **host=webserver** indicates that you are searching for events with **host** fields that have values of **webserver**. When you run this search, events with different **host** field values are not retrieved, nor are events that contain other fields that share **webserver** as a value. This means that this search returns a more focused set of results than you might get if you just searched for **webserver** in the search bar.

For more information, read [About fields](#) in the Knowledge Manage Manual.

Index-time and search-time fields

As Splunk software processes event data, it extracts and defines fields from that data, first at index time, and again at search time.

See [Index time versus search time](#) in the *Managing Indexers and Clusters* manual.

Field extraction at index time

At **index time**, Splunk software extracts a small set of fields. This set of fields includes **default fields**, custom indexed fields, and fields indexed from structured data.

Default fields exist in all events. Three important default fields are host, source, and source type. They describe where the event originated. Other default fields include datetime fields, which provide additional searchable granularity to event timestamps. Splunk software also automatically adds default fields classified as internal fields.

Custom indexed fields are fields that you have manually configured for index-time extraction. See [Create custom fields at index time](#) in the *Getting Data In* manual.

Finally, when Splunk software indexes structured data, it creates index-time field extractions for the fields that it finds. Examples of structured data include:

- comma-separated value files (CSV)
- tab-separated value files (TSV)
- pipe-separated value files
- JavaScript Object Notation (JSON) data sources

When searching for default field values and custom indexed field values you can use the standard `<field>=<value>` syntax. This syntax matches default fields, custom indexed fields, and search-time fields.

However if you are searching specifically for a field that has been extracted at index-time from structured data, you can search more efficiently if you exchange the equal sign for a double colon, as follows:

```
<field>::<value>
```

This syntax works best in searches for fields that were indexed from structured data. However, you can use it to search for default and custom indexed fields as well. You cannot use it to search on **Search-time** fields.

For more information about extracting fields from structured data files, see [Extract data from files with headers](#) in the *Getting Data In* manual.

Field extraction at search time

At **search time**, Splunk software extracts additional fields, depending on its **Search Mode** setting and whether or not that setting enables field discovery given the type of search being run.

Search examples

Example 1: Search for events on all "corp" servers for accesses by the user "strawsky". It then reports the 20 most recent events.

```
host=corp* eventtype=access user=strawsky
```

In this example, **host** is a default field, while **eventtype** and **user** are additional fields that might have been automatically extracted or that you defined.

In general, an event type is a user-defined field that simplifies search by letting you categorize events. You can save a search as an event type and quickly retrieve those events using the **eventtype** field. For more information, read [About event types](#) in the Knowledge Manager Manual.

Example 2: Search for events from the source `"/var/www/log/php_error.log"`.

```
source="/var/www/log/php_error.log"
```

The source of an event is the name of the file, stream, or other input from which the event originates.

Example 3: Search for all events that have an Apache web access source type.

```
sourcetype="access_*"
```

The source type of an event is the format of the data input from which it originates. In this search uses a wildcard to match any Apache web access log that begins with "access_". This includes access_common and access_combined (and you might also see access_combined_wcookie).

Example 4: Search indexed information from various CSV files to get a list of Plano-based employees.

```
employee_office::Plano
```

You have indexed several CSV files of employee records. Each of these CSV files share the same fields. You want to search for the employees from these files that are affiliated with the office in Plano, Texas.

This example uses the `<field>::<value>` syntax to find the fields from those CSV files, which are extracted at index time. This syntax works best for fields extracted from indexed structured data, although it can handle other kinds of index time fields as well. It cannot find fields that are extracted at search time.

Example 5: Search corp1 for events that have more than 4 lines, and omit events that contain the term 400.

```
host=corp1 linecount>4 NOT 400
```

You can use comparison expressions to match field/value pairs. Comparison expressions with "=" and "!=" work with all field/value pairs. Comparison expressions with < > <= >= work only with fields that have numeric values. This example specifies a search for events that have more than 4 lines, **linecount>4**.

Example 6: Searching with the boolean "NOT" versus the comparison operator "!=" is not the same. The following search returns events where **field** is undefined (or NULL).

```
NOT field="value"
```

The following search returns events where **field** exists and does not have the value "value".

```
field!="value"
```

In the case where the value in question is the wildcard "*", **NOT field=*** will return events where field is null/undefined, and **field!=*** will never return any events.

Example 7: Search for events that match a particular CIDR notation.

Suppose the **ip** field contains these IP address values:

10.10.10.12

50.10.10.17

10.10.10.23

The following search returns the events with the first and last values: 10.10.10.12 and 10.10.10.23

```
ip="10.10.10.0/24"
```

More about fields

This topic only discussed a handful of searches with fields.

- You can restrict searches to [specific indexes](#) and, in distributed topologies, to [specific search peers](#).
- You can see more search examples [Using default fields](#) in the Knowledge Manager Manual.

Fields become more important when you start using the Splunk search language to summarize and transform your data into reports. For more information, read [About reporting commands](#).

Event sampling

By default, a Splunk search retrieves all events. However in some situations you might want to retrieve a sample set of events, instead of retrieving the entire event set. There are several reasons why you might want to use event sampling.

- To perform a quick search to ensure the correct events are being returned

- To determine the characteristics of a large data set without processing every event
- To test that the data selection, formatting, calculations, and other components of the search are working correctly

For most searches, event sampling can greatly increase search performance without decreasing functionality.

The event sampling ratio

The sampling ratio is the likelihood of any event being included in the sample result set. The formula for the ratio is `1/sample_ratio_value`.

For example, if the sample ratio value is 100, each event has a 1 in 100 chance of being included in the result set. The selection of each event is independent of the selection of all another events. It is possible that many events are included from the first 100 events, or none at all.

If a search matches 1,000,000 events when sampling is not used, using a sample ratio value of 100 would result in returning approximately 10,000 events.

If you to rerun a sampling search many times, the exact number of returned results is modeled by a binomial distribution with $n=1000000$ and $p=0.01$. This distribution looks like a normal distribution, with the mean=10000 and the standard deviation (stdev)=99.5.

In Splunk Web, the sampling ratio that you specify must be a positive integer that is greater than 1. To disable sampling in Splunk Web, set the ratio to 1.

Set the default sampling ratio

Splunk Cloud Platform

To set the default sampling ratio, request help from Splunk Support. If you have a support contract, file a new case using the Splunk Support Portal at [Support and Services](#). Otherwise, contact [Splunk Customer Support](#).

Splunk Enterprise

Prerequisites

- Only users with file system access, such as system administrators, can set the default sampling ratio.
- Review the steps in [How to edit a configuration file](#) in the *Admin Manual*.

Note: Never change or copy the configuration files in the `default` directory. The files in the `default` directory must remain intact and in their original location. Make the changes in the `local` directory.

Steps

1. Open the local `ui-prefs.conf` file for the Search app. For example, `$SPLUNK_HOME/etc/apps/<app_name>/local`.
2. Under the [Preference options] stanza, set `display.prefs.customSampleRatio` to a positive integer.

How event sampling works

By default, event sampling is not active. When you run a search, every event that matches your criteria is returned. When you specify a ratio, sampling remains in effect for the active search window. Sampling also remains in effect when you save a search as a report or dashboard panel.

When you specify a ratio value, your value overrides the default value configured for your Splunk deployment and remains in effect until you change it.

If you open a new search window, event sampling is no longer active. However, the last custom ratio that you used appears in the **Sampling** drop-down.

Commands and functions to avoid with event sampling

Typically, searches that use the **transaction**, **stats**, or **streamstats** commands are not good candidates for sampling.

When you calculate statistics using a sample set of events, the statistical values will not be accurate. To get an approximation of the true statistical values, you must scale the values that the search returns. To get the most accurate value approximation, the scaling factor should be the same as the sampling ratio.

For example, you create a report using this search with event sampling enabled.

```
... | stats sum(x)
```

Because you used event sampling, the returned value is not the complete sum of all of the events. It is only the sum of the sample set of events. If the sampling ratio is 100, the true sum is approximately 100 times the value returned by the search.

Statistical calculations that fall into this situation are **count**, **sum**, and **sumsq**.

Other statistics that are difficult to interpret when event sampling is used include:

- **distinct_count**
- **earliest**
- **latest**
- **max**
- **min**

Specify a sampling ratio

You activate event sampling for a search by specifying a sampling ratio.

1. In Splunk Web, below the Search bar, click **No Event Sampling**.
2. You can use one of the default ratios or specify a custom ratio.
 - a. To use one of the default ratios, click the ratio in the **Sampling** drop-down.
 - b. To specify a custom ratio, click **Custom** and type the ratio value. Then click **Apply**. The ratio value must be a positive integer greater than 1.

Event sampling indicators

There are several indicators in the Search & Reporting App window which show that event sampling is active. After you run a search, the **Sampling** drop-down appears in the event count line. The label for the **Sampling** drop-down specifies the ratio that is applied to the search. Additionally, if a sampling ratio is being used, the **Jobs** drop-down specifies the ratio that is applied to the search.

Event sampling with reports and dashboard panels

You can save a search that uses event sampling as a report or dashboard panel. Use the **Save As** drop-down to save the search.

When the search is saved as a report, the sampling ratio is used when the report is run.

When the search saved as a dashboard panel, the panel is powered by an inline search. When the dashboard is refreshed, the sampling ratio that was saved with the inline search is used.

If you open a report and add the report to a dashboard panel, you can specify how the panel is powered. You can specify that the panel is powered by the inline search that the report is based on. Or you can specify that the panel is powered by the report itself.

Panels powered by reports

When you view the source for the panel in Simple XML, there is no indication if the report uses event sampling.

Panels powered by inline searches

When you view the source for the panel in Simple XML, if the underlying search uses event sampling there is `<sampleRatio>` entry. For example:

```
<event>
  <title>sample events</title>
  <search>
    <query>buttercupgames</query>
    <earliest>@d</earliest>
    <latest>now</latest>
    <sampleRatio>500</sampleRatio>
  </search>
</event>
```

Accelerated reports

You cannot accelerate reports that are based on event sampling searches. See [Accelerate reports](#) in the *Reporting Manual*.

Retrieve events from indexes

With the Splunk platform, you have always been able to create new indexes and manage where you want to store your data. Additionally, when you have data split across different indexes, you can search multiple indexes at once, using the **index** field.

Specify one or multiple indexes to search

The Splunk administrator can set the default indexes that a user searches. Based on the roles and permissions, the user might have access to one or many indexes. For example the user might be able to only search main or all public indexes. The user can then specify a subset of these indexes, either an individual index or multiple indexes, to search. For more information about setting up users and roles, see ["About users and roles"](#) in *Securing Splunk Enterprise*.

For more information about managing your indexes and setting up multiple indexes, see the ["About managing indexes"](#) in the *Managing Indexers and Clusters* manual.

Control index access using Splunk Web

1. Navigate to **Settings > Roles**.
2. Click the role that the User has been assigned to.
3. Click on "3. Indexes".
4. Control the indexes that particular role has access to, as well as the default search indexes.

Syntax

You can specify different indexes to search in the same way that you specify field names and values. In this case, the field name is **index** and the field value is the name of a particular index:

```
index=<indexname>
```

Specify groups of indexes using wildcards

You can use a wildcard (*****) to specify groups of indexes. For example, if you want to search both "mail" and "main" indexes, search for:

```
index=mai*
```

To match internal indexes using a wildcard, use **__*** in your search, like this:

```
index=__*
```

You can use a wildcard to match all of the non-internal indexes or all of the internal indexes. But, you can't use a wildcard to match both types of indexes at the same time.

Partition different searches using parentheses

You can also use parentheses to partition different searches to certain indexes. See Example 3 for details.

Note: When you type "index=" into the search bar, typeahead indicates all the indexes that you can search, based on your roles and permissions settings.

Examples

Example 1: Search across all public indexes.

```
index=*
```

Example 2: Search across all indexes, public and internal.

```
index=* OR index=_*
```

Example 3: Partition different searches to different indexes; in this example, you're searching three different indexes: main, _internal, and mail. You want to see events that match "error" in all three indexes; but also, errors that match "warn" in main or "failed" in mail.

```
(index=main (error OR warn)) OR (index=_internal error) OR (index=mail (error OR failed))
```

Example 4: Search across multiple indexes on different distributed Splunk servers.

```
(splunk_server=local index=main 404 ip=10.0.0.0/16) OR (splunk_server=remote index=mail user=admin)
```

Not finding the events you're looking for?

When you add an input, the input gets added relative to the app you're in. Some apps write input data to their own specific index (for example, the Splunk App for Unix and Linux uses the 'os' index).

Search across one or more distributed search peers

When performing a distributed search from a search head, you can restrict your searches to specific search peers (also known as "indexer nodes") by default and in your saved and scheduled searches. The names of your Splunk search peers are saved as values in the "splunk_server" field. For more information about distributed search, see "About distributed search" in the Distributed Search manual.

If no search peer is specified, your search accesses all search peers you have permission to access. The default peers that you can access are controlled by the roles and permissions associated with your profile and set by your Splunk admin. For more information, see ["About users and roles"](#) in *Securing Splunk Enterprise*.

The ability to restrict your searches to specific peers can be useful when there is high latency to certain search peers and you do not want to search them by default. When you specify one or more peers, those are the only servers that are included in the search.

You can specify different peers to search in the same way that you specify other field names and values. In this case, the field name is "splunk_server" and the field value is the name of a particular distributed peer:

```
splunk_server=<peer_name>
```

Note: You can use the value "local" to refer to the Splunk instance that you are searching from; in other words, the search head itself.

```
splunk_server=local
```

Keep in mind that **field names are case sensitive**; Splunk will not recognize a field name if the case doesn't match.

Examples

Example 1: Return results from specified search peers.

```
error (splunk_server=NYsplunk OR splunk_server=CAsplunk) NOT splunk_server=TXsplunk
```

Example 2: Search different indexes on distributed search peers "foo" or "bar".

```
(splunk_server=foo index=main 404 ip=10.0.0.0/16) OR (splunk_server=bar index=mail user=admin)
```

Classify and group similar events

An event is not the same thing as an event type. An event is a single instance of data — a single log entry, for example. An event type is a classification used to label and group events.

The names of the matching event types for an event are set on the event, in a multivalue field called **eventtype**. You can search for these groups of events (for example, SSH logins) the same way you search for any field value.

This topic discusses how to classify events (save a search as an event type) and search for tagged fields. For more information about events, how Splunk software recognizes them, and what it does when it processes them for indexing, see the [Overview of event processing](#) topic in the *Getting Data In* manual.

Important: You cannot save a search pipeline as an event type; that is, when saving a search as an event type, it cannot include a search command.

Save a search as a new event type

When you search your event data, you are essentially filtering out all unwanted events. The results of your search are events that share common characteristics, and you can give them a collective name.

For example, if you often search for failed logins on different host machines, you can save an event type for the events and call it **failed_login**:

```
"failed login" OR "FAILED LOGIN" OR "Authentication failure" OR "Failed to authenticate user"
```

To save this search as an event type:

1. Click **Save As** and select **Event Type**.
2. In **Save As Event Type** window, give your event type a **Name**.
In this example the name is **failed_login**.

Save As Event Type X

Name

Tags

Color

Priority
Determines which style wins, when an event has more than one event type.

You can add a list of tags that should be applied to the event type in the **Tag(s)** field. For more about tags see the section **Use tags to group and find similar events** below.

3. Click **Save** to save your event type name.

Now, you can quickly search for all the events that match this event type the same way you can search for any field, by specifying the event type in your search criteria.

For example, you might be interested in finding failed logins on specific host machines. Your search might look something like this:

```
host=target eventtype=failed_login
```

Or you might want to investigate suspicious user activity. Your search might look something like this:

```
user=suspicious eventtype=failed_login
```

Use findtypes to discover new event types

Pass any of your searches into the **findtypes** command to display suggestions for event types. The **findtypes** command compares the events resulting from the search and groups those events that have similar punctuation and terms together.

For more information and examples, see ["findtypes"](#) in the search command reference.

Use tags to group and find similar events

In your data, you might have groups of events with related field values. To help you search more efficiently for these groups of fields, you can assign tags to their field values. You can assign one or more tags to any extracted field (including event type, host, source, or source type).

Event types can have one or more tags associated with them. You can add these tags while you save a search as an event type and from the event type manager, located in **Manager > Event types**. From the list of event types in this window, select the one you want to edit.

After you add tags to your event types, you can search for them in the same way you search for any tag. Let's say you saved a search for firewall events as the event type **firewall_allowed**, and then saved a search for login events as the event type **login_successful**. If you tagged both of these event types with *allow*, all events of either of those event types can be retrieved by using the search:

```
tag::eventtype="allow"
```

You can tag field/value pairs. You can also alias field names. See [Tag field value pairs in Search](#) and [Create field aliases in Splunk Web](#).

Search for tagged field values

There are two ways to search for tags. If you are searching for a tag associated with a value on any field, you can use the following syntax:

```
tag=<tagname>
```

Or, if you are looking for a tag associated with a value on a specific field, you can use the following syntax:

```
tag::<field>=<tagname>
```

Use wildcards to search for tags

You can use the asterisk (*****) wildcard when searching keywords and field values, including for eventtypes and tags.

For example, if you have multiple event-type tags for various types of IP addresses, such as **IP-src** and **IP-dst**, you can search for all of them with:

```
tag::eventtype=IP-*
```

If you wanted to find all hosts whose tags contain "local", you can search for the tag:

```
tag::host=*local*
```

Also, if you wanted to search for the events with eventtypes that have no tags, you can search for the Boolean expression:

```
NOT tag::eventtype=*
```

Use the timeline to investigate events

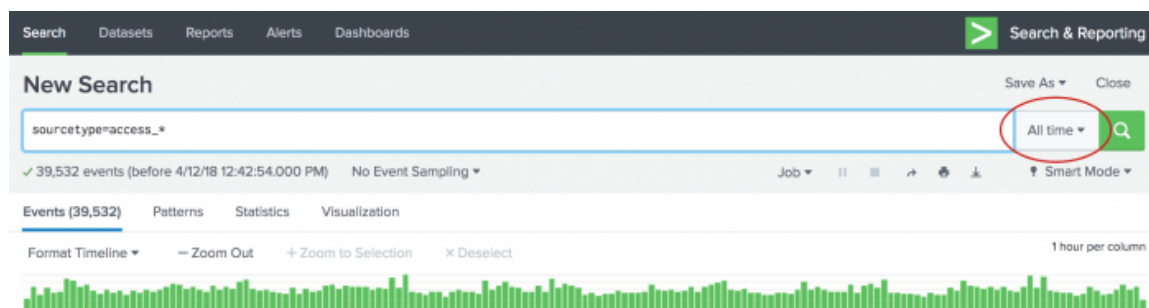
The timeline is a visual representation of the number of events in your search results that occur at each point in time. The timeline shows the distribution of events over time.

Note: When you use the timeline to investigate events, you are not running a new search. You are filtering the existing search results.

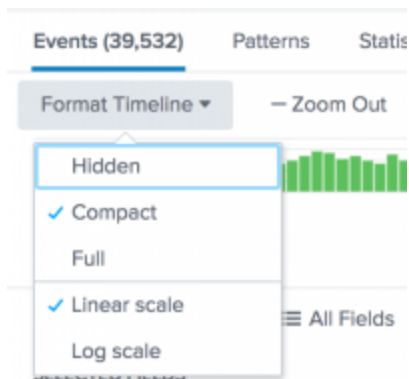
You can use the timeline to highlight patterns or clusters of events or investigate peaks (spikes in activity) and lows (possible server downtime) in event activity. Position your mouse over a bar to see the count of events. Click on a bar to drill-down to that time range.

Change the timeline format

The timeline is located in the Events tab above the events listing. It shows the count of events over the time range that the search was run. Here, the timeline shows web access events over **All time**.



Format options are located in the **Format Timeline** menu:



You can hide the timeline, or display a Compact or Full view of the timeline. You can also toggle the timeline scale between Linear scale or Log scale (logarithmic).

When **Full** is selected, the timeline view is taller to accommodate the labels on the axis. The count is on the Y-axis and time is on the X-axis.

Zoom in and zoom out to investigate events

Above the timeline are the zoom options. By default, the timeline is zoomed in. The following image shows the timeline display in Full view and zoomed in. The **Zoom Out** option is available.



Timeline legend

The timeline legend is on the top right corner of the timeline. This indicates the scale of the timeline. For example, **1 hour per column** indicates that each column represents a count of events during that hour. Zooming in and out changes the time scale.

Zoom in

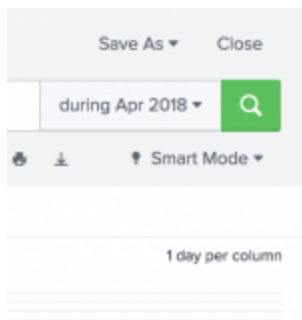
To zoom in on one or more columns in the timeline, you can either click on the columns and select **Zoom to Selection** or you can change the time range to a smaller time range in the Time Range Picker.

The smallest time unit that you can zoom in to is 1 millisecond.

Zoom out

When you click **Zoom Out**, the legend indicates that each column now represents **1 day per column** instead of an hour.

Zooming out changes not only the timeline but the value in the Time Range Picker.

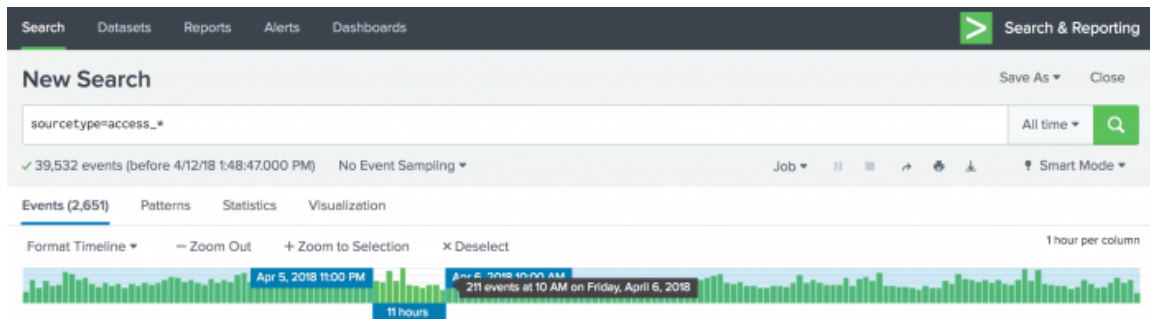


Reset the zoom

To reset the zoom or to zoom in, change the value in the Time Range Picker. For example, if you searched using **All time** and then zoomed out, select **All time** in the Time Range Picker to return to the original timeline time scale.

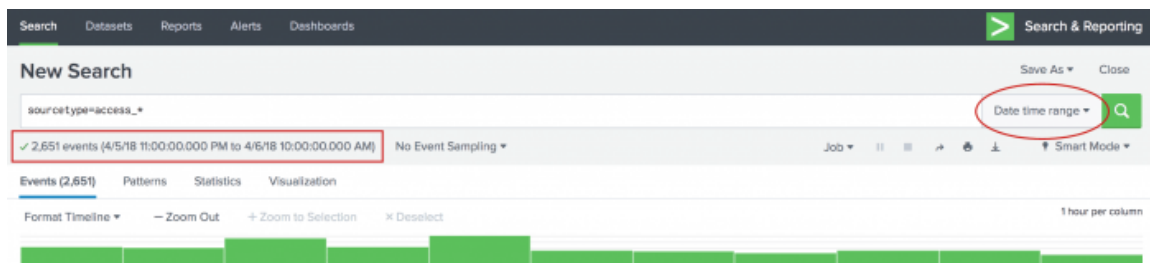
Zoom to a selection

When you mouse over and select bars in the timeline, the **Zoom to Selection** or **Deselect** options above the timeline become available.



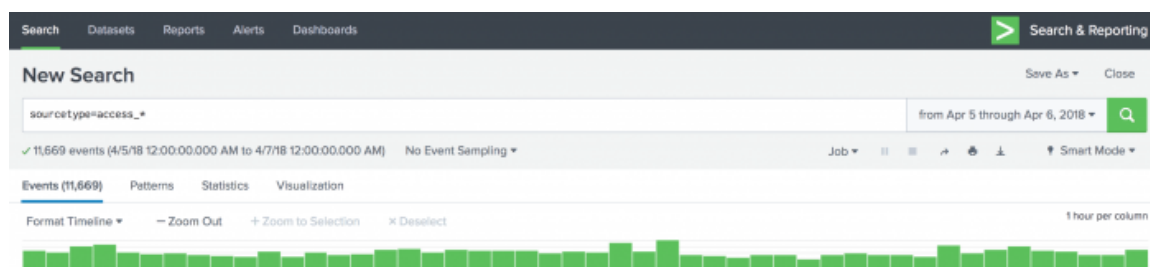
Mouse over and click on one of the bars or drag your mouse over a cluster of bars in the timeline. The events list updates to display only the events that occurred in that selected time range. The time range picker also updates to the selected time range. You can cancel this selection by clicking **Deselect**.

When you select a set of bars on the timeline and click **Zoom to Selection**, your search results are filtered to show only the selected time period. The timeline and events list update to show the results of your selection.



The dates and times that correspond to the bars you selected, along with the number of events in that time range, is reflected in the information just below the Search bar.

You cannot **Deselect** after you zoomed into a selected time range. But, you can **Zoom Out** again or change the time in the Time Range Picker.



Drill down on event details

After running a search that returns events in the **Events** tab, click on parts of those events to run different kinds of secondary **drilldown** searches that use the event detail that you have selected.

In the **Events** tab you can run a drilldown search when you click on these parts of an event:

- **Field value.** Fields are **extracted** from events at index time and search time. See ["About fields"](#) in the *Knowledge Manager Manual*.
- **Tag.** A **tag** is associated with one or more field-value pairs. A field-value pair can be associated with multiple tags. See [About tags and aliases](#) in the *Knowledge Manager Manual*.
- **Segment** (can be a connected string of segments). A **segment** is a searchable part of an event. See ["About event segmentation"](#) in the *Getting Data In Manual* to learn how segments are configured and created.
- **Timestamp.** A **timestamp** is date and time information in an event.

Drilldown search actions

The drilldown searches can perform the following actions for fields, tags, and event segments.

Drilldown search action	Description	Result
Add to search	Add a focus on the selected event detail to the original search string and run it. Splunk software discards transforming commands from the search string, as well as anything that follows them.	A dataset similar to the one returned by the original search, filtered to include only events that have the selected field, tag, or segment(s).
Exclude from search	Add an exclusion of the selected event detail to the original search and run it. Splunk software discards transforming commands from the search string, as well as anything that follows them.	A dataset similar to the one from the original search, filtered to include only events that do not have the selected field, tag, or segment(s).
Remove from search	Run a new search that is identical to the original search, except that it no longer searches for the highlighted field value, tag, or segment. Splunk software discards transforming commands from the search string, as well as anything that follows them.	A new dataset that is not as restrictive as the dataset returned by the original search. It no longer excludes events without the highlighted field value, tag, or segment.
New search	Run a new search that focuses exclusively on the selected field, tag, or segment(s).	A new dataset that contains any event which includes the selected field, tag, or segment(s).

Examples of drilldown search actions

These examples use the sample data from the Search Tutorial but should work with any format of Apache web access log. To try these examples on your own Splunk instance, you must download the sample data and follow the instructions to [get the tutorial data into Splunk](#). Use the time range **Last 7 days** when you run the searches.

Add to search

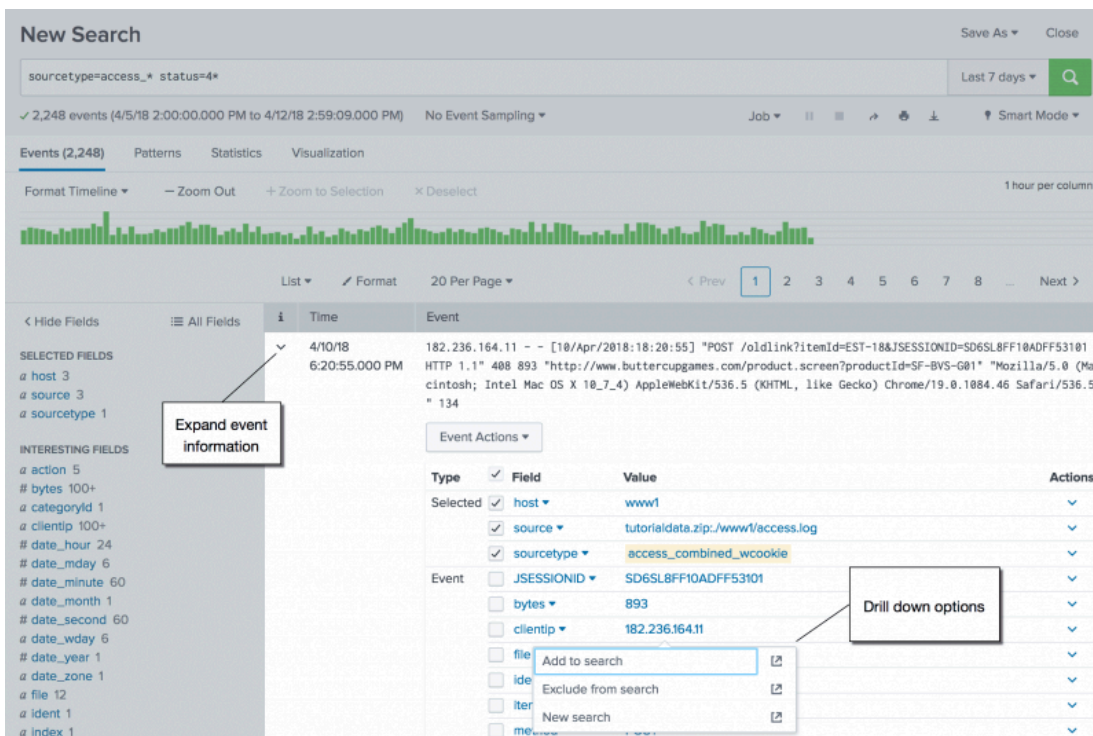
1. Run a search or report that returns an event listing in the **Events** tab.

If your search includes **transforming commands**, set the **Search Mode** to **Verbose**.

Let's start with a basic search that searches the Apache web access log files and looks for HTTP errors that start with 4, such as 404.

```
sourcetype=access_* status=4*
```

2. In the search results, expand the event information for the first event.
3. Select the **clientip** field value. In this example the value is **182.236.164.11**.
Because the **clientip** field is not currently in the search, the options are: **Add to search**, **Exclude from search**, and **New search**.



The screenshot shows the Splunk Search interface. At the top, the search bar contains the query `sourcetype=access_* status=4*` and the time range is set to `Last 7 days`. Below the search bar, a timeline view shows the distribution of events over time. The **Events (2,248)** tab is selected, and the **Format Timeline** view is active. The event list shows the first event expanded, displaying fields like `host`, `source`, `sourcetype`, `JSESSIONID`, `bytes`, and `clientip`. The `clientip` field value `182.236.164.11` is highlighted, and a dropdown menu is shown with options: **Add to search**, **Exclude from search**, and **New search**. Annotations point to 'Expand event information' and 'Drill down options'.

Field values, tags, and segments that are explicitly included in the search string appear with yellow highlighting in the event information.

4. Click **Add to search**. The field-value pair is added to the search. The search is run using the same time range. For example:

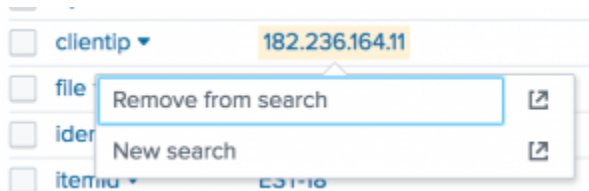
```
sourcetype=access_* status=4* clientip="182.236.164.11"
```

Remove from search

You can use the **Remove from search** drilldown option to remove specific criteria. You cannot remove criteria that includes a wildcard.

Continuing with the same search, let's remove the `clientip` from the search.

1. To remove the criteria from the search, expand the event information for the first search result.
2. Select the value for the `clientip` field. The drilldown options are **Remove from search** and **New search**.



3. Click **Remove from search**. The search is updated and runs using the same time range. For example:

```
sourcetype=access_* status=4*
```

Suppose the original search looks like this:

```
sourcetype=access_* status=404
```

Expand any event information that contains the `status` criteria. Click on the value `404` for the status field. From the drilldown options, select **Remove from search**.

Exclude from search

You can use the **Exclude from search** drilldown option to exclude a specific criteria from the search.

1. Expand the event information for a search result that contains the criteria that you want to exclude.
Suppose the original search look like this:

```
sourcetype=access_* status=4*
```

You want to exclude all **view** actions.

- For the **action** field, select the **view** value and click **Exclude from search**. The field-value pair is added to the search with the **not equal to** syntax. The search is run using the same time range. For example:

```
sourcetype=access_* status=4* action!=view
```

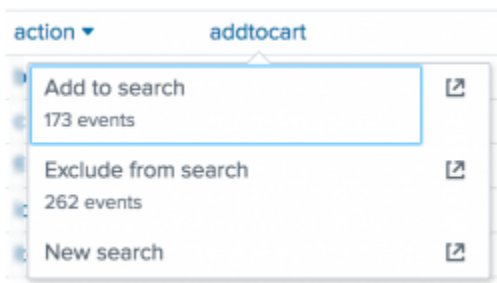
Selecting segments

To select event segments or sets of connected segments before you click on them, Splunk software identifies your segment selection with yellow highlighting.

Common values

If the value you select is among the top ten values found for the field, the **Add to search** and **Exclude from search** options show event numbers below the option names.

- The event number under **Add to search** indicates how many events contain the value for the field.
- The event number under **Exclude from search** indicates how many events do not contain the value for the field.



Run the drilldown search in a new tab

When you select one of the drilldown options, the search is run in the current tab and replaces your current search.

However, you can run the drilldown search in a new tab and leave your current search results intact. To run the drilldown search in a new tab, click the **Open In New Tab** icon for the option.



Drilldown searches for event timestamps

Event timestamp drilldown searches can help you find event correlations and perform root cause analysis.

Click on an event **timestamp** to run a secondary search that returns events which are chronologically close to that event.

When you open an event you can also click on the **_time** field to run this kind of drilldown search.

The controls for this search are called a **time accelerator**. See [Use time to find nearby events](#) in this manual for details on how the **_time** accelerator is used.

Identify event patterns with the Patterns tab

Events in search results can be grouped into **event patterns**. Events that belong to an event pattern share common characteristics, and usually can be returned by a specific search string. Event pattern analysis is useful for searches that return a diverse range of events because it quickly shows you the most common kinds of events in your search result dataset.

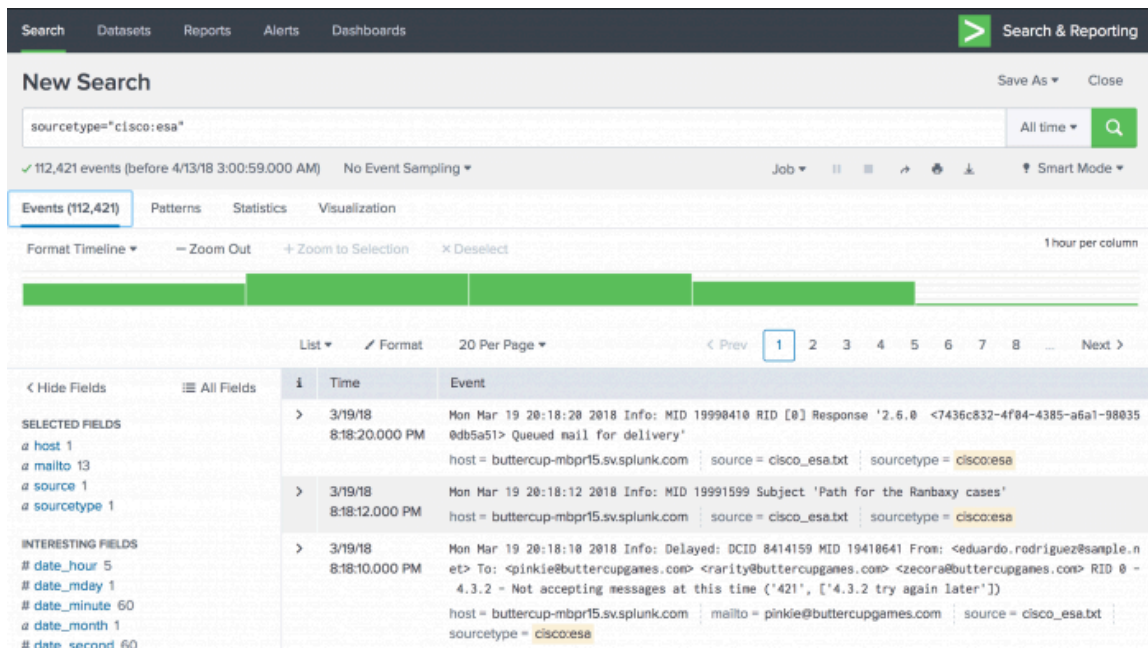
The Patterns tab simplifies event pattern identification. Click the Pattern tab to view a list of the most common patterns among the set of events returned by your search. Each of these patterns represents a set of events that share a similar structure.

Click on a pattern to:

- View the approximate number of events in your results that fit the pattern.
- View the search that returns events with this pattern.
- Save the pattern search as an event type, if possible. Not all event patterns can be saved as event types.
- Create an alert based on the pattern. For example, you can create alerts that trigger when certain patterns increase or decrease in frequency.

An event patterns example

A search that uses `sourcetype=cisco:esa` runs for All time and returns 112,421 events.

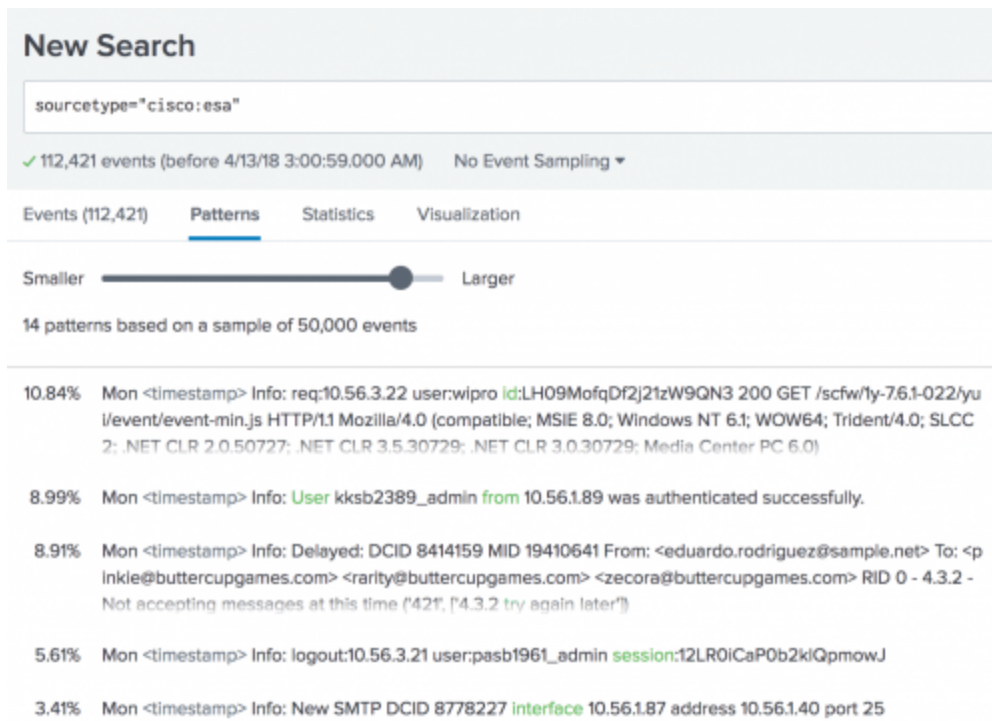


The screenshot shows the Splunk Search & Reporting interface. The search bar contains `sourcetype=cisco:esa` and the results are filtered for 'All time'. The search returned 112,421 events. The 'Patterns' tab is selected, showing a list of event patterns. The interface includes a timeline view at the top and a list of patterns below. The patterns are listed with their event counts and search strings.

Pattern	Events	Search
Mon Mar 19 20:18:20 2018 Info: MID 19998410 RID [0] Response '2.6.0 <7436c832-4f04-4385-a6a1-980350db5a51> Queued mail for delivery'	1	host = buttercup-mbpr15.svsplunk.com source = cisco_esa.txt sourcetype = cisco:esa
Mon Mar 19 20:18:12 2018 Info: MID 19991599 Subject 'Path for the Ranbaxy cases'	1	host = buttercup-mbpr15.svsplunk.com source = cisco_esa.txt sourcetype = cisco:esa
Mon Mar 19 20:18:10 2018 Info: Delayed: DCID 8414159 MID 19410641 From: <eduardo.rodriguez@sample.net> To: <pinkie@buttercupgames.com> <rarity@buttercupgames.com> <zecora@buttercupgames.com> RID 0 - 4.3.2 - Not accepting messages at this time ('421', ['4.3.2 try again later'])	1	host = buttercup-mbpr15.svsplunk.com mailto = pinkie@buttercupgames.com source = cisco_esa.txt sourcetype = cisco:esa

The patterns are based on a sample of 50,000 events.

To view all of the patterns in the list of events, click the **Patterns** tab. Initially the results identify 45 patterns. You can move the Smaller to Larger slider to change how narrow or broad the event patterns should be. If you drag the slider closer to the **Larger** side, 14 patterns are returned.



New Search

sourcetype="cisco:esa"

✓ 112,421 events (before 4/13/18 3:00:59.000 AM) No Event Sampling ▼

Events (112,421) **Patterns** Statistics Visualization

Smaller  Larger

14 patterns based on a sample of 50,000 events

- 10.84% Mon <timestamp> Info: req:10.56.3.22 user:wipro id:LH09MofqDf2j21zW9QN3 200 GET /scfw/ty-7.6.1-022/yl/event/event-min.js HTTP/1.1 Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1; WOW64; Trident/4.0; SLCC 2; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR 3.0.30729; Media Center PC 6.0)
- 8.99% Mon <timestamp> Info: User kksb2389_admin from 10.56.1.89 was authenticated successfully.
- 8.91% Mon <timestamp> Info: Delayed: DCID 8414159 MID 19410641 From: <eduardo.rodriquez@sample.net> To: <pinkie@buttercupgames.com> <rarity@buttercupgames.com> <zecora@buttercupgames.com> RID 0 - 4.3.2 - Not accepting messages at this time (42f; [4.3.2 try again later])
- 5.61% Mon <timestamp> Info: logout:10.56.3.21 user:pasb1961_admin session:12LR0iCaP0b2kiQpmowJ
- 3.41% Mon <timestamp> Info: New SMTP DCID 8778227 interface 10.56.1.87 address 10.56.1.40 port 25

The threat level event pattern is the most common one. Some of the listed patterns are relatively rare in this dataset, and finding them in the Events tab listing might be difficult. The Patterns tab makes it easier to see these event patterns and save them as event types if necessary.

How the Patterns tab works

When you click the Patterns tab, Splunk software runs a secondary search on a subset of the search results that have been received up to that point. This search job analyzes those results and derives the most common event patterns in those results. It then lists those patterns in descending order from most prevalent to least prevalent. It may not include outlier patterns that are based on extremely small groups of events, because they are statistically unreliable.

The secondary search can take a long time to complete when the original dataset contains an extremely large variety of event patterns. For example, some searches return datasets containing over 500 patterns, where most of those patterns represent very small collections of events. The algorithm that identifies these patterns is designed to avoid doing too much work for small patterns, but it also attempts to be as accurate as possible.

The Patterns tab only accepts **transforming searches** when you set their **search mode** to **Verbose**. The Patterns tab cannot find patterns for **real-time searches** in any search mode.

Event pattern keywords

The Patterns tab defines patterns by the presence or absence of one or more keywords. If the keywords identified for a pattern are added to or excluded from the original search, the search returns events that fit that pattern. Keywords present in a pattern are identified with green text in the pattern list. Excluded keywords are not identified in the event list.

In the preceding event pattern example, the threat level event pattern has "threat" as its keyword, meaning that the search that returns events fitting the pattern would look like this:

```
sourcetype=cisco_esa threat
```

If this event pattern were also identified by the exclusion of the keyword "verified," the search that returns events fitting the pattern would look like this:

```
sourcetype=cisco_esa threat ( NOT verified )
```

To see all of the keywords associated with a pattern, click on the pattern.

Use the Patterns tab

1. From the Search view, run a search that returns more than 5000 results.

Searches returning more than 5000 results produce reliable patterns.

2. Click the Patterns tab.

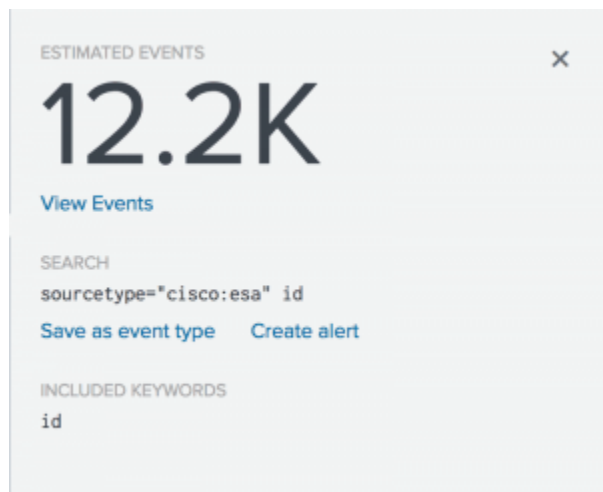
You do not need to wait for the search to complete, but the pattern listing is more accurate with finalized search results.

3. (Optional) If there appear to be too many patterns or too few, or if you do not see the patterns you expect, move the slider.

Dragging the slider to **Larger** runs a secondary search job that consolidates some patterns together, resulting in event patterns that represent larger numbers of events, and a wider variety of events inside each pattern group.

Dragging the slider to **Smaller** runs a secondary search job that increases the granularity of the results. The event patterns it finds represent smaller numbers of events.

4. (Optional) Click on a pattern to view information for that pattern.



Estimated Events is the estimated count of events in the dataset returned by the original search that fit the event pattern. In this example, the original search had 112k events. This pattern accounts for an estimated 12,200 events or 10.84% of the total number of events.

Included Keywords identifies keywords that should be added to the base search to return the pattern. If the Patterns tab identifies keywords that should be excluded from the base search, they appear under an **Excluded Keywords** section.

You can see the search that returns events fitting the event pattern under **Search**.

5. (Optional) In the pattern information area, click **View Events** to run the search displayed under **Search**.

When it runs, this search uses the same time range as your original search.

6. (Optional) In the pattern information area, click **Save as event type** to save the search as an **event type**.

Save as event type is available only for event patterns based on searches that do not include pipe characters and additional search commands. See "[About event types](#)" in this manual.

7. (Optional) In the pattern information area, click **Create alert** to create an **alert** based on the pattern.

For example, create a scheduled alert that is triggered when the frequency of the event pattern rises above or drops below a threshold. If you know that events that fit an event pattern tend to appear at a steady rate of approximately 100 events per hour, set the alert to run on an hourly schedule and trigger when 150 or more events are returned. See the [Alerting Manual](#).

Numbers in the Patterns tab

When the secondary search finishes, the Patterns tab displays a message explaining how many events it analyzed to obtain the displayed results.

The Patterns tab analyzes a subset of the total number of events returned by the original search. The maximum number of events in this subset is 50k. This maximum reduces processing times for the Pattern tab secondary search. If your original search returns less than 50k events, the secondary search analyzes up to 1000 events per timeline bar spanned by the original search. For example, if the original search spans 14 timeline bars, the secondary search analyzes 14,000 events to obtain its patterns listing.

You can control the maximum number of events analyzed by the secondary search by updating the **maxevents** setting, in the [findkeywords] stanza, in **limits.conf**. This setting defaults to **50000**. Do not change this value. A number less than 50,000 reduces the accuracy of your events. A number higher than 50,000 increases the processing time required by the secondary search.

The estimated event count provided in the pattern information area does not apply to the number of events analyzed in the Patterns tab secondary search. It applies to the total number of events returned by the original search. If an event pattern is estimated to represent 7,350 events and the original search returned 265k events, the pattern accounts for 2.7% of the events returned by the search.

Restricting Patterns tab usage

All roles, including the user role, have permission to use the Patterns tab by default. To restrict usage of the Patterns tab, remove the **pattern_detect capability**. Roles without this capability do not see the Patterns tab option after they run a search.

For more information about capabilities, see "[About defining roles with capabilities](#)" in the *Securing Splunk* manual.

Preview events

In a distributed environment, by default, when you run a search the results are not displayed until all of the search peers begin to return event data for the time range that you specify. In a distributed environment with a large number of peers, or where some of the peers are slow, there can be a delay in displaying search results.

The events preview mode displays an event as soon as the event is returned, instead of waiting until all of the events are returned to see the search results. This mode displays events that are in-memory and not yet committed.

Limitations using the preview mode

There are some limitations in the Events viewer when you enable the events preview mode.

You cannot expand the Events viewer to see detailed information about an event until all of the events from your search are returned. When you position your mouse over the information icon, a message informs you that the events preview mode is enabled.

As results are returned and displayed in the Events viewer, the order of the events changes. As new results are added to the Events viewer, the events are inserted into the correct time order.

The Events viewer provides the option to display events in a list, as a table, or as raw events. When the Events viewer is set to **Table** and the events preview mode is enabled, you cannot sort the list of events until the search completes.

Enable events preview mode

To see the events more quickly, you can enable the events preview mode in the Search app.

When you enable events preview, it is enabled for everyone using the Search app, not just for you.

Splunk Cloud Platform

To enable the events preview mode request help from Splunk Support. If you have a support contract, file a new case using the Splunk Support Portal at [Support and Services](#). Otherwise, contact [Splunk Customer Support](#).

Splunk Enterprise

Prerequisites

- Only users with file system access, such as system administrators, can enable the events preview mode.
- Review the steps in [How to edit a configuration file](#) in the *Admin Manual*.

Note: Never change or copy the configuration files in the **default** directory. The files in the **default** directory must remain intact and in their original location. Make the changes in the **local** directory.

Steps

1. Open the local **limits.conf** file for the Search app. For example, `$SPLUNK_HOME/etc/apps/<app_name>/local`.
2. Under the [search] stanza, set **timeline_events_preview** to **true**.

Specify Time Ranges

About searching with time

You can use time to narrow your search and to group events in your search by time.

How timestamps are used

Timestamp processing is a key step in **event processing**. Splunk software uses **timestamps** to:

- Correlate **events** by time
- Create the timeline histogram in Splunk Web
- Set time ranges for searches

Timestamps are stored in UNIX time

Regardless of how time is specified in your events, timestamps are converted to UNIX time and stored in the **_time** field when your data is indexed. If your data does not have timestamps, the time at which your data is indexed is used as the timestamp for your events.

UNIX time is the number of seconds that have elapsed since 00:00:00 Coordinated Universal Time (UTC), 1 January 1970. This moment in time is sometimes referred to as epoch time. UNIX time appears as a series of numbers, for example **1518632124**. You can use any UNIX time converter to convert the UNIX time to either GMT or your local time.

GMT and UTC

GMT (Greenwich Mean Time) is sometimes confused with UTC (Coordinated Universal Time). However GMT is a *time zone* and UTC is a *time standard*.

- GMT is a time zone officially used in some European and African countries as their local time. The time is displayed in either the 24-hour format (00:00-23:59) or the 12-hour format (00:00-12:00 AM/PM).
- UTC is a time standard that is the basis for time and time zones worldwide. No country uses UTC as a local time.
- Neither GMT nor UTC ever change for Daylight Saving Time (DST). However, some of the countries that use GMT switch to different time zones during their DST period. For example, the United Kingdom uses GMT for most of the year, but switches to British Summer Time (BST) during the summer months. BST is one hour ahead of GMT.

The **_time** field

The **_time** field appears in a human readable format in Splunk user interfaces. However, the values in the **_time** field are stored in UNIX time.

Specify narrow time ranges

Splunk user interfaces use a default time range when you create a search. This range helps to avoid running searches with overly-broad time ranges that waste system resources and produce more results than you really need.

Whether you are running a new search, a report, or creating a dashboard, it is important to narrow the time range to only the dates or times that you really need.

Time is also crucial for determining what went wrong. You often know when something happened, if not exactly what happened. Looking at events that happened around the same time that something went wrong can help correlate results and find the root cause of the problem.

See also

Related information

[Select time ranges to apply to your search](#)

[Specify time modifiers in your search](#)

[Specify time ranges for real-time searches](#)

[Use time to find nearby events](#)

[How time zones are processed by the Splunk platform](#)

[Date and time format variables](#) in the *Search Reference*

[Time modifiers](#) in the *Search Reference*

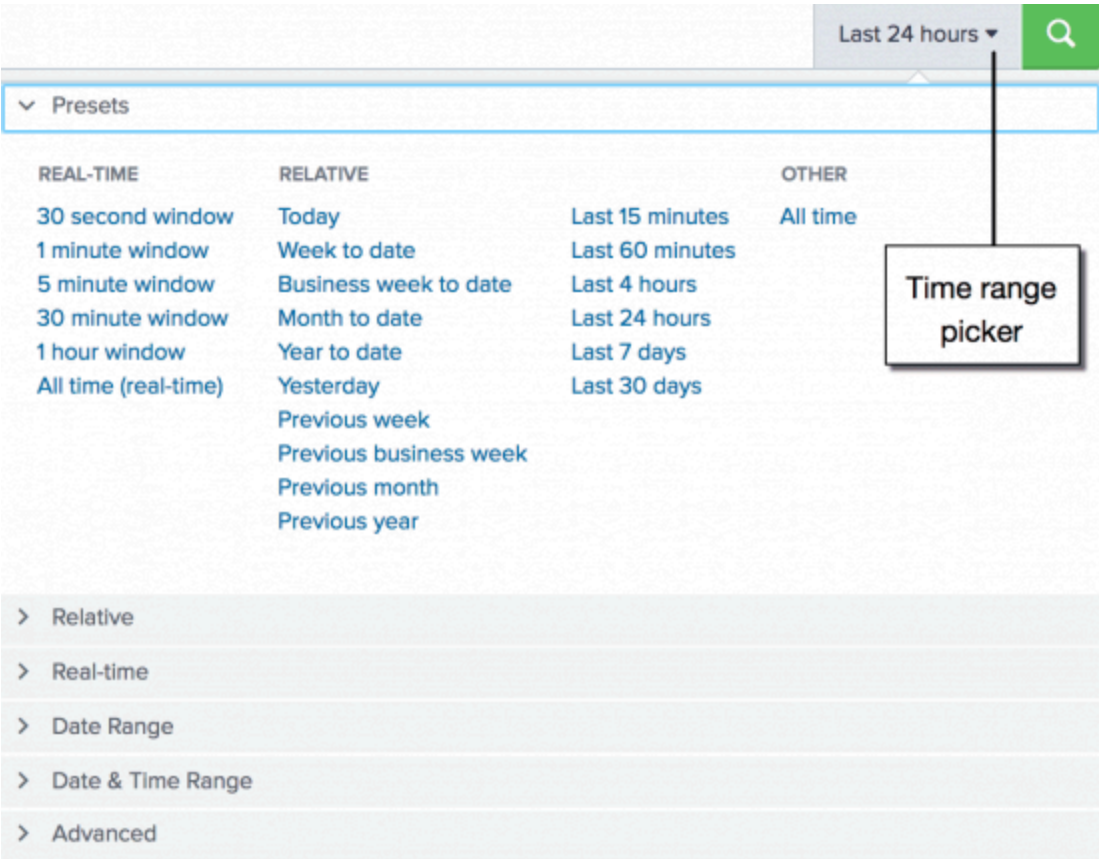
Select time ranges to apply to your search

Use the **time range picker** to set time boundaries on your searches. You can restrict a search with preset time ranges, create custom time ranges, specify time ranges based on date or date and time, or work with advanced features in the time range picker. These options are described in the following sections.

Note: If you are located in a different timezone, time-based searches use the timestamp of the event from the Splunk instance that indexed the data. See [How time zones are processed by the Splunk platform](#).

Select from a list of Preset time ranges

The time range picker includes many built-in time ranges options that are defined by default in the `times.conf` file. You can select from a list of Real-time windows, Relative time ranges, and search over All Time.



Real-Time Preset time ranges

CAUTION: The number of concurrent real-time searches can greatly affect indexing performance. See [About real-time searches and reports](#).

Note: Users must have the Admin role to run and save real-time searches. For more information on managing roles and assigning roles to users, see [Create and manage roles with Splunk Web](#) in *Securing Splunk Enterprise*.

The Real-Time Preset time ranges apply to real-time searches and are listed in the following table. To learn about relative time modifiers, see [Specify time modifiers in your search](#).

Real-Time Preset time range	Description	Equivalent relative time modifiers
30 second window	Events in the last 30 second window.	earliest_time = rt-30s latest_time = rt
1 minute window	Events in the last 1 minute window.	earliest_time = rt-1m latest_time = rt
5 minute window	Events in the last 5 minute window.	earliest_time = rt-5m

Real-Time Preset time range	Description	Equivalent relative time modifiers
		latest_time = rt
30 minute window	Events in the last 30 minute window.	earliest_time = rt-30m latest_time = rt
1 hour window	Events in the last 1 hour window.	earliest_time = rt-1h latest_time = rt
All time (real-time)	Total events for all real-time searches.	earliest_time = rt latest_time = rt

Relative Preset time ranges

The Relative Preset time ranges are listed in the following table. To learn more about relative time modifiers, see [Specify time modifiers in your search](#).

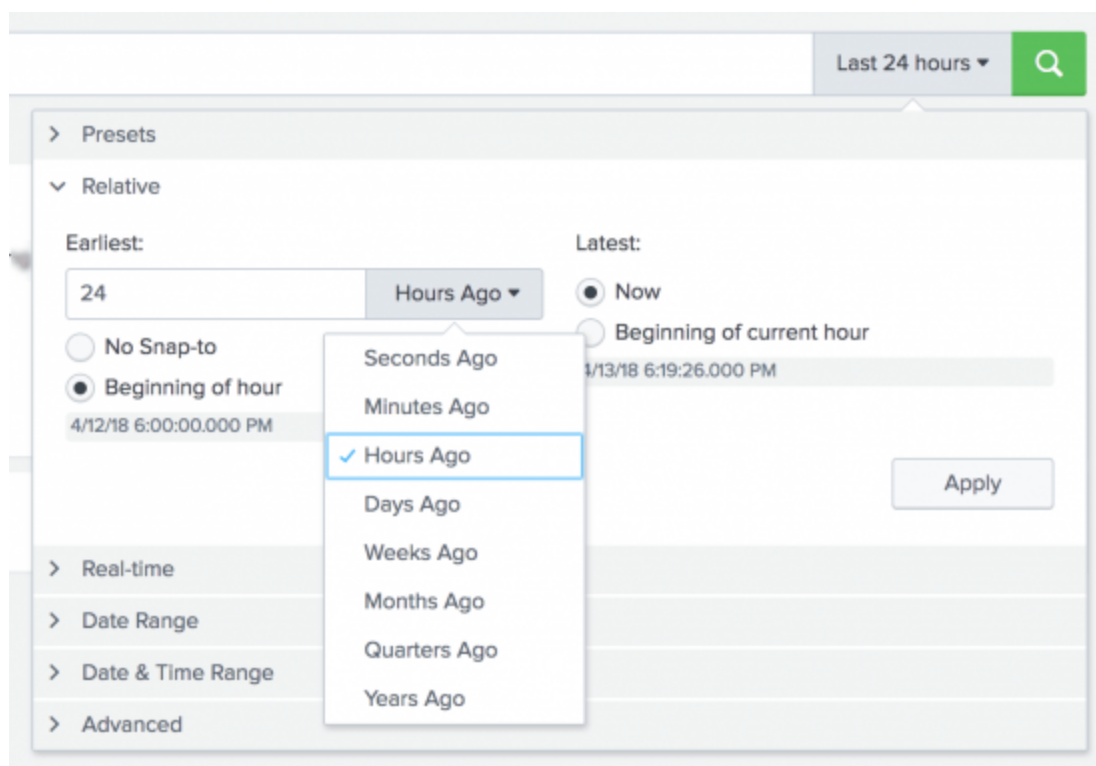
Relative Preset time range	Description	Equivalent relative time modifiers
Today	Events from today.	earliest_time = @d latest_time = now
Week to date	Events from this week to the current date.	earliest_time = @w0 latest_time = now
Business week to date	Events from this business week to the current date. Starts from the previous Monday at midnight (00:00:00) to now.	earliest_time = @w1 latest_time = now
Month to date	Events from this month to the current date.	earliest_time = @mon latest_time = now
Year to date	Events from this year to the current date.	earliest_time = @y

Relative Preset time range	Description	Equivalent relative time modifiers
		latest_time = now
Yesterday	Events from yesterday to today.	earliest_time = -1d@d latest_time = @d
Previous week	Events from the previous week.	earliest_time = -7d@w0 latest_time = @w0
Previous business week	Events from the previous business week. If you run a search with this time range on a Sunday, the earliest time value will be the previous Monday. However, if you run this time range on a Saturday, the earliest time will be Monday 2 weeks ago.	earliest_time = -6d@w1 latest_time = -1d@w6
Previous month	Events from the previous month.	earliest_time = -1mon@mon latest_time = @mon
Previous year	Events from the complete calendar year of the previous year. For example, if you run a search on any day in 2024, search results include events from Jan 1, 2023 at 00:00:00 to Jan 1, 2024 at 00:00:00.	earliest_time = -1y@y latest_time = @y
Last 15 minutes	Events from the last 15 minutes.	earliest_time = -15m latest_time = now
Last 60 minutes	Events from the last 60 minutes.	earliest_time = -60m@m latest_time = now
Last 4 hours	Events from the last 4 hours.	earliest_time = -4h@m

Relative Preset time range	Description	Equivalent relative time modifiers
		latest_time = now
Last 24 hours	Events from the last 24 hours.	earliest_time = -24h@h latest_time = now
Last 7 days	Events from the last 7 days.	earliest_time = -7d@h latest_time = now
Last 30 days	Events from the last 30 days.	earliest_time = -30d@d latest_time = now

Define custom Relative time ranges

Use Relative time range options to specify a custom time range for your search that is relative to Now or the Beginning of the current hour. You can select from the list of time range units: Seconds Ago, Minutes Ago, and so on.



Presets

Relative

Earliest:

24

Hours Ago ▾

Seconds Ago

Minutes Ago

✓ Hours Ago

Days Ago

Weeks Ago

Months Ago

Quarters Ago

Years Ago

Latest:

Now

Beginning of current hour

4/13/18 6:19:26.000 PM

No Snap-to

Beginning of hour

4/12/18 6:00:00.000 PM

Apply

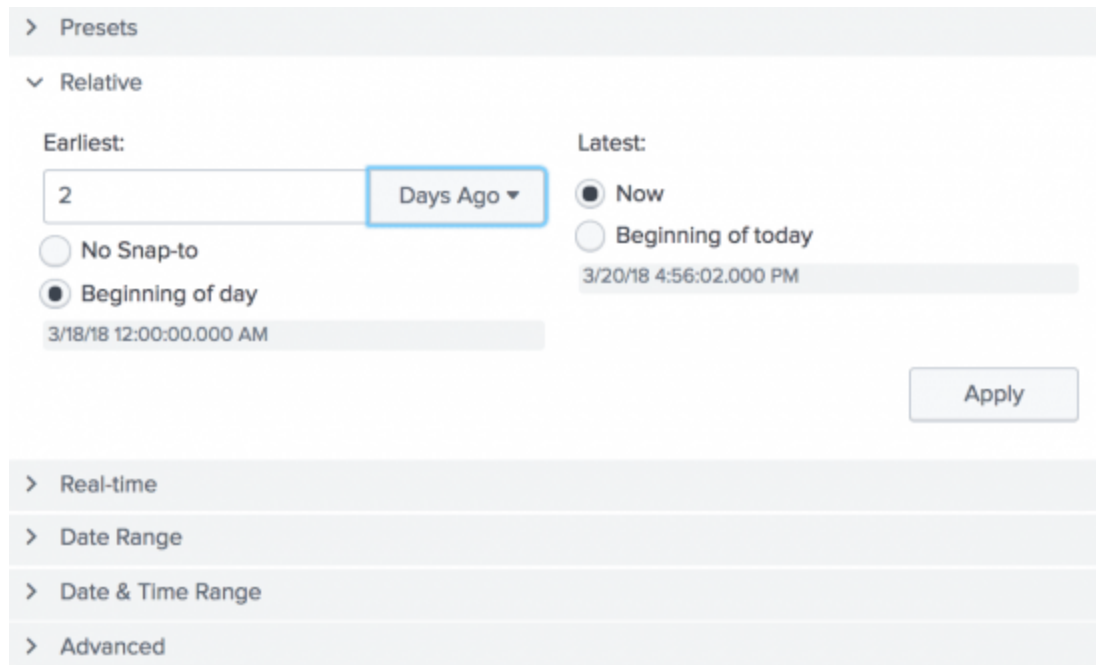
Real-time

Date Range

Date & Time Range

Advanced

By default, Earliest is set to **No Snap-to** and Latest is set to **Now**. If you specify the snap-to option for **Earliest** or **Latest**, the time range will snap to beginning of the time frame that you select. For example, if you select **Days Ago**, the Earliest snap to value is **Beginning of today**.



Presets

Relative

Earliest:

2

Days Ago ▾

Seconds Ago

Minutes Ago

Hours Ago

Days Ago

Weeks Ago

Months Ago

Quarters Ago

Years Ago

Latest:

Now

Beginning of today

3/20/18 4:56:02.000 PM

No Snap-to

Beginning of day

3/18/18 12:00:00.000 AM

Apply

Real-time

Date Range

Date & Time Range

Advanced

The preview boxes below the fields update to the time range as you make the selections.

To learn more about relative time ranges, see [Specify time modifiers in your search](#).

Define custom Real-time time ranges

Note: Users must have the Admin role to run and save real-time searches. For more information on managing roles and assigning roles to users, see [Create and manage roles with Splunk Web](#) in *Securing Splunk Enterprise*. In Splunk Cloud Platform on Victoria Experience, real-time searches are enabled by default. In Splunk Cloud Platform on Classic Experience, you must open a support ticket to enable real-time search. For more information, see [About real-time searches and reports](#) in the *Search Manual*.

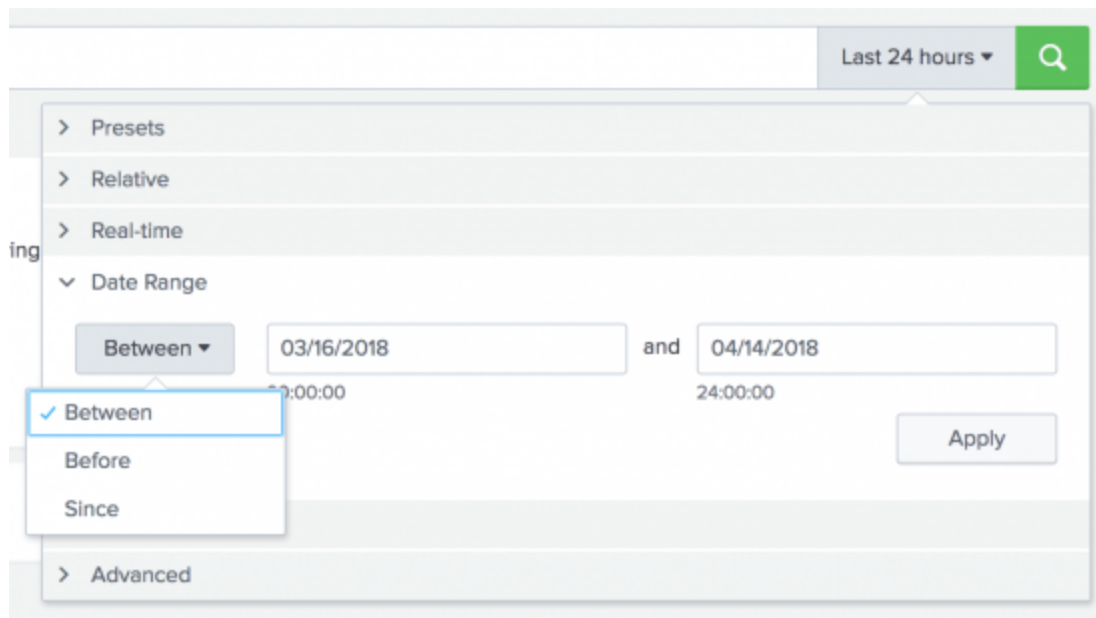
Users can use the real-time option to specify a custom Earliest time for a **real-time search**. Because this time range is for a real-time search, a Latest time is not relevant.

The screenshot shows the Splunk search interface. At the top right, there is a dropdown menu set to "Last 24 hours" and a green search button with a magnifying glass icon. Below this, a sidebar menu on the left lists search options: "Presets", "Relative", "Real-time" (which is selected and highlighted with a blue border), "Date Range", "Date & Time Range", and "Advanced". The "Real-time" section is expanded, showing two input fields: "Earliest:" and "Latest:". The "Earliest:" field contains the value "24" and a dropdown menu set to "Hours Ago". Below this field, a timestamp "4/13/18 9:53:05.000 PM" is displayed. The "Latest:" field contains the value "now". An "Apply" button is located at the bottom right of the "Real-time" section.

To learn more about time ranges for real-time searches, see [Specify real-time time range windows in your search](#).

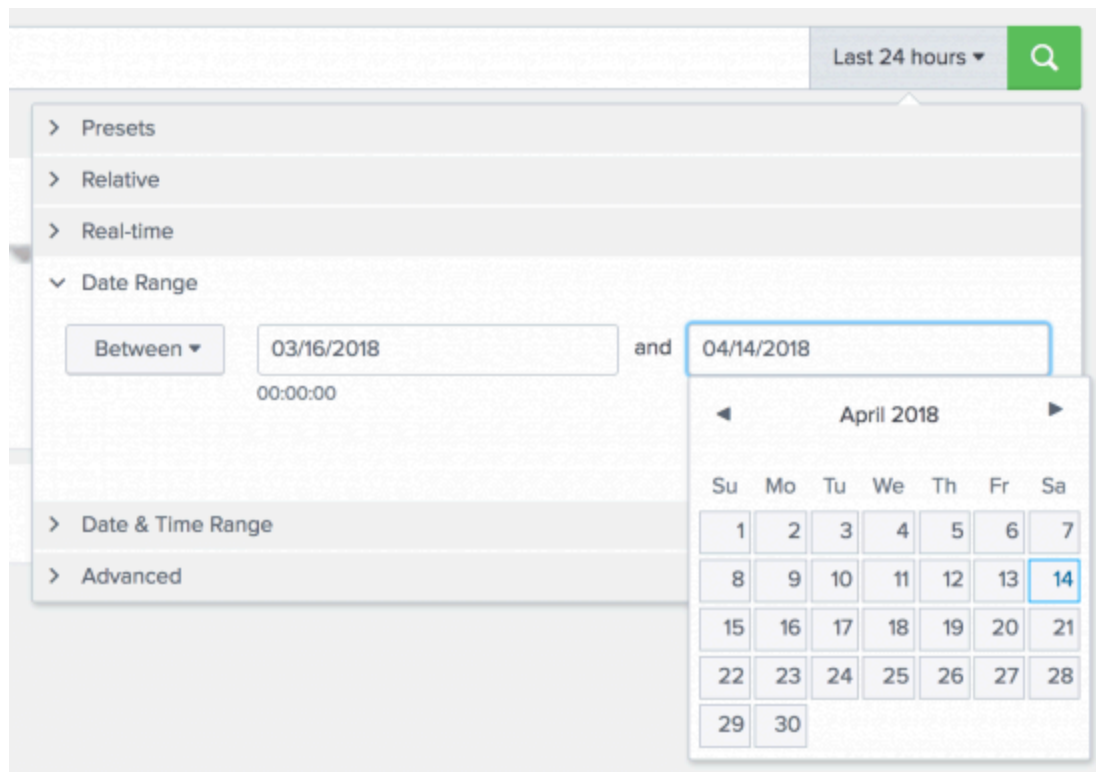
Define custom Date ranges

Use the Date Range option to specify custom calendar dates in your search. You can choose among options to return events: **Between** a beginning and end date, **Before** a date, and **Since** a date.



The screenshot shows the Splunk search interface with the 'Last 24 hours' filter and a search button. The 'Date Range' section is expanded, showing a dropdown menu with options: 'Between' (selected), 'Before', and 'Since'. The date range is set to '03/16/2018' and '04/14/2018' with a time range of '00:00:00' to '24:00:00'. An 'Apply' button is visible.

For these fields, you can type the date into the text box or select the date from a calendar.



The screenshot shows the Splunk search interface with the 'Date Range' section expanded. The date range is set to '03/16/2018' and '04/14/2018' with a time range of '00:00:00'. A calendar overlay is visible, showing the month of April 2018. The date '14' is selected in the calendar.

Su	Mo	Tu	We	Th	Fr	Sa
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30					

Define custom Date & Time ranges

Use the Date & Time Range option to specify custom calendar dates and times for the beginning and ending of your search.

The screenshot shows the Splunk search bar with a dropdown menu open. The menu has several options: Presets, Relative, Real-time, Date Range, and Date & Time Range. The Date & Time Range option is expanded, showing a sub-menu with three options: Between, Before, and Since. The Since option is selected and highlighted with a blue border. In the background, the search bar shows a date range from 04/13/2018 22:00:00.000 to (up to now). The format HH:MM:SS.SSS is displayed below the time field. An Apply button is visible at the bottom right of the menu.

You can type the date into the text box or select the date from a calendar.

Use Advanced time range options

Use the Advanced option to specify the earliest and latest search times. You can write the times in UNIX time or relative time notation, such as `-3d@d`. The UNIX time value you type is converted to local time.

The UNIX time or relative time that you specify is displayed as a timestamp under the text field so that you can verify your entry.

The screenshot shows the Splunk search bar with a dropdown menu open. The menu has several options: Presets, Relative, Real-time, Date Range, Date & Time Range, and Advanced. The Advanced option is expanded, showing two input fields: Earliest and Latest. The Earliest field contains the relative time notation `-3d@d`, and the Latest field contains `-1h@h`. Below each input field, the corresponding UNIX timestamp is displayed: 4/11/18 12:00:00.000 AM for Earliest and 4/14/18 9:00:00.000 PM for Latest. A Documentation link and an Apply button are visible at the bottom right of the menu.

Customize the list of Preset time ranges

You can customize the set of time ranges that appear in the **Presets** list the time range picker in Splunk Web. You can create a time range based on an existing time range, or you can hide time ranges.

Create a time range based on an existing time range

The easiest way to create a new time range is to use an existing time range as the basis for a new time range. For example, the Relative time range list contains the **Last 15 minutes** time range. You want to create a time range for the last 30 minutes. You start by creating a duplicate, or clone, of the **Last 15 minutes** time range. In the clone, you change the **Earliest** setting from -15min to -30min.

1. From the **Settings** menu, under the Knowledge list select **User interface**.
2. In the User Interface window, select **Time ranges**.
3. Locate the time range that you want to use.
4. In the Actions column click **Clone**.
5. A copy of the specifications for the time range appear. Make the changes to the time range specifications and click **Save**.

The new time range appears in the Relative list in the Presets menu.

Create a new Preset time range

You can create a new time range for the Presets menu. For example, you want to create a time range that shows searches yesterday from the hours of 12:00 to 15:00. You need to specify relative times in the Earliest and Latest fields. In the **Earliest** field you specify `-1d@d+12h`. In the **Latest** field you specify `-1d@d+15h`.

1. From the **Settings** menu, under the Knowledge list select **User interface**.
2. In the User Interface window, select **Time ranges**.
3. Click **New**.
4. Complete the fields in the Add New window and click **Save**.

The new time range appears in the Relative list in the Presets menu.

Hide a time range on the Presets list

1. From the **Settings** menu, under the Knowledge list select **User interface**.
2. In the User Interface window, select **Time ranges**.
3. Locate the time range you want to hide. In the Status column click **Disable**.

Setting default time ranges for the API or CLI

You can set time ranges manually in the `times.conf` file when you want to specify a time range for a REST API endpoint or for the command line interface (CLI).

Splunk Cloud Platform

To set the default time ranges for the API, request help from Splunk Support. If you have a support contract, file a new case using the Splunk Support Portal at [Support and Services](#). Otherwise, contact [Splunk Customer Support](#). Splunk Cloud Platform users don't have shell access to the Splunk Cloud Platform deployment and can't use the CLI to set default time ranges.

Splunk Enterprise

Prerequisites

- Only users with file system access, such as system administrators, can change time ranges manually in the `times.conf` file.
- Review the steps in [How to edit a configuration file](#) in the *Admin Manual*.

Note: Never change or copy the configuration files in the `default` directory. The files in the `default` directory must remain intact and in their original location. Make the changes in the `local` directory.

Steps

1. Open the local `times.conf` file for the Search app. For example, `$SPLUNK_HOME/etc/apps/<app_name>/local`.
2. Create a stanza for the time range that you want to specify. For examples, see the [times.conf reference](#) in the *Admin Manual*.

Change the default time range

The default time range for ad hoc searches in the Search & Reporting App is set to **Last 24 hours**.

In Splunk Enterprise, an administrator can set the default time range globally, across all apps. See [Change default values](#) in the Splunk Enterprise *Admin Manual*.

In Splunk Cloud Platform, contact Splunk customer support to request a change to the default time range.

See also

Related information

[About searching with time](#)

[Specify time ranges for real-time searches](#)

[How time zones are processed by the Splunk platform](#)

Specify time modifiers in your search

When searching or saving a search, you can specify absolute and relative time ranges using the following time modifiers:

```
earliest=<time_modifier>
```

```
latest=<time_modifier>
```

An absolute time range uses specific dates and times, for example, from 12 A.M. April 1, 2024 to 12 A.M. April 13, 2024.

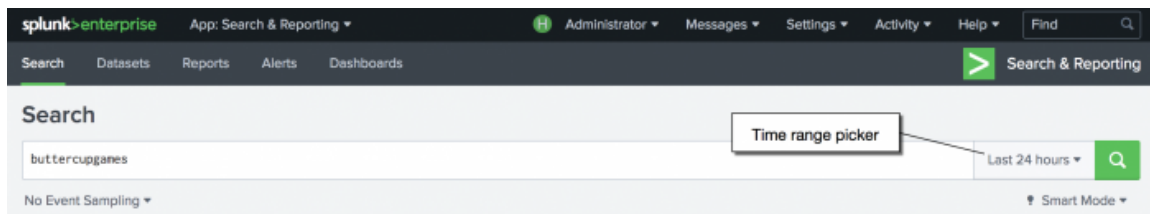
A relative time range is dependent on when the search is run. For example, a relative time range of -60m means 60 minutes ago. If the current time is 3 P.M., the search returns events from the last 60 minutes, or 2 P.M. to 3 P.M. today.

The current time is referred to as "now".

Note: `earliest` and `latest` time modifiers for real-time search that use the `rt` prefix, for example `earliest=-rt-5m latest=rt`, are not intended for use in search strings. Specifying `earliest` and `latest` time modifiers using real-time modifier syntax in a search string returns an error. For more information, see [Specify time ranges for real time searches](#).

Time modifiers and the Time Range Picker

A time range that you specify in the Search bar, or in a saved search, overrides the time range that is selected in the Time Range Picker.



For example, if you specify a time range of **Last 24 hours** in the Time Range Picker and in the Search bar you specify `earliest=-30m latest=now`, the search only looks at events that have a timestamp within the last 30 minutes.

This applies to any of the options you can select in the Time Range Picker,

However, this does not apply to subsearches.

Time ranges and subsearches

Time ranges selected from the Time Range Picker apply to the main search and to subsearches, unless a time range is specified in the Search bar.

Time ranges that you specify directly in the Search bar apply only to that portion of the search.

- The time ranges specified in the main search do not apply to subsearches.
- Time time ranges specified in a subsearch applies only to that subsearch. The time range does not apply to the main search or any other subsearch.

Specify absolute time ranges

For exact time ranges, the syntax for the time modifiers is `%m/%d/%Y:%H:%M:%S`. For example, the following search specifies a time range from 12 A.M. April 19, 2022 to 12 A.M. April 27, 2022.

```
earliest=04/19/2022:00:00:00 latest=04/27/2022:00:00:00
```

If you specify only the **earliest** time modifier, **latest** is set to the current time **now** by default. If you specify a **latest** time modifier, you must also specify an **earliest** time.

Specify relative time ranges

You define the relative time in your search by using a string of characters that indicate the amount of time. The syntax is an integer and a time unit.

1. Begin your string with a minus (-) or a plus (+) to indicate the offset before or after the time amount.
2. Specify the amount of time by using a number and a time unit. When you specify single time amounts, the number is implied. For example **s** is the same as **1s**, **m** is the same as **1m**, and so on. The supported time units are listed in the following table.

Time range	Valid values
seconds	s, sec, secs, second, seconds
minutes	m, min, mins, minute, minutes
hours	h, hr, hrs, hour, hours
days	d, day, days
weeks	w, week, weeks
months	mon, month, months
quarters	q, qtr, qtrs, quarter, quarters
years	y, yr, yrs, year, years

When specifying relative time, use **now** to refer to the current time.

Relative time modifiers that snap to a time

With relative time, you can specify a **snap to** time, which is an offset from the relative time. The snap to time unit rounds down to the nearest or latest time for the time amount that you specify. To do this, separate the time amount from the snap to time unit with an "@" character.

The syntax for the snap to time unit is **[+|-]<time_integer><time_unit>@<time_unit>**.

When snapping to the nearest or latest time, Splunk software always snaps backwards or rounds down to the latest time that is not after the specified time. For example, the current time is 15:45:00 and the snap to time is **earliest=-h@h**. The time modifier snaps to 14:00.

You can also define the relative time modifier using only the snap to time unit. For example, to snap to a specific day of the week, use `@w0` for Sunday, `@w1` for Monday, and so forth. For Sunday, you can specify either `w0` or `w7`.

If you do not specify a snap to time unit, the search uses seconds as the snap to time unit.

The snap to option becomes very useful in a range of situations. For example, if you want to search for events in the previous month, specify `earliest=-mon@mon latest=@mon`. This example begins at the start of the previous month and ends at the start of the current month.

Difference between relative time and relative snap to time

On April 28th, you decide to run a search at 14:05.

- If you specify `earliest=-2d`, the search goes back exactly two days, starting at 14:05 on April 26th.
- If you specify `earliest=-2d@d`, the search goes back to two days and snaps to the beginning of the day. The search looks for events starting from 00:00 on April 26th.

Special time units

The following abbreviations are reserved for special cases of time units and snap time offsets.

Time Unit	Description
<code>earliest=1</code>	If you want to search events from the start of UNIX epoch time, use <code>earliest=1</code>. UNIX epoch time 1 is UTC January 1, 1970 at 12:00:01 AM. <code>earliest=0</code> in the search string indicates that time is not used in the search. When <code>earliest=1</code> and <code>latest=now</code> or <code>latest=<a_large_number></code>, the search will run over all time. The difference is that: • Specifying <code>latest=now</code> (which is the default) does not return future events. • Specifying <code>latest=<a_large_number></code> returns future events, which are events that contain timestamps later than the current time, <code>now()</code>.
<code>earliest=now</code> <code>latest=now</code>	Specify that the search starts or ends at the current time. Any search that includes <code>earliest=<relative time offset></code> should also include <code>latest=now</code>. For example, <code>earliest=-10s latest=now</code>. See Specify earliest relative time offset and latest time in ad hoc searches.
<code>@q</code> , <code>@qtr</code> , or <code>@quarter</code>	Specify a snap to the beginning of the most recent quarter: Jan 1, Apr 1, July 1, or Oct 1.
<code>w0</code> , <code>w1</code> , <code>w2</code> , <code>w3</code> , <code>w4</code> , <code>w5</code> , <code>w6</code> , and <code>w7</code>	Specify "snap to" days of the week ; where <code>w0</code> is Sunday, <code>w1</code> is Monday, etc. When you snap to a week, <code>@w</code> or <code>@week</code> , it is equivalent to snapping to Sunday or <code>@w0</code> . You can use either <code>w0</code> or <code>w7</code> for Sunday.

Specify earliest relative time offset and latest time in ad hoc searches

Ad hoc searches that use the **earliest** time modifier with a relative time offset should also include **latest=now** in order to avoid time range inaccuracies. For example, if you want to get all events from the last 10 seconds starting at 01:00:10, the following search returns all events that occur between the time of 01:00:00 and 01:00:10, as expected.

```
index=main earliest=-10s latest=now
```

Running the same search without including **latest=now** might produce unpredictable results or impact performance in certain scenarios when the search head is overloaded with ad hoc searches.

Examples of relative time modifiers

For the following examples, the current time is Wednesday, 05 June 2024, 01:37:05 P.M.

Note: 24h is usually but not always equivalent to 1d because of Daylight Savings Time boundaries.

Time modifier	Description	Resulting time	Equivalent modifiers
now	Now, the current time	Wednesday, 05 June 2024, 01:37:05 P.M.	now()
-60m	60 minutes ago	Wednesday, 05 June 2024, 12:37:05 P.M.	-60m@s
-1h@h	1 hour ago, to the hour	Wednesday, 05 June 2024, 12:00:00 P.M.	
-1d@d	Yesterday	Tuesday, 04 June 2024, 12:00:00 A.M.	
-24h	24 hours ago (yesterday)	Tuesday, 04 June 2024, 01:37:05 P.M.	-24h@s
-7d@d	7 days ago, 1 week ago today	Wednesday, 29 May 2024, 12:00:00 A.M.	
-7d@m	7 days ago, snap to minute boundary	Wednesday, 29 May 2024, 01:37:00 P.M.	
@w0	Beginning of the current week	Sunday, 02 June 2024, 12:00:00 A.M.	
+1d@d	Tomorrow	Thursday, 06 June 2024, 12:00:00 A.M.	
+24h	24 hours from now, tomorrow	Thursday, 06 June 2024, 01:37:05 P.M.	+24h@s

Examples of chained relative time offsets

You can also specify offsets from the snap-to-time or "chain" together the time modifiers for more specific relative time definitions.

Time modifier	Description	Resulting time
@d-2h	Snap to the beginning of today (12 A.M.) and subtract 2 hours from that time.	10 P.M. last night.
-mon@mon+7d	One month ago, snapped to the first of the month at midnight, and add 7 days.	The 8th of last month at 12 A.M.

Examples of searches with relative time modifiers

Search from the beginning of the week to the time of your search

This example searches for Web access errors from the beginning of the week to the time that you run your search.

```
eventtype=webaccess error earliest=@w0 latest=now
```

This search returns matching events starting from 12:00 A.M. of the Sunday of the current week to the current time. Of course, this means that if you run this search on Monday at noon, you will only see events for 36 hours of data.

Search the current business week

This example searches for Web access errors from the current business week, where **w1** is Monday and **w6** is Friday.

```
eventtype=webaccess error earliest=@w1 latest=+7d@w6
```

This search returns matching events starting from 12:00 A.M. of the Monday of the current week and ending at 11:59 P.M. of the Friday of the current week.

- If you run this search on Monday at noon, you will only see events for 12 hours of data.
- If you run this search on Friday, you will see events from the beginning of the week to the current time on Friday. The timeline however, will display for the full business week.

Search the last full business week

This example searches Web access errors from the last full business week.

```
eventtype=webaccess error earliest=-7d@w1 latest=@w6
```

This search returns matching events starting from 12:00 A.M. of last Monday and ending at 11:59 P.M. of last Friday.

Search the last 24 hours but omit 1 hour

This example searches an index for the last 24 hours but omits any events returned from Midnight to 1:00 A.M., when downtime returns false log entries.

```
index=myindex ((earliest=-24h latest<@d) OR (earliest>=@d+1h))
```

This search specifies two time ranges:

- 24 hours before the search is run, up to midnight
- The beginning of the day that the search is run, starting at 1 hour after midnight or 1:00 A.M.

Search for specific windows of time

The following table describes several ways to search for specific windows of time:

Description	Search	Search runtime	Search start window	Search end window
Search for events from seven days ago until today.	<pre>... earliest t=-7d latest=n ow</pre>	Wednesday May 11th at 3:37 PM	Wednesday May 4th at 3:37 PM	Wednesday, May 11th at 3:37 PM
Search for events from seven days ago for a five hour window.	<pre>... earliest t=-7d lates t=-7d+5h</pre>	Wednesday May 11th at 3:37 PM	Wednesday May 4th at 3:37 PM	Wednesday May 4th at 8:37 PM
Search for events from seven days ago, from the beginning of that day for a two hour window.	<pre>i... earliest t=-7d@d lates t=-7d@d+2h</pre>	Wednesday May 11th at 3:37 PM	Wednesday May 4th at 00:00 AM	Wednesday May 4th at 2:00 AM
Search for events from seven days ago. Start three hours before the beginning of that day. End at the beginning of the day, seven days ago.	<pre>... earliest t=-7d@d-3h lat est=-7d@d</pre>	Wednesday May 11th at 3:37 PM	Tuesday May 3rd at 9:00 PM	Wednesday May 4th at 00:00 AM

See also

Related information

[Time modifiers](#) in the *Search Reference*

Specify time ranges for real-time searches

Note: Users must have the Admin role to run and save real-time searches. For more information on managing roles and assigning roles to users, see [Create and manage roles with Splunk Web](#) in *Securing Splunk Enterprise*. In Splunk Cloud Platform on Victoria Experience, real-time searches are enabled by default. In Splunk Cloud Platform on Classic Experience, you must open a support ticket to enable real-time search. For more information, see [About real-time searches and reports](#) in the *Search Manual*.

Time ranges for historical searches are set at the time the search runs. With real-time searches, the time range boundaries are constantly updating and by default, the results accumulate from the start of the search. You can also specify a range that represent a sliding window of time, for example, the last 30 seconds. When you specify a sliding window, Splunk software uses that amount of time to accumulate data. For example, if your sliding window is 5 minutes, you will not start to see data until after the first 5 minutes have passed. You can override this behavior so that Splunk software backfills the initial window with historical data before running in the normal real-time search mode. See [Real-time backfill](#).

Real-time searches can consume a lot of system resources. As a result, consider running **scheduled searches** instead of real-time searches, even if you need updated results every 1 or 5 minutes. See [Scheduling searches](#).

Real-time modifier syntax

To run a search in real time, you can select from predefined Real-time time range windows in the time range range picker list or you can specify a custom real-time window using **Custom time...** and selecting **Real-time**.

Time ranges for real-time search follow the same syntax as for historical searches, except that you precede the relative time specifier with "rt", so that it's `rt<time_modifier>`: `rt[+|-]<time_integer><time_unit>@<time_unit>`. See [Specify time modifiers in your search](#).

These values are not designed to be used in a search string. If you are a Splunk Enterprise administrator, you can use these values when you edit the `times.conf` file (to add options to the time range picker), to specify the earliest/latest time ranges in the saved search dialog, or when you use the REST API to access the Splunk search engine.

When you use time range windows with real-time searches, some of the events that occur within the latest second may not be displayed. This is expected behavior and is due to the latency between the timestamps within the events and the time when the event arrives. Because the time range window is with respect to the timestamps within the events and not the time when the event arrives, events that arrive after the time window won't display.

Real-time searches over "all time"

There is a small difference between real-time searches that take place within a set time window (30 seconds, 1 minute, 2 hours) and real-time searches that are set to "all time."

- In **"windowed" real time searches**, the events in the search can disappear as they fall outside of the window, and events that are newer than the time the search job was created can appear in the window when they occur.
- In **"all-time" real-time searches**, the window spans all of your events, so events do not disappear once they appear in the window, but events that are newer than the time the search job was created can appear in the window as they occur.
- In comparison, **historical search** events never disappear from within the set range of time that you are searching and the latest event is always earlier than the job creation time (with the exception of searches that include events that have future-dated timestamps).

Note: When you run large all-time real-time searches that result in more than 1,000 events, you may notice that not all of the search results are displayed in the **Events viewer**. This is because Splunk software uses a circular buffer to quickly access data in real time and displays only about 1,000 of the most recent events. To see all of the events when you run your searches, select a time range other than **All time (real-time)**, such as **All time**.

Real-time backfill

For windowed real-time searches, you can backfill the initial window with historical data. This is run as a single search, just in two phases: first, a search on historical data to backfill events; then, a normal real-time search. Real-time backfill ensures that real-time dashboards seeded with data on actual visualizations and statistical metrics over time periods are accurate from the start.

You can enable real-time backfill in the `limits.conf` file in the `[realtime]` stanza:

```
[realtime]

default_backfill = <bool>
* Specifies if windowed real-time searches should backfill events
* Defaults to true
```

See also

- [About real-time searches and reports](#)
- [Expected performance and known limitations of real-time searches and reports](#)

Use time to find nearby events

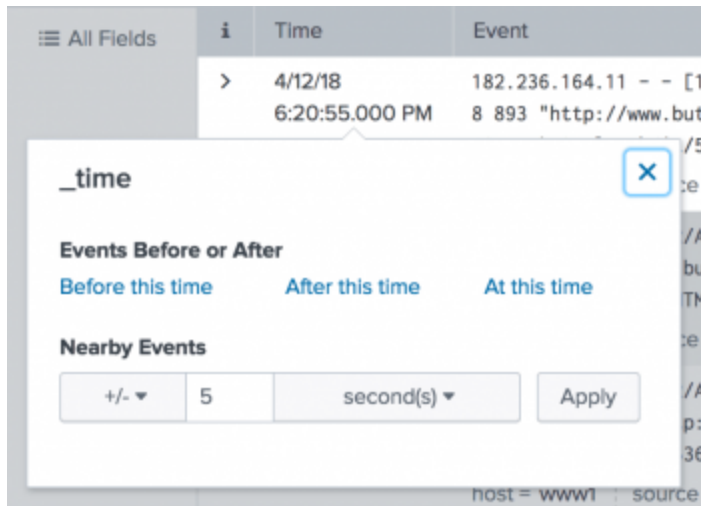
The Splunk Web timeline and time ranges for search are based on event **timestamps**.

While searching for errors or troubleshooting an issue, looking at events that happened around the same time can help correlate results and find the root cause. This topic discusses how you can search for surrounding events using an event's timestamp and using the timeline.

Use time accelerators

The `_time` field represents the timestamp of an event. When you run a search to retrieve events, the timestamp for each event is listed under the **Time** column.

You can click the timestamp of an event and open a dialog box containing controls, called a **time accelerator**. Use the `_time` accelerator to run a new search that retrieves events chronologically close to that event.



You can search for all events that occurred before or after the event time. The accelerators are **Before this time**, **After this time**, and **At this time**. In addition, you can search for nearby events. For example, you can search for + 30 seconds, - 1 minutes, +/- 5 hours, and so on.

Use the timeline

The timeline is a histogram of the number of events returned by a Splunk search over a chosen time range. The time range is broken up into smaller time intervals (such as seconds, minutes, hours, or days), and the count of events for each interval is displayed as a column.

The location of each column on the timeline corresponds to an instance when the events that match your search occurred. If there are no columns at a time period, no events were found then. The taller the column, the more events occurred at that time.

Spikes in the number of events or no events along the timeline can indicate time periods that you want to investigate.

The timeline has **drilldown** functionality similar to the table and chart drilldown. When you click on a column in the timeline, your search results update to show only the events represented by the column. If you double-click on a column, you re-run the search over the time range represented by the column. Then, you can search for all surrounding events at this time range.

Search using time bins and spans

Using commands to create time bins

You can use the `bin`, `chart`, and `timechart` commands to organize your search results into time bins.

Time bins are calculated based on <bin-options> settings, such as **bins** and **span**. For example, the **bins** option sets the maximum number of bins to discretize into. This does not set the target number of bins. It finds the smallest bin size that results in no more than N distinct bins. Even though you specify a number such as 300, the resulting number of bins might be much lower.

See the command information on the specific use of time bins in the *Search Reference*:

- [bin](#)
- [chart](#)
- [timechart](#)

Time zones and time bins

When the time bins cross multiple days or months the bins are aligned to the local day boundary.

The events returned are the same for the time range since the events are processed using UNIX time. But some events might appear in one bin in a timezone, and in another bin in a different timezone.

How specifying a time range influences time bins

When you specify a time range, either through the time range picker or explicitly in the search with the "earliest" and "latest" time modifiers, the events are processed based on which time range is used.

See [Time modifiers](#) in the *Search Reference*.

If the search specifies "Last 24 hours", then the search processes the events using UTC time. The same set of events are returned for a user in San Francisco and a user in Tokyo.

If the search specifies "Today", the search processes events based on the midnight of the timezone, not UTC time. A different set of events are returned for a user in San Francisco and a user in Tokyo, because the time that midnight occurs is different in each timezone.

If the search uses a snap-to time, such as @d or @mon, the search processes events based on the "day" or "month" of the timezone, not UTC time. A different set of events are returned for a user in SF and a user in Tokyo, because the time that midnight occurs is different in each timezone.

See [Specify a snap to time unit](#) in the *Search Reference*.

See also

Related information

[About searching with time](#)

[Select time ranges to apply to your search](#)

[Specify time modifiers in your search](#)

[How time zones are processed by the Splunk platform](#)

How time zones are processed by the Splunk platform

The Splunk platform processes time zones when data is indexed and when data is searched.

When data is indexed, the Splunk indexer looks for a timestamp in each event. The timestamp might be in one of several formats, as shown in the following table:

Type of timestamp	Timestamp example	Description
UNIX time	1523644307	In seconds
	1523644307000	In milliseconds
Human-readable format	04/13/2018 11:45:30 PDT	US Pacific Daylight Time, the timezone where Splunk Headquarters is located.
	Friday, April 13, 2018 11:45:30 AM GMT -07:00	A timestamp with an offset from GMT (Greenwich Mean Time)
	2018-04-13T11:45:30Z	A timestamp expressed in UTC (Coordinated Universal Time)
	2018-04-13T11:45:30-07:00	The local time representation of a UTC time, which is expressed with an offset.
Local time with no time zone	10:55AM	The local time is interpreted as the same time zone as the Splunk indexer where the data is indexed.

Timestamps are stored in UNIX time

Regardless of how time is specified in your events, timestamps are converted to UNIX time and stored in the `_time` field when your data is indexed. If your data does not have timestamps, the time at which your data is indexed is used as the timestamp for your events.

UNIX time is the number of seconds that have elapsed since 00:00:00 Coordinated Universal Time (UTC), 1 January 1970. This moment in time is sometimes referred to as epoch time.

GMT and UTC

GMT (Greenwich Mean Time) is sometimes confused with UTC (Coordinated Universal Time). However GMT is a *time zone* and UTC is a *time standard*.

- GMT is a time zone officially used in some European and African countries as their local time. The time is displayed in either the 24-hour format (00:00-23:59) or the 12-hour format (00:00-12:00 AM/PM).
- UTC is a time standard that is the basis for time and time zones worldwide. No country uses UTC as a local time.
- Neither GMT nor UTC ever change for Daylight Saving Time (DST). However, some of the countries that use GMT switch to different time zones during their DST period. For example, the United Kingdom uses GMT for most of the year, but switches to British Summer Time (BST) during the summer months. BST is one hour ahead of GMT.

What time zone is used for timestamps

When data is indexed and added to your Splunk instance, the Splunk indexer assumes that any timestamps in the data are in the same time zone as your Splunk instance.

Let's use a set of test data that contains 35 events with various timestamps. The data looks something like this:

timestamp	test_no
2018/10/1 00:00	tz_test0
2018/10/1 00:15	tz_test1
2018/10/1 01:00	tz_test2
2018/10/1 01:30	tz_test3
2018/10/1 01:45	tz_test4
2018/10/1 02:00	tz_test5
2018/10/1 02:30	tz_test6

You can see how the timestamp is processed in the Add Data wizard on the Set Source Type step, for example when you add a CSV file. The values in the **timestamp** field in the sample data file are converted to UNIX time and stored in the **_time** field when the data is indexed. However, for display purposes the values in the **_time** field are shown in a human-readable format.

The values in the **_time** field are the same as the values in the **timestamp** field in the sample data file because the Splunk indexer assumes that the values in the **timestamp** field are in the same time zone as the Splunk instance. The default method used to extract timestamps from your data is set to Auto.

Set Source Type

This page lets you see how the Splunk platform sees your data before indexing. If the events look correct and have the right timestamps, click "Next" to proceed. If not, use the options below to define proper event breaks and timestamps. If you cannot find an appropriate source type for your data, create a new one by clicking

Source: `tz_test.csv`

Source type: `csv` Save As

Table Format 20 Per Page < Prev 1 2 Next >

	<code>_time</code>	<code>test_no</code>	<code>timestamp</code>
1	10/1/18 12:00:00.000 AM	span_test0	2018/10/1 00:00
2	10/1/18 12:15:00.000 AM	span_test1	2018/10/1 00:15
3	10/1/18 1:00:00.000 AM	span_test2	2018/10/1 01:00
4	10/1/18 1:30:00.000 AM	span_test3	2018/10/1 01:30
5	10/1/18 1:45:00.000 AM	span_test4	2018/10/1 01:45
6	10/1/18 2:00:00.000 AM	span_test5	2018/10/1 02:00
7	10/1/18 2:30:00.000 AM	span_test6	2018/10/1 02:30

Timestamp options:

- > Timestamp
- > Delimited settings
- > Advanced

Timestamp extraction methods

Note: Most events do not require special timestamp handling. The Splunk indexer automatically recognizes and extracts the timestamps. Use these other methods only if you discover that the Splunk indexer is not extracting timestamps correctly.

In the Add Data wizard on the Set Source Type step, these are the Timestamp options:

Timestamp

Determine how timestamps for the incoming data are defined.

Extraction Auto Curr... Adva... Confi...

- > Delimited settings
- > Advanced

In addition to the Auto timestamp extraction method, there are other methods for timestamp extraction when you index data:

Current time

Sets the timestamp to the current clock time. Ignores the timestamps in the fields in the data.

Advanced

You can specify the Time Zone, the Timestamp format using the `strptime()` function, and the Timestamp fields when multiple fields comprise the complete timestamp. See the `strptime()` function in [Date and Time Functions](#) in the *Search Reference*.

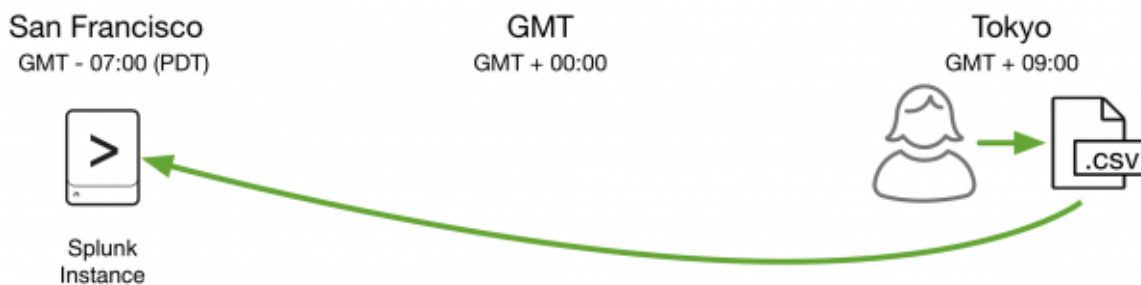
Configuration file

You can specify a configuration file for custom timestamp extractions from your event data. The configuration file must reside in `$SPLUNK_HOME` and have an XML extension.

Indexing events using a different time zone

Maybe you want some data indexed using a different time zone than the default time zone. You can specify the time zone in the Add Data wizard.

For example, let's say your Splunk instance is in San Francisco, California, which is in the US Pacific time zone. You are in an office in Tokyo, which is in the Japanese time zone. The data is from October, a time when the US uses daylight saving time. The time zone in the US is referred to as Pacific Daylight Time (PDT). Japan does not have daylight saving time. The time zone for Tokyo is Japan Standard Time (JST). You want to index the data using Japan Standard Time instead of the timezone of the Splunk instance in California.



In this example, the data is in a CSV file.

To access the Add Data wizard in Splunk Web:

1. From the **Settings** menu click **Upload**.
2. In the **Set Source Type** step of the Add Data wizard, click **Timestamp, Advanced**, and then **Time Zone**.
3. Select the time zone that you want to use. In this example, the selected time zone is (GMT+09:00) Osaka, Sapporo, Tokyo.

Set Source Type

This page lets you see how the Splunk platform sees your data before "Next" to proceed. If not, use the options below to define proper event breaks and timestamps. If you cannot find an appropriate source type for your data, create a new one by clicking "Save As".

Source: **tz_test.csv**

Source type: csv Save As

Timestamp

Determine how timestamps for the incoming data are defined.

Extraction Auto Curr... Adva... Confi...

Time Zone -- Default System Timezone --

(GMT+08:00) Taipei

(GMT+08:00) Perth

(GMT+08:45) Euclypsus

(GMT+09:00) Osaka, Sapporo, Tokyo

(GMT+09:00) Seoul

When you select a different time zone, the values in the `_time` field reflect the timezone you specify when you index the data. In this example, this is the Japanese time zone. These timestamps are equivalent to the timestamps where the Splunk instance is located. In this example, this is the Pacific Daylight Time (PDT).

Add Data

Select Source Set Source Type Input Settings Review Done < Back Next >

Set Source Type

This page lets you see how the Splunk platform sees your data before indexing. If the events look correct and have the right timestamps, click "Next" to proceed. If not, use the options below to define proper event breaks and timestamps. If you cannot find an appropriate source type for your data, create a new one by clicking "Save As".

Source: **tz_test.csv**

Source type: csv Save As

Timestamp

Determine how timestamps for the incoming data are defined.

Extraction Auto Curr... Adva... Confi...

Time Zone (GMT+09:00) Osaka, Sapporo, Tokyo

Timestamp format

A string in strptime() format that helps Splunk recognize timestamps. [Learn More](#)

Timestamp fields

Specify all the fields which constitute the timestamp. ex: field1,field2,...,fieldn

	<code>_time</code>	test_no	timestamp
1	9/30/18 8:00:00.000 AM	span_test0	2018/10/1 00:00
2	9/30/18 8:15:00.000 AM	span_test1	2018/10/1 00:15
3	9/30/18 9:00:00.000 AM	span_test2	2018/10/1 01:00
4	9/30/18 9:30:00.000 AM	span_test3	2018/10/1 01:30
5	9/30/18 9:45:00.000 AM	span_test4	2018/10/1 01:45
6	9/30/18 10:00:00.000 AM	span_test5	2018/10/1 02:00
7	9/30/18 10:30:00.000 AM	span_test6	2018/10/1 02:30

Pacific Daylight Time (PDT)

Japanese Standard Time (JST)

Remember, timestamps are stored in UNIX time. However, time is displayed in the `_time` field in a human-readable format. In this example, the values in the `_time` field are 16 hours earlier than the timestamp values in the Tokyo time zone.

Location	Timestamp	UNIX time	Description
Toyko (JST)	2018/10/1 00:00	1538352000	Midnight or 2400 hours. GMT + 09:00 hours.
GMT	2018/09/30 15:00	1538352000	Nine hours behind Japan, which is equivalent to 15:00 hours on September 30th. GMT + 00:00 hours.
San Francisco (PDT)	2018/09/30 08:00	1538352000	Seven hours behind GMT. 16 hours behind Japan, which is equivalent to 08:00 hours on September 30th. GMT - 07:00 hours.

Notice that the UNIX time does not change. What changes is the time in each time zone that is equivalent to the UNIX time.

How time is interpreted when you search

When you specify a time in your search, either by using the time range picker or using time modifiers, the time that you specify is converted into UNIX time for processing. See [Select time ranges to apply to your search](#) and [Specify time modifiers in your search](#).

Because event timestamps are stored in UNIX time, your searches return a consistent set of results regardless of the time zone you are in.

For example, if you search from 12:00 to 14:00 PDT (Pacific Daylight Time), that is the same as searching from 19:00 to 21:00 GMT (Greenwich Mean Time) which is 7 hours ahead of PDT. When daylight saving time is over, Pacific Standard Time (PST) is used. The difference between GMT and PST is 8 hours.

Note: In Splunk Web, the values in the `_time` field appear in a human-readable format in the UI. However, the values in the `_time` field are actually stored in UNIX time.

How time zones impact search results

The time range that you specify for a search might return different sets of events in different time zones. This can occur for time ranges that you specify using the time range picker and time ranges that you specify explicitly in the search with the **earliest** and **latest** time modifiers. Here are some examples:

- If you use the default **Last 24 hours** time range picker setting, the search processes the events using UNIX time. The same set of events are returned for a user in San Francisco and a user in Tokyo.
- If you use a time range that refers to a time associated with today such as **Since 00:00:00**, the search processes events based on midnight of your time zone, not UNIX time. A different set of events are returned for a user in San Francisco and a user in Tokyo, because the time that midnight occurs is different in each timezone. There are several settings in the time range picker that fall into this category, such as the Preset setting **Today** and the Date Range setting **Since <today's date>**.
- If you use a snap-to time, such as **@d** or **@mon**, the search processes events based on the beginning of the day or month of your timezone, not UNIX time. A different set of events are returned for a user in San Francisco and a user in Tokyo,

because the beginning of a day or month in one time zone is not the same UNIX time as the beginning of a day in another time zone.

To mitigate the issues with time zones, specify time based on the time zone where the Splunk indexer resides.

See also

Getting Data In

- * [How timestamp assignment works](#)
- * [Specify time zones for timestamps](#)
- * [The Set Source Type page](#)

Securing Splunk Platform

- [Create and manage users with Splunk Web](#)

Subsearches

About subsearches

Using subsearches

A subsearch is a search within a primary, or outer, search. When a search contains a subsearch, the subsearch typically runs first.

Subsearches must be enclosed in square brackets in the primary search.

Consider the following search.

```
sourcetype=access_* status=200 action=purchase [search sourcetype=access_* status=200 action=purchase | top limit=1 clientip | table clientip] | stats count, dc(productId), values(productId) by clientip
```

The subsearch portion of the search is enclosed in square brackets.

```
[search sourcetype=access_* status=200 action=purchase | top limit=1 clientip | table clientip]
```

The first command in a subsearch must be a **generating command**, such as **search**, **eventcount**, **inputlookup**, and **tstats**. For a list of generating commands, see [Command types](#) in the *Search Reference*.

One exception is the **foreach** command, which accepts a subsearch that does not begin with a generating command, such as **eval**.

Note: When a search contains a subsearch, the Splunk software processes the subsearch first as a distinct search job. Then it runs the search that contains it as another search job. Keep this in mind if you include subsearches in searches that are run frequently and you are concerned about search concurrency issues or excess load on your **search scheduler**.

How subsearches work

A subsearch looks for a single piece of information that is then added as a criteria, or argument, to the primary search. You use a subsearch because the single piece of information that you are looking for is dynamic. The single piece of information might change every time you run the subsearch.

For example, you want to return all of the events from the host that was the most active in the last hour. The host that was the most active might be different from hour to hour. You need to identify the most active host before you can return the events from that host.

Break this search down into two parts.

- The most active host in the last hour. This is the subsearch.
- The events from that host. This is the primary search.

You could run two searches to obtain the list of events. The following search identifies the most active host in the last hour.

```
sourcetype=syslog earliest=-1h | top limit=1 host | fields host
```

This search returns only one host value. Assume that the result is the host named **crashy**. To return all of the events from the host **crashy**, you need to run a second search.

```
sourcetype=syslog host=crashy
```

The drawback to running two searches is that you cannot set up reports and dashboard panels to run automatically. You must run the first search to identify the piece of information that you need, and then run the second search with that piece of information.

You can combine these two searches into one search that includes a subsearch.

```
sourcetype=syslog [search sourcetype=syslog earliest=-1h | top limit=1 host | fields + host]
```

The subsearch is in square brackets and is run first. The subsearch in this example identifies the most active host in the last hour. The result of the subsearch is then provided as a criteria for the main search. The main search returns the events for the host.

Note: By default, subsearches return a maximum of 10,000 results and have a maximum runtime of 60 seconds. If a subsearch runs for more than 60 seconds, its search results are automatically finalized.

Time ranges and subsearches

Time ranges selected from the Time Range Picker apply to the base search and to subsearches.

However, time ranges specified directly in the base search do not apply to subsearches. Likewise, a time range specified directly in a subsearch applies only to that subsearch. The time range does not apply to the base search or any other subsearch.

For example, if the Time Range Picker is set to **Last 7 days** and a subsearch contains **earliest=2d@d**, then the earliest time modifier applies only to the subsearch and **Last 7 days** applies to the base search.

When to use subsearches

Subsearches are mainly used for two purposes:

- Parameterize one search, using the output of another search. The previous example of searching for the most active host in the last hour is an example of this use of a subsearch.
- Run a separate search and add the output to the first search using the `append` command.

A subsearch can be used only where the explicit action that you are trying to accomplish is with the search and not a transformation of the data. For example, you cannot use a subsearch with "`sourcetype=top | multikv`", because the `multikv` command does not expect a subsearch as an argument. Certain commands, such as `append` and `join` can accept a subsearch as an argument.

Multiple subsearches in a search string

You can use more than one subsearch in a search.

If a search has a set of nested subsearches, the inner most subsearch is run first, followed by the next inner subsearch, working out to the outermost subsearch and then the primary search.

For example, you have the following search.

```
index=* OR index=_*
[search index=* | stats count by component
 | search [search index=* | stats count by user
 | search [search index=* | stats by ipaddress]]]
```

Note: You must have a command after the pipe and before the subsearch. In this example, the `search` command is used for the second and third subsearches. The first subsearch is used as a predicate to the main search.

The order in which this search is processed is:

1. `|search [search index=* | stats by ipaddress]`
2. `|search [search index=* | stats count by user]`
3. `[search index=* | stats count by component]`

4. (An implied search command) `index=* OR index=_*` (and the results of the nested subsearches)

Here is another example.

```
index=foo error [ search index=bar baz [search index=* | stats count by user | search count>100] | stats count by host ]
```

Be certain to analyze your search syntax when you find yourself using subsearches frequently. It is often possible to rewrite the search and omit the subsearch.

If the subsearches are sequential instead of nested, the subsearch farthest to the left, or beginning of the search, is run first. Then working towards the right, or end of the search, the next subsearch is run. When all of the subsearches are run, then the primary search is run.

For example, you have the following search.

```
index=* OR index=_* | [search index=* | stats count by customerID] | [search index=* | stats by productName]
```

The order in which the search is processes is:

1. `search index=* | stats by customerID`
2. `search index=* | stats count by productName`
3. (An implied search command) `index=* OR index=_*` (the results of the subsearches)

Initiating subsearches with search commands

A subsearch can be initiated through a search command such as the **map** command. For example, consider the following search:

```
|makeresults  
|map search="search index=* OR index=_* earliest=0 latest=now"
```

The **map** command runs a subsearch for the option

`search="search index=* OR index=_* earliest=0 latest=now"`. Other commands that can run a subsearch include **search**, **join** and **union**. These commands can start one or more subsearches. What's more, each subsearch, not the command itself, is limited by the **maxtime** setting in the `limits.conf` file, which specifies the maximum number of seconds that a subsearch can run before finalizing. In our example, the **maxtime** setting limits the runtime of the subsearch and is applied to the subsearch, not the **map** command. However, the subsearch may stop at the **map** command.

Subsearch examples

These examples show you the difference between searching your data with and without a subsearch.

These examples use the sample data from the Search Tutorial but should work with any format of Apache web access log. To try this example on your own Splunk instance, you must download the sample data and follow the instructions to [get the tutorial data into Splunk](#). Use the time range **All time** when you run the search.

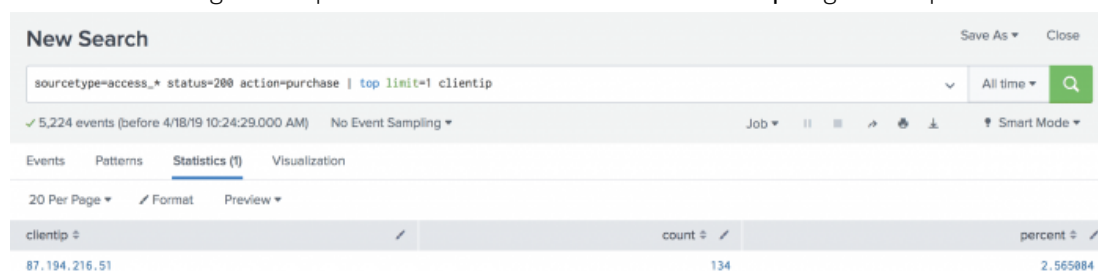
Example 1: Without a subsearch, find what the most frequent shopper purchased

You want to find the single most frequent shopper on the Buttercup Games online store and what that shopper has purchased. Use the **top** command to return the most frequent shopper.

- To find the shopper who accessed the online shop the most, use this search.

```
sourcetype=access_* status=200 action=purchase | top limit=1 clientip
```

The **limit=1** argument specifies to return 1 value. The **clientip** argument specifies the field to return.



New Search Save As Close

sourcetype=access_* status=200 action=purchase | top limit=1 clientip All time Q

✓ 5,224 events (before 4/18/19 10:24:29.000 AM) No Event Sampling Job || ↗ ⬇ ⬆ Smart Mode

Events Patterns **Statistics (1)** Visualization

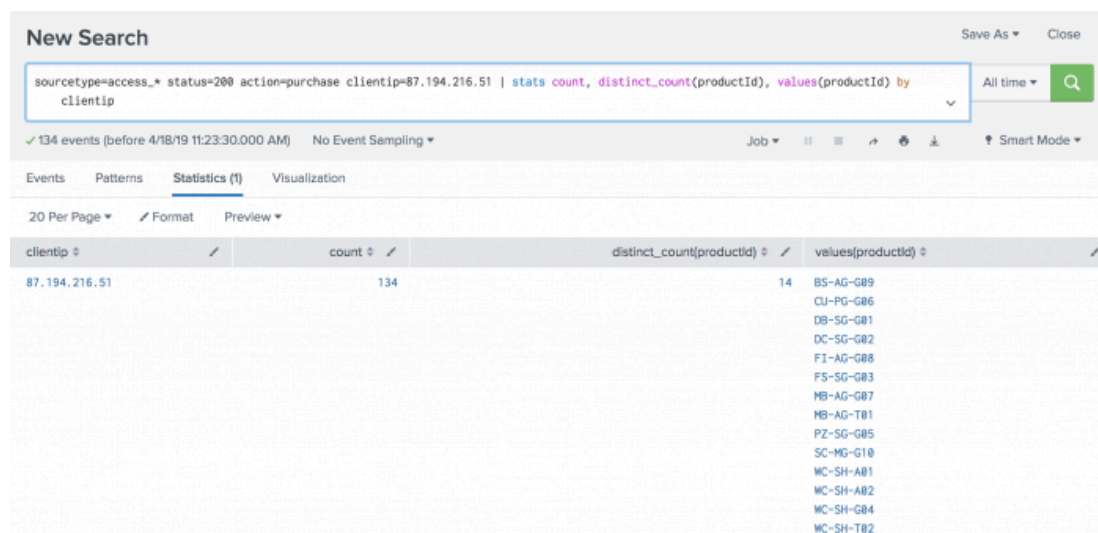
20 Per Page Format Preview

clientip	count	percent
87.194.216.51	134	2.565084

This search returns one **clientip** value, 87.194.216.51, which you will use to identify the VIP shopper.

- You now need to run another search to determine how many different products the VIP shopper has purchased. Use the **stats** command to count the purchases by this VIP customer.

```
sourcetype=access_* status=200 action=purchase clientip=87.194.216.51 | stats count, dc(productId), values(productId) by clientip
```



New Search Save As Close

sourcetype=access_* status=200 action=purchase clientip=87.194.216.51 | stats count, distinct_count(productId), values(productId) by clientip All time Q

✓ 134 events (before 4/18/19 11:23:30.000 AM) No Event Sampling Job || ↗ ⬇ ⬆ Smart Mode

Events Patterns **Statistics (1)** Visualization

20 Per Page Format Preview

clientip	count	distinct_count(productId)	values(productId)
87.194.216.51	134	14	BS-AG-G09 CU-PG-G06 DB-SG-G01 DC-SG-G02 FI-AG-G08 FS-SG-G03 MB-AG-G07 MB-AG-T01 PZ-SG-G05 SC-MG-G10 WC-SH-A01 WC-SH-A02 WC-SH-G04 WC-SH-T02

This search uses the **count()** function to return the total count of the purchases for the VIP shopper. The **dc()** function is the **distinct_count** function. Use this function to count the number of different, or unique, products that the shopper bought. The **values** function is used to display the distinct product IDs as a multivalue field.

The drawback to this approach is that you have to run two searches each time you want to build this table. The top purchaser is not likely to be the same person at any given time range.

Example 2: Using a subsearch, find what the most frequent shopper purchased

Let's start with our first requirement, to identify the single most frequent shopper on the Buttercup Games online store.

1. Copy and paste the following search into the Search bar and run the search. Make sure the time range is **All time**.

```
sourcetype=access_* status=200 action=purchase | top limit=1 clientip | table clientip
```

This search returns the clientip for the most frequent shopper, **clientip=87.194.216.51**. This search is almost identical to the search in Example 1 Step 1. The difference is the last piped command, **| table clientip**, which displays the clientip information in a table.

To find what this shopper has purchased, you run a search on the same data. You provide the result of the most frequent shopper search as one of the criteria for the purchases search.

The most frequent shopper search becomes the **subsearch** for the purchases search. The purchases search is referred to as the **outer** or primary search. Because you are searching the same data, the beginning of the outer search is identical to the beginning of the subsearch.

A subsearch is enclosed in square brackets **[]** and processed first when the search is parsed.

2. Copy and paste the following search into the Search bar and run the search.

```
sourcetype=access_* status=200 action=purchase [search sourcetype=access_* status=200 action=purchase | top limit=1 clientip | table clientip] | stats count, distinct_count(productId), values(productId) by clientip
```

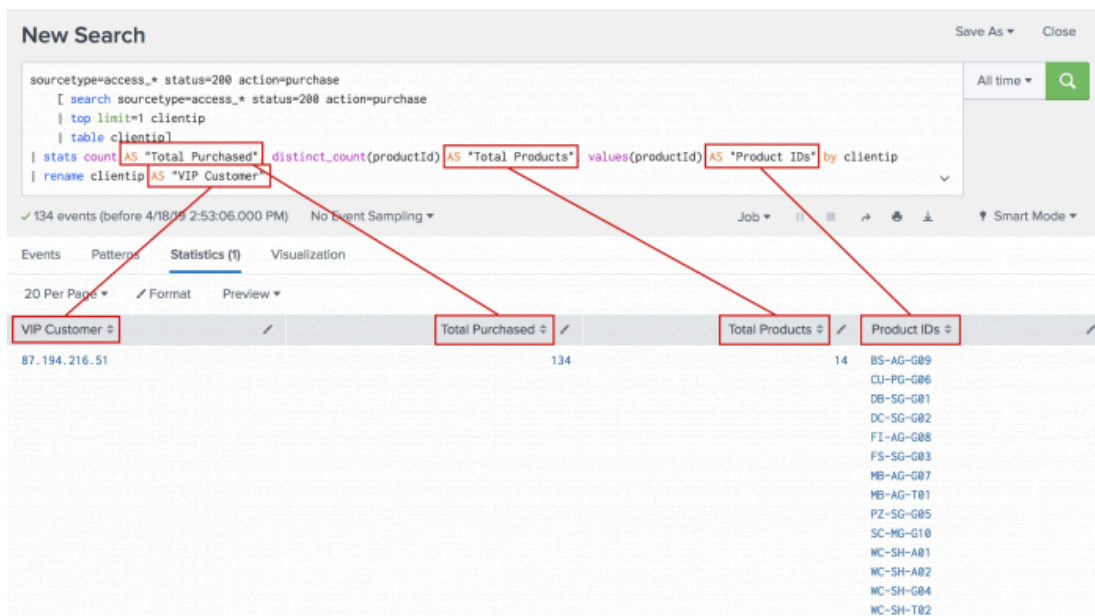
Because the **top** command returns the **count** and **percent** fields, the **table** command is used to keep only the **clientip** value.

Column	Rename
count	Total Purchased
dc(productId)	Total Products
values(productId)	Product IDs
clientip	VIP Customer

You rename columns by using the AS operator on the fields in your search. If the rename that you want to use contains a space, you must enclose the rename in quotation marks.

- To rename the fields, copy and paste the following search into the Search bar and run the search.

```
sourcetype=access_* status=200 action=purchase [search sourcetype=access_* status=200 action=purchase |
top limit=1 clientip | table clientip] | stats count AS "Total Purchased", distinct_count(productId) AS
"Total Products", values(productId) AS "Product IDs" by clientip | rename clientip AS "VIP Customer"
```



The screenshot shows the Splunk Search interface. The search bar contains the query: `sourcetype=access_* status=200 action=purchase [search sourcetype=access_* status=200 action=purchase | top limit=1 clientip | table clientip] | stats count AS "Total Purchased", distinct_count(productId) AS "Total Products", values(productId) AS "Product IDs" by clientip | rename clientip AS "VIP Customer"`. The results table shows four columns: **VIP Customer**, **Total Purchased**, **Total Products**, and **Product IDs**. Red boxes highlight the column names in the search bar and the results table, with red lines connecting them to show the mapping.

Make the search syntax easier to read

Make the search results easier to understand

Subsearch performance considerations

A subsearch can be a performance drain if the search returns a large number of results.

Consider this search.

```
sourcetype=access_* status=200 action=purchase [search sourcetype=access_* status=200 action=purchase | top limit=1 clientip | table clientip] | stats count, dc(productId), values(productId) by clientip
```

The performance of this subsearch depends on how many distinct IP addresses match **status=200 action=purchase**. If there are thousands of distinct IP addresses, the **top** command has to keep track of all of them before the top 1 is returned, impacting performance.

Additionally, by default subsearches return a maximum of 10,000 results and have a maximum runtime of 60 seconds. In large production environments it is quite possible that the subsearch in this example will timeout before it completes.

There are several alternatives you can use to control the results:

- Try to rewrite the query to limit the number of events the subsearch must process.
- You can change the number of results that the format command operates over inline with your search by appending the **format** command to the end of your subsearch.

```
...| format maxresults = <integer>
```

For more information, see the [format](#) command in the *Search Reference*.

- If you are using Splunk Enterprise, you can also control the subsearch by editing settings in the [subsearch] stanza in the [limits.conf](#) file. For example, you can edit the **maxout** setting to adjust the size of results that are returned from your subsearches. See [How to edit a configuration file](#).

After running a search, you can click the **Job** menu and select **Inspect Job** to open the Search Job Inspector. Scroll down to the **remoteSearch** component to see the actual query that resulted from your subsearch. For more information, see [View search job properties](#) in this manual.

Output settings for subsearch commands

By default, subsearches return a maximum of 10,000 results. You will see variations in the actual number of output results because every command can change what the default **maxout** is when the command invokes a subsearch. Additionally, the default applies to subsearches that are intended to be expanded into a search expression, which is not the case for some commands such as [join](#), [append](#), and [appendcols](#).

- For example, the **append** command can override the default maximum if the **maxresultrows** argument is specified, unless you specify **maxout** as an argument to the **append** command.
- The output limit of the **join** command is controlled by **subsearch_maxout** in the **limits.conf** file. This defaults to 50,000 events.

Use subsearch to correlate events

A subsearch takes the results from one search and uses the results in another search. This enables sequential state-like data analysis. You can use subsearches to correlate data and evaluate events in the context of the whole event set, including data across different indexes or Splunk Enterprise servers in a distributed environment.

For example, say you have two or more indexes for different application logs. The event data from these logs share at least one common field. You can use the values of this field to search for events in one index based on a value that is not in another index:

```
sourcetype=some_sourcetype NOT [search sourcetype=another_sourcetype | fields field_val]
```

That search is equivalent to the SQL "NOT IN" functionality:

```
SELECT * from some_table  
WHERE field_value  
NOT IN (SELECT field_value FROM another_table)
```

Example

To identify the IP address of the top customer at Buttercup Games with the most purchases, you could run the following search:

```
sourcetype=access_* status=200 action=purchase  
| top limit=1 clientip
```

Then, you could search the customer's purchase history by running the following search on the customer's IP address, which is 87.194.216.51:

```
sourcetype=access_* status=200 action=purchase clientip=87.194.216.51  
| stats count, distinct_count(productId), values(productId) by clientip
```

But, what if the next time you run this search, someone else is the top customer? You would have to run the first search again to find out the new top customer's IP address and then rewrite the second search with that new IP address. Instead of going to all of that trouble, you could get the same results by using a subsearch to correlate the events with the IP address and pass the top customer's IP address to the main search every time you run the search:

```
sourcetype=access_* status=200 action=purchase  
[ search sourcetype=access_* status=200 action=purchase  
  | top limit=1 clientip  
  | table clientip  
  ]  
| stats count, distinct_count(productId), values(productId) by clientip
```

The search results look something like this:

clientip	count	dc(productId)	values(productId)
87.194.216.51	134	14	BS-AG-G09 CU-PG-G06 DB-SG-G01 DC-SG-G02 FI-AG-G08 FS-SG-G03 MB-AG-G07 MB-AG-T01 PZ-SG-G05 SC-MG-G10 WC-SH-A01 WC-SH-A02 WC-SH-G04 WC-SH-T02

Change the format of subsearch results

When you use a subsearch, the **format** command is implicitly applied to your subsearch results. The **format** command changes the subsearch results into a single linear search string. This is used when you want to pass the values in the returned fields into the primary search.

If your subsearch returned a table, such as:

```

      | field1 | field2 |
      -----
event/row1 | val1_1 | val1_2 |
event/row2 | val2_1 | val2_2 |

```

The **format** command returns:

```
(field1=val1_1 AND field2=val1_2) OR (field1=val2_1 AND field2=val2_2)
```

For more information, see the [format](#) command.

Formatting exceptions

There are a couple of exceptions to the formatting that the **format** command performs.

- All internal fields, fields that begin with a leading underscore (`_`) character, are ignored and not formatted as a linear search string.
- If the name of a field is either **search** or **query**, the values of fields are rendered directly in the reformatted search string.

The search and query fields

You can rename a field to either **search** or **query** to change the format of the subsearch results. Renaming a field to **search** or **query** is a special use case. When you rename your fields to anything else, the subsearch returns the new field names that you specify.

Using the search field name

Use the **search** field name and the **format** command when you need to append some static data or apply an evaluation on the data in the subsearch. You can then pass the data to the primary search. For example, you rename the second field in the search results to **search**, as shown in the following table:

	field1	search	
event/row1	val1_1	val1_2	
event/row2	val2_1	val2_2	

Then using the **format** command returns:

```
(field1=val1_1 AND val1_2) OR (field1=val2_1 AND val2_2)
```

Instead of

```
(field1=val1_1 AND field2=val1_2) OR (field1=val2_1 AND field2=val2_2)
```

For multivalue fields, when you use the **search** field name, the first value of the field is used as the actual search term.

Using the query field name

Use the **query** field name when you want the values in the fields returned from the subsearch, but not the field names.

The **query** field name is similarly to using the **format** command. Instead of passing the field and value pairs to the main search, such as:

```
(field1=val1_1 AND field2=val1_2) OR (field1=val2_1 AND field2=val2_2)
```

Using the **query** field name passes only the values:

```
(val1_1 AND val1_2) OR (val2_1 AND val2_2)
```

Examples

The following search looks for a value in the **cID** field that is associated with a name token or field value. The cID value is then used to search for several sources.

```
index=myindex [search index=myindex host=myhost MyName | top limit=1 cID | fields cID ]
```

The subsearch returns the field and value in the format: ((cID="0050834ja"))

To return only the value, **0050834ja**, rename the **cID** field to **search** in the subsearch. For example:

```
index=myindex [search index=myindex host=myhost MyName | top limit=1 cID | fields cID | rename cID as search  
]
```

When the field is named **search** or **query**, the field name is dropped and the implicit **| format** command at the end of the subsearch returns only the value.

If you return multiple values, such as specifying **...| top limit=3**, the subsearch returns each of the values with the boolean OR operator between the values. For example, if the previous search example used **...| top limit=3**, the values returned from the subsearch are ((value1) OR (value2) OR (value3)).

Create Statistical Tables and Chart Visualizations

About transforming commands and searches

To create charts visualizations, your search must transform event data into statistical data tables. These statistical tables are required for charts and other kinds of data visualizations. This section discusses how to use **transforming commands** to transform event data.

This section describes the major categories of transforming commands and provides examples of how they can be used in a search.

Transforming commands

The primary transforming commands are:

- **chart** : creates charts that can display any **series** of data that you want to plot. You can decide what field is tracked on the x-axis of the chart.
- **timechart** : used to create "trend over time" reports, which means that **_time** is always the x-axis.
- **top** : generates charts that display the most common values of a field.
- **rare** : creates charts that display the least common values of a field.
- **stats** : generates a report that display summary statistics.

See [Transforming commands](#) in the *Search Reference* to learn more.

Note: As you will see in the following examples, you always place your transforming commands after your search commands, linking them with a **pipe operator** (`|`).

The **chart**, **timechart**, and **stats** commands are all designed to work with statistical functions. The list of available statistical functions includes:

- count, distinct count
- mean, median, mode
- min, max, range, percentiles
- standard deviation, variance
- sum
- first occurrence, last occurrence

For more information about statistical functions, see [Statistical and charting functions](#) in the *Search Reference*. Some statistical functions only work with the **timechart** command.

Note: All searches with transforming commands generate specific data structures. The different chart types require these data structures to be set up in particular ways. For example, not all searches that enable you to generate bar, column, line, and area charts can be used to generate pie charts. See [Data structure requirements for visualizations](#) in the *Dashboard and Visualizations* manual to learn more.

Table, chart, and report examples

The following examples use transforming commands to create tables, charts, and reports:

- [Create time based charts](#)
- [Create charts that are not \(necessarily\) time-based](#)
- [Create reports that display summary statistics](#)
- [Build a chart of multiple data series](#)

Real-time reporting

You can use real-time search to calculate metrics in real time on large incoming data flows without the use of summary indexing. However, because you are reporting on a live and continuous stream of data, the timeline will update as the events stream in and you can only view the table or chart in preview mode. Also, some search commands will be more applicable (for example, `streamstats` and `rtorder`) for use in real-time. See [About real-time searches and reports](#).

See also

[Types of commands](#)

[Types of searches](#)

Create time-based charts

This topic discusses using the [timechart](#) command to create time-based reports.

The timechart command

The timechart command generates a table of summary statistics. This table can then be formatted as a chart visualization, where your data is plotted against an x-axis that is always a time field. Use the timechart command to display statistical trends over time. You can split the data with another field as a separate series in the chart. Timechart visualizations are usually line, area, or column charts.

When you use the **timechart** command, the x-axis represents time. The y-axis can be any other field value, count of values, or statistical calculation of a field value.

For more information, see the [Data structure requirements for visualizations](#) in the *Dashboards and Visualizations* manual.

Examples

Example 1: This report uses internal Splunk log data to visualize the average indexing throughput (indexing kbps) of Splunk processes over time. The information is separated, or split, by processor:

```
index=_internal "group=throughput" | timechart avg(instantaneous_eps) by processor
```

See also

[Build a chart of multiple data series](#)

Create charts that are not (necessarily) time-based

This topic discusses using the [transforming command](#), [chart](#), to create visualizations that are not time-based.

The chart command

The [chart command](#) returns your results in a data structure that supports visualization of your data [series](#) as a chart such as a column, line, area, and pie chart.

Unlike the **timechart** command, which uses the **_time default field** as the x-axis, charts created with the **chart** command use an arbitrary field as the x-axis. With the chart command, you use the **over** keyword to determine what field takes the x-axis.

Examples

Example 1: Use web access data to show you the average count of unique visitors over each weekday.

```
sourcetype=access_* | chart avg(clientip) over date_wday
```

One of the options you have is to split the data by another field, meaning that each distinct value of the "split by" field is a separate series in the chart. If your search includes a "split by" clause, place the **over** clause before the "split by" clause.

The following report generates a chart showing the sum of kilobytes processed by each **clientip** within a given timeframe, split by **host**. The finished chart shows the **bytes** value taking the y-axis while **clientip** takes the x-axis. The delay value is broken out by host. After you run this search, format the report as a stacked bar chart.

```
sourcetype=access_* | chart sum(bytes) over clientip by host
```

Example 2: Create a stacked bar chart that splits out the http and https requests hitting your servers.

To do this, first create **ssl_type**, a search-time **field extraction** that contains the inbound port number or the incoming URL request, assuming that it is logged. The finished search would look like this:

```
sourcetype=access_* | chart count over ssl_type
```

After you run the search, format the results as a stacked bar chart.

Visualize field value highs and lows

This topic discusses how to use the **transforming commands**, **top** and **rare**, to create charts that display the most and least common values.

The top and rare commands

The **top command** returns the most frequent values of a specified field in your returned events. The **rare command**, returns the least common value of a specified field in your returned events. Both commands share the same syntax. If you don't specify a limit, the default number of values displayed in a **top** or **rare** is ten.

Examples

Example 1: Generate a report that sorts through firewall information to list the top 100 destination ports used by your system:

```
sourcetype=firewall | top limit=100 dst_port
```

Example 2: Generate a report that shows you the source ports with the lowest number of denials.

```
sourcetype=firewall action=Deny | rare src_port
```

A more complex example of the top command

Say you're indexing an alert log from a monitoring system, and you have two fields:

- **msg** is the message, such as **CPU at 100%**.
- **mc_host** is the host that generates the message, such as **log01**.

How do you get a report that displays the top **msg** and the values of **mc_host** that sent them, so you get a table like this:

Messages by mc_host
CPU at 100%
log01
log02
log03
Log File Alert
host02
host56
host11

To do this, set up a search that finds the top **message** per **mc_host** (using **limit=1** to only return one) and then sort by the message **count** in descending order:

```
sourcetype=alert_log | top 1 msg by mc_host | sort count
```

Create reports that display summary statistics

This topic discusses using the [stats](#) and [eventstats](#) **transforming commands** to create reports that display summary statistics related to a field.

The stats and eventstats commands

The **eventstats** command works in exactly the same manner as the **stats** command, except that the aggregation results of the command are added inline to each event, and only the aggregations that are pertinent to each event.

Using split-by clauses

To fully utilize the **stats** command, you need to include a "split by" clause. For example, the following report won't provide much information:

```
sourcetype=access_combined | stats avg(kbps)
```

It gives you the average of **kbps** for all events with a sourcetype of **access_combined**—a single value. The resulting column chart contains only one column.

But if you break out the report with a split by field, Splunk software generates a report that breaks down the statistics by that field. The following report generates a column chart that sorts through the **access_combined** logs to get the average throughput (kbps), broken out by host:

```
sourcetype=access_combined | stats avg(kbps) by host
```

Examples

Example 1: Create a report that shows you the CPU utilization of Splunk processes, sorted in descending order:

```
index=_internal "group=pipeline" | stats sum(cpu_seconds) by processor | sort sum(cpu_seconds) desc
```

Example 2: Create a report to display the average kbps for all events with a sourcetype of **access_combined**, broken out by host.

You specify the field name for the **eventstats** results by adding the **as** argument. So the first example above could be restated with "avgkbps" being the name of the new field that contains the results of the **eventstats avg(kbps)** operation:

```
sourcetype=access_combined | eventstats avg(kbps) as avgkbps by host
```

When you run this set of commands, Splunk software adds a new **avgkbps** field to each **sourcetype=access_combined** event that includes the **kbps** field. The value of **avgkbps** is the average kbps for that event.

Look for associations, statistical correlations, and differences in search results

This topic discusses **transforming commands** that find associations, similarities, and differences among field values in your search results.

The `associate` command

The `associate` command identifies events that are associated with each other through field/field value pairs. For example, if one event has a `referer_domain` of "<http://www.google.com/>" and another event has a `referer_domain` with the same URL value, then they are associated.

"Tune" the results gained by the `associate` command with the `supcnt`, `supfreq`, and `improv` arguments. For more information about these arguments see the [associate command reference](#) topic.

Example: Search the web access sourcetypes and identify events that share at least three field/field-value pair associations.

```
sourcetype=access* | associate supcnt=3
```

The `correlate` command

The `correlate` command calculates the statistical correlation between fields. It uses the `cocur` operation to calculate the percentage of times that two fields exist in the same set of results.

Example: Search across all events where `eventtype=goodaccess`, and calculates the co-occurrence correlation between all of those fields.

```
eventtype=goodaccess | correlate type=cocur
```

The `diff` command

Use the `diff` command to compare the differences between two search results. By default it compares the raw text of the search results you select, unless you use the `attribute` argument to focus on specific field attributes.

Example: Compare the IP addresses for the 44th and 45th events returned in the search.

```
eventtype=goodaccess | diff pos1=44 pos2=45 attribute=ip
```

Build a chart of multiple data series

Splunk **transforming commands** do not support a direct way to define multiple data **series** in your charts (or timecharts). However, you CAN achieve this using a combination of the `stats` and `xyseries` commands.

The `chart` and `timechart` commands both return tabulated data for graphing, where the x-axis is either some arbitrary field or `_time`, respectively. When these commands are used with a split-by field, the output is a table where each column represents a distinct value of the split-by field.

In contrast, the `stats` command produces a table where each row represents a single unique combination of the values of the group-by fields. You can then use the `xyseries` command to redefine your data series for graphing.

For most cases, you can simulate the results of "... | chart n by xy" with "... | stats n by xy | xyseries x y n". (For the **timechart** equivalent of results, x = **_time**.)

Scenario

Let's say you want to report on data from a cluster of application servers. The events gathered from each server contain information such as counts of active sessions, requests handled since last update, etc. and are placed in the **applications_servers** index. You want to display each server instance and the number of sessions per instance on the same timechart so that you can compare the distributions of sessions and load.

Ideally, you want to be able to run a timechart report, such as:

```
index=application_servers
| timechart sum(handledRequests) avg(sessions) by source
```

However, timechart does not support multiple data series; so instead, you need run a search similar to the following:

```
index=application_servers
| bin _time
| stats sum(handledRequests) as hRs, avg(sessions) as ssns by _time, source
| eval s1="handledReqs sessions"
| makemv s1
| mvexpand s1
| eval yval=case(s1=="handledReqs",hRs,s1=="sessions",ssns)
| eval series=source+": "+s1
| xyseries _time,series,yval
```

Walkthrough

Let's break this search down so it's easier to understand what each part of the search is doing:

The first thing that you need to do, before the **stats** command, is to separate the events by time.

```
... | bin _time
```

The **stats** command is used to calculate statistics for each source value: The sum of **handledRequests** values are renamed as **hRs**, and the average number of **sessions** are renamed as **ssns**.

```
... | stats sum(handledRequests) as hRs, avg(sessions) as ssns by _time, source
```

The following portion of the search uses the **eval** command to add a single-valued field called "s1" to each result from the **stats** command. Then, the **makemv** command converts the values in the **s1** field into a multivalued field, where the first

value is "handledRequests" and the second value is "sessions". The **mvexpand** command then creates separate series for each value of **s1**.

```
... | eval s1="handledRequests sessions" | makemv s1 | mvexpand s1
```

Then the search uses the **eval** command to define a new field called "yval", and assign values to the field based on the case that it matches. So, if the value of **s1** is "handledRequests", the **yval** field is assigned the "hRs" value. And, if the value of the **s1** field is "sessions", the **yval** field is assigned the "ssns" value.

```
... | eval yval=case(s1=="handledRequests",hRs,s1=="sessions",ssns)
```

Then the search uses the **eval** command to define a new field called "series", which concatenates the value of the **sessions** and **s1** fields.

```
... | eval series=source+": "+s1
```

Finally, the **xyseries** command is used to define a chart with **_time** on the x-axis, **yval** on the y-axis, and data defined by the **series** field.

```
... | xyseries _time, yval, series
```

See also

Commands

[bin command](#) in the *Search Reference*

[eval command](#) in the *Search Reference*

[makemv command](#) in the *Search Reference*

[mvexpand command](#) in the *Search Reference*

[stats command](#) in the *Search Reference*

[xyseries command](#) in the *Search Reference*

Compare hourly sums across multiple days

The [timechart command](#) creates charts that show trends over time. It has strict boundaries limiting what it can do. There are times when you should use the [chart command](#) command, which can provide more flexibility.

This example demonstrates how to use **chart** to compare values collected over several days. You cannot do this with **timechart**

Scenario

These two searches are almost identical. They both show the hourly sum of the **P** field over a 24-hour period. The only difference is that one search covers a period ten days in the past, while the other covers a period nine days into the past:

Search 1:

```
earliest=-10d latest=-9d | timechart span="1h" sum(P)
```

Search 2:

```
earliest=-9d latest=-8d | timechart span="1h" sum(P)
```

Create a column chart that combines the results of these two searches, so you can see the sum of **P** for 3pm, ten days ago side-by-side with the sum of **P** for 3pm, nine days ago.

Solution

Using the chart command, set up a search that covers both days. Then, create a "sum of P" column for each distinct **date_hour** and **date_wday** combination found in the search results.

The finished search looks like this:

```
earliest=-10d latest=-8d | chart sum(P) by date_hour date_wday
```

This produces a single chart with 24 slots, one for each hour of the day. Each slot contains two columns that enable you to compare hourly sums between the two days covered by the time range of the report.

For a primer on reporting searches and how they're constructed, see ["Use reporting commands"](#) in the *Search Manual*.

For more information about **chart**> and **timechart** functions, see ["Statistical and charting functions"](#) in the *Search Reference*.

Drill down on tables and charts

After running a search, you can run different kinds of secondary **drilldown** searches. The drilldown search options depend on the type of element you click on.

In the **Statistics** tab you can run a drilldown search when you click on a field value or calculated search result.

For information on drilling down on field-value pairs, see [Drill down on event details](#).

You can also click on elements of charts and visualizations to run drilldown searches. For more information see [Use drilldown for dashboard interactivity](#) in *Dashboards and Visualizations*.

Open a non-transforming search in Pivot to create tables and charts

Searches that do not contain **transforming commands** return event lists that you can view in the **Events** tab, and you can use the **Patterns** tab to see the dominant patterns amongst those events. However, these non-transforming searches cannot return results in the form of statistical tables. Without statistical tables, Splunk software cannot create charts or other visualizations. This means that when you run a non-transforming search you do not get results in the **Statistics** or **Visualization** tabs.

If you run a non-transforming search and want to make tables or charts based on it, go to the **Statistics** or **Visualization** tab and open the search in Pivot.

In the **Pivot Editor**, you can build tables and charts without editing the original search string. As you work with the Pivot Builder to refine your visualizations, the underlying search is rerun as required so you can see the effect of your changes.

When you save a visualization that you create in the Pivot Editor as a report or dashboard panel, a corresponding **data model** is created. This data model is the foundation of the saved report or dashboard panel. It defines the underlying search and the fields involved in the report or dashboard panel. Without it you cannot rerun the report or view the panel that you saved.

Open a search in Pivot

1. In the Search view, run a non-transforming search. For example:

```
sourcetype=access_* status=200 action=purchase
```

2. Go to the **Statistics** or **Visualization** tab and click **Pivot**.
3. Select the set of fields that you want to use to build your pivot table or chart in the Pivot Builder.

Each option displays the number of fields it represents in parenthesis:

All Fields provides all of the fields that were discovered by the search.

Selected Fields restricts you to the fields identified as **Selected Fields** for the search on the **Fields** tab. If you open a search in Pivot without making changes or selecting fields, the **Selected Fields** option provides the default selected fields: **host**, **source**, and **sourcetype**. To build your pivot table or chart using a different set of fields, go to the **Fields** tab and select the fields in the Selected Fields list before you move to the **Statistics** or **Visualization** tab and open the search in Pivot.

Fields with at least lets you set a coverage threshold for your fieldset. For example, to work with fields that apply to the majority of your events, set the threshold to something high, like 70%. The fieldset you get in Pivot only includes fields that exist in 70% (or more) of the events returned by the search.

4. Click **Ok** to go to the **Pivot Editor**.
5. Build your pivot table or chart.

The **Attributes** list in the pivot element types (filters, split rows, split columns, and column values) contains the fieldset that you selected in step 3.

Note: If you navigate away from the Pivot Editor without saving your table or chart, your work is lost.

To save your work, see the next subtopic, "Save the finished pivot table or chart."

While in the Pivot Editor, you can click **Open in Search** to open the pivot search in the Search interface and edit that search. This action takes you out of the Pivot Editor and prevents you from saving any pivot table or chart you created (see the next subtopic). For more information about using the Pivot Editor to design tables, see "[Design pivot tables with the Pivot Editor](#)." For more information about using the Pivot Editor to design charts and other visualizations, see "[Design pivot charts and visualizations with the Pivot Editor](#)."

Save the finished pivot table or chart

You can save a table or chart in the Pivot Editor as a report or dashboard panel. However, Splunk software must also create a data model to support the saved report or panel. This data model is required to access the report or panel after you have saved it.

1. In the Pivot Editor, click **Save As** and select either *Report* or *Dashboard Panel*.

Depending on which one you choose, either the Save As Report or Save As Dashboard Panel dialog box appears.

2. In the **Save As** dialog box provide information for the report or dashboard panel that you are saving.

For more information about these fields, see the documentation on [saving reports](#) or [saving dashboard panels](#).

3. In the **Save as** dialog box type the **Model Name** and **Model ID** for the data model that will support the report or dashboard panel.

You can manage the model that Splunk Enterprise creates through this process if your role has admin-level capabilities.

4. Click **Save** to save the report or dashboard panel and create the data model.

Click a button to view the new report or dashboard panel, or go to the new data model by clicking the name of the model.

Click the data model name to go to the Data Model Builder, where you can change the fields associated with the model and add data model dataset to the model.

About permissions for datasets created though this method

Newly created data models are private and can only be seen and used by the person who created them. Only users with admin or power roles (or a role with equivalent permissions) can share data models. If the data model is not shared, reports or dashboard panels created with that data model cannot be shared either. In addition, data models cannot be accelerated until they are shared.

If you have created a report or dashboard panel that is for your use only, you do not have to do anything. If you want other users to be able to access the report or dashboard panel, share the related data model, if you have appropriate permissions, or have a user with admin-level permissions share it for you.

For more information about data model permissions see "[Manage data models](#)" in the *Knowledge Manager Manual*.

Search and Report in Real Time

About real-time searches and reports

With **real-time searches** and reports, you can search events before they are **indexed** and preview reports as the events stream in.

- You can design alerts based on real-time searches that run continuously in the background. Such **real-time alerts** can provide timelier notifications than alerts that are based on **scheduled reports**. For more information, see the [Alerting Manual](#).
- You can also display real-time search results and reports in **dashboards**. For more information, see the dashboard [overview](#) in *Dashboards and Visualizations*.

CAUTION: The number of concurrent real-time searches can greatly affect indexing performance. To lessen the impact on the indexer, you can enable indexed real-time searches, which is described later in this topic. For more information, see [Expected performance and known limitations of real-time searches and reports](#).

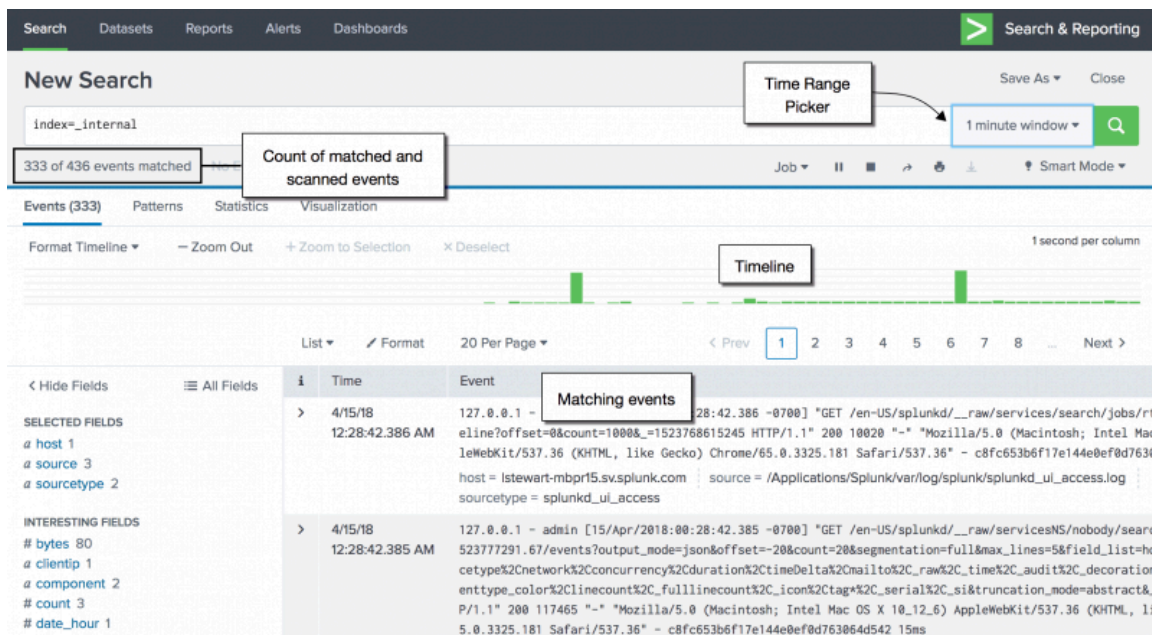
Note: Users must have the Admin role to run and save real-time searches. For more information on managing roles and assigning roles to users, see [Create and manage roles with Splunk Web](#) in *Securing Splunk Enterprise*. Real-time searches are turned on by default in Splunk Enterprise and Splunk Cloud Platform on Victoria Experience. In Splunk Cloud Platform on Classic Experience, you must open a support ticket to turn on real-time search. For more information, see [Differences between Splunk Cloud Platform and Splunk Enterprise](#) in the *Splunk Cloud Platform Service Description*.

Real-time search mechanics

Real-time searches scan events as the events arrive for indexing. When you kick off a real-time search, Splunk software scans the incoming events. The scan looks for events that contain index-time fields that indicate the event *could* be a match for your search.

As the real-time search runs, the software periodically evaluates the scanned events against your search criteria to find actual matches within the sliding **time range window** that you have defined for the search. The number of matching events can fluctuate up or down over time as the search discovers matching events at a faster or slower rate. If you are running the search in Splunk Web, the search timeline also displays the matching events that the search has returned within the chosen time range.

Here is an example of a real-time search with a one minute time range window. At the point that the following screen capture was taken, the search had scanned a total of 436 events since it was launched. The matching event count of 333 represents the number of events matching the search criteria that were identified in the past minute. This number fluctuated between 312 and 357 for the following minute. If the number spiked or dropped dramatically, that could indicate that something interesting was happening that requires a closer look.



The screenshot shows the Splunk Search interface. At the top, there's a navigation bar with 'Search', 'Datasets', 'Reports', 'Alerts', and 'Dashboards'. Below this, the 'New Search' section displays the search query 'index=_internal'. A 'Time Range Picker' is set to '1 minute window'. A box highlights '333 of 436 events matched', and another box highlights 'Count of matched and scanned events'. The 'Events (333)' tab is active, showing a list of events. A 'Timeline' view is also visible, showing events as green bars on a timeline. The event list includes fields like 'Time', 'Event', and 'source'.

As you can see, the newest events are on the right side of the timeline. As time passes, the events move left until the events move off the left side, disappearing from the time range window entirely.

A real-time search should continue running until you or another user stops the search or deletes the search job. The real-time search should not "time out" for any other reason. If your events are stopping it could be a performance-related issue (see [Expected performance and known limitations](#)).

Real-time searches can take advantage of all search functionality, including advanced functionality like lookups, transactions, and so on. There are also search commands that are to be used specifically in conjunction with real-time searches, such as [streamstats](#) and [rtorder](#).

Note that real-time searches are resource intensive and can impact the overall health and performance of your searches.

Indexed real-time search

By default, when you run a real-time search, the search runs before events are indexed. Alternatively, you can configure your real-time searches to run after the events are indexed, which can greatly improve indexing performance. This is especially true if you're running a lot of concurrent real-time searches because **indexed real-time search** decreases the impact on the indexer. Indexed real-time search runs searches like **historical searches**, but also continually updates the search with new events as the events appear on disk.

Use indexed real-time search when up-to-the-second accuracy is not needed.

Splunk Cloud Platform

For Splunk Cloud Platform on Victoria Experience, indexed real-time search is turned on.

Splunk Enterprise

For Splunk Enterprise, indexed real-time search is turned off by default. To turn on indexed real-time search, follow these steps.

Prerequisites

- Only users with file system access, such as system administrators, can turn on indexed real-time search.

- Review the steps in [How to edit a configuration file](#) in the *Admin Manual*.

Note: Never change or copy the configuration files in the `default` directory. The files in the `default` directory must remain intact and in their original location. Make the changes in the `local` directory.

Steps

1. Open the local `limits.conf` file for the Search app. For example,
`$SPLUNK_HOME/etc/apps/<app_name>/local`.
2. Under the `[realtime]` stanza, set `indexed_realtime_use_by_default` to `true`.

Indexed real-time search and newly added search peers

If your deployment includes newly added **search peers** and you are using indexed real-time search, all events from the new search peers show up by default in **historical searches**. However, existing indexed real-time searches will not pick up events from the search peers until you kick off another indexed real-time search. As a result, you should restart your indexed real-time searches every time you add new search peers.

If you want to automatically pick up all events in your real-time searches after you've added new search peers to your deployment, use real-time search instead of *indexed* real-time search. You can do this by changing the `indexed_realtime_use_by_default` setting in the local `limits.conf` file from `true` back to the default, which is `false`. See [How to edit a configuration file](#) in the Splunk Enterprise *Admin Manual*.

Note: Using real-time search by setting `indexed_realtime_use_by_default` to `false` makes events available to searches with lower latency, but reduces indexing throughput.

About the sync delay lag time

The results returned by an *indexed* real-time search will always lag behind a real-time search. Built into indexed real-time searches is a sync (synchronizing) delay. The sync delay is a precaution so that none of the data is missed.

Indexed data does not necessarily appear on disk in the order that the data is indexed because:

- Multiple threads are used for indexing simultaneously
- The sync delay ordering that is on your operating system

An indexed real-time must remember the latest indexed event that is returned for the current iteration of the time range window. That event is used as the start point for the next iteration of the time range window. If a sync delay is not imposed, some of the events before the latest event might not be searchable yet. These events are not returned during that iteration of the time range window and will never be returned. The likelihood of an unreturned event increases as the indexing and system load increases.

You can control the number of seconds of sync delay lag time with the `indexed_realtime_disk_sync_delay = <int>` setting. By default, this delay is set to 60 seconds.

The default of 60 seconds is fairly conservative. For most systems a 30 second delay will probably work successfully. If, for your system and usage, it is acceptable for indexed real-time searches to miss some events, you can set a very low or 0 sync delay. However, you will not be able to tell if you are missing events, except for searches that should match all events.

Other indexed real time settings

There are other settings that you can use to configure indexed real-time search behavior, including:

- `indexed_realtime_default_span`
- `indexed_realtime_maximum_span`
- `indexed_realtime_cluster_update_interval`

These settings are described in the [limits.conf.spec](#) file.

See also

- [Real-time searches and reports in Splunk Web](#)
- [Real-time searches and reports in the CLI](#)
- [Expected performance and known limitations of real-time searches and reports](#)
- [How to restrict usage of real-time searches](#)

Blogs

- [Splunk: Limiting Real-Time Searches and Maximizing Performance Gainz](#) by Hurricane Labs

Real-time searches and reports in Splunk Web

Real-time searches in Splunk Web

You run a **real-time search** in exactly the same way you run **historical searches**. However, because you are searching a live and continuous stream of data, the timeline updates as the events stream in and you can only view the report in preview mode. Also, some search commands are more applicable to real-time searches than historical searches. For example, [streamstats](#) and [rtorder](#) were designed for use in real-time searches.

To kick off a real-time search in Splunk Web, use the time range menu to select a preset real-time **time range window**, such as **30 seconds** or **1 minute**. You can also [specify a sliding time range window](#) to apply to your real-time search.

If you have Apache web access data, run the following search to see web traffic events as they stream in.

```
sourcetype=access_*
```

The raw events that are streamed from the input pipeline are not time-ordered. You can use the [rtorder](#) command to buffer the events from a real-time search and emit them in ascending time order.

The following example keeps a buffer of the last 5 minutes of web traffic events, emitting events in ascending time order once they are more than 5 minutes old. Newly received events that are older than 5 minutes are discarded if an event after that time has already been emitted.

```
sourcetype=access_* | rtorder discard=t buffer_span=5m
```

Real-time search relies on a stream of events. Thus, you cannot run a real-time search with any other leading search command, such as `| metadata` which does not produce events or `| inputcsv` which just reads in a file. Also, if you try to send the search results to `| outputcsv`, the CSV file will not be written until the real-time search is Finalized.

Real-time reports in Splunk Web

Run a report to preview the IP addresses that access the most web pages. In this case, the `top` command returns a table with three columns: clientip, count, and percent. As the data streams in, the table updates with new values.

```
sourcetype=access_* | top clientip
```

For each web traffic event, add a `count` field that represents the number of events seen so far (but do not include the current event in the count).

```
sourcetype=access_* | streamstats count current=false
```

You can also drilldown into real-time reports. However, real-time drilldown does not spawn another real-time search. Instead, it spawns a historic search, as you will drilldown into the events that have already been retrieved and indexed. For more information, see [Use drilldown for dashboard interactivity](#) in *Dashboards and Visualizations*.

See also

- [About real-time searches and reports](#)
- [Real-time searches and reports in the CLI](#)
- [Expected performance and known limitations of real-time searches and reports](#)
- [How to restrict usage of real-time searches](#)

Real-time searches and reports in the CLI

To run a real-time search in the CLI, replace the command "search" with "rtsearch":

```
./splunk rtsearch 'eventtype=pageview'
```

Use the `highlight` command to emphasize terms in your search results. The following example highlights "GET" in your page view events:

```
./splunk rtsearch 'eventtype=pageview | highlight GET'
```

By default, search results have line wrapping enabled. Use the **-wrap** option to turn off line wrapping:

```
./splunk rtsearch 'eventtype=pageview' -wrap 0
```

Real-time reports in the CLI will also display in preview mode and update as the data streams in.

```
./splunk rtsearch 'error | top clientip'
```

Use the **-preview** option to suppress the results preview:

```
./splunk rtsearch 'error | top clientip' -preview false
```

If you turn off preview, you can still manage (Save, Pause, Finalize, or Delete) the search from the Jobs page in Splunk Web. After you finalize the search, the report table will display. For more information, see ["Supervise jobs with the Jobs page"](#) in this manual.

To run a windowed real-time search, use the **earliest_time** and **latest_time** parameters. Real-time parameter values must be enclosed in single quotation marks.

```
rtsearch 'index=_internal' -earliest_time 'rt-30s' -latest_time 'rt+30s'
```

Note: Real-time searches can only be set at the API level, so the search does not run if you try to specify the time range modifiers within the search string. The **earliest_time** and **latest_time** parameters should set the same-name arguments in the REST API.

See [Search endpoint descriptions](https://docs.splunk.com/Documentation/Splunk/RESTREF/RESTsearch). No Content found for https://docs.splunk.com/Documentation/Splunk/RESTREF/RESTsearch. Then navigate to **search/jobs**, under POST click **Expand**, and locate **search_mode**.

You can view all CLI commands by accessing the CLI help reference. For more information, see ["Get help with the CLI"](#) in this manual.

See also

- [About real-time searches and reports](#)
- [Real-time searches and reports in Splunk Web](#)
- [Expected performance and known limitations of real-time searches and reports](#)
- [How to restrict usage of real-time searches](#)

Expected performance and known limitations of real-time searches and reports

Real-time search matches events that have arrived at the port but have not been persisted to disk. The rate of arrival of events and number of matches can determine how much memory is consumed and the impact on the indexing rate.

Indexing throughput

Splunk software performance is expected to be acceptable as long as the indexers are not currently heavily loaded and do not have more than a few concurrent real-time searches.

CAUTION: Real-time searches will have a significant impact on performance in high volume environments and network load when you have many concurrent real-time searches.

When planning your real-time searches, you should consider how it will affect the performance of both:

- The **search peer** that must stream the live events.
- The **search head** that must process the aggregated stream of live events.

The more work that is done on the search peer, the less that is required on the search head, and vice versa. The search peer is important to the overall system function, so you do not want to burden it with too much filtering of live events. However, if the search peer does not filter at all, the processing power and bandwidth required to send all the live events to the search head may prove costly, especially when you have multiple real-time searches running concurrently.

In cases where the search head cannot keep up with the search peer, the queue on the index processor will cease to flag events for the search. However, the events will have a sequence number that you can use to tell when and how many events were omitted from search consideration.

Concurrent real-time and historical searches

You can run real-time and historical searches concurrently, within the limits of your hardware. There are no restrictions on separate searches for the same or different users.

Concurrent real-time searches

Running multiple real-time searches will negatively impact indexing capacity. The real-time search feature is optimized for real-time alerting on sparse, or rare-term, searches and sacrifices indexing capacity for improved latency and reliability.

Indexed real-time searches

The number of concurrent real-time searches can greatly affect indexing performance. To lessen the impact on the indexer, you can enable indexed real-time search. This will basically run the search like a historical search, but will also continually update it with new events as they appear on disk.

Read more about how to enable indexed real-time search in [About real-time searches and reports](#).

Real-time search windows

Windowed real-time searches are more expensive than non-windowed. The operations required to manage and preview the window contents can result in a windowed real time search not keeping up with a high rate of indexing. If your windowed search does not display the expected number of events, try a non-windowed search. If you are interested only in event counts, try using "timechart count" in your search.

See [Specify time ranges for real-time searches](#).

See also

- [About real-time searches and reports](#)
- [Real-time searches and reports in Splunk Web](#)
- [Real-time searches and reports in the CLI](#)
- [How to restrict usage of real-time searches](#)

How to restrict usage of real-time search

Because overuse of real-time search can result in performance costs, you may find it necessary to restrict its usage.

Options for restricting real-time search are as follows:

- Disable real-time search at the indexer level by editing `indexes.conf` for specific indexes.
- Disable real-time search for particular roles and users.
- Edit `limits.conf` to reduce the number of real-time searches that can be run concurrently at any given time.
- Edit `limits.conf` to restrict indexer support for real-time searches.

Note: For Splunk Cloud Platform, if you contacted Support to enable real-time searches, you must contact Support to restrict real-time searches.

Disable real-time search in `indexes.conf`

Searching in real time may be very expensive on the indexer. If you want to disable it on an indexer, you can edit a `[default]` setting in that indexer's `indexes.conf`. Note that this setting cannot be overridden on an index-by-index basis, it applies to all indexes located on the indexer.

```
[default]
enableRealtimeSearch = <bool>
```

Note: A *search head* that connects to multiple indexers will still be able to get real-time search results from the indexers that do have it enabled.

CAUTION: Consult with Support before changing the `enableRealtimeSearch` setting in the `indexes.conf` file. Setting `enableRealtimeSearch=false` might prevent the ITSI Rules Engine from working properly. See [Real-time search requirements](#) in the *IT Service Intelligence Install and Upgrade Manual*.

Disable real-time search for a user or role

Real-time search is a **capability** that you can map to specific users or roles in Splunk Web from **Manager > Access Controls**. By default, the **rtsearch** capability is assigned to the Admin and Power roles and not the User role. A role without the **rtsearch**

capability will not be able to run a real-time search on that search head, regardless what indexers that search head is connected to.

Set search limits on real-time searches

You can use the `[search]` stanza in `limits.conf` to change the maximum number of real-time searches that can run concurrently on your system.

```
[search]
max_rt_search_multiplier = <decimal number>
realtime_buffer = <int>
```

max_rt_search_multiplier

- A number by which the maximum number of historical searches is multiplied to determine the maximum number of concurrent real-time searches. Defaults to 1.
- **Note:** The maximum number of real-time searches is computed as:
$$\text{max_rt_searches} = \text{max_rt_search_multiplier} \times \text{max_hist_searches}$$

realtime_buffer

- The maximum number of accessible events to keep for real-time searches from the UI. Must be ≥ 1 . Defaults to 10000.
- The real-time buffer acts as a circular buffer once this limit is reached.

Set indexer limits for real-time search

You can use the `[realtime]` stanza in `limits.conf` to change the default settings for indexer support of real-time searches. **These options can be overridden for individual searches via REST API arguments.**

```
[realtime]
queue_size = <int>
blocking = [0|1]
max_blocking_secs = <int>
indexfilter = [0|1]
```

queue_size = <int>

- The size of queue for each real-time search. Must be > 0 .
- Defaults to 10000.

blocking =[0|1]

- Specifies whether the indexer should block if a queue is full.
- Defaults to false (0).

max_blocking_secs = <int>

- The maximum time to block if the queue is full. This option is meaningless, if **blocking = false**.
- Means "no limit" if set to 0.
- Defaults to 60.

indexfilter = [0|1]

- Specifies whether the indexer should pre-filter events for efficiency.
- Defaults to true (1).

See also

- [About real-time searches and reports](#)
- [Real-time searches and reports in Splunk Web](#)
- [Real-time searches and reports in the CLI](#)
- [Expected performance and known limitations of real-time searches and reports](#)

Evaluate and Manipulate Fields

About evaluating and manipulating fields

This section discusses the search commands that enable you to evaluate new fields, manipulate existing fields, enrich events by adding new fields, and parse fields with multiple values.

- At the core of evaluating new fields is the **eval** command and its functions. Unlike the **stats** command, which enables you to calculate statistics based on fields in your events, the **eval** command enables you to create new fields using existing fields and an arbitrary expression. The **eval** command has many functions. See [Use the eval command and functions](#).
- You can easily enrich your data with more information at search time. See [Use lookup to add fields from external lookup tables](#).
- You can use the Splunk SPL (search processing language) to [extract fields in different ways using a variety of search commands](#).
- Your events might contain fields with more than one value. There are search commands and functions that work with multivalue fields. See [Manipulate and evaluate fields with multiple values](#).

Use the eval command and functions

The **eval** command enables you to devise arbitrary expressions that use automatically extracted fields to create a new field that takes the value that is the result of the expression's evaluation. The **eval** command is versatile and useful. Although some **eval** expressions seem relatively simple, they often can be quite complex.

This topic discusses how to use the [eval command](#) and the [evaluation functions](#).

Types of eval expressions

An **eval** expression is a combination of literals, fields, operators, and functions that represent the value of your destination field. The expression can involve a mathematical operation, a string concatenation, a comparison expression, a boolean expression, or a call to one of the **eval** functions. **eval** expressions require that the field's values are valid for the type of operation.

For example, with the exception of addition, arithmetic operations may not produce valid results if the values are not numerical. For addition, **eval** can concatenate the two operands if they are both strings. When concatenating values with a period (.), the **eval** command treats both values as strings, regardless of their actual type.

You can think of an **eval** expression as everything that follows the **eval** command, typically to the next pipe. For example, this is an **eval** expression:

```
... | eval location=city.", ".state
```

However, the **eval** expression in this search is **(eval(status=404))**, which is followed by an **AS** clause:

```
sourcetype=access_* | stats count(eval(status=404)) AS status_count
```

Example 1: Use an eval expression with a stats function

This example searches the **status** field which contains HTTP status codes like 200 and 404.

If you run this search, it returns a total count of all events with a value in the **status** field.

```
sourcetype=access_* | stats count(status)
```

This results look something like this:

count(status)
39532

You can add a BY clause to organize the count by HTTP code.

```
sourcetype=access_* | stats count(status) by status
```

This results look something like this:

status	count(status)
200	34282
400	701
403	228
404	690
406	710
408	756
500	733
503	952
505	480

However if you want the results for only one specific status, you can use an `<eval-expression>`. Because the `<eval-expression>` returns an evaluated field, you must use the AS keyword to specify a name for the evaluated field.

This search retrieves all of the events where the sourcetype is any Apache web access log and counts the number of events where the **status** field value is **404**. The results are placed into a field called **status_count**.

```
sourcetype=access_* | stats count(eval(status=404)) AS status_count
```

The results look something like this:

status_count
690

You can organize the results using a BY clause.

```
sourcetype=access_* | stats count(eval(status="404")) AS status_count BY source
```

The results look something like this:

source	status_count
tutorialdata.zip:/www1/access.log	244
tutorialdata.zip:/www2/access.log	209
tutorialdata.zip:/www3/access.log	237

Example 2: Define a field that is the sum of the areas of two circles

Use the **eval** command to define a field that is the sum of the areas of two circles, A and B.

```
... | eval sum_of_areas = pi() * pow(radius_a, 2) + pi() * pow(radius_b, 2)
```

The area of circle is πr^2 , where r is the radius. For circles A and B, the radii are `radius_a` and `radius_b`, respectively. This **eval** expression uses the **pi** and **pow** functions to calculate the area of each circle and then adds them together, and saves the result in a field named, `sum_of_areas`.

Example 3: Define a location field using the city and state fields

Use the **eval** command to define a location field using the city and state fields. For example, if the `city=Philadelphia` and `state=PA`, `location="Philadelphia, PA"`.

```
... | eval location=city.", ".state
```

This **eval** expression is a simple string concatenation.

Example 4: Use eval functions to classify where an email came from

This example uses sample email data. You should be able to run this search on any email data by replacing the `sourcetype=cisco:esa` with the `sourcetype` value and the `mailfrom` field with email address field name in your data. For example, the email might be `To`, `From`, or `Cc`).

This example classifies where an email came from based on the email address domain. The `.com`, `.net`, and `.org` addresses are considered **local**, while anything else is considered **abroad**. There are many domain names. Of course, domains that are not `.com`, `.net`, or `.org` are not necessarily from **abroad**. This is just an example.

The **eval** command in this search contains multiple expressions, separated by commas.

```
sourcetype="cisco:esa" mailfrom=*| eval accountname=split(mailfrom,"@"), from_domain=mvindex(accountname,-1),
location=if(match(from_domain, "[^\\n\\r\\s]+\\.(com|net|org)"), "local", "abroad") | stats count BY location
```

The first half of this search is similar to previous example. The `split()` function is used to break up the email address in the `mailfrom` field. The `mvindex` function defines the `from_domain` as the portion of the `mailfrom` field after the `@` symbol.

Then, the `if()` and `match()` functions are used.

- If the `from_domain` value ends with a `.com`, `.net.`, or `.org`, the `location` field is assigned the value `local`.
- If `from_domain` does not match, `location` is assigned the value `abroad`.

The `eval` results are then piped into the `stats` command to count the number of results for each `location` value.

The results appear on the Statistics tab and look something like this:

location	count
abroad	3543
local	14136

Note: This example merely illustrates using the `match()` function. If you want to classify your events and quickly search for those events, the better approach is to use event types. Read more [about event types](#) in the *Knowledge manager manual*.

Defining calculated fields

If you find that you use a particular eval expression on a regular basis, consider defining the field as a calculated field. Doing this means that when you're writing a search, you can omit the eval expression and refer to the field like you do any other extracted field. When you run the search, the fields will be extracted at search time and will be added to the events that include the fields in the eval expressions.

Read more about how to configure this in [Define calculated fields](#) in the *Knowledge Manager Manual*.

Use lookup to add fields from lookup tables

You can match fields in your events to fields in external sources, such as lookup tables, and use these matches to add more information inline to your events.

A lookup table can be a static CSV file, a KV store collection, or the output of a Python script. You can also use the results of a search to populate the CSV file or KV store collection and then set that up as a lookup table. For more information about field lookups, see [Configure CSV and external lookups](#) and [Configure KV store lookups](#) in the *Knowledge Manager Manual*.

After you configure a fields lookup, you can invoke it from the Search app with the `lookup` command.

Example

You have a field lookup named `dnslookup` which references a Python script that performs a DNS and reverse DNS lookup and accepts either a host name or IP address as arguments. You can use the `lookup` command to match the host name values in your events to the host name values in the lookup table, and add the corresponding IP address values to your events.

```
... | lookup dnslookup clienthost AS host OUTPUT clientip
```

See also

Related information

[Configure external lookups](#) in the *Knowledge Manager Manual*

Extract fields with search commands

You can use search commands to extract fields in different ways.

- The [rex](#) command performs field extractions using named groups in Perl regular expressions.
- The [extract](#) (or **kv**, for key/value) command explicitly extracts field and value pairs using default patterns.
- The [multikv](#) command extracts field and value pairs on multiline, tabular-formatted events.
- The [spath](#) command extracts field and value pairs on structured event data, such as XML and JSON.
- The [xmlkv](#) and [xpath](#) commands extract field and value pairs on XML-formatted event data.
- The [kvform](#) command extracts field and value pairs based on predefined form templates.

In Splunk Web, you can define field extractions on the **Settings > Fields > Field Extractions** page.

The following sections describe how to extract fields using regular expressions and commands. See [About fields](#) in the *Knowledge Manager Manual*.

Extract fields using regular expressions

The [rex](#) command performs field extractions using named groups in Perl regular expressions that you include in the search criteria. The **rex** command matches segments of your raw events with the regular expression and saves these matched values into a field.

In this example, values that occur after the strings **From:** and **To:** are saved into the **from** and **to** fields.

```
... | rex field=_raw "From: (?<from>.*) To: (?<to>.*)"
```

If a raw event contains **From: Susan To: Bob**, the search extracts the field name and value pairs: **from=Susan** and **to=Bob**.

For a primer on regular expression syntax and usage, see www.regular-expressions.info. The following are useful third-party tools for writing and testing regular expressions:

- [regex101](#)
- [RegExr](#)
- [Debuggex](#)

Extract fields from .conf files

The `extract` command forces field/value extraction on the result set. If you use the `extract` command without specifying any arguments, fields are extracted using field extraction stanzas that have been added to the `props.conf` file. You can also use the `extract` command to test field extractions that you add to the conf files.

Extract fields from events formatted as tables

Use the `multikv` command to force field and value extractions on multiline, tabular-formatted events. The `multikv` command creates a new event for each table row and derives field names from the table title.

Extract fields from events formatted in XML

The `xmlkv` command enables you to force field and value extractions on XML-formatted tags in event data, such as transactions from web pages.

Extract fields from XML and JSON documents

The `spath` command extracts information from structured data formats, such as XML and JSON, and store the extracted values in fields.

Extract fields from events based on form templates

The `kvform` command extracts field and value pairs from events based on form templates that are predefined and stored in `$SPLUNK_HOME/etc/system/local/`, or your own custom application directory in `$SPLUNK_HOME/etc/apps/`. For example, if `form=sales_order`, the search looks for a `sales_order.form`, and matches all processed events against that form to extract values.

Note: For Splunk Cloud Platform, you must create a private app to extract fields using form templates. If you are a Splunk Cloud administrator with experience creating private apps, see [Manage private apps in your Splunk Cloud deployment](#) in the *Splunk Cloud Admin Manual*. If you have not created private apps, contact your Splunk account representative for help with this customization.

Evaluate and manipulate fields with multiple values

About multivalue fields

A multivalue field is a field that contains more than one value. For example, events such as email logs often have multivalue fields in the To: and Cc: information.

Multivalue fields are parsed at **search time**, which enables you to process the values in the search pipeline. Search commands that work with multivalue fields include `makemv`, `mvcombine`, `mvexpand`, and `nomv`. The `eval` and `where` commands support

functions, such as `mvcount()`, `mvfilter()`, `mvindex()`, and `mvjoin()` that you can use with multivalue fields. See [Evaluation functions](#) in the *Search Reference* and the examples in this topic.

You can also use the statistical eval functions, `max` and `min`, on multivalue fields. See [Statistical eval functions](#) in the *Search Reference*.

If you are using Splunk Enterprise, you can configure multivalue fields in the `fields.conf` file to specify how Splunk software detects more than one field value in a single extracted field value. Edit the `fields.conf` in `$SPLUNK_HOME/etc/system/local/`, or your own custom application directory in `$SPLUNK_HOME/etc/apps/`. For more information on how to do this, see [Configure extractions of multivalue fields with fields.conf](#) in the *Knowledge Manager Manual*.

If your search produces results, such as a table, the results get written to the `results.csv.gz` file. The contents of the `results.csv.gz` file include fields that begin with "`__mv__`". These fields are for internal use only and are used to encode multivalue fields.

Note: For Splunk Cloud Platform, you must create a private app to configure multivalue fields. If you are a Splunk Cloud Platform administrator with experience creating private apps, see [Manage private apps in your Splunk Cloud Platform deployment](#) in the *Splunk Cloud Platform Admin Manual*. If you have not created private apps, contact your Splunk account representative for help with this customization.

Evaluate multivalue fields

One of the more common examples of multivalue fields is email address fields, which typically appear two or three times in a single sendmail event—one time for the sender, another time for the list of recipients, and possibly a third time for the list of Cc addresses.

Count the number of values in a field

Use the `mvcount()` function to count the number of values in a single value or multivalue field.

In this example, `mvcount()` returns the number of email addresses in the To, From, and Cc fields and saves the addresses in the specified "`_count`" fields.

```
eventtype="sendmail" | eval To_count=mvcount(split(To,"@"))-1 | eval From_count=mvcount(From) | eval Cc_count=mvcount(split(Cc,"@"))-1
```

This search takes the values in the `To` field and uses the `split` function to separate the email address on the @ symbol. The `split` function is also used on the `Cc` field for the same purpose.

If only a single email address exists in the `From` field, as you would expect, `mvcount(From)` returns 1. If there is no Cc address, the Cc field might not exist for the event. In that situation `mvcount(cc)` returns NULL.

Filter values from a multivalue field

Use the **mvfilter()** function to filter a multivalue field using an arbitrary Boolean expression. The **mvfilter** function works with only one field at a time.

In this example, **mvfilter()** keeps all of the values for the field **email** that end in **.net** or **.org**.

```
eventtype="sendmail" | eval email=mvfilter(match(email, "\.net$") OR match(email, "\.org$"))
```

Note: This example also uses the **match()** function to compare the pattern defined in quotes to the value of **email**. See [Evaluation functions](#) in the *Search Reference*.

Return a subset of values from a multivalue field

Use the **mvindex()** function to reference a specific value or a subset of values in a multivalue field. Since the index numbering starts at 0, if you want to reference the 3rd value of a field, you would specify it as 2.

In this example, **mvindex()** returns the first email address in the "To" field for every email sent by Sender:

```
eventtype="sendmail" from=Sender@* | eval to_first=mvindex(to,0)
```

If you want to see the top 3 email addresses that Sender writes to, use the following search.

```
eventtype="sendmail" from=Sender@* | eval top_three=mvindex(to,0,2)
```

In this example, **top_three** is, itself, a multivalue field.

Manipulate multivalue fields

Use **nomv** to convert a multivalue field into a single value

You can use the **nomv** command to convert values of the specified multivalue field into one single value. The **nomv** command overrides the multivalue field configurations that are set in **fields.conf** file.

In this example for sendmail events, you want to combine the values of the senders field into a single value.

```
eventtype="sendmail" | nomv senders
```

Use makemv to separate a multivalue field

You can use the **makemv** command to separate multivalue fields into multiple single value fields. In this example for sendmail search results, you want to separate the values of the senders field into multiple field values.

```
eventtype="sendmail" | makemv delim="," senders
```

After you separate the field values, you can pipe it through other commands. For example, you can display the top senders.

```
eventtype="sendmail" | makemv delim="," senders | top senders
```

Use mvexpand to create multiple events based on a multivalue field

You can use the **mvexpand** command to expand the values of a multivalue field into separate events for each value of the multivalue field. In this example, new events are created for each value in the multivalue field, "foo".

```
... | mvexpand foo
```

Use mvcombine to create a multivalue field from similar events

Combine the values of "foo" with ":" delimiter.

```
... | mvcombine delim=":" foo
```

See also

[Configure extractions of multivalue fields with fields.conf](#) in the *Knowledge Manager Manual*.

Calculate Statistics

About calculating statistics

This section discusses how to calculate summary statistics on events. When you think about calculating statistics with Splunk's search processing language (SPL), the **stats** command is probably what comes to mind first. The stats command generates reports that display summary statistics in a tabular format. Additionally, you can use the **chart** and **timechart** commands to create charted visualizations for summary statistics and the **geostats** command to create map visualizations for summary statistics of events that include geographical location fields.

The **stats**, **chart**, and **timechart** commands (and their related commands **eventstats**, **geostats** and **streamstats**) are designed to work in conjunction with statistical functions. For examples of searches using these commands and functions, read ["Use the stats command and functions"](#).

Later topics discuss how to:

- ["Use stats with eval expressions and functions"](#) to calculate statistics.
- ["Add sparklines to report tables"](#).

The [Advanced statistics](#) section contains topics on detecting anomalies, finding and removing outliers, detecting patterns, and time series forecasting.

Use the stats command and functions

This topic discusses how to use the statistical functions with the **transforming commands** **chart**, **timechart**, **stats**, **eventstats**, and **streamstats**.

- For more information about the stat command and syntax, see the ["stats"](#) command in the *Search Reference*.
- For the list of stats functions, see ["Statistical and charting functions"](#) in the *Search Reference*.

About the stats commands and functions

The [stats](#), [streamstats](#), and [eventstats](#) commands each enable you to calculate summary statistics on the results of a search or the events retrieved from an index. The **stats** command works on the search results as a whole. The **streamstats** command calculates statistics for each event at the time the event is seen, in a streaming manner. The **eventstats** command calculates statistics on all search results and adds the aggregation inline to each event for which it is relevant. See more about the differences between these commands in the next section.

The [chart command](#) returns your results in a data structure that supports visualization as a chart (such as a column, line, area, and pie chart). You can decide what field is tracked on the x-axis of the chart. The [timechart command](#) returns your results formatted as a time-series chart, where your data is plotted against an x-axis that is always a time field. Read more about visualization features and options in the [Visualization Reference](#) of the Data Visualization Manual.

The **stats**, **chart**, and **timechart** commands (and their related commands **eventstats** and **streamstats**) are designed to work in conjunction with statistical functions. The list of statistical functions lets you count the occurrence of a field and calculate sums, averages, ranges, and so on, of the field values.

For the list of statistical functions and how they're used, see ["Statistical and charting functions"](#) in the *Search Reference*.

Stats, eventstats, and streamstats

The **eventstats** and **streamstats** commands are variations on the **stats** command.

The **stats** command works on the search results as a whole and returns only the fields that you specify. For example, the following search returns a table with two columns (and 10 rows).

```
sourcetype=access_* | head 10 | stats sum(bytes) as ASumOfBytes by clientip
```

The **ASumOfBytes** and **clientip** fields are the only fields that exist after the stats command. For example, the following search returns empty cells in the **bytes** column because it is not a result field.

```
sourcetype=access_* | head 10 | stats sum(bytes) as ASumOfBytes by clientip | table bytes, ASumOfBytes, clientip
```

To see more fields other than **ASumOfBytes** and **clientip** in the results, you need to include them in the stats command. Also, if you want to perform calculations on any of the original fields in your raw events, you need to do that before the stats command.

The **eventstats** command computes the same statistics as the **stats** command, but it also aggregates the results to the original raw data. When you run the following search, it returns an events list instead of a results table, because the eventstats command does not change the raw data.

```
sourcetype=access_* | head 10 | eventstats sum(bytes) as ASumOfBytes by clientip
```

You can use the **table** command to format the results as a table that displays the fields you want. Now, you can also view the values of **bytes** (or any of the original fields in your raw events) in your results.

```
sourcetype=access_* | head 10 | eventstats sum(bytes) as ASumOfBytes by clientip | table bytes, ASumOfBytes, clientip
```

The **streamstats** command also aggregates the calculated statistics to the original raw event, but it does this at the time the event is seen. To demonstrate this, include the **_time** field in the earlier search and use **streamstats**.

```
sourcetype=access_* | head 10 | sort _time | streamstats sum(bytes) as ASumOfBytes by clientip | table _time, clientip, bytes, ASumOfBytes
```

Instead of a total sum for each **clientip** (as returned by **stats** and **eventstats**), this search calculates a sum for each event based on the time that it is seen. The **streamstats** command is useful for reporting on events at a known time range.

Examples

Example 1

This example creates a chart of how many new users go online each hour of the day.

```
... | sort _time | streamstats dc(userid) as dcusers | delta dcusers as deltadcusers | timechart sum(deltadcusers)
```

The **dc** (or **distinct_count**) function returns a count of the unique values of **userid** and renames the resulting field **dcusers**.

If you don't rename the function, for example "dc(userid) as dcusers", the resulting calculation is automatically saved to the function call, such as "dc(userid)".

The **delta** command is used to find the difference between the current and previous **dcusers** value. Then, the sum of this delta is charted over time.

Example 2

This example calculates the median for a field, then charts the count of events where the field has a value less than the median.

```
... | eventstats median(bytes) as medbytes | eval snap=if(bytes>=medbytes, bytes, "smaller") | timechart count by snap
```

Eventstats is used to calculate the median for all the values of bytes from the previous search.

Example 3

This example calculates the standard deviation and variance of calculated fields.

```
sourcetype=log4j ERROR earliest=-7d@d latest=@d | eval warns=errorGroup+"-"+errorNum | stats count as Date_Warns_Count by date_mday,warns | stats stdev(Date_Warns_Count), var(Date_Warns_Count) by warns
```

This search returns errors from the last 7 days and creates the new field, warns, from extracted fields errorGroup and errorNum. The stats command is used twice. First, it calculates the daily count of warns for each day. Then, it calculates the standard deviation and variance of that count per warns.

Example 4

You can use the calculated fields as filter parameters for your search.

```
sourcetype=access_* | eval URILen = len(useragent) | eventstats avg(URILen) as AvgURILen, stdev(URILen) as StdDevURILen | where URILen > AvgURILen+(2*StdDevURILen) | chart count by URILen span=10 cont=true
```

In this example, eventstats is used to calculate the average and standard deviation of the URI lengths from **useragent**. Then, these numbers are used as filters for the retrieved events.

Use stats with eval expressions and functions

You can embed eval expressions and functions within any of the stats functions. This is a shorthand method for creating a search without using the **eval** command separately from the **stats** command.

For example, the following search uses the **eval** command to filter for a specific error code. Then the **stats** function is used to count the distinct IP addresses.

```
status=* | eval dc_ip_errors=if(status=404,clientip,NULL()) | stats dc(dc_ip_errors)
```

As an alternative, you can embed an eval expression using eval functions in a **stats** function directly to return the same results. For example:

```
status=* | stats dc(eval(if(status=404, clientip, NULL()))) AS dc_ip_errors
```

Use eval expressions to count the different types of requests against each Web server

This example uses the sample data from the Search Tutorial but should work with any format of Apache web access log. To try this example on your own Splunk instance, you must download the sample data and follow the instructions to [get the tutorial data into Splunk](#). Use the time range **All time** when you run the search.

Run the following search to use the **stats** command to determine the number of different page requests, GET and POST, that occurred for each Web server.

```
sourcetype=access_* | stats count(eval(method="GET")) AS GET, count(eval(method="POST")) AS POST BY host
```

This example uses **eval** expressions to specify the different field values for the **stats** command to count.

- The first clause uses the **count()** function to count the Web access events that contain the **method** field value **GET**. Then, using the AS keyword, the field that represents these results is renamed GET.
- The second clause does the same for POST events.
- The counts of both types of events are then separated by the web server, using the BY clause with the **host** field.

The results appear on the Statistics tab and look something like this:

host	GET	POST
www1	8431	5197
www2	8097	4815
www3	8338	4654

Use eval expressions to categorize and count fields

This example uses sample email data. You should be able to run this search on any email data by replacing the **sourcetype=cisco:esa** with the **sourcetype** value and the **mailfrom** field with email address field name in your data. For example, the email might be **To**, **From**, or **Cc**).

Find out how much of the email in your organization comes from .com, .net, .org or other top level domains.

The **eval** command in this search contains two expressions, separated by a comma.

```
sourcetype="cisco:esa" mailfrom=*
| eval accountname=split(mailfrom,"@"), from_domain=mvindex(accountname,-1)
| stats count(eval(match(from_domain, "[^\n\r\s]+\..com"))) AS ".com",
count(eval(match(from_domain, "[^\n\r\s]+\..net"))) AS ".net",
count(eval(match(from_domain, "[^\n\r\s]+\..org"))) AS ".org",
count(eval(NOT match(from_domain, "[^\n\r\s]+\..(com|net|org)"))) AS "other"
```

- The first part of this search uses the **eval** command to break up the email address in the **mailfrom** field. The **from_domain** is defined as the portion of the **mailfrom** field after the **@** symbol.
 - The **split()** function is used to break the **mailfrom** field into a multivalue field called **accountname**. The first value of **accountname** is everything before the "@" symbol, and the second value is everything after.
 - The **mvindex()** function is used to set **from_domain** to the second value in the multivalue field **accountname**.
- The results are then piped into the **stats** command. The stats **count()** function is used to count the results of the **eval** expression.
- The **eval** expression uses the **match()** function to compare the **from_domain** to a regular expression that looks for the different suffixes in the domain. If the value of **from_domain** matches the regular expression, the **count** is updated for each suffix, **.com**, **.net**, and **.org**. Other domain suffixes are counted as **other**.

The results appear on the Statistics tab and look something like this:

.com	.net	.org	other
4246	9890	0	3543

See also

Commands

[eval command](#) in the *Search Reference*

Related information

[Statistical and charting functions](#) in the *Search Reference*

[Evaluation functions](#) in the *Search Reference*

[About evaluating and manipulating fields](#)

Add sparklines to search results

If you are working with **stats** and **chart** searches, you can increase their usefulness and overall information density by adding sparklines to their result tables. Sparklines are inline charts that appear within table cells in search results, and are designed to display time-based trends associated with the primary key of each row.

Note: The size of the sparkline is defined by settings in the `limits.conf` file. The `sparkline_maxsize` setting defines the maximum number of elements to emit for a sparkline.

For example, say you have this search, set to run over events from the **Last 15 minutes**:

```
index=_internal | chart count by sourcetype
```

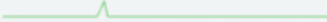
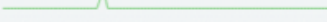
This search returns a two-column results table that shows event counts for the source types that have been indexed to `_internal` in the last 15 minutes. The first column lists each **sourcetype** found in the past hour's set of `_internal` index events; this is the primary key for the table. The second column, **count**, displays the event counts for each listed source type:

sourcetype	count
scheduler	18
splunk_archiver-2	18
splunk_web_access	223
splunk_web_service	226
splunkd	73486
splunkd_access	1745
splunkd_ui_access	5925

You can add sparklines to the results of this search by adding the **sparkline** function to the search itself:

```
index=_internal | chart sparkline count by sourcetype
```

This results in a table that is almost the same as the preceding one, except that now, for each row you have a sparkline chart that shows the event count trend for each listed source type over the past 15 minutes.

sourcetype	sparkline	count
splunk_web_access		8
splunk_web_service		6
splunkd		2421
splunkd_access		46
splunkd_ui_access		232

Now you can easily see patterns in your data that may have been invisible before. Some search activity apparently caused a bump in most `index=_internal` source types about three quarters into the 15 minute span. And **splunkd** has what almost looks like a regular heartbeat running over the entire span of time.

Note: Each sparkline in a table displays information in relation to the other events represented in that sparkline, but not in relation to the other sparklines. A peak in one sparkline does not necessarily have the same value as a peak in another sparkline.

Using sparklines with the stats and chart commands

You always use the sparklines feature in conjunction with **chart** and **stats** searches, because it is a function of those two search commands. It is not a command by itself. The functionality of sparklines is the same for both search commands.

Sparklines are not available as a dashboard chart visualization by themselves, but you can set up a dashboard panel with a table visualization that displays sparklines. For more information, see the "[Visualization reference](#)" topic in the Splunk Data Visualizations Manual.

For more information about the **chart** and **stats** commands, including details on the syntax around the **sparkline** function, see [chart](#) and [stats](#) in the *Search Reference*.

Example: Stats, sparklines, and earthquake data

Here are some examples of **stats** searches that use sparklines to provide additional information about earthquake data.

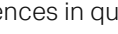
This example uses recent earthquake data downloaded from the [USGS Earthquakes website](#). The data is a comma separated ASCII text file that contains magnitude (mag), coordinates (latitude, longitude), region (place), etc., for each earthquake recorded.

You can download a current CSV file from the [USGS Earthquake Feeds](#) and add it as an input.

Let's say you want to use the USGS Earthquakes data to show the locations that had the most earthquakes over the past month, a column that shows the average quake magnitude for each location. You could use the following search:

```
source="all_month.csv" | stats sparkline count, avg(mag) by locationSource | sort count
```

This search returns the following table, with sparklines that illustrate the quake count over the course of the month for each of the top earthquake locations:

locationSource	sparkline	count	avg(mag)
bgs		1	2.7
ld		2	2.275
ott		9	2.0777777777777775
se		10	2.015
ismp		17	2.025882352941177
nm		25	1.8228000000000002
tul		28	2.7178571428571425
uu		197	1.1888324873096445
mb		202	0.7736138613861386
uw		222	1.1463963963963963
pr		269	2.7057249070631966
hv		340	1.6103235294117646
nn		858	0.6322843822843826
us		988	4.123380566801619
ci		1319	0.9805761940864288
nc		1994	0.9795634721525331
ak		3015	1.5647097844112765

Right away you can see differences in quake distribution between the different locations.

You can click on a sourceLocation to see the actual events that are included in the location calculations.

For the **avg(mag)** column, you can use the Format icon to change the number formatting in that column.



You can easily get the minimum and maximum count for a particular region by mousing over the sparkline; in this example you can see that in Southern Alaska, the minimum count of quakes experienced in a single day during the 7-day period was 1, while the maximum count per day was 6.

But what if you want your sparkline to represent not only the earthquake count, but also the relative average magnitude of the quakes affecting each region? In other words, how can you make the sparkline line chart represent average quake magnitude for each "time bucket" (segment) of the chart?

Try a search like this:

```
source="all_month.csv" | stats sparkline(avg(mag),6h) as magnitude_trend count, avg(mag) by locationSource |
sort - count
```

This search produces a sparkline for each region that shows the average quake magnitude for the quake events that fall into each segment of the sparkline. By specifying a dash (-) after **sort**, the results are sorted in descending order.

But it does a bit more than that. It also asks that the sparkline divide itself up into smaller chunks of time. The preceding table had a sparkline that was divided up by day, so each data point in the sparkline represented an event count for a full 24 hour period. This is why those sparklines were so short.

The addition of the **6h** to the search language overrides this default and displays sparklines that are broken up into discrete six-hour chunks, which makes it easier to see the distribution of events along the sparkline for the chosen time range.

The search also renames the sparkline column as **magnitude_trend** to make it easier to understand.

locationSource	magnitude_trend	count	avg(mag)
ak		3015	1.56
nc		1994	0.98
ci		1319	0.98
us		988	4.12
nn		858	0.63
hv		340	1.61
pr		269	2.71
uw		222	1.15
mb		202	0.77
uu		197	1.19

Now you can see that the quakes for the **nc** location are spread out more evenly than the previous search suggested.

Memory and stats search performance

A pair of **limits.conf** settings strike a balance between the performance of the **stats** family of search commands and the amount of memory they use during the search process, in RAM and on disk. If your **stats**, **sistats**, **geostats**, **tstats**, or **mstats** searches are consistently slow to complete, you can adjust these settings to improve their performance, but at the cost of increased search-time memory usage, which can lead to search failures.

When you run a **stats** search, the search head returns batches of results until the search result set is complete. The **max_mem_usage_mb** setting in the **[default]** stanza of **limits.conf** limits the amount of memory (RAM) that a batch of search results can take up during the **stats** search process. Its default setting is 200 MB. Lower limits to **max_mem_usage_mb** can cause a search to be slower to complete, but there should be no loss of results.

The **stats** command processor spills data from RAM to disk when the amount of RAM that the batch is estimated to use exceeds **max_mem_usage_mb**. Do not set **max_mem_usage_mb=0** as this removes the bounds to the amount of memory a batch can use.

If you run a lot of high cardinality **stats** searches (searches that must match, filter, and aggregate extremely large numbers of discrete field values), consider reducing **max_keymap_rows** to improve their performance. This setting in the **[stats | sistats]** stanza of **limits.conf** limits the number of result rows that the **stats** search processor can store in the key map during the mapping phase of the map-reduce operation in a **stats** search. The **stats** search processor stores expunged result rows in memory, subject to the **max_mem_usage_mb** setting.

Note: The **eventstats** and **streamstats** commands also use these settings, but in a slightly different way.

For more information, see the topics for [eventstats](#) and [streamstats](#) in the *Search Reference*.

Splunk Cloud Platform

To change these limitations, request help from Splunk Support. If you have a support contract, file a new case using the Splunk Support Portal at [Support and Services](#). Otherwise, contact [Splunk Customer Support](#).

Splunk Enterprise

Prerequisites

- Only users with file system access, such as system administrators, can increase the **max_mem_usage_mb** and **max_keymap_rows** settings using configuration files.

- Review the steps in [How to edit a configuration file](#) in the Splunk Enterprise *Admin Manual*.

Note: Never change or copy the configuration files in the **default** directory. The files in the **default** directory must remain intact and in their original location. Make the changes in the **local** directory.

Steps

1. Open the local **limits.conf** file for the Search app. For example, `$SPLUNK_HOME/etc/apps/<app_name>/local`.
2. Under the [default] stanza, set **max_mem_usage_mb** to the value you want to use.
3. Under the [stats | sistats] stanza, set **max_keymap_rows** to the value you want to use.

Advanced Statistics

About advanced statistics

You can use the **stats** and **eval** commands, and how to create sparkline charts. See [calculate basic statistics](#).

Additionally, you can detect anomalies in your data. This might include finding outliers to identify anomalies or spikes in your data. See [About anomaly detection](#) and [Detecting anomalies](#).

You might want to remove outliers that unnecessarily skew your calculations or the way your charts plot the data. See [Finding and removing outliers](#)

You can detect patterns in your data, grouping events based on how similar the events are to each other. See [Detecting patterns](#).

If there are patterns and correlations to events that you monitor, you can use them to predict future activity. With this knowledge, you can proactively send alerts based on thresholds and perform "what-if" analyses to compare various scenarios. See [About time series forecasting](#) and [Machine Learning Toolkit](#).

See also

Related information

[Commands for advanced statistics](#)

[Use the stats command and functions](#)

[Use stats with eval expressions and functions](#)

[Add sparklines to report tables](#)

Commands for advanced statistics

Now it is time to learn about the rich set of commands that you can use to perform advanced statistics. The following tables organize these commands by category.

Commands that find anomalies

These commands are used to find anomalies in your data. You can search for uncommon or outlying events and fields, or cluster similar events together.

Command	Description
<u>analyzefields</u>	Analyzes numeric fields for their ability to predict another discrete field.
<u>anomalies</u>	Computes an "unexpectedness" score for an event.
<u>anomalousvalue</u>	Finds and summarizes irregular, or uncommon, search results.
<u>anomalydetection</u>	Computes a probability for each event and detects unusually small probabilities.
<u>cluster</u>	Groups similar events together.
<u>kmeans</u>	Partitions the events into k clusters, with each cluster defined by its mean value. Each event belongs to the cluster with the nearest mean value.
<u>outlier</u>	Removes outlying numerical values.
<u>rare</u>	Displays the least common values of a field.

Commands for predicting and trends

These commands predict future values and calculate trendlines that can be used to create visualizations.

Command	Description
<u>predict</u>	Predicts future values of a time series.
<u>trendline</u>	Computes moving averages of fields.
<u>x11</u>	Looks for trends in a time series by removing repeating patterns.

Commands for reports, charts, and maps

These commands are used to build **transforming searches**. These commands return statistical data tables that are required for charts and other kinds of data visualizations.

Command	Description
<u>addtotals</u>	Computes the sum of all numeric fields for each search result.
<u>contingency</u>	Builds a contingency table, a co-occurrence matrix, for the values of two fields.
<u>correlate</u>	Calculates the correlation between different fields.
<u>eval</u>	Calculates mathematical, string, or boolean expressions. Puts the value into a new field.
<u>eventstats</u>	Adds summary statistics to all search results.
<u>geostats</u>	Generates statistics to display geographic data and summarize the data on maps.
<u>outlier</u>	Removes outlying numerical values.
<u>rare</u>	Displays the least common values of a field.
<u>stats</u>	Calculates aggregate statistics over the results set, such as average, count, and sum.
<u>streamstats</u>	Adds summary statistics about the preceding events to each search result.
<u>timechart</u>	Create a time series chart and corresponding table of statistics. See " Statistical and charting functions " in the <i>Search Reference</i> .
<u>trendline</u>	Computes moving averages of fields.

See also

- [About advanced statistics](#)
- [Finding and removing outliers](#)
- [Detecting anomalies](#)
- [Detecting patterns](#)
- [About time series forecasting](#)

About anomaly detection

This section describes anomaly detection. For a complete list of topics on detecting anomalies, finding and removing outliers, detecting patterns, and time series forecasting see [About advanced statistics](#), in this manual.

Overview of anomaly detection

An anomaly is a deviation from the expected behavior of the system. An anomaly can be:

- A single event
- A sequence of events
- A sequence of transactions
- Complex patterns

Examples of common use cases for anomaly detection include:

Industry	Use case example
IT	Identifying a distributed denial of service (DDoS) attack from IP address ranges.
Marketing	Rare but high-value customer purchase patterns.
Product	Rare or previously unknown method of using a product that yields better results or yields results more efficiently than known methods.
Security	Faster-than-human transactions. Detecting when transactions are being performed much more quickly by one user than by others. This could indicate a bot or an attempt to probe security measures.

Effective anomaly detection

To perform effective anomaly detection, put all of the data in one place. If you do not have your machine and business data in the same place, you cannot perform a comprehensive analysis.

Begin tracking IT and business performance metrics. Additionally, create a baseline data image which shows the current state of your system.

See also

- [About advanced statistics](#)
- [Finding and removing outliers](#)
- [Detecting anomalies](#)
- [Detecting patterns](#)

Finding and removing outliers

This section describes outliers. For a complete list of topics on detecting anomalies, detecting patterns, and time series forecasting see [About advanced statistics](#), in this manual.

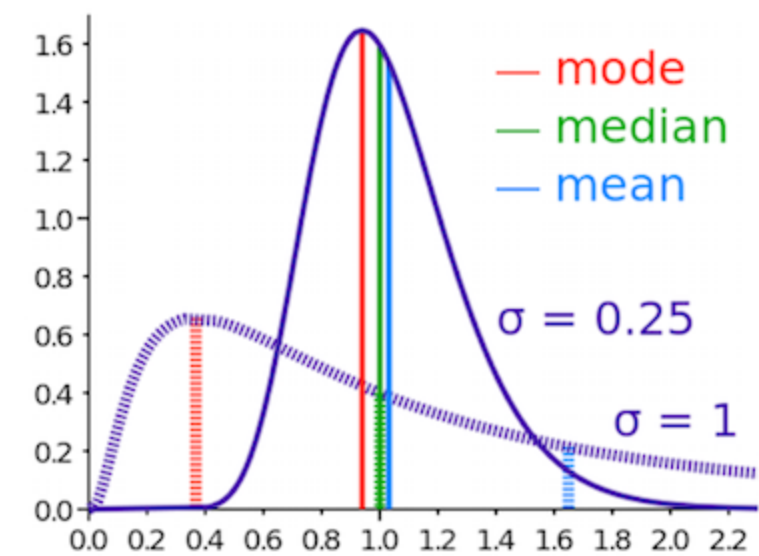
What is an outlier?

An outlier is a data point that is far removed from the typical distribution of data points.

In some situations you might want to simply identify the outliers. In other situations you might want to remove the outliers so that they do not skew your statistical results or create issues displaying data in your charts.

Statistical outliers

A *statistical outlier* is a data point that is far removed from some measure of centrality. Typical measures of centrality are mean, median, and mode. Mean is the average value. Median is the value at the middle of a sorted list of all values. Mode is the most frequent value.



Centrality measure	Splunk centrality commands
Mean	<code>stats avg(<i>field</i>)</code>
Median	<code>stats median(<i>field</i>)</code>
Mode	<code>stats mode(<i>field</i>)</code>

There are different types of statistical outliers. Outliers can be far from the mean, far from the median, or a small number relative to the mode. An outlier can also be a data point that is far removed from the typical range of data points.

Identifying outliers

There are several methods you can use to identify outliers. Often these methods involve calculating some measure of centrality and then identifying the outliers.

In some cases outliers are identified when you notice an anomaly. For example, when you chart the data and you notice that the axis is skewed. An outlier can be the culprit.

Use the number of statistical outliers as a bellwether. If you see more statistical outliers than usual, that phenomenon itself is an anomaly.

To calculate the centrality measure, you can use the following commands.

Centrality measure	Splunk centrality commands
Standard deviation	<code>stats stdev(field)</code>
Quartiles and percentiles	<code>stats perc75(field) perc25(field)</code>
Top 3 most frequent values	<code>top 3 field</code>

In many cases, you need additional commands to calculate the information that you are looking for. The following sections contain some of these examples.

Calculate the lower and upper boundaries of an acceptable range to identify outliers

The following example takes the first 500 events from the `quote.csv` file. The `streamstats` command and a moving window of 100 events are used to calculate the average and standard deviation. The average and standard deviation are used with the `eval` command to calculate the lower and upper boundaries. A sensitivity is added into the calculation by multiplying the `stdev` by 2. The `eval` command uses those boundaries to identify the outliers. The outliers are then sorted in descending order.

```
| inputlookup quote.csv
| head 500
| eval _time=(round(strptime(time, "%Y-%m-%d %H:%M:%SZ")))
| streamstats window=100 avg("price") as
  avg stdev("price") as stdev
| eval lowerBound=(avg-stdev*2)
| eval upperBound=(avg+stdev*2)
| eval isOutlier=if('price' < lowerBound
  OR 'price' > upperBound, 1, 0)
| fields "_time", "symbol", "sourcetype", "time", "price",
  "lowerBound", "upperBound", "isOutlier"
| sort - isOutlier
```

Use the interquartile range (IQR) to identify outliers

The following example takes the first 500 events from the `quote.csv` file. The `eventstats` command is used to calculate the median, the 25th percentile (p25), and the 75th percentile (p75). The IQR is calculated with the `eval` command by subtracting the percentiles. The median and the IQR are used with the `eval` command to calculate the lower and upper boundaries. A sensitivity is added into the calculation by multiplying the IQR by 20. The `eval` command uses those boundaries to identify the outliers. The outliers are then sorted in descending order.

```
| inputlookup quote.csv
| head 500
| eval _time=(round(strptime(time, "%Y-%m-%d %H:%M:%SZ")))
| eventstats median("price") as median p25("price")
  as p25 p75("price") as p75
| eval IQR=(p75-p25)
| eval lowerBound=(median-IQR*20)
| eval upperBound=(median+IQR*20)
| eval isOutlier=if('price' < lowerBound
  OR 'price' > upperBound, 1, 0)
| fields "_time", "symbol", "sourcetype", "time", "price",
  "lowerBound", "upperBound", "isOutlier"
| sort - isOutlier
```

Removing outliers in charts

You can use the [outlier](#) command to remove outlying numerical values from your search results.

You have the option to remove or transform the events with outliers. The **remove** option removes the events. The **transform** option truncates the outlying value to the threshold for outliers. The threshold is specified with the **param** option.

A value is considered an outlier if the value is outside of the **param** threshold multiplied by the inter-quartile range (IQR). The default value for **param** is 2.5.

Create a chart of web server events, transform the outlying values

For a timechart of web server events, transform the outlying average CPU values.

```
host='web_server' 404
| timechart avg(cpu_seconds) by host
| outlier action=transform
```


Remove outliers that interfere with displaying the y-axis in a chart

Sometimes when you create a chart, a small number of values are so far from the other values that the chart is rendered unreadable. You can remove the outliers so that the chart values are visible.

```
index=_internal source=*access*
| timechart span=1h max(bytes)
| fillnull
| outlier
```

Remove outliers using the three-sigma rule across transactions

This example uses the **eventstats** command to calculate the average and the standard deviation. The three-sigma limit is then calculated. The **where** command filters search results. Only the events with the duration less than the three-sigma limit are returned.

```
... | eval durationMins = (duration/60)
| eventstats avg(durationMins) as Avrg, stdev(durationMins) as StDev
| eval threeSigmaLimit = (Avrg + (StDev * 3))
| where durationMins < threeSigmaLimit
```

Manage alerts for outliers

When setting up an alert, it is important to review the outlier threshold you have set.

- * If the threshold value is too low, too many alerts are returned for non-critical outliers
- * If the threshold value is too high, not enough alerts are returned and you might not identify the outliers

Typically a small percentage of events in your data are outliers. If you have 1,000,000 events a day and 5% of the events are outliers, setting an alert would trigger 50,000 alert actions unless you specify a throttle. A *throttle* suppresses additional alerts that have the same field value in a given time range.

For example, your search returns on average 100 events every minute. You only want to be alerted when the status for an event is 404. You can setup the alert to perform the alert action one time every 60 seconds instead of alerting you for every event that has a 404 status in that 60 second window.

Set alert throttling

You set throttling as part of setting an alert.

1. Determine what percent of your events are outliers.
2. Under **Settings**, choose **Searches, reports, and alerts**.

3. Under Schedule and alert, click the **Schedule this search** check box. The screen expands to display the scheduling and alerting options.
4. Specify the alert condition and mode.
5. Mark the **Throttling** check box and specify when the throttling expires.
6. Specify the alert action.
7. Click **Save**.

See also

Related information

[About advanced statistics](#)

[Detecting patterns](#)

Commands

[outlier](#)

[stats](#)

[top](#)

Detecting anomalies

There are a set of commands that you can use to perform anomaly detection. See [Commands for advanced statistics](#).

Finding spikes in your data

You want to identify spikes in your data. Spikes can show you where you have peaks (or troughs) that indicate that some metric is rising or falling sharply. There are all sorts of spikes. Traffic spikes, sales spikes, spikes in the number of returns, spikes in database load. Whatever type of spike you are interested in, you want to watch for it and then perhaps take some action to address those spikes.

You can use a moving trendline to help you see the spikes. Run a search followed by the [trendline](#) command using a field that you want to create a trendline for.

For example, on web access data, you could chart an average of the **bytes** field.

```
sourcetype=access* | timechart avg(bytes) as avg_bytes
```

To add another line or bar series to the chart for the simple moving average (sma) of the last 5 values of **bytes**, use this command:

```
... | trendline sma5(avg_bytes) as moving_avg_bytes
```

If you want to clearly identify spikes, you might add an additional series for spikes. The following search adds a field called "spike" that indicates when the average number of bytes exceeds twice the moving average.

```
... | eval spike=if(avg_bytes > 2 * moving_avg_bytes, 10000, 0)
```

The 10000 here is arbitrary and you should choose a value relevant to your data that makes the spike noticeable. Changing the formatting of the y-axis to Log scale also helps.

Putting this all together, the search is:

```
sourcetype=access*  
| timechart avg(bytes) as avg_bytes  
| trendline sma5(avg_bytes) as moving_avg_bytes  
| eval spike=if(avg_bytes > 2 * moving_avg_bytes, 10000, 0)
```

This search uses a simple moving average for the last 5 results (sma5). Explore with different simple moving average values to determine the best simple moving average to use to identify the spikes.

The [trendline](#) command also supports the exponential moving average (ema) and the weighted moving average (wma).

Alternatively, you can bypass the charting altogether and replace the **eval** command with the **where** command to filter your results.

```
... | where avg_bytes > 2 * moving_avg_bytes
```

And by looking at the table view or setting an alert, you will see when the **avg_bytes** spiked.

See also

Related information

[About advanced statistics](#)

Commands

[eval](#)

[trendline](#)

Detecting patterns

This section describes detecting patterns in your data. For a complete list of topics on detecting anomalies, finding and removing outliers, and time series forecasting see [About advanced statistics](#), in this manual.

Detecting patterns in events

The `cluster command` is a powerful command for detecting patterns in your events. The command groups events based on how similar they are to each other. The `cluster` command groups events based on the contents of the `_raw` field, unless you specify another field.

When you use the `cluster` command, two new fields are appended to each event.

- The `cluster_count` is the number of events that are part of the cluster. This is the cluster size.
- The `cluster_label` specifies which cluster the event belongs to. For example, if the search returns 10 clusters, then the clusters are labeled from 1 to 10.

Anomalies come in small or large groups (or clusters) of events. A small group might consist of 1 or 2 login events from a user. An example of a large group of events might be a DDoS attack of thousands of similar events.

Use the cluster command parameters wisely

- Use the `labelonly=true` parameter to return all of the events. If you use `labelonly=false`, which is the default, then only one event from each cluster is returned.
- Use the `showcount=true` parameter so that a `cluster_count` field is added to all of the events. If `showcount=false`, which is the default, the event count is not added to the event.
- The threshold parameter `t` adjusts the cluster sensitivity. The smaller the threshold value, the fewer the number of clusters.

Other commands to use with the cluster command

- Use the `dedup command` on the `cluster_label` column to see the most recent grouped events within each cluster.
- To group the events and make the results more readable, use the `sort command` with the cluster columns. Sort the `cluster_count` column based on the number of clusters.
 - For small groups of events, sort the `cluster_count` column in ascending order.
 - For large groups of events sort the `cluster_count` column in descending order.
 - Sort the `cluster_label` column in ascending order. Cluster labels are numeric. Sorting in ascending order organizes the events by label, in numerical order.

Return the 3 most recent events in each cluster

The following search uses the `CustomerID` in the `sales_entries.log` file. Setting `showcount=true` ensures that all events get a `cluster_count`. The cluster threshold is set to 0.7. Setting `labelonly=true` returns the incoming events. The `dedup` command is used to see the 3 most recent events within each cluster. The results are sorted in descending order to group the events.

```
source="/opt/log/ecommstv1/sales_entries.log" CustomerID
| cluster showcount=true t=0.7 labelonly=true
```

```
| table _time, cluster_count, cluster_label, _raw  
| dedup 3 cluster_label  
| sort -cluster_count, cluster_label, - _time
```

If you do not set `labelonly=true`, then only one event from each cluster is returned.

See also

Related information

[About advanced statistics](#)

Commands

[dedup, cluster](#)

[sort](#)

About time series forecasting

You can forecast time series data in a number of ways. For example:

- Capacity planning to determine hardware requirements for virtual environments and forecast energy consumption
- Forecast earnings and other business metrics
- Enhanced monitoring of key components which can detect system failures and prevent outages before they occur

You can use reports and dashboards to monitor activity as it is happening, then drill down into events and do a root-cause analysis to learn why something happened. If there are patterns and correlations in events that you monitor, you can use them to predict future activity. With this knowledge, you can proactively send alerts based on thresholds and perform "what-if" analyses to compare various scenarios.

Commands for time series forecasting

The Splunk search language includes two forecasting commands: `predict` and `x11`.

- The [predict command](#) enables you to use different forecasting algorithms to predict future values of single and multivalue fields.
- The [x11 command](#), which is named after the X11 algorithm, removes seasonal fluctuations in fields to expose the real trend in your underlying data series.

Forecasting algorithms

You can select from the following algorithms with the `predict` command: LL, LLP, LLT, LLB, and LLP5. Each of these algorithms are variations of the Kalman filter.

Algorithm option	Algorithm name	Description
LL	Local level	This is a univariate model with no trends and no seasonality. Requires a minimum of 2 data points.
LLP	Seasonal local level	This is a univariate model with seasonality. The periodicity of the time series is automatically computed. Requires the minimum number of data points to be twice the period.
LLT	Local level trend	This is a univariate model with trend but no seasonality. Requires a minimum of 3 data points.
LLB	Bivariate local level	This is a bivariate model with no trends and no seasonality. Requires a minimum of 2 data points. LLB uses one set of data to make predictions for another. For example, assume it uses dataset Y to make predictions for dataset X. If the holdback=10, the LLB algorithm uses the last 10 data points of Y to make predictions for the last 10 data points of X.
LLP5		Combines LLT and LLP models for its prediction.

For more information, see the [predict](#) command in the *Search Reference*.

Forecasting seasonality with the x11 command

The seasonal component of your time-series data is either additive or multiplicative, which is reflected in the two types of seasonality that you can calculate with the x11 command: **add()** for additive and **mult()** for multiplicative.

How do you know which type of seasonality to adjust from your data? The best way to describe the difference between an additive and a multiplicative seasonal component is with an example: The annual sales of flowers will peak on and around certain days of the year, such as Valentine's Day and Mother's Day.

During Valentine's Day, the sale of roses might increase by X dollars every year. This dollar amount is independent of the normal level of the series, and you can add X dollars to your forecasts for Valentine's Day every year, making this time series a candidate for an additive seasonal adjustment. In an additive seasonal adjustment, each value of a time series is adjusted by adding or subtracting a quantity that represents the absolute amount by which that value differs from normal in that season.

Alternatively, in a multiplicative seasonal component, the seasonal effect expresses itself in percentage terms. The absolute magnitude of the seasonal variations increases as the series grows over time. For example, the number of roses sold during Valentine's Day might increase by 40% or a factor of 1.4. When the sales of roses is generally weak, the absolute (dollar) increase in Valentine's Day sales will also be relatively weak. However, the percentage will be constant. And, if the sales of roses are strong, then the absolute (dollar) increase will be proportionately greater. In a multiplicative seasonal adjustment, this pattern is removed by dividing each value of the time series by a quantity that represents the percentage from normal or divided by a factor that is typically observed in that season.

When plotted on a chart, these two types of seasonal components show distinguishing characteristics:

- The additive seasonal series shows steady seasonal fluctuations, regardless of the overall level of the series.
- The multiplicative seasonal series shows varying size of seasonal fluctuations that depend on the overall level of the series.

For more information, see the ["x11"](#) command in the *Search Reference*.

See also

Related information

[About advanced statistics](#)

Commands

[predict](#)

[x11](#)

Machine Learning

This section describes the Splunk Machine Learning Toolkit. For a complete list of topics on detecting anomalies, finding and removing outliers, detecting patterns, and time series forecasting see [About advanced statistics](#).

Splunk Machine Learning Toolkit

The [Splunk Machine Learning Toolkit](#) app delivers new SPL commands, custom visualizations, assistants, and examples to explore a variety of machine learning concepts.

Each assistant includes end-to-end examples with datasets, plus the ability to apply the visualizations and SPL commands to your own data. You can inspect the assistant panels and underlying code to see how it all works. See the [Splunk Machine Learning Toolkit User Guide](#) for information.

Group and Correlate Events

About event grouping and correlation

Event correlation is finding relationships between seemingly unrelated events in data from multiple sources to answer questions like, "how far apart in time did a specific set of events occur?" or "what's the total amount of time it took for a transaction to complete?"

Splunk software supports event correlations using time and geographic location, transactions, sub-searches, field lookups, and joins.

- Identify relationships based on the time proximity or geographic location of the events. Use this correlation in any security or operations investigation, where you might need to see all or any subset of events that take place over a given time period or location.
- Track a series of related events, which may come from separate IT systems and data sources, together as a single transaction. Identify the amount of time it took to complete the transaction and the number of events within a single transaction.
- Use a sub-search to take the results of one search and use them in another. Create conditional searches, where you see the results of a search only if the sub-search meets certain thresholds.
- Correlate your data to external sources with lookups.

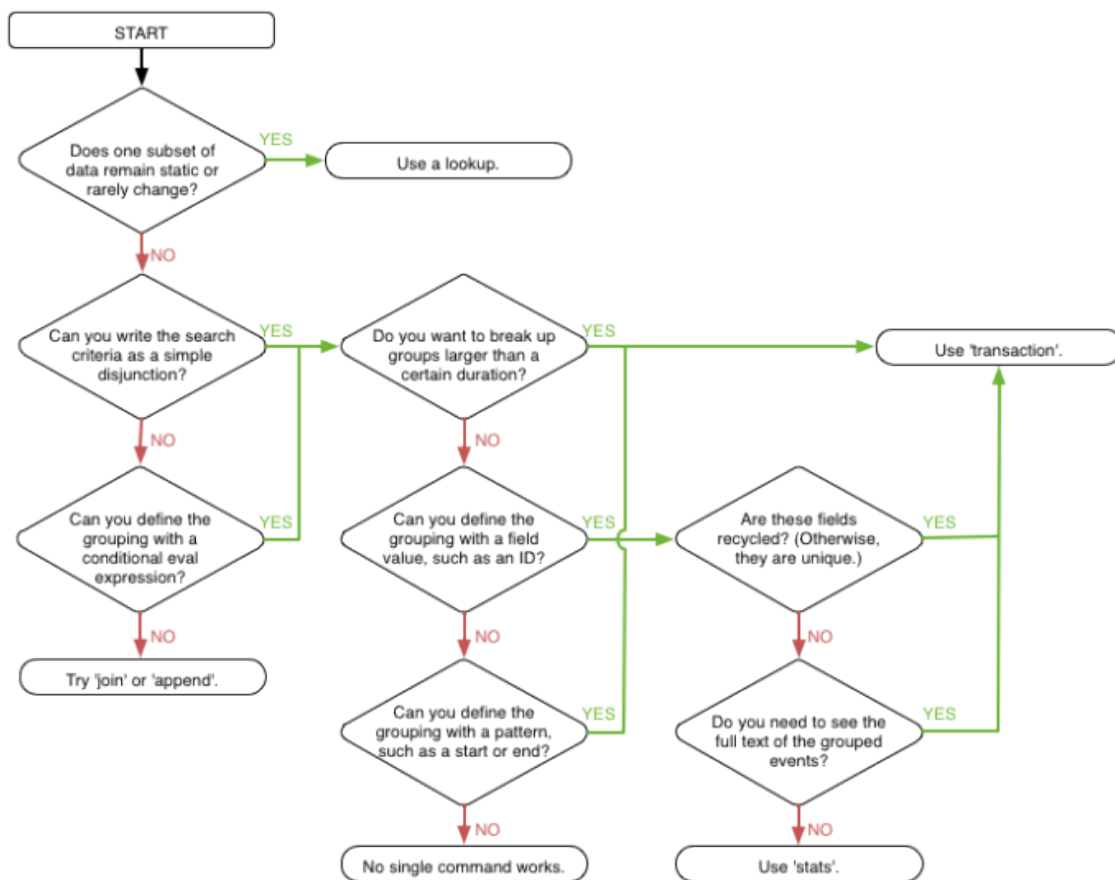
- Use SQL-like inner and outer joins to link two completely different data sets together based on one or more common fields.

This chapter discusses three methods for correlating or grouping events:

- [Use time to identify relations between events](#)
- [Use subsearch to correlate events](#)
- [Use transactions to identify and group related events](#)

You can also use [field lookups](#) and other features of the search language. Depending on your search criteria and how you want to define your groupings, you may be able to use a search command, such as [append](#), [associate](#), [contingency](#), [join](#), or [stats](#). Sometimes, there is no single command that you can use.

If you're not sure where to start, the following flow chart can help you decide whether to use a lookup, define a transaction, or try another search command to define your event grouping.



In most cases, you can accomplish more with [the stats command](#) or [the transaction command](#); and these are recommended over using the **join** and **append** commands. You can read more about when to use **stats** and **transaction** in the topic "[About transactions](#)" later in this chapter. You can also read more about the stats commands in the "[Calculate Statistics](#)" chapter of this manual.

Note: The information for this diagram was provided by [Nick Mealy](#).

Use time to identify relationships between events

Time is crucial for determining what went wrong – you often know when. Splunk software enables you to identify baseline patterns or trends in your events and compare it against current activity.

You can run a series of time-based searches to investigate and identify abnormal activity and then use the timeline to drill into specific time periods. Looking at events that happened around the same time can help correlate results and find the root cause.

Read more about how to ["Use the timeline to investigate events"](#) in this manual.

About transactions

A **transaction** is any group of conceptually-related events that spans time, such as a series of events related to the online reservation of a hotel room by a single customer, or a set of events related to a firewall intrusion incident. A **transaction type** is a configured transaction, saved as a **field** and used in conjunction with the **transaction** command. Any number of data sources can generate transactions over multiple log entries.

Transaction search

A transaction search is useful for a single observation of any physical event stretching over multiple logged events. Use the **transaction** command to define a transaction or override transaction options specified in [transactiontypes.conf](#).

One common use of a transaction search is to group multiple events into a single meta-event that represents a single physical event. For example, an **out of memory problem** could trigger several database events to be logged, and they can all be grouped together into a transaction.

To learn more, see [Identify and group events into transactions](#) in this manual.

Transaction search example

This example uses the sample data from the Search Tutorial but should work with any format of Apache web access log. To try this example on your own Splunk instance, you must download the sample data and follow the instructions to [get the tutorial data into Splunk](#). Use the time range **All time** when you run the search.

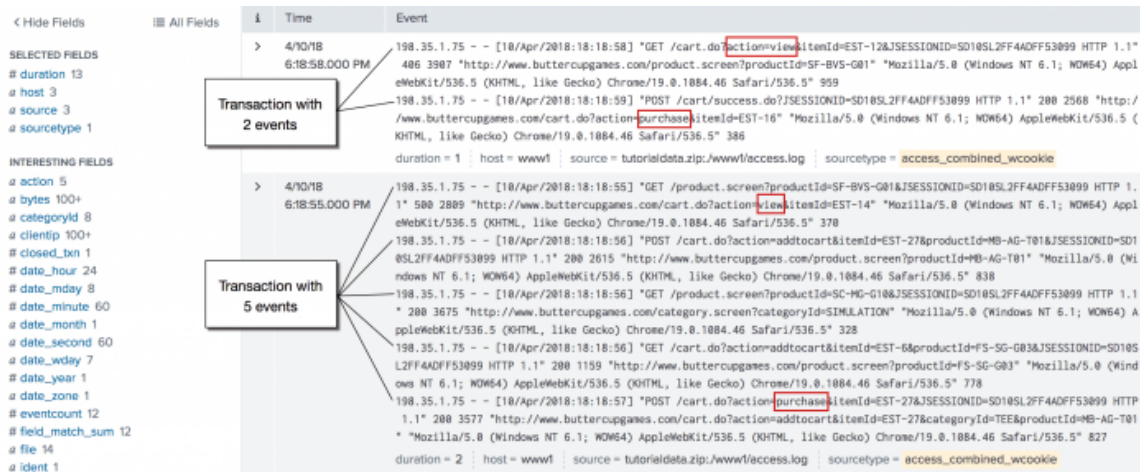
This example searches for transactions with the same session ID and IP address. This example defines a transaction as a group of events that have the same session ID, **JSESSIONID**, and come from the same IP address, **clientip**, and where the first event contains the string, "view", and the last event contains the string, "purchase".

```
sourcetype=access_* | transaction JSESSIONID clientip startswith="view" endswith="purchase" | where duration>0
```

The search defines the first event in the transaction as events that include the string, "view", using the **startswith="view"** argument. The **endswith="purchase"** argument does the same for the last event in the transaction.

This example then pipes the transactions into the **where** command and the **duration** field to filter out all of the transactions that took less than a second to complete. The **where** filter cannot be applied before the **transaction**

command because the **duration** field is added by the **transaction** command. The values in the **duration** field show the difference, in seconds, between the timestamps for the first and last events in the transaction.



< Hide Fields	All Fields	i	Time	Event
Transaction with 2 events				
4/10/18 6:18:58.000 PM 198.35.1.75 - - [10/Apr/2018:18:18:58] "GET /cart.do?action=view&itemId=EST-12&JSESSIONID=SD18SL2FF4ADFF53899 HTTP 1.1" 200 2568 "http://www.buttercupgames.com/cart.do?action=purchase&itemId=EST-16" "Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/536.5 (KHTML, like Gecko) Chrome/19.0.1084.46 Safari/536.5" 386				
duration = 1 host = wwwf source = tutorialdata.zip:wwwf/access.log sourcetype = access_combined_wcookie				
Transaction with 5 events				
4/10/18 6:18:55.000 PM 198.35.1.75 - - [10/Apr/2018:18:18:55] "GET /product.screen?productId=SF-BVS-G01&JSESSIONID=SD18SL2FF4ADFF53899 HTTP 1.1" 200 2809 "http://www.buttercupgames.com/cart.do?action=view&itemId=EST-14" "Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/536.5 (KHTML, like Gecko) Chrome/19.0.1084.46 Safari/536.5" 378				
198.35.1.75 - - [10/Apr/2018:18:18:56] "POST /cart.do?action=addtocart&itemId=EST-27&productId=MB-AG-T01&JSESSIONID=SD18SL2FF4ADFF53899 HTTP 1.1" 200 2615 "http://www.buttercupgames.com/product.screen?productId=MB-AG-T01" "Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/536.5 (KHTML, like Gecko) Chrome/19.0.1084.46 Safari/536.5" 838				
198.35.1.75 - - [10/Apr/2018:18:18:56] "GET /product.screen?productId=SC-MG-G10&JSESSIONID=SD18SL2FF4ADFF53899 HTTP 1.1" 200 3675 "http://www.buttercupgames.com/category.screen?categoryId=SIMULATION" "Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/536.5 (KHTML, like Gecko) Chrome/19.0.1084.46 Safari/536.5" 328				
198.35.1.75 - - [10/Apr/2018:18:18:56] "GET /cart.do?action=addtocart&itemId=EST-64&productId=FS-SG-G03&JSESSIONID=SD18SL2FF4ADFF53899 HTTP 1.1" 200 1159 "http://www.buttercupgames.com/product.screen?productId=FS-SG-G03" "Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/536.5 (KHTML, like Gecko) Chrome/19.0.1084.46 Safari/536.5" 778				
198.35.1.75 - - [10/Apr/2018:18:18:57] "POST /cart.do?action=purchase&itemId=EST-27&JSESSIONID=SD18SL2FF4ADFF53899 HTTP 1.1" 200 3577 "http://www.buttercupgames.com/cart.do?action=addtocart&itemId=EST-27&categoryId=TEE&productId=MB-AG-T01" "Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/536.5 (KHTML, like Gecko) Chrome/19.0.1084.46 Safari/536.5" 827				
duration = 2 host = wwwf source = tutorialdata.zip:wwwf/access.log sourcetype = access_combined_wcookie				

You might be curious about why the transactions took a long time, so viewing these events might help you to troubleshoot. You won't see it in this data, but some transactions may take a long time because the user is updating and removing items from his shopping cart before he completes the purchase. Additionally, this search is run over all events. There is no filtering before the **transaction** command. Anytime you can filter the search before the first pipe, the faster the search runs.

For more examples, see the [transaction](#) command.

Using stats instead of transaction

Both the [stats](#) command and the [transaction](#) command are similar in that they enable you to aggregate individual events together based on field values.

The **stats** command is meant to calculate statistics on events grouped by one or more fields and discard the events (unless you are using [eventstats](#) or [streamstats](#)). On the other hand, except for the duration between first and last events and the count of events, the **transaction** command does not compute statistics over the grouped events. Additionally, it retains the raw event and other field values from the original event and enables you to group events using much more complex criteria, such as limiting the grouping by time span or delays and requiring terms to define the start or end of a group.

The **transaction** command is most useful in two specific cases:

1. When a unique ID (from one or more fields) alone is not sufficient to discriminate between two transactions. This is the case when the identifier is reused, for example web sessions identified by cookie or client IP. In this case, time spans or pauses are also used to segment the data into transactions. In other cases, when an identifier is reused, for example in DHCP logs, a particular message may identify the beginning or end of a transaction.
2. When it is desirable to see the raw text of the events combined rather than an analysis on the constituent fields of the events.

In other cases, it's usually better to use the **stats** command, which performs more efficiently, especially in a distributed environment. Often there is a unique ID in the events and **stats** can be used.

For example, to compute the statistics on the duration of trades identified by the unique ID **trade_id**, the following searches will yield the same answer:

```
... | transaction trade_id | chart count by duration span=log2
```

and

```
... | stats range(_time) as duration by trade_id | chart count by duration span=log2
```

If however, the **trade_id** values are reused but each trade ends with some text, such as "END", the only solution is to use this **transaction** search:

```
... | transaction trade_id endswith=END | chart count by duration span=log2
```

On the other hand, if **trade_id** values are reused, but not within a 10 minute duration, the solution is to use the following **transaction** search:

```
... | transaction trade_id maxpause=10m | chart count by duration span=log2
```

Read more about "[About event grouping and correlation](#)" in an earlier chapter in this manual.

Transactions and macro search

Transactions and macro searches are a powerful combination that allow substitution into your transaction searches. Make a transaction search and then save it with **\$field\$** to allow substitution.

For an example of how to use macro searches and transactions, see [Define and use search macros](#) in the *Knowledge Manager Manual*.

Identify and group events into transactions

You can search for related events and group them into one single event, called a *transaction* (sometimes referred to as a session).

Transactions can include:

- Different events from the same source and the same host.
- Different events from different sources from the same host.
- Similar events from different hosts and different sources.

Search for transactions using the [transaction](#) command either in Splunk Web or at the CLI. The **transaction** command yields groupings of events which can be used in reports. To use **transaction**, either call a transaction type (that you [configured via transactiontypes.conf](#)), or define transaction constraints in your search by setting the search options of the **transaction** command.

Transaction search options

Transactions returned at search time consist of the raw text of each event, the shared event types, and the field values.

Transactions also have additional data that is stored in the fields: **duration** and **transactiontype**.

- **duration** contains the duration of the transaction (the difference between the timestamps of the first and last events of the transaction).
- **transactiontype** is the name of the transaction (as defined in **transactiontypes.conf** by the transaction's stanza name).

You can add **transaction** to any search. For best search performance, craft your search and then pipe it to the transaction command. For more information see the topic on the [transaction](#) command in the Search Reference manual.

Follow the **transaction** command with the following options. **Note:** Some **transaction** options do not work in conjunction with others.

name=<transaction-name>

- Specifies the name of a stanza from **transactiontypes.conf**. Use this to invoke a **transaction type** that you have already configured for reuse. If other arguments are provided, they overrule values specified for the same arguments in the transaction rule. For example, if **web_purchase**, the transaction rule you're invoking, is configured with **maxevents=10**, but you'd like to run it with a different value for **maxevents**, add **maxevents** to the search string with the value you want:

```
sourcetype=access_* | transaction name=web_purchase maxevents=5
```

[field-list]

- This is a comma-separated list of fields, such as **... | transaction host,cookie**
- If set, each event must have the same field(s) to be considered part of the same transaction.
- Events with common field names and different values will not be grouped.
 - For example, if you add **... | transaction host**, then a search result that has **host=mylaptop** can never be in the same transaction as a search result with **host=myserver**.
 - A search result that has no **host** value can be in a transaction with a result that has **host=mylaptop**.

match=closest

- Specify the matching type to use with a transaction definition.
- The only value supported currently is **closest**.

maxspan=[<integer>s | m | h | d]

- Set the maximum duration of one transaction.
- Can be in seconds, minutes, hours or days.
 - For example: 5s, 6m, 12h or 30d.
- Defaults to **maxspan=-1**, for an "all time" timerange.

maxpause=[<integer> s|m|h|d]

- Specifies the maximum pause between transactions.
- Requires there be no pause between the events within the transaction greater than maxpause.
- If the value is negative, the maxspause constraint is disabled.
- Defaults to `maxpause=-1`.

startswith=<string>

- A search or eval-filtering expression which, if satisfied by an event, marks the beginning of a new transaction.
- For example:
 - `startswith="login"`
 - `startswith=(username=foobar)`
 - `startswith=eval(speed_field < max_speed_field)`
 - `startswith=eval(speed_field < max_speed_field/12)`
- Defaults to `""`.

endswith=<transam-filter-string>

- A search or eval-filtering expression which, if satisfied by an event, marks the end of a transaction.
- For example:
 - `endswith="logout"`
 - `endswith=(username=foobar)`
 - `endswith=eval(speed_field < max_speed_field)`
 - `endswith=eval(speed_field < max_speed_field/12)`
- Defaults to `""`.

For **startswith** and **endswith**, **<transam-filter-string>** is defined with the following syntax:

"<search-expression>" | (<quoted-search-expression>) | eval(<eval-expression>

- **<search-expression>** is a valid search expression that does not contain quotes.
- **<quoted-search-expression>** is a valid search expression that contains quotes.
- **<eval-expression>** is a valid eval expression that evaluates to a boolean.

Examples:

- search expression: `(name="foo bar")`
- search expression: `"user=mildred"`
- search expression: `("search literal")`
- eval bool expression: `eval(distance/time < max_speed)`

Example transaction search

Run a search that groups together all of the web pages a single user (or client IP address) looked at over a time range.

This search takes events from the access logs, and creates a transaction from events that share the same `clientip` value that occurred within 5 minutes of each other (within a 3 hour time span).

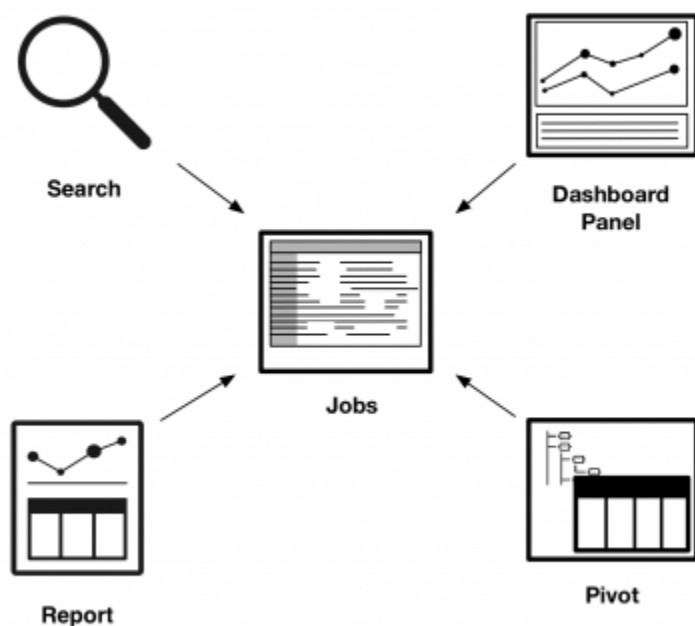
```
sourcetype=access_combined | transaction clientip maxpause=5m maxspan=3h
```

Refer to the [transaction command](#) topic in the Search Reference Manual for more examples.

Manage Jobs

About jobs and job management

Each time you run a search, create a pivot, open a report, or load a dashboard panel, the Splunk software creates a **job** in the system. When you run a search, you are creating an **ad hoc search**. Pivots, reports, and panels are powered by **saved searches**.



A job is a process that tracks information about the ad hoc search or saved search. The information that is tracked includes the owner of the job, the app that the job was run on, how many events were returned, and how long the job took to run.

Each job process creates a **search artifact**. The artifact contains the results and associated metadata that are returned at the time that the ad hoc search or saved search was run.

Inspecting jobs and managing jobs

There are several ways that you can look at information about your jobs. You can inspect a job or you can manage a job.

Search Job Inspector

Use the Search Job Inspector to view information about the current job, such as job execution costs and search job properties. See [View search job properties](#).

Job Details dashboard

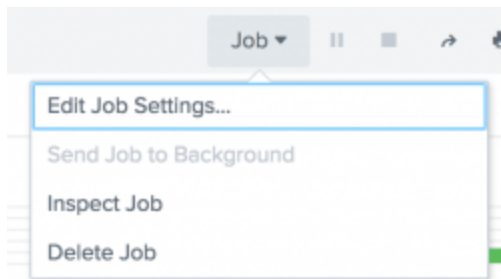
The Job Details dashboard provides a clear and concise overview of a search job process. You can access the Job Details dashboard through the Search Job Inspector. See [View search job properties](#).

Jobs manager page

Use the Jobs manager page to view information about recent jobs. If you have the Admin role or a role with an equivalent set of capabilities, you can manage the search jobs run by other users. See [Manage search jobs](#).

Job menu

After you run a search or open a report in Splunk Web, you can access and manage information about the search job without leaving the Search page. While the search is running, paused, or finalized, click **Job** and choose from the available options.



- **Edit the job settings.** Select this to open the Job Settings dialog, where you can change the job read permissions, extend the job lifetime, and get a URL for the job. You can use the URL to share the job with others, or to create a bookmark to the job from your web browser.
- **Send the job to the background.** Select this if the search job is slow to complete and you want to work on other Splunk activities, including running a new search job. The job continues to run in the background.
- **Inspect the job.** Opens a separate window and displays information and metrics for the search job using the Search Job Inspector.
- **Delete the job.** Use this to delete a job that is currently running, is paused, or which has finalized. After you delete the job, you can still save the search as a report.

Edit search job settings

You can open the Job Settings dialog when a search job is running, paused, or finalized. Just click **Job** and select **Edit Job Settings**.

Job Settings ×

Owner

admin

App

search

Read Permissions

Private

Everyone

Lifetime ?

10 minutes

7 days

Link To Job

http://localhost:8000/en-US/app/search/searc

🔗

Copy or bookmark the link by right-clicking the icon, or drag the icon into your bookmarks bar.

Cancel

Save

Sharing jobs

There are several ways to share a job with other Splunk users. You can change the job permissions or send a link to the job. This can be handy if you want another user to see the results returned by the job. See [Sharing and exporting jobs](#).

Job lifetimes

When you run a new search, a job is retained in the system for a period of time, called the job **lifetime**. The default lifetime is 10 minutes. The lifetime starts from the moment the job is run. See [Extending job lifetimes](#) in this manual.

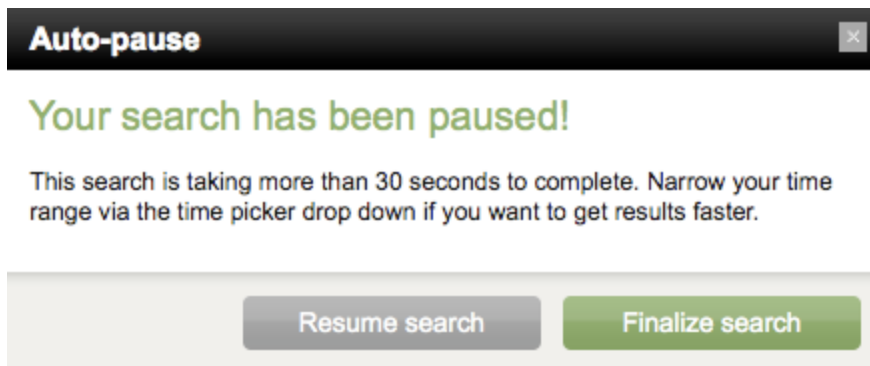
Managing long-running jobs

Sometimes a search job runs for a long time. You might want to edit the search to change the search criteria, or you might want to pause the search or run the search in the background.

Autopause long-running jobs

To handle inadvertently long-running search jobs, you can autopause a job. This feature is enabled by default only for summary dashboard clicks, to deal with the situation where a user mistakenly initiates "all time" searches.

When autopause is enabled for a particular search view, the search view includes an autopause countdown field during the search. If the search time limit has been reached, an information window will appear to inform the user that the search has been paused. It offers the user the option of resuming or finalizing the search. By default, the limit before autopause is 30 seconds.



Managing jobs when a computer goes into sleep mode

When a search is run in Splunk Web from a computer that is not a Splunk server and the computer changes to sleep or hibernate mode, the underlying search process is stopped. The Splunk software interprets the change to sleep or hibernate mode as if the browser tab in which the software is running has been closed and is no longer being used.

To avoid this issue, use one of the following techniques:

- Send the job to the background. The job continues to run in the background even when your computer goes into sleep or hibernate mode. From the **Job** menu, select **Send Job to Background**.
- Save and schedule the search. The search runs independently from the computer that was used to create the search. You will need to decide if you want to save and schedule the search as a report, dashboard or an alert. See [Saving searches](#) and [Scheduling searches](#).
- Share the job. The lifetime of the job is automatically extended to 7 days and read permissions are set to Everyone. See [Share jobs and export results](#).
- Change the settings on the computer to extend the time before the computer goes into sleep or hibernate mode.

Administering jobs

Users with the Admin role, or a role with equivalent capabilities, can restrict how many jobs a given user can run, and how much space their job artifacts can take up.

You must define a role with the desired restrictions and assign the user to the role. You can apply a high level of granularity by giving unique roles to each user in your system.

Edit search restriction settings

To edit the search restrictions setting for a role:

1. In Splunk Web, go to **Settings** and then **Roles**.
2. Edit an existing role and use the **4. Restrictions** tab to limit the scope of search results that are returned when users with the role run searches. See [Use roles to limit search results](#).

For more information about roles, see [Add and edit roles](#) in the *Securing Splunk Enterprise* manual.

Edit search restrictions using `authorize.conf`

If you're using Splunk Enterprise, you can manually edit search restrictions using the `authorize.conf` file.

1. Review the contents of the [authorize.conf.example](#) file in the *Admin Manual*. This example explains some the attributes that you might want to use.
2. Create the configuration file.

Scope	Description
System-wide	Create the <code>authorize.conf</code> file in local directory for the system. The location of the system local directory is <code>\$SPLUNK_HOME/etc/system/local</code> .
Application-specific	Create the <code>authorize.conf</code> file in the local directory for the application. The location of an application local directory is <code>\$SPLUNK_HOME/etc/apps/<app_name>/local</code> .

3. Edit the local `authorize.conf` file. To restrict the jobs that users can run, add the following information to the file:
 - a. Add a stanza for the role that you want to create. Use the format `[role_<roleName>]`. Role names must be in lowercase characters. For example, `[role_ninja]`.
 - b. Optional. Add the `importRoles` attribute. Importing a role also imports the other aspects of that role, such as the indexes that the role is allowed to search. For example, `importRoles = user`.
 - c. Add the `srchDiskQuota` attribute and value. This is the maximum amount of disk space (MB) that search jobs can use, for a user that belongs to this role. The default value is 100MB. For example, `srchDiskQuota = 500`.
 - d. Add the `srchJobsQuota` attribute and value. This is the maximum number of concurrently running searches that a user of this role can have. The default value is 3. `srchJobsQuota = 10`.
 - e. Optional. Add the `rtsearch` attribute to specify if the user is authorized to run **real-time searches**. If you enable real-time searches for the user, you should also specify the `rtSrchJobsQuota` attribute.

For attribute descriptions and information about the default values, see [role_name stanza](#) for the `authorize.conf` file in the *Admin Manual*.

See also

- [Create and edit reports](#) in the *Reporting Manual*

Extending job lifetimes

When you run a new search job, the job is retained in the system for a period of time, called the job **lifetime**. During the lifetime, you can access the job and view the data returned by the job. If the job is not accessed within the specified lifetime, the job expires and is removed from the system.

There are two lifetime settings, 10 minutes and 7 days. The lifetime starts from the moment the job is run.

Default job lifetimes

The default lifetime for a search job depends on whether the search job is an artifact of an unscheduled or **scheduled search**.

For example, dashboard panels that are based on an inline search, use unscheduled searches. Panels that are based on a report, use saved searches. The saved search can be unscheduled or scheduled.

Default lifetimes for unscheduled searches

When you run an **ad hoc search** and the search is finalized or completes on its own, the resulting search job has a default lifetime of 10 minutes. Other knowledge objects, such as real-time alerts and panels based on inline searches that use unscheduled searches have the same default lifetime.

Default lifetimes for scheduled searches

Scheduled searches launch search jobs on a regular interval. By default, these jobs are retained for the interval of the scheduled search multiplied by two. For example, if the search runs every 6 hours, the resulting jobs expire in 12 hours.

Automatic lifetime extensions

Whenever you access an active job, such as when you view the results of a search job, the lifetime is reset. The reset happens whether the job lifespan is 10 minutes or 7 days. Here are a few examples of how this works.

- If the lifetime is set to 10 minutes and you run the search job at 11:00 AM, the job lifetime is set to end at 11:10 AM. If you run the job again at 11:07 AM, the job lifetime is reset to end at 11:17 AM.
- If you set the lifetime for a job to 7 days and then access the job 4 days later, the job lifetime is reset and will not expire for another 7 days from the current day and time.

Changing the lifetime for the current job

You change the lifetime setting for the current ad hoc search job in the Search app.

1. Select the **Job** drop-down.
2. Select **Edit Job Settings** to display the Job Settings.
3. For **Lifetime**, select either 10 Minutes or 7 Days.

Changing the lifetime for active jobs

You can't change the lifetimes for jobs resulting from previously run unscheduled or scheduled searches. You can only change the lifetimes for active search jobs. See [Manage search jobs](#).

Expired jobs

After the job lifetime ends, the job expires and is deleted from the system.

Note: It is possible that while you are looking at the list of jobs that a job will expire. When you try to extend the lifetime of the expired job, a message appears explaining that the job no longer exists. You cannot extend the lifetime of an expired job.

Changing the default lifetime values

You can change the default value for the job lifetime for unscheduled and scheduled searches.

Change the default lifetime value for unscheduled searches

You can change the default value for the job lifetime for unscheduled searches.

Splunk Cloud Platform

To change the default value for the job lifetime for unscheduled searches, request help from Splunk Support. If you have a support contract, file a new case using the Splunk Support Portal at [Support and Services](#). Otherwise, contact [Splunk Customer Support](#).

Splunk Enterprise

Prerequisites

- Only users with file system access, such as system administrators, can change the default lifetime values.
- Review the steps in [How to edit a configuration file](#) in the *Admin Manual*.

Note: Never change or copy the configuration files in the **default** directory. The files in the **default** directory must remain intact and in their original location. Make the changes in the **local** directory.

1. Open the local **limits.conf** file for the Search app. For example, `$SPLUNK_HOME/etc/apps/<app_name>/local`.
2. In the **[search]** stanza, change the **default_save_ttl** value to a number that is appropriate for your needs. The acronym TTL is an abbreviation for "time to live." The **default_save_ttl** setting measures search job time to live in seconds, and defaults to 604800 seconds, or one week.

Change the default lifetime value for scheduled searches

You can change the default lifetime for jobs resulting from a specific scheduled search.

Splunk Cloud Platform

You can use Splunk Web to make this change.

Prerequisites

- Only system administrators with the **sc_admin** role should change default lifetime values.
- Review the information about the **dispatch.ttl** setting in the Dispatch search options section of the [Savedsearches.conf.spec file](#) in the *Admin Manual*.

Steps:

1. Select **Settings > Searches, Reports, and Alerts**.
2. Locate the name of the scheduled search in the list.
3. In the **Actions** column, select **Edit, Advanced Edit**.
4. Locate the **dispatch.ttl** setting. The default setting is **2p**.
5. Change the setting, based on the information provided in the savedsearches.conf.spec file.

Splunk Enterprise

You can change this default either through Splunk Web, as described previously for Splunk Cloud Platform, or through the configuration files.

Prerequisites

- Only users with file system access, such as system administrators, can <describe the customization> using configuration files.
- Review the steps in [How to edit a configuration file](#) in the *Splunk Enterprise Admin Manual*.
- You can have configuration files with the same name in your default, local, and app directories. Read [Where you can place \(or find\) your modified configuration files](#) in the *Splunk Enterprise Admin Manual*.

Note: Never change or copy the configuration files in the **default** directory. The files in the **default** directory must remain intact and in their original location. Make the changes in the **local** directory.

Steps:

1. Open the local **savedsearches.conf** file. For example, `$SPLUNK_HOME/etc/apps/<app_name>/local`.
2. Locate the scheduled search, and change the **dispatch.ttl** setting to a different interval multiple.

Share jobs and export results

You can share a job with other Splunk users, or export the event data to archive or to use with a third-party charting application.

Share a job with others

When you share a job, you are sharing the results of a specific run of a search.

There are several ways that you can share a specific job with other Splunk platform users. You can change the permissions for a search job to share that job with other users. You can also share a job by sending the URL for a search job to a Splunk platform user.

You can only change permissions or share a link to the current job.

Change job permissions

You can share a job by changing the permissions on that job. By default, all jobs are **Private**.

1. From the **Job** menu, select **Edit Job Settings** to display the Job Settings dialog box.
2. Change **Read Permissions** to **Everyone**.
3. Click **Save**

Job Settings

×

Owner

admin

App

search

Read Permissions

Private

Everyone

Lifetime ?

10 minutes

7 days

Link To Job

http://localhost:8000/en-US/app/search/search



Copy or bookmark the link by right-clicking the icon, or drag the icon into your bookmarks bar.

Cancel

Save

Share a job URL

You can share a job with other Splunk users by sending them a link to the job. This is handy when you want another user to see the results returned by the job.

The users that you send the link to must have permissions to use the app that the job belongs to.

Decide which method you want to use to obtain a job link. You can use the **Share** icon or the **Job** menu.

1. To use the Share icon:
 - a. Click the icon.  The Share icon is one of the search action icons.
 - b. In the **Link To Job** text box, copy the URL and send the link to the users that you want to share the job results with. The permissions on the job are automatically changed to **Everyone** and the lifetime of the job is automatically extended to **7 days**.

Share Job



The job's lifetime has been extended to 7 days and read permissions have been set to Everyone. Manage the job via [Job Settings](#).

Link To Job

<http://localhost:8000/en-US/app/search/search?sid=152>



Copy or bookmark the link by right-clicking the icon, or drag the icon into your bookmarks bar.

2. To use the **Job** menu:
 - a. From the **Job** menu, select **Edit Job Settings** to display the Job Settings dialog box.
 - b. Change **Read Permissions** to **Everyone**. If the permissions for a job are set to **Private**, other users cannot access the job with the link.
 - c. Change **Lifetime** to **7 days**.
 - d. Copy the link and send the link to the users you want to share the job results with.

Job Settings



Owner **admin**

App **search**

Read Permissions

Private

Everyone

Lifetime ?

10 minutes

7 days

Link To Job

<http://localhost:8000/en-US/app/search/search>



Copy or bookmark the link by right-clicking the icon, or drag the icon into your bookmarks bar.

Bookmark icon

Cancel

Save

You can also save the link for your own use by using the Bookmark icon. The Bookmark icon appears in both the Job Settings dialog box and the Share Jobs dialog box. You can click and drag the Bookmark icon to the bookmarks bar in your Web browser.

Export job results to a file

You can export your job results in a variety of format such as CSV, JSON, PDF, Raw Events, and XML. You can then archive the file, or use the file with a third-party charting application. The format options depend on the type of job artifact that you are working with.

- If the search generates calculated data that appears on the Statistics tab, you cannot export using the Raw Events format.
- If the search is a saved search, such as a Report, you can export using the PDF format.

The export file is saved in the default download directory for your browser or operating system.

There are several methods that you can use to export search results. A few of these methods include Splunk Web, CLI, SDKs, and REST. Some of the methods are optimized for speed, while others are good for extremely large event sets.

For a complete list of the export methods and links to the specific steps, see [Export search results](#).

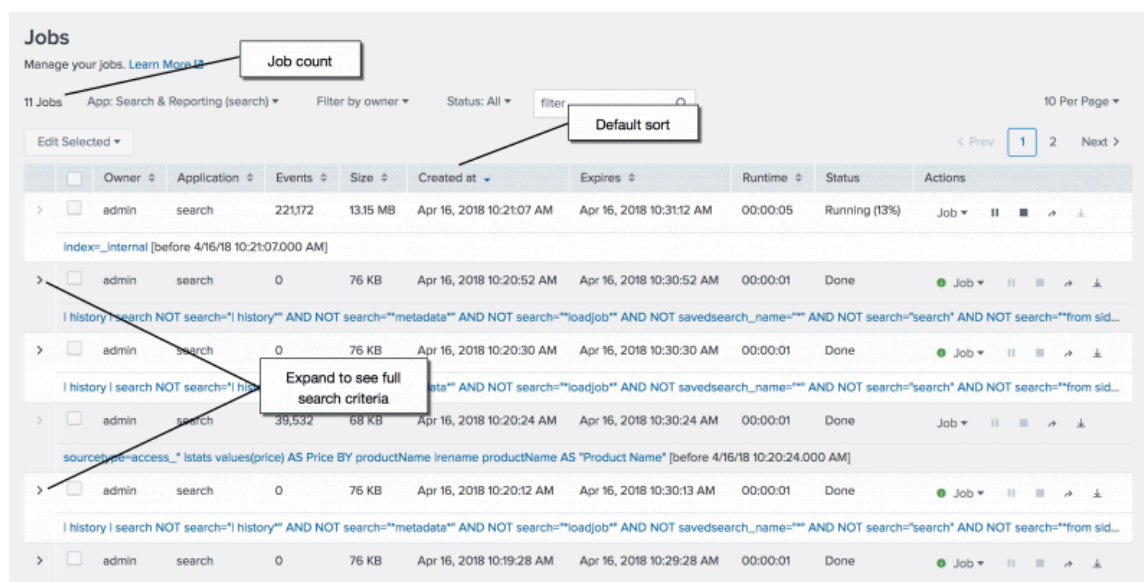
Manage search jobs

You can use the Jobs page to review and manage any **job** that you own.

If you have the Admin role, or a role with an equivalent set of capabilities, you can manage the search jobs run by all users of your Splunk implementation.

Opening the Jobs page

- In Splunk Web, to view a list of your jobs select **Activity > Jobs**. This opens the **Jobs** page.



The screenshot shows the Splunk Jobs page. At the top, there's a header 'Jobs' with a sub-header 'Manage your jobs. [Learn More](#)'. Below this, there's a 'Job count' box highlighting '11 Jobs'. The main table has columns: Owner, Application, Events, Size, Created at, Expires, Runtime, Status, and Actions. A 'Default sort' box highlights the 'Created at' column header. An 'Expand to see full search criteria' box highlights the search criteria for a job: 'index=_internal [before 4/16/18 10:21:07:000 AM]'. The table shows several jobs, including one that is 'Running (13%)' and others that are 'Done'.

The Jobs page displays a list of different types of search jobs.

- Jobs resulting from ad hoc searches or pivots that you have recently run manually.
- Jobs for searches that are run when dashboards are loaded or reports are opened.
- Jobs for scheduled searches.

Refreshing the jobs list

The list of jobs in the Jobs page does not automatically refresh.

- Jobs that are created after you display the Jobs page are not visible until you reload the Jobs page.
- If a job expires while you have the Jobs page open, the job appears in the Jobs page list, but you cannot view the job results.

To refresh the Jobs page, reload the page.

Job actions

You can use the Actions column to perform actions on a job.

Runtime	Status	Actions
00:00:05	Running (13%)	Job ▾ ■ ↻ ⬇
00:00:01	Done	● Job ▾ ■ ↻ ⬇

Use the **Job** drop-down to edit the job settings, pause the job, extend the job lifetime, inspect the job, or delete the job.

Use the action icons to pause, stop, share, and export jobs.

To perform these actions on multiple jobs, select the jobs and click **Edit Selected**. Then select the action that you want to perform.

Note: When you pause a search job, it might take some time for indexers to stop running remote searches and clear out search results that are buffered in memory.

View and compare jobs

You can see a list of the jobs you have recently dispatched or saved for later review. Use the list to compare job statistics such as run time, total count of events matched, size, and so on.

Active job count

In the upper right corner of the Jobs page there is a count of the total number of jobs in the list.

The count reflects the number of jobs at the moment you opened the Jobs page. If a job expires while the Jobs page is open, the job count does not refresh.

Sort the job list

By default, the list of jobs is sorted by the **Created at** column.

You can sort the list by any column that displays a sort button in the column heading. For example, you can sort the list by the job expiration or by the job owner.

- Click once on the column heading to sort the list in ascending order. Click again to sort the list in descending order.

Filter the job list

You can filter the list of jobs by application, by owner, and by status.

- In the **Filter** box, type a term or expression that appears in the search criteria to filter the list.

For example, you can specify `diskUsage`, `EMBED AND diskUsage=8*`, or `label=EMBED AND diskUsage=8*` in the Filter box.

View job search results

You can view the results of a search that is listed on the Jobs page.

1. Click on the search link to view the results associated with a specific job.
 - For ad hoc searches the link is the search criteria.
 - For saved searches the link is the name of the report, dashboard panel, or pivot.

The results open in the Search app view.

Check the progress of ongoing jobs

You can check on jobs that are dispatched by scheduled searches, real-time searches, and long-running historical searches.

Use the **Status** column to check on the progress of ongoing jobs. The **Status** column shows the percent of the events that have been processed. Current jobs have a status of **Running**. Jobs that are running in the background have a status of **Backgrounded**.

Change the per page job count

You can change the number of jobs that appear on each page in the list. The default is to display ten jobs on each page. On the right side of the window, you can display 10, 20 or 50 jobs per page.

Inspect jobs

You can inspect a job to take a closer look at what a search is doing and see where the Splunk software is spending most of its processing time.

Use the Search Job Inspector to view information about the current job, such as the search ID (sid), job execution costs, and search job properties.

1. In the **Actions** column for the specific job, select **Job**.
2. Select **Inspect Job**.

For more information about the using the Search Job Inspector, see [View search job properties](#).

Extending search job lifetimes

From the Jobs page, there are several ways to change the lifetime of a job. To learn more about lifetimes for specific types of jobs, see [Extending job lifetimes](#).

Quickly extend job lifetimes

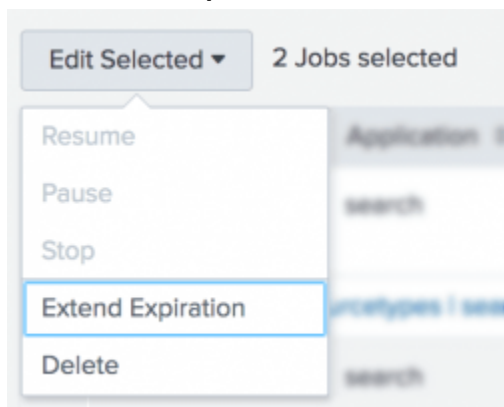
You can quickly extend the lifetime of a job.

1. In the **Actions** column for the specific job, select **Job**.
2. Select **Extend Job Expiration**.

Extend the lifetimes for multiple jobs

You can extend the lifetimes of multiple jobs at the same time.

1. Select the jobs whose lifetimes you want to extend.
2. Above the job list, click **Edit Selected**.
3. Select **Extend Expiration**.



Choose the period of time for the lifetime extension

Use this method to select the time period for the extension.

1. In the **Actions** column for the specific job, select **Job**.
2. Select **Edit Job Settings**.
3. Select either 10 minutes or 7 days.
4. Click **Save**.

Share jobs

You can share a job by changing the job permissions, or by sharing a URL to the job.

Change job permissions

You can share a job by changing the permissions on that job. By default, all jobs are set to Private.

1. From the **Job** menu, select **Edit Job Settings** to display the Job Settings dialog box.
2. Change **Read Permissions** to **Everyone**.
3. Click **Save**.

Share a job URL

You can share a job with other Splunk users by sending them a link to the job.

The users that you send the link to must also have permissions to use the app that the job belongs to.

1. In the **Actions** column, click the Share icon for the job that you want to share.
2. In Share Job dialog box, copy the URL in the **Link To Job**.
3. Send the link to the users that you want to share the job results with.

The permissions on the job are changed to **Everyone** and the lifetime of the job is extended to **7 days**.

You can also obtain the URL from the **Job Settings** window. If you use this method to share a job, you must change the **Read Permissions** to **Everyone** and the **Lifetime** to **7 days**.

Export jobs

You can export the event data from a job in a variety of formats such as CSV, JSON, PDF, Raw Events, and XML. You can then archive the file, or use the file with a third-party charting application. The format options depend on the type of job artifact that you are working with.

1. In the **Actions** column, click the Export icon for the event data that you want to export.
2. Click **Format** and select the format that you want the search results to be exported in.
3. In the **File Name** field, type a name for the export file where the event data will be stored.
4. In the **Number of Results** field, specify the number of results that you want to export.
5. Click **Export** to save the job events in the export file.

The export file is saved in the default download directory for your browser or operating system.

Note: If your search returns a large number of results, you can access all of the results in the Search app. However, the full set of results might not be stored with the **search job artifact**. When you export search results, the export process is based on the search job artifact, not the results in the Search app. If the artifact does not contain the full set of results, a message appears at

the bottom of the Export Results dialog box to tell you that the search will be rerun by the Splunk software before the results are exported.

For more information, see [Export search results](#).

If the Export icon is not visible, the icon has been hidden by your system administrator to prevent data export.

Extend the session timeout when exporting large amounts of data

When you try to export large amounts of data using the Export button, the session might timeout.

Users with the Admin role, or a role with an equivalent set of capabilities, can use the following procedure to extend the session timeout limit.

1. Click **Settings > Server Settings > General Settings**.
2. In the **Splunk Web** section, increase the number in the Session timeout field.
3. Click **Save**.

Increasing the timeout settings allows Splunk Web more time for the connection between your browser and Splunk Web.

Delete jobs

You can delete one or more jobs from the jobs list.

Delete a single job

You can delete a job from the job list.

1. In the **Actions** column for the specific job, select **Job**.
2. Select **Delete Job**.

Delete multiple jobs

You can delete multiple jobs at one time.

1. Select the jobs that you want to delete.
2. Above the job list, click **Edit Selected**.
3. Select **Delete**.

View search job properties

The **Search Job Inspector** and the Job Details dashboard are tools that let you take a closer look at what your search is doing and see where the Splunk software is spending most of its time.

This topic discusses how to use the Search Job Inspector to both troubleshoot the performance of a search **job** and understand the behavior of knowledge objects such as event types, tags, lookups and so on within the search. For more information, see [Manage search jobs](#) in this manual.



Here's a Splunk Education video about [Using the Splunk Search Job Inspector](#). This video shows you how to determine if your search is running efficiently, event types, searches in a distributed environment, search optimization, and disk usage.

This topic also shows you how to access the Job Details dashboard after you open the Search Job Inspector for a search job. The Job Details dashboard gives you a clear and concise overview of a search job's process and performance. You may want to review the Job Details dashboard before you try to parse the itemized report provided by the Search Job Inspector.

The Job Details dashboard displays basic search job facts and metrics, shows you the search strings that were run in the background to carry out the search, and gives you a concise overview of search costs and indexer usage metrics.

Open the Search Job Inspector

You can access the Search Job Inspector for a search job, as long as the search job has not expired (which means that the search artifact still exists). The search does not need to be running to access the Search Job Inspector.

1. Run the search.
2. From the **Job** menu, select **Inspect Job**.

This opens the Search Job Inspector in a separate window.

Change the search job

After the Search Job Inspector opens in a separate window, you can use the URL for that window to inspect any valid search job artifact if you have its search ID (SID). You can find the SID of a search on the Jobs page (select **Activity > Jobs**) or listed in the dispatch directory, `$SPLUNK_HOME/var/run/splunk/dispatch`. For more information about the Jobs page, see [Manage search jobs](#) in this manual.

If you look at the URI path for the Search Job Inspector window, you will see something like this at the end of the string:

```
.../manager/search/job_inspector?sid=1299600721.22
```

The **sid** is the SID number. The **namespace** is the name of the app that the SID is associated with. In this example, the SID is 1299600721.22.

Type the search artifact SID in the URI path, after **sid=** and press **Enter**. As long as you have the necessary ownership permissions to view the search, you will be able to inspect it.

What the Search Job Inspector shows you

At the top of the Search Job Inspector window, an information message appears. The message depends on whether the job is paused, running, or finished. For example, if the job is finished the message tells you how many results it found and the time it took to complete the search. Any error messages are also displayed at the top of the window.

Below these messages you should find the SID for the search job and two links:

- The **search.log** link takes pulls up a text file with the raw search.log data for this search job. The displays of the Search Job Inspector and Job Details dashboard are based on this search.log data.
- The **Job Details Dashboard** link goes to the Job Details dashboard, which provides a condensed summary of important metrics and facts about the search job you are investigating. You may want to look at this dashboard before you try to do a deep dive into the Search Job Inspector's comprehensive report.

Open the Job Details dashboard to get a concise overview of your search job

After opening the Search Job Inspector for a search, click the **Job Details Dashboard** link to open the Job Details dashboard.

Note: You can also reach the Job Details dashboard from the Dashboards page.

You can easily reload the dashboard for any search job currently listed on the Jobs page by placing its SID in the provided field and pressing the **Return** key.

The dashboard supplies the essential properties of the search job in a format that is designed for quick scanning. It breaks down into four sections:

Summary

The Summary presents basic metrics and facts about the search job, including the number of events scanned per second, the result count, and the **search mode** of the search.

Search Strings

The Search Strings section shows the search string that was provided to run the search alongside the string for the optimized version of the search that was actually run behind the scenes to improve search performance. The Search String section also includes the search strings for the map and reduce phases of the search, if applicable.

Search Costs

The Search Costs section provides simple breakdowns of search costs by command and phase, as well as the cumulative startup handoff time.

Indexers

The Indexers section lists the indexers that processed the search job. The shading of duration values on the Time Spent Running Search Per Indexer table helps you quickly identify indexers that did not process the search as efficiently as their peers. Click on an indexer name in that table to see the time the indexer spent on the search, broken out by search process.

The following Job Details dashboard panels will not display results for users with roles that do not have access to the `_introspection` index:

- Search Costs by Command
- Approximate Time Spent in Reduce Phase
- Map Phase Search Costs by Command
- Reduce Phase Search Costs by Command

Contact your administrator if you require access to this information.

To return to the main Search Job Inspector, right-click on the dashboard and select **Back**.

Review the specific execution costs and properties of your search job

The key information that the Search Job Inspector displays are the execution costs and the search job properties.

Execution costs

The Execution costs section lists information about the components of the search and how much impact each component has on the overall performance of the search.

Search job properties

The Search job properties section lists other characteristics of the job.

Execution costs

With the information in the **Execution costs** section, you can troubleshoot the efficiency of your search. You can narrow down which processing components are impacting the search performance. This section contains information about the search processing components that were used to process your search.

- The component durations in seconds.
- How many times each component was invoked while the search ran.
- The input and output event counts for each component.

The Search Job Inspector lists the components alphabetically. The number of components that you see depend on the search that you run.

The following tables describes the significance of each individual command and distributed component in a typical keyword search.

Execution costs of search commands

In general, for each command that is part of the search job, there is a parameter `command.<command_name>`. The values for these parameters represent the time spent in processing each `<command_name>`. For example, if the table command is used, you will see `command.table`.

Search command component name	Description
<code>command.search</code>	After the Splunk software identifies the events that contain the indexed fields matching your search, the events are analyzed to identify which events match the other search criteria. These are concurrent operations, not consecutive. • <code>command.search.index</code> - tells how long it took to look into the TSIDX files for the location to read in the raw data. This is the time spent identifying, from the tokens in the base search, what events to retrieve. • <code>command.search.rawdata</code> - tells how long it took to read the actual events from the rawdata files. • <code>command.search typer</code> - tells how long it took to assign event types to events.

	• command.search.kv - tells how long it took to apply field extractions to the events. • command.search.fieldalias - tells how long it took to rename fields based according to props.conf. • command.search.lookups - tells how long it took to create new fields based on existing fields (perform field lookups). • command.search.filter - tells how long it took to filter out events that do not match, for example fields and phrases. • command.search.tags - tells how long it took to assign tags to events.
--	--

There is a relationship between the type of commands used and the numbers you can expect to see for Invocations, Input count, and Output count. For searches that generate events, you expect the input count to be 0 and the output count to be some number of events X. If the search is both a generating search and a filtering search, the filtering search would have an input (equal to the output of the generating search, X) and an output=X. The total counts would then be input=X, output=2*X, and the invocation count is doubled.

Execution costs of dispatched searches

Distributed search component name	Description
dispatch.h.check_disk_usage	The time spent checking the disk usage of this job.
dispatch.h.create_dSearchResultInfrastructure	The time to create and set up the collectors for each peer and execute the HTTP post to each peer.
dispatch.h.earliest_time	Specifies the earliest time for this search. Can be a relative or absolute time. The default is an empty string.
dispatch.h.emit_prereport_files	When running a transforming search , Splunk Enterprise cannot compute the statistical results of the report until the search completes. After it fetches events from the search peers (dispatch.fetch), it, writes out the results to local files. dispatch.emit_prereport_files provides the time that it takes for Splunk Enterprise to write the transforming search results to those local files.
dispatch.h.evaluate	The time spent parsing the search and setting up the data structures needed to run the search. This component also includes the time it takes to evaluate and run subsearches. This is broken down further for each search command that is used. In general, dispatch.evaluate.<command_name> tells you the time spent parsing and evaluating the <command_name> argument. For example, dispatch.evaluate.search indicates the time spent evaluating and parsing the search command argument.

<code>dispatch.fetch</code>	The time spent by the search head waiting for or fetching events from search peers. The <code>dispatch.fetch</code> value is different than the <code>command.search</code> value. The <code>command.search</code> value includes time spent by all indexers, which can be greater than the actual elapsed time of the search. If you have only a single node, then the <code>dispatch.fetch</code> and the <code>command.search</code> values will be similar. In a distributed environment, depending on the search, these values can be very different.
<code>dispatch.preview</code>	The time spent generating preview results.
<code>dispatch.process_remote_timeline</code>	The time spent decoding timeline information generated by search peers.
<code>dispatch.reduce</code>	The time spent reducing the intermediate report output.
<code>dispatch.stream.local</code>	The time spent by search head on the streaming part of the search.
<code>dispatch.stream.remote</code>	The time spent executing the remote search in a distributed search environment, aggregated across all peers. Additionally, the time spent executing the remote search on each remote search peer is indicated with: <code>dispatch.stream.remote.<search_peer_name>.output_count</code> represents bytes sent rather than events in this case.
<code>dispatch.timeline</code>	The time spent generating the timeline and fields sidebar information.
<code>dispatch.writeStatus</code>	The time spent periodically updating <code>status.csv</code> and <code>info.csv</code> in the job's dispatch directory.
<code>startup.configuration</code>	The time spent generating the startup configuration.
<code>startup.handoff</code>	The time elapsed between the forking of a separate search process and the beginning of useful work of the forked search processes. In other words it is the approximate time it takes to build the search apparatus. This is cumulative across all involved peers. If this takes a long time, it could be indicative of I/O issues with <code>.conf</code> files or the dispatch directory.

Search job properties

The **Search job properties** fields provide information about the search job. The **Search job properties** fields are listed in alphabetical order.

Parameter name	Description
<code>cursorTime</code>	The earliest time from which no events are later scanned. Can be used to indicate progress. See description for <code>doneProgress</code> .
<code>delegate</code>	For saved searches, specifies jobs that were started by the user. Defaults to scheduler.
<code>diskUsage</code>	The total amount of disk space used, in bytes.
<code>dispatchState</code>	The state of the search. Can be any of the following states: • QUEUED • PARSING • RUNNING • PAUSE • FINALIZING • INTERNAL_CANCEL • USER_CANCEL • BAD_INPUT_CANCEL • QUIT • FAILED • DONE
<code>doneProgress</code>	A number between 0 and 1.0 that indicates the approximate progress of the search. $\text{doneProgress} = (\text{latestTime} - \text{cursorTime}) / (\text{latestTime} - \text{earliestTime})$
<code>dropCount</code>	For real-time searches only, the number of possible events that were dropped due to the <code>rt_queue_size</code> (defaults to 100000).
<code>earliestTime</code>	A time string that sets the earliest (inclusive), respectively, time bounds for the search. Can be used to indicate progress. See description for <code>doneProgress</code> .
<code>eai:acl</code>	Describes the app and user-level permissions. For example, is the app shared globally, and what users can run or view the search?
<code>eventAvailableCount</code>	The number of events that are available for export.
<code>eventCount</code>	The number of events returned by the search. In other words, this is the subset of scanned events (represented by the <code>scanCount</code>) that actually matches the search terms.
<code>eventFieldCount</code>	The number of fields found in the search results.
<code>eventIsStreaming</code>	Indicates if the events of this search are being streamed.

eventIsTruncated	Indicates if events of the search have not been stored, and thus not available from the events endpoint for the search.
eventSearch	Subset of the entire search that is before any transforming commands. The timeline and events endpoint represents the result of this part of the search.
eventSorting	Indicates if the events of this search are sorted, and in which order. asc = ascending; desc = descending; none = not sorted
isBatchMode	Indicates whether or not the search is running in batch mode. This applies only to searches that include transforming commands.
isDone	Indicates if the search has completed.
isFailed	Indicates if there was a fatal error executing the search. For example, if the search string had invalid syntax.
isFinalized	Indicates if the search was finalized (stopped before completion).
isPaused	Indicates if the search has been paused.
isPreviewEnabled	Indicates if previews are enabled.
isRealTimeSearch	Indicates if the search is a real time search.
isRemoteTimeline	Indicates if the remote timeline feature is enabled.
isSaved	Indicates that the search job is saved, storing search artifacts on disk for 7 days from the last time that the job has been viewed or touched. Add or edit the default_save_ttl value in limits.conf to override the default value of 7 days.
isSavedSearch	Indicates if this is a saved search run using the scheduler.
isTimeCoursored	Specifies if the cursorTime can be trusted or not. Typically this parameter is set to true if the first command is search.
isZombie	Indicates if the process running the search is dead, but with the search not finished.
keywords	All positive keywords used by this search. A positive keyword is a keyword that is not in a NOT clause.
label	Custom name created for this search.
latestTime	A time string that sets the latest (exclusive), respectively, time bounds for the search. Can be used to indicate progress. See description for doneProgress .

numPreviews	Number of previews that have been generated so far for this search job.
messages	Errors and debug messages.
optimizedSearch	The restructured syntax for the search that was run. The built-in optimizers analyze your search and restructure the search syntax, where possible, to improve search performance. The search that you ran is displayed under the search job property.
performance	This is another representation of the Execution costs .
remoteSearch	The search string that is sent to every search peer.
reportSearch	If reporting commands are used, the reporting search.
request	GET arguments that the search sends to splunkd .
resultCount	The total number of results returned by the search.
resultIsStreaming	Indicates if the final results of the search are available using streaming (for example, no transforming operations).
resultPreviewCount	The number of result rows in the latest preview results.
runDuration	Time in seconds that the search took to complete.
scanCount	The number of events that are scanned or read off disk.
search	The search string.
searchCanBeEventType	If the search can be saved as an event type, this will be 1, otherwise, 0. Only base searches (no subsearches or pipes) can be saved as event types.
searchProviders	A list of all the search peers that were contacted.
searchTelemetry	A JSON object that contains search job metadata. The structure of this JSON object is likely to change from release to release.
sid	The search ID number.

statusBuckets	Maximum number of timeline buckets.
ttl	The time to live, or time before the search job expires after it completes.
Additional info	Links to further information about your search. These links may not always be available. • timeline • field summary • search.log

Note: When troubleshooting search performance, it's important to understand the difference between the **scanCount** and **resultCount** costs. For dense searches, the **scanCount** and **resultCount** are similar (**scanCount** = **resultCount**); and for sparse searches, the **scanCount** is much greater than the result count (**scanCount** >> **resultCount**). Search performance should not so much be measured using the **resultCount** /time rate but **scanCount** /time instead. Typically, the **scanCount** /second event rate should hover between 10k and 20k events per second for performance to be deemed good.

Debug messages

Configure the Search Job Inspector to display DEBUG messages when there are errors in your search. For example, DEBUG messages can warn you when there are fields missing from your results.

The Search Job Inspector displays DEBUG messages at the top of the Search Job Inspector window, after the search has completed.

By default the Search Job Inspector hides DEBUG messages.

Prerequisites

- Only users with file system access, such as system administrators, can configure the Search Job Inspector to display DEBUG messages.
- Review the steps in [How to edit a configuration file](#) in the Splunk Enterprise *Admin Manual*.
- You can have configuration files with the same name in your default, local, and app directories. Read [Where you can place \(or find\) your modified configuration files](#) in the Splunk Enterprise *Admin Manual*.

CAUTION: Never change or copy the configuration files in the default directory. The files in the default directory must remain intact and in their original location. Make changes to the files in the local directory.

1. To configure DEBUG messages to display, open **limits.conf**.
2. Set the **infocsv_log_level** parameter in the **[search_info]** stanza to **DEBUG**.

```
[search_info]
infocsv_log_level = DEBUG
```

Examples of Search Job Inspector output

Here's an example of the execution costs for a dedup search, run over All time:

```
* | dedup punct
```

The search commands component of the **Execution costs** panel might look something like this:

Execution costs

Duration (seconds)		Component	Invocations	Input count	Output count
	0.039	command.dedup	20	3,506	1,860
	0.019	command.fields	19	56,843	56,843
█	0.651	command.pdedup	19	56,843	3,506
█	2.546	command.search	19	-	56,843
█	1.282	command.search.rawdata	17	-	-
█	0.641	command.search.typer	19	56,843	56,843
█	0.442	command.search.kv	17	-	-
	0.055	command.search.index	19	-	-
	0.019	command.search.fieldalias	17	56,843	56,843
	0.019	command.search.tags	19	56,843	56,843
	0.017	command.search.lookups	17	56,843	56,843

The `command.search` component, and everything under it, gives you the performance impact of the `search` command portion of your search, which is everything before the pipe character.

The `command.pdedup` gives you the performance impact of processing the results of the `search` command before passing it into the `dedup` command.

- The Input count of `command.pdedup` matches the Output count of `command.search`.
- The Input count of `command.dedup` matches the Output count of `command.pdedup`.

In this case, the Output count of `command.pdedup` should match the number of events returned at the completion of the search. This is the value of `resultCount`, under **Search job properties**.

See also

Video

[Using the Splunk Search Job Inspector](#)

Dispatch directory and search artifacts

Each search or alert you run creates a **search artifact** that must be saved to disk. The artifacts are stored in directories under the `dispatch` directory. For each **search job**, there is one search-specific directory. When the job expires, the search-specific directory is deleted.

See [About jobs and job management](#) for information about search jobs. See [Extending job lifetimes](#) for information changing the lifetimes for search jobs.

Dispatch directory location

The **dispatch** directory stores artifacts on nodes where the searches are run. The nodes include search heads, search peers, and standalone Splunk Enterprise instances. The path to the **dispatch** directory is `$SPLUNK_HOME/var/run/splunk/dispatch`.

Dispatch directory contents

In the **dispatch** directory, a search-specific directory is created for each search or alert. Each search-specific directory contains several files including a .srs file of the search results, a **search.log** file with details about the search execution, and more.

View dispatch directory contents

From a command-line window, or UI window such as Windows Explorer or Finder, you can list the search-specific directories.

For example to view a list in a command-line window, change to the dispatch directory and list the contents in that directory. The following list contains both ad hoc, real-time, and scheduled search-specific directories.

```
# cd $SPLUNK_HOME/var/run/splunk/dispatch
# ls
1346978195.13
rt_scheduler__admin__search__RMD51cfb077d0798f99a_at_1469464020_37.0
1469483311.272
1469483309.269
1469483310.27
scheduler__nobody_c3Bsdw5rX2FyY2hpdmVy__RMD5473cbac83d6c9db7_at_1469503020_53
admin__admin__search__count_1347454406.2
rt_1347456938.31
1347457148.46
subsearch_1347457148.46_1347457148.1
```

The contents of a search-specific directory includes files and subdirectories. The following example shows the contents of a search-specific directory named **1346978195.13**.

```
#ls 1346978195.13/
args.txt
audited
buckets/
events/
generate_preview
```



```

info.csv
metadata.csv
peers.csv
request.csv
results.srs.zst
runtime.csv
search.log
status.csv
timeline.csv

```

Note: Windows users should use **dir** instead of **ls** in a command-line window, or Windows Explorer, to see the contents of the **dispatch** directory.

File descriptions for search-specific directories

The files or subdirectories that appear in a search-specific directory depend on the type of search that you run. The following table lists the files and subdirectories that might appear in your search-specific directories.

File name	Contents
args.txt	The arguments that are passed to the search process.
alive.token	The status of the search process. Specifies if the search is alive or not alive.
audited	A flag to indicate the events have been audit signed.
buckets	A subdirectory that contains the field picker statistics for each-bucket. The buckets are typically the chunks that are visible in the search histogram UI. This is not related to index buckets.
custom_prop.csv	Contains custom job properties, which are arbitrary key values that can be added to search job, retrieved later, are mostly used by UI display goals.
events	A subdirectory that contains the events that were used to generate the search results.
generate_preview	A flag to indicate that this search has requested preview. This file is used mainly for Splunk Web searches.
info.csv	A list of search details that includes the earliest and latest time, and the results count.
metadata.csv	The search owner and roles.
peers.csv	The list of peers asked to run the search.
pipeline_sets	The number of pipeline sets an indexer runs. The default is 1.

File name	Contents
remote_events or events_num_num.csv.gz	Used for the remote-timeline optimization so that a reporting search that is run with <code>status_buckets>0</code> can be MapReduced.
request.csv	A list of search parameters from the request, including the fields and the text of the search.
results.srs.zst	An archive file that contains the search results in a binary serialization format.
rtwindow.czv.gz	Events for the latest real-time window when there are more events in the window than can fit in memory. The default limit is 50K.
runtime.csv	Pause and cancel settings.
search.log	The log from the search process.
sort...	A sort temporary file, used by the <code>sort</code> command for large searches.
srtmpfile...	A generic search tempfile, used by facilities which did not give a name for their temporary files.
status.csv	The current status of the search, for example if the search is still running. Search status can be any of the following: QUEUED, PARSING, RUNNING, PAUSED, FINALIZING, FAILED, DONE.
timeline.csv	Event count for each timeline bucket.

The dispatch directory also contains ad hoc data model acceleration summaries. These are different from persistent data model acceleration summaries, which are stored at the index level.

Working with the results.srs.zst file

The `results.srs.zst` file is an archive file that is created when you run a search and contains the search results in a binary serialization format.

To view search results in the `results.srs.zst` file you must convert the file into a CSV format. See the [toCsv](#) CLI utility in the *Troubleshooting Manual* for the command line steps to convert the file.

Dispatch directory naming conventions

The names of the search-specific directories in the dispatch directory are based on the type of search. For saved and scheduled searches, the name of a search-specific directory is determined by the following conditions.

- The name of the search is less than 20 characters and contains only ASCII alphanumeric characters. The search-specific directory name includes the search name.
- The name of the search is 20 characters or longer, or contains non-alphanumeric characters. A hash is used for the name. This is to ensure a search-specific directory named by the search ID can be created on the filesystem.

Note: A search that contains multiple subsearches might exceed the maximum length for the dispatch directory name. When the maximum length is exceeded, the search fails.

Type of search	Naming convention	Examples
Local ad hoc search	The UNIX time of the search.	An ad hoc search. <pre>1347457078.35</pre> A real-time ad hoc search. <pre>rt_1347456938.31</pre> An ad hoc search that uses a subsearch, which creates two dispatch directories. <pre>1347457148.46</pre> <pre>subsearch_1347457148.46_1347457148.1</pre>
Saved search	The user requesting the search, the user context the search is run as, the app the search came from, the search string, and the UNIX time.	"count" – run by admin, in user context admin, saved in app search <pre>admin__admin__search__count_1347454406.2</pre> "Errors in the last 24 hours" – run by somebody, in user context somebody, saved in app search <pre>somebody__somebody__search_RMD5473cbac83d6c9db7_1347455134.20</pre>
Scheduled	The user requesting the search, the user context the search is run as, the	"foo" – run by the scheduler , with no

Type of search	Naming convention	Examples
search	app the search came from, the search string, UNIX time, and an internal ID added at the end to avoid name collisions.	user context, saved in app unix <pre>scheduler__nobody__unix__foo_at_1347457380_051d958b8354c580</pre> "foo2" - remote peer search on search head sh01, with admin user context, run by the scheduler, saved in app search <pre>remote_sh01_scheduler__admin__search__foo2_at_1347457920_79152a9a8bf33e5e</pre>
Remote search	Searches that are from remote peers start with the word "remote".	"foo2" - remote peer search on search head sh01 <pre>remote_sh01_scheduler__admin__search__foo2_at_1347457920_79152a9a8bf33e5e</pre>
Real-time search	Searches that are in real-time start with the letters "rt".	Ad hoc real-time search <pre>rt_1347456938.31</pre>
Replicated search	Searches that are replicated search results (artifacts) start with "rsa_". These SIDs occur in search head clusters when a completed search is replicated to a search head other than the search head that originally ran the search	

Type of search	Naming convention	Examples
Replicated scheduled search	The "rsa_scheduler_" prefix is for replicated results of searches dispatched by the scheduler.	The search "foo" is run by the rsa_scheduler, with no user context, saved in app unix. <pre>rsa_scheduler_nobody_unix_fo o_at_1502989576_051d958b8354 c580</pre>
Report acceleration search	These are the probe searches created by datamodel acceleration to retrieve the acceleration percentage from all peers.	<pre>SummaryDirector_1503528948.2 4878_D12411CE-A361-4F75-B90 A-28AFDA88151B</pre>

Dispatch directory maintenance

The dispatch directory reaper iterates over all of the artifacts every 30 seconds. The reaper deletes artifacts that have expired, based on the last time that the artifacts were accessed and their configured time to live (TTL), or lifetime.

If no search artifacts are eligible to be reaped and the dispatch volume is full, artifacts are not prematurely reaped to recover space. When the dispatch volume is full, new searches cannot be dispatched. You must manually reap the dispatch directory to make space for new search artifacts.

See [Extending job lifetimes](#) for information about changing the default lifetime for the search artifact using Splunk Web.

Search artifact lifetime in the dispatch directory

Default lifetime values depend on the type of search. The lifetime countdown begins when the search completes. The following table lists the default lifetime values by type of search.

Search type	Default lifetime
Manually run saved search or ad hoc search	10 minutes
Remote search from a peer	10 minutes

Search type	Default lifetime
Scheduled search	The lifetime varies by the selected alert action, if any. If an alert has multiple actions, the action with the longest lifetime becomes the lifetime for the search artifact. Without an action, the value is determined by the dispatch.ttl attribute in the savedsearches.conf file. The default value for the dispatch.ttl attribute is twice (2x) the scheduled period. Alert actions determine the default lifetime of a scheduled search. - Email, RSS, and tracking alert actions have a default lifetime of 24 hours. - Script alert actions have a default lifetime of 10 minutes. - Summary indexing and populate lookup alert actions have a default lifetime of 2 minutes.
Show source scheduled search	30 seconds
Subsearch	5 minutes Subsearches generate two search-specific directories. There is a search-specific directory for the subsearch and a search-specific directory for the search that uses the subsearch. These directories have different lifetime values.

Change search artifact lifetime

There are a number of ways to change the search artifact lifetime. Modifying the default search behavior affects searches with no other lifetime value or TTL applied.

See [How to edit a configuration file](#) for Splunk Enterprise.

Search behavior type	Process
Global search behavior	In the limits.conf file, set ttl or remote_ttl in the [search] stanza or ttl in the [subsearch] stanza. Modifying limits.conf provides the default for searches, so it affects searches with no other lifetime value applied.
Search specific behavior	In the savedsearches.conf file, set dispatch.ttl for an individual search. Or set an individual value for a search when you save the search in Splunk Web. This overrides the default search behavior.
Searches with alert actions	In the alert_actions.conf, set a ttl value to specify the minimum lifetime of a search artifact if a certain alert action is triggered. This overrides any shorter lifetime applied to a search.

Clean up the dispatch directory based on the age of directories

As more and more artifacts are added to the dispatch directory, it is possible that the volume of artifacts will cause an adverse effect on search performance or that a warning appears in the UI. The warning threshold is based on the `dispatch_dir_warning_size` attribute in the `limits.conf` file.

The default value for the `dispatch_dir_warning_size` attributes is 5000.

You can move search-specific directories from the dispatch directory to another, destination, directory. You move search-specific directories by using the `clean-dispatch` command. You must specify a time that is later than the last modification time for the search-specific directories. The destination directory must be on the same file system as the dispatch directory.

CAUTION: The `clean-dispatch` command is not suitable for production environments and should be used only in certain scenarios as directed by Splunk Support.

Run the command `$SPLUNK_HOME/bin/splunk clean-dispatch help` to learn how to use the `clean-dispatch` command.

See [Too many search jobs](#) in the *Troubleshooting Manual* for more information about cleaning up the dispatch directory.

Limit search process memory usage

Splunk software can be configured to automatically terminate search job processes that exceed a threshold of a configured quantity of resident memory in use.

You might use this feature if:

- You want to be proactive and avoid a scenario where one runaway search causes one or several of your search peers to crash.
- You already have encountered this scenario and do not want it to happen again.
- In the Distributed Management Console, the **Search Activity: Instance** view exposes one or more searches that consume dangerous amounts of physical memory. You can see this information in the **Top 10 memory-consuming search** panel.

What does this threshold do?

Enabling this threshold limits the maximum memory permitted for each search process. A search process that is an outlier in memory size is automatically killed off, limiting damage.

This threshold uses process resource usage information that is recorded by platform instrumentation. So this feature works only on *nix, Solaris, and Windows platforms.

- See [Introspection endpoint descriptions](#) in the *REST API Reference Manual*.
- See [About the platform instrumentation framework](#) in the *Troubleshooting Manual*.

Search memory is checked periodically, so a rapid spike might exceed the configured limit.

The functionality is wired into the DispatchDirectoryReaper, so stalls in the reaper components also cause stalls in how often the memory of searches are checked.

Enable a search process memory threshold

The search process memory tracking is disabled by default.

Splunk Cloud Platform

To enable the `search process memory threshold`, request help from Splunk Support. If you have a support contract, file a new case using the Splunk Support Portal at [Support and Services](#). Otherwise, contact [Splunk Customer Support](#).

Splunk Enterprise

To enable the `search process memory threshold` in the `limits.conf` file, follow these steps.

Prerequisites

- Only users with file system access, such as system administrators, can edit configuration files.
- Review the steps in [How to edit a configuration file](#) in the *Splunk Enterprise Admin Manual*.

CAUTION: Never change or copy the configuration files in the default directory. The files in the default directory must remain intact and in their original location. Make changes to the files in the local directory.

Steps

1. Open or create a local `limits.conf` file in the desired path. For example, use the `$SPLUNK_HOME/etc/apps/search/local` path to apply this change only to the Search app.
2. Under the `[search]` stanza, change the setting for the `enable_memory_tracker` setting to `true`.
3. Review and adjust the memory limit.

You can set the limit to an absolute amount or a percentage of the identified system maximum, using `search_process_memory_usage_threshold` or `search_process_memory_usage_percentage_threshold`, respectively. Searches are always tested against both values, and the lower value applies. See [limits.conf.spec](#) in the **Admin Manual**.

4. To enable the configuration changes, restart Splunk Enterprise.

Where is threshold activity logged?

Threshold activity is logged in different places depending on whether the threshold causes a search process to be stopped on a search head or a search peer.

Search head logging

If the threshold causes a search process to be stopped on a search head, an error is inserted into the `search.log` and the search artifact file `info.csv` on the search head. The error message is also displayed in Splunk Web below the search bar and logged in the `splunkd.log` file in the `DispatchReaper` category.

The error states that the process was terminated and specifies the limit setting and value. The error message differs depending on whether the physical memory usage (in megabytes) or relative physical memory usage (in percent), or both, exceeded the threshold. The message looks something like this:

```
The search process with sid=<sid name> was forcefully terminated because both its physical memory usage (
<specified in MB> ) and its relative physical memory usage ( <specified in percent> ) have exceeded the
'search_process_memory_usage_threshold' ( <specified in MB> ) and
'search_process_memory_usage_percentage_threshold' (<specified in percent>) settings in limits.conf.
```

Search peer logging

If the threshold causes a search process to be stopped on a search peer, an error message is logged in the **splunkd.log** file in the StreamedSearch category and the **splunkd.log** file in the Dispatcher category.

The error states that the process was terminated and specifies the limit setting and value. The error message differs depending on whether the physical memory usage (in megabytes) or relative physical memory usage (in percent), or both, exceeded the threshold. The message looks something like this:

```
Forcefully terminated search process with sid=<sid name> since both its physical memory usage ( <specified in
MB> ) and the relative physical memory usage (<specified in percent>) has exceeded the physical memory
thresholds specified in limits.conf / search_process_memory_usage_threshold ( <specified in MB> ) and
limits.conf/search_process_memory_usage_percentage_threshold <specified in percent>) respectively.
```

Manage Splunk Enterprise jobs from the OS

If you have Splunk Enterprise on Microsoft Windows or *nix, you can manage search jobs from the operating system as described in this topic. For information on how to manage search jobs in Splunk Web, see [Manage search jobs](#) in this manual.

Manage jobs in *nix

When a search job runs, it will manifest itself as a process in the OS called **splunkd search**. You can manage the job's underlying processes at the OS command line.

To see the job's processes and its arguments, type:

```
> top
> c
```

This will show you all the processes running and all their arguments.

Typing **ps -ef | grep "search"** will isolate all the Splunk search processes within this list. It looks like this:

```
[pie@fflanda ~]$ ps -ef | grep 'search'
530369338 71126 59262  0 11:19AM ??          0:01.65 [splunkd pid=59261] search --id=rt_1344449989.64 --
maxbuckets=300 --ttl=600 --maxout=10000 --maxtime=0 --lookups=1 --reduce_freq=10 --rf=* --user=admin --pro --
roles=admin:power:user AhjH8o/Render TERM_PROGRAM_VERSION=303.2
530369338 71127 71126  0 11:19AM ??          0:00.00 [splunkd pid=59261] search --id=rt_1344449989.64 --
maxbuckets=300 --ttl=600 --maxout=10000 --maxtime=0 --lookups=1 --reduce_freq=10 --rf=* --user=admin --pro --
roles=
```

There will be two processes for each search job; the second one is a "helper" process used by the **splunkd** process to do further work as needed. The main job is the one using system resources. The helper process will die on its own if you kill the main process.

The process info includes:

- the search string (search=)
- the job ID for that job (id=)
- the ttl, or length of time that job's artifacts (the output it produces) will remain on disk and available (ttl=)
- the user who is running the job (user=)
- what role(s) that user belongs to (roles=)

When a job is running, its data is being written to `$SPLUNK_HOME/var/run/splunk/dispatch/<job_id>/`. Scheduled jobs (scheduled saved searches) include the saved search name as part of the directory name.

The value of **ttl** for a process will determine how long the data remains in this spot, even after you kill a job. When you kill a job from the OS, you might want to look at its job ID before killing it if you want to also remove its artifacts.

Manage jobs in Windows

On Windows, each search likewise runs as a separate process. Windows does not have a command-line equivalent for the ***nix top** command, but there are several ways in which you can view the command line arguments of executing search jobs.

- Use the **Process Explorer** utility (<http://technet.microsoft.com/en-us/scriptcenter/dd742419.aspx>) to find the command line of the process that is performing the search.
- Use the **TASKLIST** and **Get-WMIObj** commands in the **Powershell** (<http://technet.microsoft.com/en-us/scriptcenter/dd742419.aspx>) command line environment to get the **ProcessID** and **CommandLine** arguments for a search job.

When a search runs, the data for that search is written into the

`%SPLUNK_HOME\var\run\splunk\dispatch\<epoch_time_at_start_of_search>.<number_separator>` directory. Saved searches are written to similar directories that have a naming convention of " **admin__admin__search_** " and a randomly-generated hash of numbers in addition to the UNIX time.

Use the filesystem to manage jobs

You can manage a job by creating and deleting files in the job's artifact directory:

- To cancel a job, go into that job's artifact directory create a file called 'cancel'.
- To preserve that job's artifacts (and ignore its ttl setting), create a file called 'save'.

- To pause a job, create a file called 'pause', and to unpause it, delete the 'pause' file.

Save and Schedule Searches

Saving searches

If you are using reports, also referred to as "saved searches," in the Splunk Dashboard Studio see [Use reports and saved searches with ds.savedSearch](#) in the *Splunk Dashboard Studio* manual.

To learn more using ad hoc searches see [Create search-based visualizations with ds.search](#) in the *Splunk Dashboard Studio* manual.

When you create a search, you have several options to choose from to save the search. In the Search app, the choices are listed under the **Save As** drop-down.

Save as option	Description	More information
Report	When you create a search that you would like to run again, you can save the search as a report.	See Create and edit reports in the <i>Reporting Manual</i> . If you are using reports, also referred to as "saved searches," in the Splunk Dashboard Studio, see Use reports and saved searches with ds.savedSearch in the <i>Splunk Dashboard Studio</i> manual for information on how to use them.
Dashboard panel	You can also save a search as a dashboard panel. Dashboards can have one or more panels which can show search results in tables or in graphical visualizations.	See Getting started in the <i>Dashboards and Visualizations</i> manual. These searches are also referred to as "ad hoc" searches. If you are using these searches in the Splunk Dashboard Studio, see, Create search-based visualizations with ds.search in the <i>Splunk Dashboard Studio</i> manual.
Alert	Some searches provide timely information that you want to be notified about. You can save a search as an alert. An alert is an action that a saved search triggers, based on the results of the search. The action might be to send an email or run a script.	See About alerts in the <i>Alerting Manual</i> .
Event type	You can save a search as an event type. Event types are a categorization system to help you make sense of your data. Event types let you sift through huge amounts of data, find similar patterns, and create alerts and reports.	See About event types in the <i>Knowledge Manager Manual</i> .

See Also

[Scheduling searches](#)

Scheduling searches

You can **[schedule searches](#)** to run on a regular basis.

Option	Description	More information
Report	After you save a search as a report, you can convert that report into a scheduled report. A scheduled report is a report that runs on a scheduled interval, and which can trigger an action each time the report runs. There are two actions available for scheduled reports: Send email and Run a script.	See Schedule reports in the <i>Reporting Manual</i> .
Dashboard panel	There are several options to create a scheduled report: - You can create a dashboard panel that is based on that scheduled report. - When you save an ad hoc search as a dashboard panel, the panel refers to the search as an inline search. You can edit the dashboard panel to convert the search to a report and then schedule the report.	See Working with dashboard panels in the <i>Dashboards and Visualizations</i> manual.
Alert	You can create a scheduled alert to search for events on a regular schedule. You can configure scheduling, trigger conditions, and throttling to customize the alert.	See Create scheduled alerts in the <i>Alerting Manual</i> .

See Also

[Saving searches](#)

Run Federated Searches Across Multiple Splunk Deployments

About federated search

You can run federated searches to search datasets outside of your local Splunk platform deployment. From your local search head, federated search gives you a holistic view of datasets across multiple Splunk platform deployments.

Federated search is topology-agnostic, so it works despite the complexity of the Splunk platform deployments involved. You can run a federated search across any remote Splunk Cloud Platform or Splunk Enterprise deployment.

Do you use **hybrid search**? See [Migrate from hybrid search to federated search](#).

Components of a typical federated search setup

Federated search introduces a set of terms. Familiarize yourself with them before you attempt to dig into setting up and running federated searches.

Federated search

A search of one or more remote datasets on one or more federated providers.

Local deployment

The Splunk platform deployment from which you run federated searches. The federated search head for your federated search resides on your local deployment.

In this context, "local" does not refer to your physical location. If you are in London and are logging into a Splunk platform deployment located in New York City when you run a federated search, that New York City deployment is the local deployment for your federated search.

Federated search head

A search head residing on your local deployment that initiates federated searches.

Federated provider

A remote Splunk platform deployment. Contains the data that you search with your federated searches.

Before you can run federated searches, you must create federated provider definitions on the local deployment. A federated provider definition serves several purposes:

- It enables the federated search head to make network connections to the federated provider and run searches on a remote search head on that provider through a service account.
- It determines whether the federated provider runs in standard or transparent mode, which in turn affects how you write and run federated searches.

See [About standard and transparent mode](#).

See [Define a federated provider](#).

Remote search head

A search head on a federated provider.

Federated index

An index you create on your federated search head to run federated searches over standard mode federated providers. Each federated index maps to a specific remote dataset on a standard mode federated provider. Federated indexes do not ingest data or events. They provide a logical mapping to remote datasets. See [Create a federated index](#).

Remote dataset

A dataset on a standard mode federated provider. Currently, only events indexes, data models, and saved searches qualify as remote datasets.

How federated search works

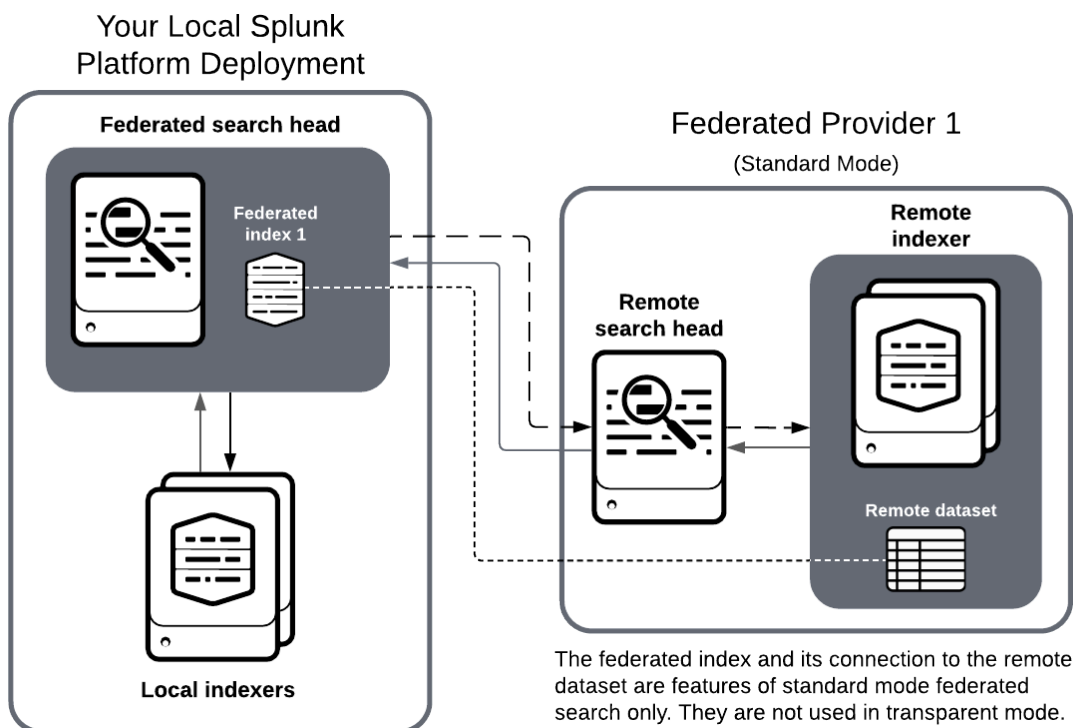
The federated search process works similarly to [distributed search](#). On a distributed search, the initial processing of a search is handled by the indexers of a Splunk platform deployment, and then the results are aggregated on the search head for that deployment to produce a final result set.

Federated searches, however, are broken up into parts that are processed on a "local" Splunk platform deployment and parts that are processed on one or more remote deployments. Each of these remote deployments is a federated provider.

For example, say you have a simple federated search that involves only one federated provider. In this case, the federated search process sends the remote portion of the search to the federated provider. On the federated provider, the remote search head and its indexers process the search independently, performing a pre-aggregation of the results. The remote search head then sends the results back to the federated search head on the local deployment, where the local search head aggregates the remote results into the final result set for the complete federated search.

The following diagram illustrates a federated search over a remote deployment. The remote deployment is a standard mode federated provider. The federated provider has an events index dataset that is available for federated searches. On the local deployment, a federated index on the federated search head maps to a remote dataset.

Note: The federated index in this example is there because the federated provider in this example is a standard mode federated provider. Transparent mode federated providers do not require federated indexes.



A simple federated search for this setup might look like this:

```
index=federated:provider1_fedindex1 | stats count
```

This search references a federated index named **provider1_fedindex1**. The **provider1_fedindex1** federated index maps to the remote dataset stored on Federated Provider 1. The remote search head uses this mapping to send back events from its remote index dataset to the federated search head on your local deployment. The federated search head runs the **stats count** operation on those events. When this **stats count** aggregation is complete, the federated search head presents the results without additional processing, as there are no additional datasets involved in the search.

See [Run federated searches](#) to learn how to write federated searches.

Kinds of federated searches you can set up

This table lists the four kinds of federated searches that you can set up, and the Splunk Enterprise or Splunk Cloud Platform versions that those types of federated searches require.

Kind of federated search	Local deployment	Remote deployment
Splunk Enterprise to Splunk Enterprise	Splunk Enterprise (version 8.2.0 or higher)	Splunk Enterprise (version 8.2.0 or higher)
Splunk Cloud Platform to Splunk Cloud Platform	Splunk Cloud Platform (version 8.1.2103 or higher)	Splunk Cloud Platform (version 8.1.2103 or higher)
Splunk Enterprise to Splunk Cloud Platform	Splunk Enterprise (version 8.2.0 or higher)	Splunk Cloud Platform (version 8.2.2104 or higher)
Splunk Cloud Platform to Splunk Enterprise	Splunk Cloud Platform (version 8.2.2203 or higher)	Splunk Enterprise (version 9.0.0 or higher)

If you have a Splunk Enterprise deployment that is lower than 8.2 and want to run federated searches without upgrading the entire deployment, you can upgrade a single search head in that deployment to 8.2 and run federated searches from that search head.

Splunk Cloud Platform environment and region support

Federated search supports Splunk Cloud Platform deployments in AWS and Google Cloud.

For the conditions and limitations that apply to region support for federated search in AWS and Google Cloud, including search between regions, see the coverage of federated search in the [Splunk Cloud Platform Service Description](#).

About the standard and transparent modes

When you define a federated provider, you must decide what mode you want that provider to use. Federated provider modes offer different federated search experiences, and you must select the mode that best fits your needs.

You have two federated provider mode options: standard and transparent.

- Choose standard mode if you want to restrict data access to specific remote datasets such as indexes, saved searches, or data models. Standard mode is the best fit for federated search users who are not migrating from a hybrid search setup.
- Choose transparent mode if you use **hybrid search** and want to migrate to federated search. Transparent mode lets you run your hybrid mode searches without syntax changes.

Note: Federated search does not support setting up a mix of transparent mode and standard mode federated providers for the same local deployment, as this practice can introduce unexpected complications. All of your federated providers should use the same provider mode.

Transparent mode is available in Splunk Cloud Platform version 8.2.2107 and higher and Splunk Enterprise version 9.0.0 and higher. The following table describes the differences between the two modes.

Category	Standard mode federated search	Transparent mode federated search
Kinds of federated search	Applies to the following kinds of federated search: • Splunk Cloud Platform to Splunk Cloud Platform • Splunk Enterprise to Splunk Enterprise • Splunk Cloud Platform to Splunk Enterprise • Splunk Enterprise to Splunk Cloud Platform, if you are not migrating to federated search from a hybrid search setup.	Applies to Splunk Enterprise to Splunk Cloud Platform federated search, if you are migrating from a hybrid search setup.
Provider setup	Requires: • A federated provider definition. • A separate federated index definition for each dataset on the federated provider that you want to search. You can designate events indexes, data models, and saved searches as searchable datasets. You can associate a single remote deployment with multiple standard mode federated provider definitions. For example, for one remote deployment you might set up different standard mode federated provider definitions for different application contexts.	Requires federated provider definition only. You can associate a single remote deployment with only one transparent mode federated provider definition. See Define a federated provider .
User permissions applied to remote portion of search	The federated search runs on the federated provider with the permissions of the service account user you define on the federated provider.	The federated search runs on the federated provider with the permissions of the user who initiates the search on the local deployment.
Application context of	Uses the application context set in the federated provider definition.	Uses the application context of the local search.

Category	Standard mode federated search	Transparent mode federated search
remote portion of search		
Knowledge objects applied to remote portions of searches	Uses knowledge objects that are defined on the remote search head of the federated provider. See Custom knowledge object coordination for standard mode federated providers .	Through bundle replication, uses knowledge objects from the federated search head of the local deployment.
Security	The role-based access control permissions for the service account user on the federated provider determine what your local users can search on the federated provider. In addition, access to federated indexes is role-based, which allows you to restrict your local users' ability to search remote datasets on the federated provider.	The role-based access control permissions for your local users determine what your users can search on the federated provider. In addition, to activate transparent mode federated search capabilities for the federated provider, the service account must have the fsh_manage capability.
Which local searches run as federated searches on the federated provider?	Only local searches that invoke federated indexes run over remote datasets on federated providers. Searches that do not invoke federated indexes run only on your local deployment.	When you connect your local instance to a transparent mode federated provider, all of your local searches run over that federated provider as federated searches, whether or not you intend for them to search remote datasets on that provider. This might reduce the performance of searches that you intend to run only over your local deployment.
Special search processing language (SPL) syntax required?	Yes	No
Can send only specific subsearches to the remote search head?	Yes	No
Can run entire federated search on the remote search head?	Yes	No
Provides separate namespace for remote	Yes	No

Category	Standard mode federated search	Transparent mode federated search
indexes (to avoid name collisions)?		
Can run remote saved searches?	Yes	No
Can search unaccelerated data models?	Yes. In your search, reference a local federated index that maps to a remote data model on the federated provider.	Yes. In your search, reference a local data model to get data from your local deployment as well as remote data from the federated provider.
Can search accelerated data models?	Yes. In your search, reference a local federated index that maps to a remote accelerated data model on the federated provider. Searches that use tstats in conjunction with prestats=t are not supported.	No
SPL limitations	Standard mode searches cannot include: • Generating commands other than search, from, and tstats. • Metrics commands such as mpreview or mstats. • Wildcards in references to federated indexes.	Transparent mode federated searches have the following limitations: • You cannot use the from command to reference remote saved search datasets. • You cannot use the datamodel command to search remote data model datasets. • You cannot use the rest command in transparent mode federated searches.
Dataset availability	You can search the following dataset types on a federated provider: • events indexes • saved searches • data models	You can search events and metrics indexes on a federated provider.

About federated search and Splunk security and IT products

Federated search supports Splunk IT Service Intelligence version 4.16.0 and higher, for transparent mode federated search only. For more information, see [New features in Splunk IT Service Intelligence](#) in the ITSI 4.16.0 *Release Notes*.

Federated search does not currently support Splunk Enterprise Security.

Set up federated search between Splunk platform deployments

Complete the following steps to set up federated search between a local deployment and a remote deployment.

Note:

To run federated searches, Splunk Cloud Platform deployments require additional configuration from Splunk Support. This is true whether the Splunk Cloud Platform deployment is on the local or remote side of the federated search. If you are setting up federated search between two Splunk Cloud Platform deployments, you must contact Splunk Support for both deployments.

If you have a support contract, file a new case using the Splunk Support Portal at [Support and Services](#). Otherwise, contact [Splunk Customer Support](#).

Number	Task	For more information see
1	Determine the federated provider mode of the remote deployment.	About standard and transparent mode
2	Set up a service account on the remote deployment.	Service accounts and federated search security
3	Apply a federated provider definition to the remote deployment. Set the provider mode.	Define a federated provider
4	If you have defined a standard mode federated provider, define one or more federated indexes for it.	Create a federated index
5	If you have defined a standard mode federated provider and you intend to run federated searches that are dependent on custom knowledge objects, ensure those knowledge objects exist on the remote search head.	Custom knowledge object coordination for standard mode federated providers
6	Run federated searches.	Run federated searches

Migrate from hybrid search to federated search

Do you run **hybrid searches** from your Splunk Enterprise search head that combine data from your Splunk Enterprise instance with data from a Splunk Cloud Platform environment? Now you can migrate your hybrid searches to the federated search solution. Federated search expands your cross-deployment search capabilities.

Comparing hybrid search and federated search

The following table shows you how hybrid search and transparent mode federated search match up.

Feature	Hybrid search	Federated search in transparent mode
Environments spanned in a search	Hybrid searches can span a single Splunk Enterprise deployment and a single Splunk Cloud Platform deployment.	Transparent mode federated searches can span a single Splunk Enterprise deployment and multiple Splunk Cloud Platform deployments.
Splunk Cloud	Hybrid search supports only SCP environments with	Federated search supports both Classic and Victoria

Feature	Hybrid search	Federated search in transparent mode
Platform (SCP) experience designation support	the Classic Experience designation. Hybrid search does not support SCP environments with the Victoria Experience designation.	Experience SCP environments.
Ad-hoc search	Yes	Yes
Scheduled search	No	Yes
Workload management (WLM)	No	Yes
Search processing language (SPL) coverage	No special syntax required. All commands allowed.	No special syntax required. All commands allowed.
Security (RBAC)	Hybrid search enforces all security at the Splunk Enterprise search head.	Transparent mode federated searches enforce all security at the Splunk Enterprise search head. There is no need to configure additional security on the indexer tier. See Service accounts and federated search security .
Search head architecture	For hybrid search, the Splunk Cloud Platform requires a single search head. Hybrid search does not let you search Splunk Cloud Platform environments with search head cluster configurations.	Federated search works with all search management tier architecture options and combinations.
Version compatibility and upgrades	There are strict version dependencies for hybrid search between Splunk Enterprise and Splunk Cloud Platform environments. An upgrade on either side can break hybrid searches until you upgrade the corresponding deployment to a compatible version.	For transparent mode federated search, you need to have Splunk Enterprise 9.0 or higher and Splunk Cloud Platform 8.2.2107 or higher. There isn't a strict versioning dependency between the two platforms. Splunk Cloud Platform upgrades do not break federated searches.
Operability	To activate and configure hybrid search between a Splunk Enterprise environment and Splunk Cloud Platform environment, you must contact your Splunk representative.	You should be able to activate and configure federated search between a Splunk Enterprise environment and a Splunk Cloud Platform environment by following the steps outlined in this topic.

Transparent or standard mode?

Federated search offers two modes of operation: standard and transparent. These modes provide two different experiences of federated search.

Transparent mode gives hybrid search users a smooth transition to federated search. Transparent mode requires the least amount of change to existing saved searches and search workflow.

For an overview of federated search terminology and a comparison of the two federated search modes, see [About federated search](#).

Move to federated search

To move to federated search, you must first contact Splunk Support to get your Splunk Cloud Platform deployment configured for federated search. Then you follow a few self-service setup steps. Afterwards, you can run federated searches that combine data from your local Splunk Enterprise deployment and a remote Splunk Cloud Platform deployment.

1. [Activate token authentication for the Splunk Cloud Platform deployment](#)
2. [Configure the IP allow list for the Splunk Cloud Platform deployment](#)
3. [Create a service account for the Splunk Cloud Platform deployment](#).
4. [Disable hybrid search on your Splunk Enterprise deployment](#).
5. [Create a federated provider definition for the Splunk Cloud Platform deployment](#).
6. [Write and run federated searches](#).

Activate token authentication for the Splunk Cloud Platform deployment

You must activate token authentication for your Splunk Cloud Platform deployment, if it isn't already activated. See [Enable or disable token authentication](#) in *Securing Splunk Cloud Platform*.

Configure the IP allow list for the Splunk Cloud Platform deployment

You must configure the IP allow list for the Splunk Cloud Platform deployment through the IP allow list page in Splunk Web. For details, see [Configure IP allow lists using Splunk Web](#) in the *Splunk Cloud Platform Admin Manual*.

The IP allow list use case is **Search head API access**.

In the IP allow list, provide the Splunk Enterprise deployment search head or search head cluster subnets using CIDR notation following this format: `<ip_address>/32`.

Create a service account for the Splunk Cloud Platform deployment

A service account is a dedicated user account that you create on the remote Splunk Cloud Platform deployment over which you want to run federated searches. The service account allows users on the local Splunk Enterprise deployment to apply their own data access privileges and permissions to the searches they run over the remote deployment. The service account must have a role with the `fsh_manage` capability.

See [Service accounts and federated search security](#).

Turn off hybrid search on your Splunk Enterprise and Splunk Cloud Platform deployments

Turn off hybrid search on your local Splunk Enterprise deployment before you set up federated provider definitions on that deployment. See [Disable hybrid search](#) in the *Splunk Cloud Platform Admin Manual*.

Turn hybrid search off on your remote Splunk Cloud Platform deployment before you define your remote deployment as a federated provider for your local Splunk Enterprise deployment.

Splunk Customer Support will assist you in turning off hybrid search functionality configured for your Splunk Cloud Platform deployment. If you have a support contract, log in and file a new case using the [Splunk Support Portal](#). Otherwise, contact [Splunk Customer Support](#).

Note: If you skip this step, you risk getting duplicated or incomplete search results during the period when hybrid search and transparent mode federated search are simultaneously operating on your system.

Create a federated provider definition for the Splunk Cloud Platform deployment

You create a **federated provider** definition for your Splunk Cloud Platform deployment through the Federated Provider page. The Federated Provider page is available in Settings. These settings determine how the **federated search head** on your Splunk Enterprise deployment collaborates with the remote search heads on your federated provider to run a federated search.

See [Define a federated provider](#).

Add Federated Splunk Provider
×

Federated Provider Details

Designate a remote data source as a federated provider and supply requirements for secure data transfer.

Provider Mode
☐ Standard
☒ Transparent

You cannot change this provider mode selection after you save the federated provider definition. [Learn more](#)

Provider Name

Remote Host

Enter host name with port number or IP address with port number.
Example: buttercup.splunkcloud.com:8089 or 127.0.0.1:8089

Service Account Username

Service Account Password

Warning and Consent
☒ Agree

By using the Federated Search feature you acknowledge and agree that the deployment initiating the federated search will receive and store search result data from the queried remote deployment(s). As such, you acknowledge and agree that you have discussed the risks of your Federated Search feature usage with your company's appropriate authorities before proceeding, particularly in regard to

Test Connection
Cancel
Save

When you set up a Splunk Cloud Platform environment as a federated provider, you:

Determine whether the provider uses standard mode or transparent mode

If you are transitioning from hybrid search to federated search, use transparent mode.

Help your local federated search head connect to the remote federated provider

Supply the Splunk Cloud Platform environment host name and management port number (8089).

Provide the service account credentials

Supply the service account user id and password you defined previously.

Now you are ready to run federated searches.

Write and run federated searches

Under transparent mode you can run the same kinds of searches that you used for hybrid search, without changes to syntax.

See [Run federated searches](#) for more information about writing federated searches and about restrictions on federated searches.

Service accounts and federated search security

Before you define a remote Splunk platform deployment as a **federated provider**, create a service account on that remote deployment. The service account enables secure communication between the **federated search head** on your local Splunk platform deployment and the federated provider.

This topic also discusses the fact that [federated search supports HTTPS with TLS 1.2 encryption](#).

Federated search security models

A service account enables different security models depending on whether you define it on the remote search head of a standard mode federated provider or transparent mode federated provider.

Federated provider mode	Security model
Standard mode	The role-based access control permissions for the service account user on the federated provider determine what your local users can search on the federated provider. In addition, access to federated indexes is role-based, which allows you to restrict your local users' ability to search remote datasets on the federated provider.
Transparent mode	The role-based access control permissions for your local users determine what your users can search on the federated provider. In addition, to activate transparent mode federated search capabilities for the federated provider, the service account must have the fsh_manage capability.

For more information about the standard and transparent federated provider modes, see [About federated search](#).

Step one: Create a service account role on the remote deployment

To set up a federated provider service account on a remote deployment, you must first create an appropriate service account role on that deployment. This task differs depending on whether the federated provider you are setting up the service account for will use standard mode or transparent mode.

Note: It is not necessary for service account roles to inherit capabilities from or be otherwise equivalent to the admin, sc_admin, or power roles. They need only to have the capabilities necessary to run searches, which are applied to the user role.

If the federated provider will use standard mode

If you plan to define the remote deployment as a standard mode federated provider, create a new service account role on the remote deployment. This is the role you'll give to the service account user for the federated provider in the following task. This role sets the data access privileges and restrictions for all federated searches run over this federated provider.

See [Create and manage roles with Splunk Web](#), in the *Securing the Splunk Platform* manual.

1. On the remote deployment, in Splunk Web, navigate to **Settings > Roles**.
2. Select **New Role**.
3. Give the role a unique **Name**.

Note: Role names must use lowercase characters only. They cannot contain spaces, colons, or forward slashes. You cannot edit the names of existing roles.

4. Select the role name to continue role setup.
5. On the **Inheritance** tab of Edit Roles, ensure that the service account role has the essential capabilities for running searches by selecting the User role.
Do not have the service account role inherit from the admin, sc_admin or power roles. Do not give the service account role capabilities that are equivalent to those roles.
6. Use the other Edit Roles tabs to ensure that the role has appropriate access to data on the remote deployment for the federated searches your users will be running. Specify role capabilities, searchable indexes, search restrictions, and search-related limits.
7. Select **Save**.

Note: Service account roles for standard mode federated providers must also have read permissions for any remote datasets that you expect your federated search users to access through **federated indexes**. For example, if you are going to set up a federated index that maps to a data model on a federated provider, make sure that the service account role for that federated provider has read permissions for that data model.

For more information about setting permissions for knowledge objects like saved searches and data models, see [Manage knowledge object permissions](#).

If the federated provider will use transparent mode

If you plan to define the remote deployment as a transparent mode federated provider, create a new service account role on the remote deployment and give the role the fsh_manage and search capabilities. This is the role you'll give to the service account user for the federated provider.

Note: When you give the federated provider service account a role with the `fsh_manage` capability, you grant the admin of the federated search head on the local deployment the privilege to authorize access to indexes and data on the federated provider. The search capability ensures that searches can run over the transparent mode provider. If the service account user for a transparent mode federated provider does not have a role with the `fsh_manage` and search capabilities, that federated provider rejects all federated search requests that reach it.

See [Create and manage roles with Splunk Web](#), in the *Securing the Splunk Platform* manual.

1. On the remote deployment, in Splunk Web, navigate to **Settings > Roles**.
2. Select **New Role**.
3. Give the role a unique **Name**.

Note: Role names must use lowercase characters only. They cannot contain spaces, colons, or forward slashes. You cannot edit the names of existing roles.

4. Select the role name to continue role setup.
5. Open the **Capabilities** tab and select the `fsh_manage` and search capabilities.
No other role settings are required. When you run a federated search over this provider, the remote search head applies the role of the user running the search. This service account role facilitates access to the federated provider, nothing more.
6. Select **Save**.

Step two: Create a new service account user on the remote deployment and assign the role to it

The next step in creating a federated provider service account is creating a service account user on the remote deployment. This user is the service account for the federated provider. Assign the role you identified or created in the first step to this service account user.

Note: This step is the same whether your federated provider will use standard mode or transparent mode.

See [Create and manage users with Splunk Web](#), in the *Securing the Splunk Platform* manual.

1. On the remote deployment, in Splunk Web, navigate to **Settings > Users**.
2. Select **New user**.

Note: The service account user must be native to the remote Splunk deployment. Federated search does not support setup of service account users that are provisioned through identity providers like Active Directory and authentication schemes like Lightweight Directory Access Protocol (LDAP) or Security Assertion Markup Language (SAML).

3. Give the service account user a name, password, and time zone.
4. Give this user the remote deployment role you defined or identified in the previous task.

5. Deselect the **Require password change on first login** option.
6. Select **Save**.
7. Save a record of the username and password for the service account.
You need these credentials for the **Service Account Username** and **Service Account Password** fields when you create the federated provider definition for the remote deployment.

See [Define a federated provider](#).

Additional security for standard mode federated providers: Federated indexes

When you define a remote deployment as a standard mode federated provider, you also create **federated indexes** on the federated search head of your local deployment. See [Create a federated index](#).

On your local deployment, you must define additional role-based access control rules that identify the federated indexes to which your users have access. Each federated index on your local deployment maps to a single dataset on a standard mode federated provider, so this practice ensures that specific roles have access only to specific remote datasets.

After you create federated indexes, follow these steps.

1. On the local deployment, in Splunk Web, navigate to **Settings > Roles**.
2. Select the name of a role that you have associated to users who run federated searches.
3. Select **3. Indexes** to display the contents of the Indexes tab.
4. Locate the federated indexes you have defined. All federated index names in the **Indexes** list begin with **federated:**.
5. Select **Included** for a federated index to enable users with this role to see search results from that index.
If you do not select **Included** for any federated indexes, users with this role cannot run federated searches over standard mode federated providers.

Note: Do not add any federated indexes to the **Default** index column for a role. Users who run standard mode federated searches must always reference federated indexes by name in those searches.

6. To save all of the changes you have made and close the dialog box, select **Save**.

About HTTPS with TLS 1.2 encryption for federated search

For the purposes of federated search, an internal REST API endpoint on port 8089 facilitates communication between local and remote Splunk platform search heads using HTTPS with Transport Layer Security (TLS) 1.2 encryption. You can set up HTTPS proxy data transmission for federated search. Federated search does not support HTTP proxy data transmission.

For more information about configuring an HTTPS proxy server for a Splunk Enterprise deployment, see [Configure splunkd to use your HTTP Proxy Server](#) in the Splunk Enterprise *Admin Manual*.

For more information about configuring TLS encryption for a Splunk Enterprise deployment, see the following links in in *Securing Splunk Enterprise*.

- [About TLS encryption and cipher suites](#)
- [Introduction to securing the Splunk platform with TLS](#)

- [Configure TLS certificates for inter-Splunk communication](#) for details on activating mutually-authenticated TLS, or mTLS.

Note: To set up an HTTPS proxy server and TLS encryption for a Splunk Cloud Platform deployment, contact your Support representative.

Set the app context for standard mode federated providers

All Splunk searches run within the context of an **app** such as Search and Reporting. The app context determines the set of **knowledge objects** that the search head applies to the search during search processing. For example, when you run Splunk searches while you use the Search app, the app context of those searches guarantees that your search head applies Search-specific knowledge objects such as **calculated fields**, **lookups**, and **field extractions** to those searches.

The concept of app context is also crucial to federated searches. When you run a federated search, you must ensure that the Splunk software applies the same app context to the local and remote portions of the search. However, when you run a federated search over a standard mode federated provider, there is no way for the provider's remote search head to detect the app context of the local search head.

Resolve this issue by identifying the app context of the standard mode federated provider when you define the provider: Set the federated provider's **Application short name** to the short name of the desired app. Install the app on the federated provider if it isn't currently installed there.

Later, when you run a federated search over a standard mode federated provider, make sure the provider has the same **Application short name** as the app context of the search. If there is an app context mismatch, your federated searches might fail or return incorrect results because they are using different sets of knowledge objects on the local and remote sides of the federated search.

Note: Transparent mode federated providers do not require the **Application short name** field. In transparent mode, federated searches apply the application context that is active when the search is run on the local search head to the remote search heads on the federated providers.

For more information about federated provider setup, see [Define a federated provider](#).

For help with app installation:

- If you use Splunk Cloud Platform, see [Install apps on your Splunk Cloud Platform deployment](#) in the *Splunk Cloud Platform Admin Manual*.
- If you use Splunk Enterprise, see [Where to get more apps and add-ons](#) in the *Admin Manual*.

Benefits of setting an app context

Setting an app context for a standard mode federated provider serves three purposes.

Purpose	Details
Knowledge object scoping	Provision of an app context ensures that federated searches that use the federated provider are limited to the knowledge objects that are associated with the named app.
Provision of workload management limits	An app context allows administrators of the federated provider to set app-based <u>workload management</u> limits on your federated searches.
Logging and auditing	The app context simplifies logging and auditing of the federated provider.

Find installed apps and their application short names

To see which apps are installed on a specific Splunk platform deployment, in Splunk Web on that deployment, select **Apps > Manage Apps** to go to the Apps listing page. On the Apps listing page, the short names of the installed apps appear in the **Folder name** column.

Create standard mode federated providers that have the same host name but different app contexts

If a standard mode federated provider has the Search app as its app context, federated searches, the portions of federated searches that run over that provider can use or reference only the tags, aliases, search-time field extractions, and other knowledge objects that are associated with the Search app. If you run a federated search that references an alias or calculated field that is associated with another app, it fails on the federated provider because it can't find those knowledge objects in the Search app.

Remedy this problem by creating multiple standard mode federated provider definitions that use the same host and port but which have different app contexts. You can do this as long as each federated provider configuration has a unique name. For example, you can use the following three standard mode federated provider configurations concurrently even though they all search data on the same remote host:

Federated provider name	Remote host	Application short name
buttercupgames-search	buttercup.games.splunkcloud.com:8089	search
buttercupgames-custom1	buttercup.games.splunkcloud.com:8089	your_custom_app1
buttercupgames-custom2	buttercup.games.splunkcloud.com:8089	your_custom_app2

You can set up multiple standard mode providers with the same host and port as long as they have different provider names.

See [Define a federated provider](#).

Duplicate knowledge objects on local and remote search heads

When you run federated searches that involve knowledge objects over standard mode federated providers, you must ensure that the same sets of knowledge objects are present on the local and remote search heads. If you do not do this, you risk

having your searches fail or return incorrect results. See [Custom knowledge object coordination for standard mode federated providers](#).

Custom knowledge object coordination for standard mode federated providers

If you plan to run federated searches that invoke your custom **knowledge objects** over a standard mode **federated provider**, identify the knowledge objects that you want to use in your searches and verify that they are present on the **federated search head** and the remote search head. Ensuring that custom knowledge objects are present on the local and remote search heads helps your federated searches to complete without errors and return correct results.

For example, if you are running a standard mode federated search that references a custom **calculated field**, the calculated field definition must be present on the local and remote sides of the federated search. If the calculated field doesn't exist on the remote search head, the remote search head can't apply the calculated field to search results from the federated provider. If the calculated field exists only on the remote search head, the search fails.

Note: Skip this topic if you are using a transparent mode federated provider. When you use transparent mode federated search, the Splunk software brings your local custom knowledge objects to the remote search head through automatic bundle replication.

Example of a custom lookup in a standard mode federated search

Say you are using standard mode federated search, and you want to run a federated search that includes a custom CSV file-based **lookup** named `empAddress`. This lookup finds events in your search results with `employeeID` fields and adds corresponding `address`, `city`, `country` and `postal_code` field-value pairs to those events.

All CSV file-based lookups have two parts: a lookup definition, and a lookup table file. In this case, the lookup definition and lookup table file have the names `empAddress` and `employee_addresses.csv`, respectively.

For this example, you run two searches. The first search applies the lookup to results from a remote index on the federated provider:

```
| search index=federated:remote_index  
| lookup empAddress employeeID
```

The second search applies the lookup to search results from an index on the local deployment as well as a remote index on the federated provider.

```
| search index=local_index OR index=federated:remote_index  
| lookup empAddress employeeID
```

The following table tells you how these standard mode federated searches run, depending on the location of their lookup definitions and lookup table files.

Location of lookup definition and lookup table file	Result of running the lookup search over a standard mode federated provider
Only on the local federated search head (FSH)	The first search completes successfully but does not return results or error messages. The second search completes successfully, returning results only from the federated search head. It does not display error messages.
Only on the remote search head (RSH)	The Splunk software terminates both searches and displays error messages for them.
Same definition and file exist on both the FSH and the RSH	Both searches succeed with results from both the FSH and the RSH.

Before you run a standard mode federated search that involves a lookup, you must ensure that the lookup table file and lookup definition are duplicated on the local and remote sides of the search. If you are searching over multiple standard mode federated providers, you need to manually upload the same lookup table file to each federated provider, and you need to create identical lookup definitions on each federated provider.

The importance of duplication over naming

When you prepare to run federated searches with knowledge objects over standard mode federated providers, you can arrange for your searches to run without knowledge object errors by ensuring that there are knowledge objects with the same names on the local and remote sides of the search. However, if these identically-named local and remote knowledge objects have different definitions, this practice might cause your searches to return incorrect results.

Improve your chance of getting correct results from a standard mode federated search that involves knowledge objects by duplicating the definitions of those knowledge objects on the local and remote search heads. When the knowledge object definitions and related files (such as CSV files, for CSV file lookups) are in sync on the local and remote sides of the search, you get consistent search results.

Ensure a custom knowledge object exists on federated and remote search heads

After you identify the custom knowledge objects that you are using in your federated searches, make sure those knowledge objects are present on the federated search head as well as the remote search head on the federated provider. In most cases the easiest way to do this is through Splunk Web.

Prerequisites

- Knowledge object verification requires admin access to the local and remote search heads involved. If you do not have admin access to a Splunk platform deployment where you must duplicate knowledge objects, coordinate this work with the administrator of that deployment.
- Learn about federated provider service accounts. See [Service accounts and federated search security](#).

Steps

1. Identify a knowledge object that you want to use in your federated searches.

2. Verify that the knowledge object exists with identical definitions on the local and remote deployments involved in the search by looking it up in **Settings** on each deployment. See [Help with knowledge objects](#).
3. If the knowledge object does not exist on a deployment involved in the search, duplicate its definition on the deployment.
4. Ensure that the remote instance of the knowledge object has its permissions set so that the federated provider service account can access it. See [Manage knowledge object permissions](#) in the *Knowledge Manager Manual*.
5. If the knowledge object is a lookup, duplicate the lookup file or collection and upload or install it in the federated provider.

Repeat this process for each knowledge object you intend to use in your federated searches.

Help with knowledge objects

The following table lists knowledge object definitions, files, and collections that you must duplicate on your federated and remote search heads if you want to use them in federated searches. You can verify the existence of a knowledge object by looking it up in **Settings** for your local deployment and the remote deployments involved in the federated search.

All links go to topics in the *Knowledge Manager Manual* unless otherwise indicated.

Type of knowledge object	Items that must be duplicated among the federated and remote search heads	For more information
Custom search-time field extraction	Field extraction configurations	About fields
Calculated field	Calculated field definition	About calculated fields
Field alias	Field alias definition	Create field aliases in Splunk Web
CSV file lookup	- Lookup definition - CSV lookup table file	Define a CSV lookup in Splunk Web
External lookup	- Lookup definition - External lookup script	Create external lookups for apps in Splunk Cloud Platform or Splunk Enterprise in the <i>Developer Guide</i> on the Developer Portal
KV Store lookup	- Lookup definition - KV store collection	Define a KV store lookup in Splunk Web
Geospatial lookup	- Lookup definition - The .kmz or .kml lookup table file	Define a geospatial lookup in Splunk Web
Event type	Event type definition	About event types
Search macro	Search macro definition	Define search macros in Settings
Tag	Tag definition	Define and manage tags in Settings

Define a federated provider

To set up federated search on your local Splunk platform deployment, you must define one or more **federated providers** for that deployment. A federated provider definition enables your local deployment to establish a connection with a remote Splunk platform deployment. The federated provider definition is also where you select the provider mode, which determines how you will run your federated searches over that provider.

Prerequisites

- Read [About federated search](#) to familiarize yourself with federated search concepts and terminology.
- You must have a role with the `admin_all_objects` and `indexes_edit` **capabilities** to create a federated provider.
 - If you use Splunk Cloud Platform, the `sc_admin` role has these capabilities by default. See [Define roles on the Splunk platform with capabilities](#) in the *Securing Splunk Cloud Platform* manual.
 - If you use Splunk Enterprise, the `admin` role has these capabilities by default. See [Define roles on the Splunk platform with capabilities](#) in the *Securing Splunk Enterprise* manual.
- Gather the unique host name of the remote deployment that you want to set up as a federated provider. The format of the host name depends on whether your local deployment uses search head clustering. See the following table for the right host name format for your deployment type.

Deployment type	Uses search head clustering?	Host name format	Host name example
Splunk Cloud Platform	No	<stack name>.splunkcloud.com	buttercupgames.splunkcloud.com
Splunk Cloud Platform	Yes	shc1.<stack name>.splunkcloud.com	shc1.buttercupgames.splunkcloud.com
Splunk Enterprise	No	<deployment name>.splunk.com	buttercupgames.splunk.com
Splunk Enterprise	Yes	<deployment name>-shc.splunk.com or shc-<deployment name>.splunk.com	shc-buttercupgames.splunk.com or buttercupgames-shc.splunk.com

You can find the <stack name> or <deployment name> in the URL for the main stack of a Splunk platform deployment.

When you connect to a Splunk Cloud Platform federated provider that uses search head clustering, in most cases you will connect to the load balancer for the cluster when you use the URLs described above. The load balancer can handle the distribution of federated search data to and from the search head cluster members. The load balancer can also manage disruptions if individual search heads within the cluster go offline.

- Create a service account user on each remote deployment that you want to set up as a federated provider. You need the service account user name and password for the federated provider definition. See [Service accounts and federated search security](#).

Note: To run federated searches, Splunk Cloud Platform deployments require additional configuration from Splunk Support. This is true whether the Splunk Cloud Platform deployment is on the local or remote side of the federated search. If you are setting up federated search between two Splunk Cloud Platform deployments, you must contact Splunk Support for both deployments.

If you have a support contract, file a new case using the Splunk Support Portal at [Support and Services](#). Otherwise, contact [Splunk Customer Support](#).

Steps

1. On your local deployment, in Splunk Web, go to **Settings > Federated Search**.
2. On the **Federated Providers** tab, select **Add Federated Provider**.
3. Using the following table, specify the settings for your federated provider.

Setting	Description	Default value
Provider Type	Determines the federated provider type. Currently, this setting is fixed. You can define only federated providers that are remote deployments.	Splunk
Provider Mode	Select the mode of the federated provider. For a comparison of the standard and transparent modes, see About federated search. Transparent mode is recommended only if you are migrating to federated search from a Splunk Enterprise to Splunk Cloud Platform hybrid search setup. Note: Federated search does not support setting up a mix of transparent mode and standard mode federated providers for the same local deployment, as this practice can introduce unexpected complications. All of the federated providers for a specific local deployment must use the same provider mode. A remote deployment can be a transparent mode federated provider for one local deployment and a standard mode federated provider for a different local deployment.	Standard
Provider Name	Select a unique name for the federated provider. The provider name can contain only alphanumeric characters and underscores. The provider name cannot be the string splunk by itself. You can use this string with other alphanumeric characters. For example, abcsplunk is a valid provider name. When you run federated searches, this Provider Name is added to your search results as the value of a field named splunk_federated_provider, enabling you to group or filter results by the federated providers that produced them. The splunk_federated_provider field appears in the Interesting Fields list in the Fields sidebar. See Federated provider names added to search results.	No default

Setting	Description	Default value
Remote Host	Provide the host name and port number for the federated provider, separated by a colon character. For example: buttercupgames.splunkcloud.com:8089. You can provide an IP address instead of a host name. You can provide any legitimate port number. 8089, the standard management port number, works for any federated provider. Note: If you can't connect to port 8089 on a remote Splunk Cloud Platform deployment, contact your Splunk representative to check that the management port is open on the federated provider. For the purposes of federated search, communication between local and remote Splunk platform search heads is facilitated by an internal REST API endpoint.	No default
Service Account Username and Service Account Password	If you do not already have a service account on the federated provider, create one. A service account is a dedicated user account that allows the federated search head on your local Splunk instance to search datasets on the federated provider. See Service accounts and federated search security. Note: If you save a transparent mode federated provider definition with incorrect Service Account Username or Service Account Password values, you risk being locked out of the service account, which can prevent you from running searches. Be sure to use Test Connection to verify that you have provided accurate credentials. Fix connection issues before you save the definition.	No default
Application Short Name	Applies only to standard mode federated providers. Specify the short name of an app to apply an app context to searches on the federated provider. When you run a federated search with this federated provider, the federated search applies the app context set by Application Short Name to the portion of the search that takes place on the federated provider. It ignores the app context of the local search head that the search originates from. If you leave this setting blank, Splunk software applies search, the short name of the Search & Reporting app, to this setting. See Set the app context for standard mode federated providers.	search

4. Select **Test Connection** to test the connection to the remote deployment that this federated provider definition is meant to set up.
 You should see a "Connection successful" message at the top of the dialog if the values that you have provided for the **Provider Name**, **Remote Host**, **Service Account Username**, and **Service Account Password** fields are correct. If you get an error message instead, it means one or more of those fields has been set incorrectly. Update the fields and repeat this step until you get the **Connection successful** message. If you are having trouble making a connection, see [Troubleshoot a federated provider connection](#).
5. Select **Save** to save the federated provider configuration.

Troubleshoot a federated provider connection

If you are not able to get the **Test Connection** button to verify a connection between the federated provider and your local Splunk instance, try these troubleshooting methods.

- Make sure you have provided correct values for the **Remote Host**, **Service Account User Name**, and **Service Account Password** fields. Verify that you have created a service account user for the federated provider.
- There may be setup issues that require assistance from Splunk Customer Support, especially if you are trying to set up federated search from or to a Splunk Cloud Platform deployment. If you have a support contract, log in and file a new case using the [Splunk Support Portal](#). Otherwise, contact [Splunk Customer Support](#).

Avoid transparent mode lockouts

If you save a transparent mode federated provider definition with incorrect **Remote Host**, **Service Account Username** or **Service Account Password** information, you run the risk of being locked out of the service account by the Splunk Cloud Platform, which might remove your capability to run searches.

When the system locks your account, you have two options for unlocking it:

- If you have administrator permissions, you can unlock the account through Splunk Web. See [Unlock a user account in Splunk Web](#) in *Securing the Splunk Platform*.
- You can wait for the lockout timeout threshold to pass. When the lockout timeout threshold is reached, the system unlocks the account. The default lockout timeout threshold for the Splunk platform is 30 minutes. Your Splunk administrator may have defined a different threshold for your deployment.

Even after you unlock your account, you run the risk of being locked out again if you do not resolve the federated provider credential issue that caused the system to lock you out.

About creating multiple federated provider definitions for the same host name and port

The manner in which the Splunk software handles multiple federated provider definitions sharing the same host name and port differs depending on whether the federated provider definitions use standard mode or transparent mode.

Standard mode

You can create multiple standard mode federated provider definitions that share the same host name and port as long as they all have different **Provider name** values. You might do this if you want to create separate provider definitions for different app contexts on the same remote deployment. For more information, see [Set the app context for standard mode federated providers](#).

Transparent mode

There is no use case for having multiple transparent mode federated providers share a host name and port. If you configure multiple transparent mode federated provider definitions for the same remote deployment, each time you run a search the Splunk software delivers an error message about this misconfiguration and randomly chooses which transparent mode federated provider to run the search over.

If the remote deployments you search over use Splunk Enterprise, this functionality is extended to detect when two or more transparent mode federated providers share the same search head cluster ID. For this extension to work, the service accounts for the transparent mode federated providers must have roles with the `list_search_head_clustering` capability.

Next step for a standard mode federated provider

After you define a standard mode federated provider, create one or more **federated indexes** and associate remote datasets from the federated provider to those federated indexes. For more information, see [Create a federated index](#).

Next step for a transparent mode federated provider

After you define a transparent mode federated provider, you are ready to run federated searches. See [Run federated searches](#).

Create a federated index

After you set up one or more remote Splunk platform deployments as standard mode **federated providers** for your local Splunk platform deployment, you need to create **federated indexes** for use in federated searches. Each federated index you create maps to one remote dataset on a standard mode federated provider.

Note: Federated indexes do not ingest or store data or events. Federated indexes cannot be targets for data inputs. The function of a federated index is to route your federated search to a specific dataset on a standard mode federated provider.

The Splunk platform creates federated indexes on the **federated search head** of your local deployment.

In this task, you:

- Provide the name of the federated index.
- Select a standard mode federated provider. The federated provider must contain the remote dataset to which the federated index is mapped.
- Specify the remote dataset to which the federated index is mapped.

You can map a federated index to only one remote dataset at a time. If a federated provider contains several remote datasets over which you want to run federated searches, define a separate federated index for each dataset.

Note: Transparent mode federated providers do not use federated indexes. If you are running all of your federated searches in transparent mode, you can skip this topic.

See [About federated search](#) for an overview of the standard and transparent modes of federated search.

Specifying remote datasets

When you create a federated index, you map the index to a specific remote dataset on a standard mode federated provider. Remote datasets can be events indexes, saved searches, or data models.

Remote dataset type	Definition
Index	Each events index on a federated provider is a searchable dataset.
Saved search	The result set produced by a run of a saved search on a federated provider is a searchable dataset.
Data model	The set of events defined by a <u>data model</u> on a federated provider is a searchable dataset.

You can map a federated index to an **accelerated data model** and then search it with the `tstats` command. See [Run federated searches](#).

Use saved search datasets to route around federated search limitations

You can use saved search datasets to get around certain limitations of federated searches over a standard mode federated provider. For example, standard mode federated searches cannot be the following kinds of searches:

- Searches that use metrics search commands such as `mstats` to search data in metrics indexes.
- Searches that use any **generating commands** other than `search`, `from`, or `tstats`.

However, you can create federated indexes that map to saved search datasets that use these commands. Then you can write federated searches that reference those federated indexes. See [Run federated searches](#).

Remote dataset restrictions

The following kinds of indexes, saved searches, and data models cannot be used as remote datasets for federated searches. Do not map federated indexes to them.

- Metrics indexes
- Federated indexes
- Saved searches that contain references to federated indexes
- Data models with constraint searches that refer to federated indexes

The saved search and data model limitations relate to the fact that federated search does not support federated index chaining.

Review the permission settings on saved searches or data models that you want to use as federated search datasets. These knowledge objects must either be shared globally, or they must have the same app context as the federated provider that the federated index is associated with. In either case they must be shared with read permissions enabled.

For example, if you are creating a federated index for a federated provider that is associated with the Search app, any saved search dataset for that index must be shared with the Search app as well, or shared globally.

See [Manage knowledge object permissions](#) in the *Knowledge Manager Manual*.

Ensure federated index replication to search head cluster members in your local Splunk Enterprise deployment

Note: If your local deployment uses Splunk Cloud Platform, you can ignore this section.

If your local deployment uses Splunk Enterprise with a search head cluster as its search tier and you are going to run federated search with a standard mode federated provider, you must use the **deployer** to distribute an additional configuration to the **server.conf** files on your search head cluster members. This configuration enables your federated index definitions to replicate to each member of the search head cluster.

Note: Do not create federated indexes until you have pushed this configuration to your search head cluster members. Federated index definitions that you create before you push this configuration are not replicated to search head cluster members after you push the configuration. Searches using federated indexes that have not been properly replicated can fail or return incorrect search results.

Use the deployer to push a **configuration bundle** that adds `conf_replication_include.indexes = true` to the **[shclustering]** stanza of **server.conf** on each member of your search head cluster. To use the deployer you must have an admin or similar role with the `admin_all_objects` capability.

For more information about using the deployer to push updates to search head cluster members, see [Use the deployer to distribute apps and configuration updates](#) in the *Distributed Search* manual.

Prerequisites for federated index creation

- Read [About federated search](#) to familiarize yourself with federated search concepts and terminology.
- To create federated indexes, you must have a role with the `admin_all_objects` and `indexes_edit` **capabilities**.
 - If you use Splunk Cloud Platform, the `sc_admin` role has these capabilities by default. See [Define roles on the Splunk platform with capabilities](#) in the *Securing Splunk Cloud Platform* manual.
 - If you use Splunk Enterprise, the `admin` role has these capabilities by default. See [Define roles on the Splunk platform with capabilities](#) in the *Securing Splunk Enterprise* manual.
- You must define one or more federated providers. See [Define a federated provider](#).
- If your local deployment uses Splunk Enterprise and has a search head cluster as its search tier, you must use the **deployer** to distribute an additional configuration to the **server.conf** files on your search head cluster members. This configuration enables your federated index definitions to replicate to each member of the search head cluster. See [Ensure federated index replication to search head cluster members in your local Splunk Enterprise deployment](#).
- Know the names of the remote datasets to which you want your federated indexes to map.

Steps

1. On the local deployment, in Splunk Web, go to **Settings > Federated Search**.
2. On the **Federated Indexes** tab, select **Add Federated Index**.
3. Using the following table, specify the settings for your federated index.

Setting	Description	Default value
Federated Index Name	Specify the name of the federated index you're creating. The name must reference the remote dataset it maps to. Federated index names have the following restrictions: • They may contain only lower-case letters, numbers, underscores, and hyphens. • They must begin with a letter or number. • They cannot be more than 2048 characters in length. • They cannot contain the string "kvstore".	No default
Federated Provider	Select the standard mode federated provider that contains the dataset to which this federated index will map.	No default
Remote Dataset	Specify the remote Dataset Type that this federated index maps to and provide the Dataset Name . For Dataset Name , provide the name of a dataset of the selected Dataset Type that currently exists on the selected federated provider.	Dataset Type defaults to Index . Dataset Name has no default.

4. Select **Save** to save the federated index configuration.

The index is created on the federated search head of your local deployment.

In Splunk Web, you can view the federated indexes that you create for your deployment by selecting **Settings > Federated Search > Federated Indexes**.

Note: Do not designate federated indexes as default indexes for roles or data inputs.

Give your users access to federated indexes

After you create a federated index, give your federated search users access to the index. If you do not do this, your users cannot search the remote dataset that the federated index maps to.

Just as with normal Splunk indexes, you grant access to federated indexes at the role level. This lets you grant federated index access to certain groups of users while disallowing access to other user groups.

To learn how to add a federated index to the set of searchable indexes for a role, see the section on federated indexes in [Service accounts and federated search security](#).

Ensure service account roles can access remote datasets

Service account roles for standard mode federated providers must also have read permissions for any remote datasets that you expect your federated search users to access through **federated indexes**. For example, if you are going to set up a federated index that maps to a data model on a federated provider, make sure that the service account role for that federated provider has read permissions for that data model.

For more information about setting permissions for knowledge objects like saved searches and data models, see [Manage knowledge object permissions](#).

Reference federated indexes in federated searches

After you create your federated indexes, you can reference them in federated searches. When you reference a federated index in a search, you are searching over the remote dataset to which the federated index maps. See [Run federated searches](#).

Run federated searches

A **federated search** lets you search across specific datasets on multiple Splunk platform deployments. These deployments include your local Splunk platform deployment and remote Splunk platform deployments that you designate as **federated providers**. A federated search is processed partly on your local instance and partly on the federated providers. The remote and local results are then brought to the **federated search head** of your local deployment, where they are aggregated together to produce a final result set.

The federated search process includes search optimizations that help federated searches run as efficiently as possible. These optimizations filter results on the remote search heads, which reduces the amount of data that must be sent over the network and processed on the federated search head, improving overall search performance.

The experience of writing and running federated searches differs depending on whether your federated providers use standard mode or transparent mode.

For an overview of federated search and federated search terminology, see [About federated search](#).

General requirements for federated search

Before you can run a federated search, your administrator must designate one or more remote deployments as federated providers. See [Define a federated provider](#). Find out whether the providers you are searching over use standard mode or transparent mode.

Federated provider names added to search results

When you run federated searches, the names of the federated providers you search are added to your search results as the values of a field named **splunk_federated_provider**. You can use the **splunk_federated_provider** field to group or filter results by the federated providers that produced them.

For example, if you run transparent mode federated searches over multiple federated providers, you can use the **splunk_federated_provider** field to discern how many results are coming from each provider. Without this field it can be hard to tell where your results are coming from. The following transparent mode federated search breaks out result counts by provider name and then again by product name.

```
sourcetype=access_* | stats count(eval(splunk_federated_provider="ProviderWest")) AS Provider_West,
count(eval(splunk_federated_provider="ProviderEast")) AS Provider_East BY productName
```

This feature requires that your local Splunk platform deployment and all remote deployments set up as federated providers be upgraded to the following Splunk platform versions:

- Splunk Cloud Platform 9.0.2305 or higher.
- Splunk Enterprise 9.0.6 or higher Splunk Enterprise 9.0.x versions.

- Splunk Enterprise 9.1.1 or higher.

Search over a standard mode federated provider

Federated searches that run over a standard mode federated provider must use a special search syntax to invoke the federated search. If you are only working with standard mode federated providers and you run a search string that does not include this syntax, you are running a local search over your local deployment.

Standard mode federated searches are subject to specific requirements and restrictions.

Additional requirements for standard mode federated search

- Your administrator must create one or more **federated indexes** on your local federated search head. Each federated index maps to a specific remote dataset on a federated provider. See [Create a federated index](#).
- Your role must have permissions for the federated indexes on your local federated search head that you intend to search. See the section on federated indexes in [Service accounts and federated search security](#).
- Any custom **knowledge objects** in your search such as **lookups**, **calculated fields**, or **event types** must be present on the local search head and the remote search head. If this duplication of knowledge objects is not present or is incorrectly applied, searches may fail or return incorrect results. See [Custom knowledge object coordination for standard mode federated providers](#).

Write standard mode federated searches

The basic syntax for a standard mode federated search differs depending on the type of remote dataset that you are referencing in the search. A federated search of an events index dataset requires different syntax than a federated search of a saved search or data model dataset.

However, all standard mode federated searches require a reference to at least one federated index that you have defined on your federated search head. This federated index maps to a remote dataset on the federated provider such as an events index, saved search, or data model. See [Create a federated index](#).

You can use Boolean operators such as **AND** and **OR** to reference multiple federated indexes in a subsearch.

If your role has the `admin_all_objects` and `indexes_edit` capabilities, you can view the federated indexes to which you have access and the remote datasets that those federated indexes map to on the Federated Indexes listing page at **Settings > Federated Search > Federated Indexes**. If your role does not have this capability, get the names of the federated indexes that you can search from your administrator.

Syntax for standard mode federated searches of remote datasets

The following table provides the search commands and syntax required to search various dataset types on a standard mode federated provider. All of the examples use the **`federated:`** prefix to invoke a federated index on the federated search head that maps to a dataset on the remote search head.

Remote dataset type	Required syntax	More information
events index	<pre> search index=federated:<federated_index_for_event s_index></pre> or <pre>index=federated:<federated_index_for_event s_index></pre>	
saved search	<pre> from federated:<federated_index_for_save d_search></pre>	A saved search dataset is composed of the results of an ad-hoc run of a saved search on the remote search head. For a comparison of saved search and last job datasets, see Create a federated index.
data model	- Use from to search an unaccelerated remote data model dataset. <pre> from federated:<federated_index_for_da ta_model></pre> - Use tstats to search an accelerated remote data model dataset. <pre> tstats <stats-func> from datamodel=fed erated:<federated_index_for_data_model></pre>	Searches that use tstats in conjunction with prestats=t are not supported.

Restrictions for standard mode federated search

Standard mode federated search does not support the following:

- Generating commands** other than [search](#), [from](#), or [tstats](#). For example, federated searches cannot include the [datamodel](#) or [inputlookup](#) commands. You can find a list of generating commands in [Command types](#), in the *Search Reference*.
- Using [from](#) to reference datasets that are not saved searches or data models.
- Searches of accelerated data models where [tstats](#) has **prestats=t**.

- **Real-time search.**
- Usage of wildcard symbols (*) to reference multiple federated indexes.
- Metrics indexes and related metrics-specific search commands, such as **mpreview** or **mstats** . If you must include metrics data in a federated search, consider mapping a federated index to a saved search dataset that contains metric data. See [Create a federated index](#).

In addition, standard mode federated search supports the verbose and smart **search modes** only for **generating searches**: searches that generate events without transforming them. You can run other kinds of federated searches only in the **fast search mode**.

Note: The fast search mode does not allow search-time field extraction. You can ensure that search-time field extraction takes place for fast mode federated searches by appending `| fields *` to the ends of those searches.

Standard mode federated search examples

Here are examples of federated searches of remote datasets on standard mode federated providers.

Simple search of a large remote events index dataset

Say you have a federated index on your local search head named **airline_flights_SF** . This federated index is mapped to an events index on a remote deployment that is defined as a federated provider for your local deployment. The remote events index contains 2,000,000 events relating to airline flight departures and arrivals at a San Francisco Airport. To return the first 100 events of this large remote dataset, run this search:

```
index = federated:airline_flights_SF | head 100
```

Note: As a best practice, run a simple search like this on a federated index that you're unfamiliar with. This practice helps you determine whether the federated index name is valid, since you won't see data if the federated index is invalid. It also helps you understand whether the dataset that the federated index is mapped to holds data that is worth searching or correlating with information on your local deployment.

Combine local and standard mode federated search results for comparative analysis

Now, say you want to investigate the performance of specific airline carriers at competing airports. The following search uses the **union** command to combine a remote saved search dataset of San Francisco flight data and a local events index dataset of New York City flight data. Then it finds the average arrival delay between airports in the combined dataset, broken out by airline.

```
| union  
[|from federated:search_of_airline_flights_SF]
```

```
[search index = airlinedata_NYC]
| stats avg (ArrDelay) by UniqueCarrier
```

If you want to search a local events index and a remote events index, you can use an **OR** operator to combine the results:

```
index = airlinedata_NYC OR index = federated:airline_flights_SF
| stats avg (ArrDelay) by UniqueCarrier
```

Aggregate results with join for a count of unique values

The following standard mode federated search uses the **join** command to aggregate results from the remote and local deployments. It returns the total count of airline carriers running flights out of San Francisco and New York City.

```
index = federated:airline_flights_SF
| join left = L right = R where L.UniqueCarrier = R.UniqueCarrier
  [search index = airlinedata_NYC]
| stats count
```

Filter a remote data model dataset search by one or more index names

When you run a **tstats** search of a remote data model dataset on a standard mode federated provider, it is often helpful to filter the results of the search by one or more indexes. Indexes referenced by the **WHERE** keyword must be present on the remote search head.

```
| tstats count from datamodel = federated:remote_dm_1
WHERE index=index_fs_1 OR index=index_fs_2
```

You can also run **tstats** searches that mix local and federated indexes.

```
|tstats count where index=local_index_1 OR index=federated:remote_dm_1
```

Filter a remote data model dataset search on child data model datasets

When you run a **tstats** search of a remote data model dataset on a standard mode federated provider, you can filter the results on one or more of the child datasets within the data model by referencing the **nodename** of the child dataset.

```
| tstats count from datamodel = federated:remote_dm_1
WHERE nodename = DM_1_DS.DM_1_DS_CHILD_1
```

See the [tstats](#) topic in the *Search Reference* for more information about using **nodename** to filter **tstats** searches on child data model datasets.

Search over a transparent mode federated provider

When you run searches over a transparent mode federated provider, you can search as if you were searching over your local deployment. No special syntax is required. The search permissions associated with your role govern what you can search on the federated provider.

Restrictions for transparent mode federated search

Transparent mode federated search does not support the following things:

- **Real-time search.**
- Searching accelerated data models.
- Using **from** to search saved search datasets on the federated provider. You can use **from** to search saved search datasets on your local deployment.
- Usage of the **federated:** syntax to refer to specific **federated indexes**. Transparent mode searches do not use federated indexes.

Troubleshooting federated searches

Federated searches can fail to return events for a variety of reasons. The following table covers the most common error messages and conditions, and gives you some ways to resolve them.

Error message or condition	Resolution	Link to more information
Invalid federated index specified	Provide the name of a federated index that exists on your local deployment.	Create a federated index
Invalid federated provider specified.	Your search references a federated index which specifies a nonexistent federated provider in its definition. Contact your administrator and have them correct the federated index definition.	Create a federated index
Unauthorized.	The service account credentials that have been set up for a federated provider in this search may be incorrect. Verify that the Service Account Username and Service Account Password that have been designated for the federated provider are correct.	Service accounts and federated search security
	If the federated provider uses transparent mode and the service account user for the provider has a role that does not have the fsh_manage capability, searches	Service accounts and federated search

Error message or condition	Resolution	Link to more information
	might not run.	security
	If the federated provider uses standard mode, the service account user for the federated provider should have permission to search the index, saved search, and data model datasets on the federated provider that are involved in the search. For example, if the search references an index dataset on the federated provider, the service account for that federated provider should have the ability to search that index dataset. This ability is associated with the service account role.	Manage knowledge object permissions in the <i>Knowledge Manager Manual</i>
Zero results returned.	If the federated provider uses standard mode, verify that your role's permissions give you access to the federated indexes referenced in the search.	Service accounts and federated search security
	If the federated provider uses standard mode, verify that the federated index is mapped to a valid remote dataset.	Create a federated index
	If the remote dataset you are trying to search is an events index, verify that the events index is not disabled.	If you need to verify whether an index is disabled: • And you use Splunk Cloud Platform, see Manage Splunk Cloud Platform indexes in the <i>Splunk Cloud Platform Admin Manual</i>. • And you use Splunk Enterprise, see Remove indexes and indexed data in <i>Managing Indexers and Clusters of Indexers</i>.
	If you are using a standard mode federated provider and the remote dataset you are trying to search is a knowledge object such as a saved search or data model, verify that the knowledge object's permissions enable you to search it.	Manage knowledge object permissions in the <i>Knowledge Manager Manual</i>
	If possible, run a search job directly on the remote search head to verify that the search head is working correctly.	
	If you are using a standard mode federated provider, verify that the remote dataset that the federated index maps to contains more than zero events.	
Search fails to complete, possibly with the error	Splunk software can terminate federated searches when their search result preview generation duration exceeds a timeout set by another component in your network, such as an elastic load balancer (ELB). When your search is terminated in	You can change this value with a REST API call. See Federated

Error message or condition	Resolution	Link to more information
"Socket error during transaction. Socket error: Success".	this manner you can change the <code>max_preview_generation_duration</code> setting in <code>federated.conf</code> to a number of seconds lower than the timeout. For example, if you have an ELB that times out at 60 seconds, you might set the <code>max_preview_generation_duration</code> to 55.	search endpoint descriptions in the <i>REST API Reference Manual</i> .
Knowledge object issues for standard mode federated providers	The knowledge objects aren't where the federated provider definition indicates they will be. Verify that the knowledge objects belong to the application identified by Application Short Name , and that the application is installed on the federated provider.	Set the app context for standard mode federated providers
	Your search includes custom knowledge objects that are not duplicated on the remote search head of the federated provider.	Custom knowledge object coordination for standard mode federated providers
	Verify that the permissions of knowledge objects on the remote search head are set so that users running federated searches from the local search head can access them.	Manage knowledge object permissions in the <i>Knowledge Manager Manual</i> .
Knowledge object issues for transparent mode federated providers	Did you run a search within minutes of setting up a transparent mode federated provider? The process by which knowledge objects are transferred from your local deployment to a transparent mode federated provider takes a few minutes. You might be encountering issues because the knowledge objects upon which your search depends aren't yet present on the remote search head. This latency can vary depending on your network bandwidth and the size of the knowledge object bundle. Wait and try your search again.	

Export Search Results

Export search results

You can export search results from your Splunk deployment, and forward data to third-party systems, as described in this topic.

What are the available export methods?

The Splunk platform provides several export methods:

- [Export data using Splunk Web](#)
- [Export data using the CLI](#)
- [Export data using SDKs](#)
- [Export data using REST API](#)

- [The `dump` search command](#)
- [Data forwarding](#)

Splunk apps

- [Deploy and Use Splunk DB Connect](#)
- [Install and Use Splunk ODBC Driver with Microsoft Excel](#)
- [Install and Use Splunk ODBC Driver with Microsoft PowerBI](#)
- [Install and Use Splunk ODBC Driver with Tableau](#)

Export options

The export method you choose depends on the data volumes involved and your level of interactivity. For example, a single on-demand search export through Splunk Web might be appropriate for a low-volume export. Alternatively, if you want to set up a higher-volume, scheduled export, the SDK and REST options work best.

For large exports, the most stable method of search data retrieval is the Command Line Interface (CLI). From the CLI, you can tailor your search to external applications using the various Splunk SDKs. The REST API works from the CLI as well, but is recommended only for internal use.

In terms of level of expertise, the Splunk Web and CLI methods are significantly more accessible than the SDKs and REST API, which require previous experience working with software development kits or REST API endpoints.

Method	Volume	Interactivity	Remarks
Splunk Web	Low	On-Demand, Interactive	Easy to obtain on-demand exports
CLI	Medium	On-Demand, Low Interactive	Easy to obtain on-demand exports
REST	High	Automated, best for computer-to-computer	Works underneath SDK
SDK	High	Automated, best for computer-to-computer	Best for automation

Supported export formats

You can export Splunk data into the following formats:

- Raw Events (for search results that are raw events and not calculated fields)
- CSV
- JSON
- XML
- PDF (for saved searches, using Splunk Web)

Export data using Splunk Web

You can export the event data from a search, report, or pivot job to various formats. You can then archive the file, or use the file with a third-party charting application.

- After you run a search, report, or pivot, click the Export button. The Export button is one of the Search action buttons.



Note: If the button is not visible, it has been hidden by your system administrator to prevent data export.

Use the Export Results window to specify the format and name for your export file:

Export Results

X

Format

CSV ▾

File Name ?

optional

Number of Results

leave blank to export all results

Cancel

Export

Sometimes your search must be run again before the results can be exported. See [When exporting triggers your search to run again](#).

- Click **Format** and select the format that you want the search results to be exported in. The supported formats depend on the type of job artifact that you are working with.

Format	Ad hoc searches	Saved searches	Notes
CSV	X	X	
JSON	X	X	
PDF		X	If the search is a saved search, such as a Report, you can export using the PDF format.
Raw Events	X	X	If the search generates calculated data that appears on the Statistics tab, you cannot export using the Raw Events format.
XML	X	X	

- Optional. In the **File Name** field, you can type a name for the export file where the event data will be stored. If you do not specify a file name, a file is created using the search job ID as the file name. The search job ID is the UNIX time when the search was run. For example **1463687468_7.csv**.

4. Optional. In the **Number of Results** field, you can specify the number of results that you want to export. If you do not specify a number, all of the events are exported. For example, if you specify 500 in the **Number of Results** field, only the first 500 results returned from your search are exported.
5. Click **Export** to save the job events in the export file.

The file is saved in the default download directory for your browser or operating system. For example, for most Windows and Mac OS X users the export file appears in the default **Downloads** directory. On Linux, check the XDG configuration file for the download directory.

When exporting triggers your search to run again

If your search returns a large number of results, it is possible that not all of the results will be stored with the **search job artifact**.

When you export search results, the export process is based on the search job artifact, not the results in the Search app. If the artifact does not contain the full set of results, a message appears at the bottom of the Export Results dialog box to tell you that the search will be rerun by the Splunk software before the results are exported.

The search is rerun when the search head believes that it cannot retrieve all of the events from the job artifact. The search head determines when to rerun the search based on the following logic:

- If the search is not a report, and one of the following is true.
 - The search is not done
 - The search is using a remote timeline
 - The search head believes that the search has not retained all of events

Extend the session timeout when exporting large amounts of data

Note: This capability is not available to Splunk Cloud Platform users.

When you export large amounts of data using the Export button, the session might timeout before the export is complete. Splunk Enterprise users who have a role with the **edit_server** capability can extend the session timeout limit.

1. Click **Settings > Server Settings > General Settings**.
2. In the **Splunk Web** section, increase the number in the Session timeout field.
3. Click **Save**.

Forward data to third-party systems

You can forward the data that you export to third-party systems.

- For an brief overview, see [Forward data to third party systems](#) in this manual.
- For more details, see [Forward data to third party systems](#) in *Forwarding Data*.

Use reports to send results to stakeholders

You can schedule reports to run on a regular interval and send the results to project stakeholders by email. The emails can present the report results in tables in the email, and as CSV or PDF attachments. The emails can also include links to the report results in Splunk Enterprise. See [Schedule Reports](#) in the *Reporting Manual*.

Export data using the CLI

The Command Line Interface (CLI) is easy to script, can handle automation, and can process volumes of data faster and more efficiently than Splunk Web. To access Splunk Enterprise through the CLI, you either need shell access to a Splunk Enterprise server, or permission to access the correct port on a remote Splunk server.

Note: Splunk Cloud Platform users don't have shell access to the Splunk Cloud Platform deployment and therefore can't use the CLI to export data.

The syntax for exporting data using the CLI is as follows:

```
splunk search [eventdata] -preview 0 -maxout 0 -output [rawdata | table | csv | raw | auto | json] >
[myfilename.log] ...
```

If **-output** is not specified, the default output depends on the type of search. For non-transforming searches, the default output format is **rawdata**. For transforming searches, the default output format is **table**.

By default, you can export a maximum of 100 events. To increase this number, use the **-maxout** argument. For example, if you include **-maxout 300000** you can export 300,000 events. Set **-maxout** to 0 to export an unlimited number of events.

To learn more about the Splunk Enterprise CLI, read [About the CLI](#) in the *Admin Manual*.

CLI output command example

This CLI example takes events from the **_internal** index that occur within the time range specified by the search string and outputs 200,000 of them in raw data format to the file **test123.dmp**.

```
splunk search "index=_internal earliest=09/14/2024:23:59:00 latest=09/16/2024:01:00:00 " -output rawdata
-maxout 200000 > c:/test123.dmp
```

Export data using the Splunk REST API

Use the Splunk REST API to access data from the command line or a Web browser.

REST API access for Splunk Cloud Platform deployments

If you have a Splunk Cloud Platform deployment and you want to use the Splunk REST API, file a Support ticket requesting the API to be enabled. Free trial Splunk Cloud Platform accounts cannot access the REST API. See [Using the REST API with Splunk Cloud Platform](#) in the *REST API Tutorials* for more details.

Export data

Exporting data starts with running a search job to generate results. You can then export this search result data to a file.

1. Run a search job using a POST to `/services/search/jobs/`. If you are using a custom time range, pass it in with the POST request.

```
curl -k -u admin:changeme \
  https://localhost:8089/services/search/jobs/ -d search="search sourcetype=access_* earliest=-7d"
```

2. Get the search job ID (SID) for the search. The `/jobs` endpoint returns an XML response including the `<sid>`, or search job ID.

```
<?xml version='1.0' encoding='UTF-8'?>
<response>
  <sid>1423855196.339</sid>
</response>
```

You can also get the search job ID by viewing the job in the [Search Job Inspector](#) in Splunk Web. Navigate to **Activity > Jobs** to open the Job Manager. Locate the search job that you just ran and click **Inspect**. The Search Job Inspector opens in a separate window. See [View the properties of a search job](#).

3. Use a GET request on the `/results` endpoint to export the search results to a file. Ensure that you do the following in the GET request:

- **Identify your object endpoints.**

To see a list of currently available object endpoints for your user, within your app, navigate to `https://localhost:8089/servicesNS/<user>/<app>/`.

For example:

```
https://localhost:8089/servicesNS/admin/search/saved/searches/
```

- **Identify the search job user and app.**

The following example defines `<user>` as `admin` and `<app>` as `search`.

- **Identify an output format.**

Use the `output_mode` parameter to specify one of the following available output formats. Use lower case for the format name, as shown here.

```
atom | csv | json | json_cols | json_rows | raw | xml
```

This example exports search results to a JSON file.

```
curl -u admin:changeme \  
-k https://localhost:8089/servicesNS/admin/search/search/jobs/1423855196.339/results/ \  
--get -d output_mode=json -d count=5
```

See also

For more details about the `/jobs` and `/export` endpoints, see the following information in the [REST API Reference](#).

- [search/jobs](#)
- [search/jobs/export](#)

See also [Creating searches using the REST API](#) in the *REST API Tutorials*.

Export data using the Splunk SDKs

Splunk Software Development Kits (SDKs) enable software developers to create Splunk apps using common programming languages. Splunk SDKs let you integrate Splunk deployments with third-party reporting tools and portals, include search results in your application, and extract high volumes of data for archival purposes. To use Splunk SDKs, you should be proficient in SDK knowledge and development.

Splunk offers SDKs for Python, Java, and JavaScript. When you run an export-search in these SDKs, the search runs immediately, it does not create a job for the search, and it start streaming results immediately.

The Splunk SDKs are built on top of the Splunk REST API. They provide a simpler interface for the REST API endpoints. With fewer lines of code, you can write applications that can:

- Create and run authenticated searches
- Add data
- Index data
- Manage search jobs
- Configure Splunk

For more information about the Splunk SDKs, read "[Overview of the Splunk SDKs](#)" in the [Splunk Developer Portal](#).

Use Python SDK to export data

The Splunk SDK for Python lets you write Python applications that can interact with Splunk deployments. Export searches using the Python SDK can be run in historical mode and real-time mode. They start right away, and stream results instantly, letting you integrate them into your Python application.

Perform an export search using the Python SDK. This script has been made cross-compatible with Python 2 and Python 3 using [python-future](#).

1. Set the parameters of what you wish to search. The following example sets the parameters as an export search of **splunklib** in the last hour.

```
sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "lib"))
import splunklib.client as client
import splunklib.results as results
from __future__ import print_function
```

Note that **sys.path.insert** adds **lib** to the path so that the app calls the version of **splunklib** installed with this app, which you should store in the `/lib` directory of the app, as detailed in [The directory structure of a Splunk App](#) in Splunk developer docs.

2. Change or acquire these values, as necessary.

```
HOST = "localhost"
PORT = 8089
USERNAME = "admin"
PASSWORD = "changeme"
```

3. Run a normal-mode search.

```
service = client.connect(
    host=HOST,
    port=PORT,
    username=USERNAME,
    password=PASSWORD)

rr = results.ResultsReader(service.jobs.export("search index=_internal earliest=-1h | head 5"))
```

4. Get the results and display them using the ResultsReader.

```
for result in rr:
    if isinstance(result, results.Message):
        # Diagnostic messages might be returned in the results
        print('%s: %s' % (result.type, result.message))
    elif isinstance(result, dict):
        # Normal events are returned as dicts
        print(result)
assert rr.is_preview == False
```

Use Java SDK to export data

The Java SDK is able to conduct and export searches while using Java.

To perform an export search using the Java SDK, run the following example in the `/splunk-sdk-java` directory using the CLI:

```
java -jar dist/examples/export.jar main --username="admin" --password="changeme"
```

The Export application exports the "main" index to **export.out**, which is saved to the current working directory. If you want to run this application again, delete **export.out** before you try again. If you do not do this, you will get an error.

Here is a different CLI example of the Java SDK. It shows how to include a search query and change the output format to JSON.

```
java -jar dist/examples/export.jar main --search="search sourcetype=access_*" json
```

Use JavaScript Export to export data

The Javascript Export endpoint can export Splunk data in the Javascript framework. Though the Splunk Javascript SDK does not currently support the Javascript Export endpoint, you can use a node javascript (.js) application request to export data.

To perform an export search using the Javascript Export endpoint:

1. Load the request module. Request is designed to be the simplest way to make an http/https call.

```
var request = require('request');
```

2. Call **get** to issue a GET request. Enter the following parameters:

- **strictSSL** – When set to false, **strictSSL** tells the request to not validate the server certificate returned by your Splunk deployment, which by default is not a valid certificate.
- **uri** – Provide the **uri** of the Splunk host along with the path for the export endpoint. A JSON response is specified in the query string.
- **qs** – Set **qs** to supply the search parameter. By passing it this way, you do not have to URI encode the search string.

```
request.get(  
  {  
    strictSSL: false,  
    uri: 'https://localhost:8089/servicesNS/admin/search/search/jobs/  
        export?output_mode=json',  
    qs: {  
      search: 'search index=_internal'    }  
  })
```



```
}  
}  
)
```

3. Call **auth** to use HTTP Basic Auth and pass your Splunk username and password.

```
.auth('admin', 'changeme', false)
```

4. Pipe the results to **stdout**.

```
.pipe(process.stdout);
```

Export data using the dump command

You can use the [dump](#) search command to export large collections of events onto a local disk. You can use this command with the CLI, Splunk SDK, and Splunk Web.

The basic syntax of the **dump** command is:

```
dump basefilename=<string> [rollsize=<number>] [compress=<number>] [format=<string>] [fields=<comma-delimited-string>]
```

The **<format>** is the data format of the dump file that you are creating. Your format options are **raw**, **csv**, **tsv**, **xml**, and **json**.

For search examples and full explanations of the required and optional arguments, see the [dump](#) command in the *Search Reference*.

Forward data to third-party systems

Splunk software can forward data to third-party systems as follows:

- Through a plain TCP socket
- Packaged in a standard syslog

To forward data to third-party systems, you configure heavy forwarders by editing the **outputs.conf**, **props.conf** and **transforms.conf** files. This export method is similar to routing your data to other Splunk deployments. You can filter the data by host, source, or source type.

See [Forward data to third party systems](#) in the *Forwarding Data* manual.

Write Custom Search Commands

About writing custom search commands

Note: This documentation has moved and been updated. See [Create custom search commands for apps in Splunk Cloud Platform or Splunk Enterprise](#) in the Developer Guide on the Developer Portal.

If you use Splunk Cloud Platform, you do not have file system access to your Splunk deployment. If you want to create custom search commands, contact Professional Services to create the custom search command and then package the command in an app for you to install. See [Manage private apps on your Splunk Cloud Platform deployment](#) for more information.

Search Examples and Walkthroughs

Calculate sizes of dynamic fields

This search determines which fields in your events consume the most disk space, without any prior knowledge of field names and number of events.

Scenario

```
index=_internal earliest=-15m latest=now
| fieldsummary
| rex field=values max_match=0 "value\":\"(?<values>[^\"]*)"\",
| mvexpand values
| eval bytes=len(values)
| rex field=field
"^(?!date|punct|host|hostip|index|linecount|source|sourcetype|timeendpos|timestartpos|splunk_server)(?<FieldNam
e>.*)"
| stats count sum(bytes) as SumOfBytesInField values(values) as Values max(bytes) as MaxFieldLengthInBytes by
FieldName
| rename count as NumOfValuesPerField
| eventstats sum(NumOfValuesPerField) as TotalEvents sum(SumOfBytesInField) as TotalBytes
| eval PercentOfTotalEvents=round(NumOfValuesPerField/TotalEvents*100,2)
| eval PercentOfTotalBytes=round(SumOfBytesInField/TotalBytes*100,2)
| eval ConsumedMB=SumOfBytesInField/1024/1024
| eval TotalMB=TotalBytes/1024/1024
| table FieldName NumOfValuesPerField SumOfBytesInField ConsumedMB PercentOfTotalBytes PercentOfTotalEvents
| addcoltotals labelfield=FieldName label=Totals
| sort - PercentOfTotalEvents
```

The results appear on the Statistics tab and look something like this:

FieldName	NumValuesPerField	SumOfBytesInField	ConsumedMB	PercentOfTotalBytes	PercentOfTotalEvents
Totals	1802	45700	0.0436	99.87	100.29
cumulative_hits	100	587	0.0006	1.28	5.55
eps	100	1862	0.0018	4.07	5.55
kb	100	1159	0.0011	2.54	5.55
kbps	100	1881	0.0018	4.12	5.55
req_time	100	3000	0.0029	6.56	5.55
uri	100	10559	0.0101	23.11	5.55
uri_query	100	3532	0.0034	7.73	5.55
message	96	11633	0.0111	25.46	5.33
avg_age	76	280	0.0012	2.80	4.22
ev	62	140	0.0001	0.31	3.44
average_kbps	59	1071	0.0010	2.34	3.27

Note: The totals do not add up to exactly 100% because this search is adding up the values after the values have been rounded. To avoid this, you can move the rounding to the end of the search string.

Walkthrough

Let's walk through each part of the search.

1. The example begins with a search to retrieve all events in `index=_internal` within the last 15 minutes.

```
index=_internal earliest=-15m latest=now
```

Note: You can replace this with any search string and time range.

2. Next, add the `fieldsummary` command to create a summary of all the fields in the previously retrieved events.

```
| fieldsummary
```

The results appear on the Statistics tab and look something like this:

field	count	distinct_count	is_exact	max	mean	min	numeric_count	stdev	values
abandoned_channels	29	1	1	0.0	0.00	0.0	29	0.00	[{"value": "0", "count": 29}]
active	29	1	1	0.0	0.00	0.0	29	0.00	[{"value": "0", "count": 29}]
active_hist_searches	31	2	1	1.0	0.13	0.0	31	0.34	[{"value": "0", "count": 27}, {"value": "1", "count": 4}]
average_kbps	87	59	1	0.3	0.21	0.0	87	0.15	[{"value": "0", "count": 29}, {"value": "0.31239045073685034", "count": 1}, {"value": "0.31240549380412547", "count": 1}, {"value": "0.3124557194522294", "count": 1}, {"value": "0.3124707607545469", "count": 1}]

- The values of each field are extracted with a regex into a multivalue field called **values**, and then expanded. The length of each value is calculated in bytes.

```
| rex field=values max_match=0 "value\":"(?(values>[^\"]*)\"","
| mvexpand values
| eval bytes=len(values)
```

- The values of the field are extracted with another regex, with some exceptions.

```
| rex field=field
"^(!date|punct|host|hostip|index|linecount|source|sourcetype|timeendpos|timestartpos|splunk_server)(?<Field
ldName>.*)"
```

- The **stats** command is used to perform multiple calculations using stats functions, including the count and the sum of the bytes (SumOfBytesInField). The values function is used to return the list of all distinct values of the **values** field as a multivalue entry (Values). The max function calculates the maximum field length in bytes (MaxFieldLengthInBytes). The results are organized by field name.

```
| stats count sum(bytes) as SumOfBytesInField values(values) as Values max(bytes) as MaxFieldLengthInBytes
by FieldName
| rename count as NumOfValuesPerField
```

- The **eventstats** command is used to calculate several sums, the number of values in each field (TotalEvents) and the sum of the bytes in each field (Total Bytes).

```
| eventstats sum(NumOfValuesPerField) as TotalEvents sum(SumOfBytesInField) as TotalBytes
```

- Several **eval** commands are run to calculate the percentage of total events, the percentage of total bytes, the megabytes consumed, and the total megabytes.

```
| eval PercentOfTotalEvents=round(NumOfValuesPerField/TotalEvents*100,2)
| eval PercentOfTotalBytes=round(SumOfBytesInField/TotalBytes*100,2)
| eval ConsumedMB=SumOfBytesInField/1024/1024
| eval TotalMB=TotalBytes/1024/1024
```

- The **table** command is used to display on a specific set of fields. The **addcoltotals** command is used to calculate the total for each column. The **sort** command is used to sort the list in descending order by the PercentOfTotalEvents field.

```
| table FieldName NumberOfValuesPerField SumOfBytesInField ConsumedMB PercentOfTotalBytes
PercentOfTotalEvents
| addcoltotals labelfield=FieldName label=Totals
| sort - PercentOfTotalEvents
```

The results appear on the Statistics tab and look something like this:

FieldName	NumValuesPerField	SumOfBytesInField	ConsumedMB	PercentOfTotalBytes	PercentOfTotalEvents
Totals	1802	45700	0.0436	99.87	100.29
cumulative_hits	100	587	0.0006	1.28	5.55
eps	100	1862	0.0018	4.07	5.55
kb	100	1159	0.0011	2.54	5.55
kbps	100	1881	0.0018	4.12	5.55
req_time	100	3000	0.0029	6.56	5.55
uri	100	10559	0.0101	23.11	5.55
uri_query	100	3532	0.0034	7.73	5.55
message	96	11633	0.0111	25.46	5.33
avg_age	76	280	0.0012	2.80	4.22
ev	62	140	0.0001	0.31	3.44
average_kbps	59	1071	0.0010	2.34	3.27